BESCHREIBUNG

GRAFIK-Interpreter: Beschreibung

Zweck, Fenster, Geräte, Töne, Aufruf und Grenzen des GRAFIK-Modus im Wortlaut der Quelle

Dieses PDF gibt die Quelle im Wortlaut wieder; Listen, Tabellen und Codeblöcke entsprechen der Quelle, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

Quelle	C8205-Interpreter-GRAFIK_Beschreibung.md	0.95-080
Stand	11.11.2026	
Umfang	83 Seiten, A4 hoch	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Inhaltsverzeichnis

1 Zweck, Abgrenzung zu TEST . . . 4

2 Start und Schalter . . . 5

3 Bedienpult-Fenster . . . 7

3.1 Fenster, Layout, Bildschirm . . . 7
3.2 Lampen . . . 7
3.3 Tasten, Schalter und Selektoren . . . 8
3.4 Netzschalter und Netzablauf . . . 9
3.5 Lampentempo und ruhiges Bild . . . 9
3.6 Bedienablauf: vom Einschalten bis zum Start eines Programms . . . 10
3.7 Bedienung während des Programmlaufs und Ende . . . 10
3.8 E und M+ E im Halt . . . 11
3.9 Aufbau der Pult-Software und Threads . . . 12

4 Bedienung . . . 12

4.1 Auslösung und Tastenarten . . . 12
4.2 Sperren bei Stellung R . . . 13
4.3 E und M: Halten, Umschalt+Klick, E+ und E- . . . 13
4.4 Darstellung, Touchscreen, Fokus . . . 14
4.5 Computertastatur . . . 14

5 SW-Fenster . . . 15

5.1 Fenster und Anzeige . . . 16
5.2 Wann eine Eingabe angenommen wird . . . 16
5.3 Zeichenabbildung . . . 17
5.4 Lage KB/GB und Umschaltzeichen . . . 17
5.5 Puffer (seit 19.10.2026) . . . 18
5.6 Rücktaste (seit 19.10.2026) . . . 18
5.7 Fokus, Titel und Weiterleitung vom Pult . . . 19
5.8 Debug-Log und Spur . . . 19
5.9 Überholte Fassungen (zum Lesen der Altdokumente) . . . 19
5.10 Kennungen, Stand der Festlegungen . . . 20

6 Töne . . . 20

6.1 Aufbau: ein Tonstrom, ein Mischer . . . 20
6.2 Schreibwerk . . . 21
6.3 Lochstreifenleser . . . 21
6.4 Lochbandstanzer . . . 22
6.5 Rechner: Grundgeräusch und Netzablauf . . . 22
6.6 Lautstärke und Stummschaltung . . . 22
6.7 Start und Fehlerfälle . . . 23
6.8 Plattformen . . . 23
6.9 Konstanten und ihr Stand . . . 23
6.10 Befunde und Hinweise zu den Quellen . . . 24
6.11 Kennungen . . . 26
6.12 Tastengeräusche des Pults . . . 26

7 Echte Trommel . . . 27

7.1 Bausteine . . . 27
7.2 Regeln der Nachbildung . . . 27
7.3 Takt . . . 28
7.4 Plattformschicht . . . 28
7.5 Anbindung an den GRAFIK-Interpreter . . . 28
7.6 Folgen . . . 29
7.7 Prüfstand . . . 30
7.8 Zeitmodelle und Zähler der Trommelstatistik . . . 30
7.9 Kennungen . . . 32
7.10 Zusatztrommeln des experimentellen Prototyps C8205Z (--C8205Z) . . . 32
7.11 Taktprotokoll (--taktprotokoll) . . . 33

8 Hardware-Check . . . 35
8.1 Fenster . . . 35
8.2 Anzeige: sechs Zeilen untereinander . . . 35
8.3 Haltzeit (seit 19.10.2026) . . . 35
8.4 Fokus . . . 36
8.5 Stand, Annahmen, Prüfung . . . 36
8.6 Kennungen . . . 37
8.7 Zusatztrommeln im HW-Check (nur mit --C8205Z, experimenteller Prototyp C8205Z) . . . 37

9 Plattformen . . . 37
9.1 Linux . . . 38
9.2 Windows (Teilabnahme 26.10.2026) . . . 38
9.3 macOS . . . 41

10 Offene Punkte . . . 41
10.0 Gestrichene Punkte (22.10.2026) . . . 41
10.1 Windows (eigene Abnahme, Teilabnahme 26.10.2026) . . . 41
10.2 Zu entscheiden oder zu bestätigen . . . 42
10.3 Standardannahmen ohne Einzelbestätigung . . . 42
10.4 Bekannte Einschränkungen und Befunde . . . 43
10.5 Als erledigt belegt . . . 44
10.6 Arbeitsvermerk Stufe 6 (entfällt nach Abschluss) . . . 59
10.7 Kennungen . . . 60

11 Aufbau der GRAFIK-Module . . . 60
11.1 Warum eine zweite Programmfamilie . . . 60
11.2 Schichten . . . 61
11.3 Was man wissen muss . . . 62
11.4 Kennungen . . . 62

Anhang A Entscheidungsregister . . . 62

Anhang B Übersetzungstabelle alt → neu . . . 79
B.1 Dokumente . . . 79
B.2 Kennungsfamilien . . . 79
B.3 Verweise im Code und in Dokumenten (später umzustellen) . . . 82

Knotengrenze in GRAFIK (G4-014/G4-015, 26.10.2026) . . . 83

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Version: 0.95-080

Stand: Gerüst, Abschnitte 1 bis 11 geschrieben (§10 mit teilweiser Triage, siehe §10.6); Prüfverfahren §1 bis §10 liegt vor (`C8205-Interpreter-GRAFIK_Pruefverfahren.md`**).** Das PDF dazu (`C8205-GRAFIK_Beschreibung_A4.pdf`) wird mit `make_all_pdf.sh` erzeugt. Entstanden in Stufe 6 (Dokumentation konsolidieren); Plan und Anforderungen: Anforderungspaket Dokument 12 (fällt nach Abschluss weg).

Leser: Anwender und Entwickler des GRAFIK-Interpreters. Das Dokument ersetzt nach Abschluss von Stufe 6 die Anforderungsdokumente 00 bis 12. Alte Kennungen (F-13, S4-09 usw.) stehen im Text in Klammern und in Anhang B.

1 Zweck, Abgrenzung zu TEST

Inhalt: Zwei Modi, `--MODUS`, was in GRAFIK wirkt und was nicht

Quellen (alt): Dok. 00 §3.6; Dok. 11 S5-01 bis S5-07

Zwei Modi. Der Python-Interpreter `c8205_interp.py` hat zwei Betriebsarten, gewählt mit `--MODUS=TEST` oder `--MODUS=GRAFIK` (Vorgabe 1, 15.10.2026; Umsetzung S5-01). **Ohne Angabe gilt GRAFIK** (Entscheidung 1 zu `--MODUS`, 15.10.2026). Geschrieben werden `--MODUS=X`, `--MODUS X` und `--modus`; der Wert ist unabhängig von Groß- und Kleinschreibung, bei Wiederholung gilt der letzte (S5-02, Vorschlag des Entwicklers, in Stufe 5 umgesetzt).

Gegenstand	`--MODUS=TEST`	`--MODUS=GRAFIK`
Ablauf	Stapelbetrieb: Tastenfolge der Testdatei, dann Knotenschleife bis zum Halt; verhält sich wie vor Stufe 5 (S5-04)	Dauerbetrieb am Bedienpult; Start mit ausgeschaltetem Rechner, Netzablauf über den Netzschalter (Klick → B rot → `fade_in` → B grün; der Interpreter läuft erst bei B grün)
Fenster	keines; bei `--SW-TERMINAL` das Fenster „SW Ausgabe“	Bedienpult (fest 1148 × 860) und SW-Fenster (130 × 36 Zeichen), dazu das Terminal; auf 1920 × 1080 nebeneinander, Überlappung erlaubt
Töne	keine, keine Wartezeit des Hochlaufs	Schreibwerk, Lochstreifenleser, Rechner, Stanzer (§6); abschaltbar mit `--STUMM`
Trommel	persistente Textdatei `trommel.txt`, rechnerisches Modell, alle Zeiten 0, **kein C-Modul**	echte Trommel über das C-Modul (§7), mit Wartezeiten
Eingabe K0	Terminal oder `--k0 datei=PFAD`	Pult (T) und SW-Fenster (SW); `--k0` wirkungslos
Testdatei, Makros, Testprotokoll	ja	keine; eine angegebene Testdatei wird ignoriert (S5-06)
Programm laden	Trommelbelegung und Tastenfolge der Testdatei	wie am echten Rechner: Objektprogramm-Lochstreifen über Leser 1 oder 2 mit dem Minimalprogramm, danach Sofortbefehl am Pult (Entscheidung 11 zu `--MODUS`)

Was gleich bleibt (Gleichwertigkeit, entschieden 16.10.2026, Review F-04). Bei gleichem Programm liefern TEST und GRAFIK dieselben Ergebnisse (Registerinhalte, Ausgaben). Ausnahmen sind nur die Zeiten (in GRAFIK echte Trommel) und die Befehlsentnahme nach einem X-Abbruch (`Überblick_und_Gesamtaufbau.md`, 4.1). Jede andere Abweichung gilt als Fehler des GRAFIK-Zweigs. Der Prüfweg steht im Prüfverfahren §4.

TEST bleibt maßgeblich. Tests, Zeit- und Zyklenvergleiche und die Gegenprobe laufen im Modus TEST. Der Claude-Interpreter arbeitet nach den Regeln von TEST (kein C-Code, keine Fenster, keine Töne, rechnerische Trommel, Zeiten 0) und bleibt die unabhängige Gegenprobe (Entscheidung 7). Das Reproduzierbarkeitsproblem der echten Trommel betrifft nur GRAFIK.

Fenster schließen. Ein Klick auf X am Pult oder am SW-Fenster wirkt wie Abbruch mit Strg+C; „Netz aus“ bleibt der Netzschalter im Pult (Entscheidung 12, Review F-05). Das SW-Fenster zeigt jedes angenommene Zeichen selbst an (Dok. 09 S3-04 neu, 18.10.2026); die älteren Fassungen „ohne Echo“ sind damit überholt (Registerzeilen bleiben als Geschichte unverändert).

Fehlerfälle in GRAFIK. Fehlen Tkinter oder die grafische Anzeige, oder lässt sich das C-Modul nicht laden, gibt es eine Meldung im Terminal und der Interpreter beendet sich mit Rückkehrcode **16** (Entscheidung 5; S5-09).

Unbekannter Modus. Fehlermeldung und Rückkehrcode **2** (`c8205_interp.py` 0.03-017; Dok. 11 S5-03). Entschieden vom Projektinhaber am 19.10.2026: Dieser Wert gilt. Die ältere Registerzeile vom 15.10.2026 (Entscheidung 2 zu `--MODUS`) nennt 16; sie bleibt in Anhang A unverändert stehen und ist in diesem Punkt überholt.

2 Start und Schalter

Inhalt: `c8205_grafik.sh/.bat`, `python3 c8205_interp.py`, `--leser1/2`, `--stanzer1`, `--trommel`, `--STUMM`, `--BASP`, `--trommelzeit`, `--mdverz`, `--HWCHECK`, `--C8205Z`, Rückkehrcodes

Quellen (alt): Dok. 05 F-08, F-13; Dok. 11

Aufruf.

```
./c8205_grafik.sh [Schalter ...]          (Linux; Windows: c8205_grafik.bat, Teilabnahme 26.10.2026, §9.2)
python3 c8205_interp.py [Schalter ...]    (gleichwertig; ohne --MODUS startet GRAFIK)
```

`c8205_grafik.sh` reicht alle Schalter unverändert an `c8205_grafik.py` weiter. Der Stapelbetrieb mit Testdatei braucht `--MODUS=TEST`; `c8205_interpreter.sh` und `.bat` setzen das selbst (S5-08). Wer eine Testdatei ohne `--MODUS=TEST` angibt, bekommt auf `stderr` den Hinweis „Testdateien werden im Modus GRAFIK ignoriert; fuer Testlaeufe --MODUS=TEST angeben.“ und der Lauf geht als GRAFIK weiter (S5-06, bestätigt 19.10.2026, O5-1).

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Schalter, die in GRAFIK wirken (nach `c8205_grafik.py`, `hauptprogramm()`):

Schalter	Wirkung
`--MODUS`	siehe §1
`--leser1 ORDNER`, `--leser2 ORDNER`	Ordner mit nachrückenden Lochstreifendateien; gelesene wandern in `LOCHSTREIFENKORB` (Register 15.10.2026); eine gleichnamige Datei im Korb wird überschrieben (OP-34, entschieden 21.10.2026). Standard `virtuelle_Geraete/Leser1` und `/Leser2`; beide müssen verschieden sein, sonst Fehler (16)
`--stanzer1 ORDNER`	Ausgabeordner des Stanzers; Standard `virtuelle_Geraete/Stanzer1`
`--trommel PFAD`	Trommeldatei (seit 15.10.2026); Standard `Trommel/trommel.txt`. Sie ist der Speicher des Rechners, Programme verändern sie; wer die Ausgangsbelegung behalten will, sichert sie oder gibt eine eigene Kopie an. Neben ihr liegt die Schalterdatei der Bahnsperren (`trommel.basp`)
`--BASP abcdef`	Bahnsperrenschalter, sechs Ziffern 0/1; wirkt in GRAFIK über das C-Modul (`bahnsperre_setzen`), wird im Pult nicht angezeigt; ohne Angabe gilt die Stellung der Schalterdatei (anlegen mit `000000`)
`--trommelzeit alt|echt|schnell`	Zeitmodell der Trommel; Standard `alt`. `alt`: Wartezeiten nach der Wanduhr im C-Modul; `echt`: virtuelle Zeit (eine Wortzeit je Phase), auf Hardwaretempo gebremst; `schnell`: virtuelle Zeit ohne Bremsen. Einzelheiten und Zähler: §7.8
`--taktprotokoll DATEI`	Taktmitschrift der virtuellen Zeit (§7.11); nur mit `--trommelzeit echt` oder `schnell`, sonst Rückkehrcode 16. Standard: aus. Dazu `--takt-max N` (Zeilen, Standard 5000000, 0 = unbegrenzt)
`--STUMM`	schaltet alle Töne ab; damit entfällt auch die Wartezeit des Hochlaufs (Entscheidung 6)
`--TON pppp`	Lautstärke je Gruppe (Pult, SW, LL, LS), vier Ziffern 0 bis 9, 5 = Voreinstellung, 0 = stumm; wirkungslos mit `--STUMM`
`--TASTATUR de|en`	Tastaturbelegung beim Abspielen einer Klickspur und im Protokoll; Standard `de`
`--LS-LESER-TYP 303|FS751` (auch `--ls-leser-typ`)	Typ des Lochbandlesers, gilt für beide Leser K1 und K2 (G4-022): `303` = Metallbürsten, 130 Zeichen/s (Standard), `FS751` = optoelektronisch, 750 Zeichen/s; `FS715` wird als `FS751` angenommen. Ändert das Lesetempo und den Leserton (FS751 spielt `ton/sw_toene/ls_fs751_einzel.wav`, `ls_fs751_dauer.wav`, `ls_fs751_ende.wav`, G4-024). Im Modus TEST erlaubt und ohne Wirkung; ein anderer Wert ist ein Aufruffehler (Rückkehrcode 2)
`--HWCHECK`	zweites Fenster mit BZ, BR, Spur/Sektor, AC und <<AR>> (§8)
`--C8205Z[=VERZ]`	schaltet die vier Zusatztrommeln des **experimentellen Prototyps C8205Z** ein (Standard: aus; §7.10): `5k00 030` wählt ZT k (1 bis 4; 5100 = ZT0 bis 5400 = ZT3), `zzz0 bbbb 601` gibt die Haupttrommelspur bbbb auf die gewählte ZT aus, `zzz0 bbbb 661` liest sie zurück; ein Befehl dauert rund 81 bis 86 ms. Die Trommeln liegen als `zt1.txt` bis `zt4.txt` in `VERZ` (Standard `Trommel/ZT/`). Braucht das gebaute Erweiterungsmodul `c8205_ztmodul`
`--mdverz VERZ`	Ordner mit den Dokumenten, aus denen der Knotengraph gelesen wird; Standard `markdown-dateien`
`--DEBUGLOG [DATEI]`, `--AUFZEICHNUNG [DATEI]`, `--WIEDERGABE [DATEI]`	Fehlersuche: Protokoll, Aufzeichnung und Wiedergabe einer Klickspur (Prüfverfahren §5)
`--pult-protokoll`, `--netz-schnell`, `--ohne-fenster`, `--entwickler-tasten`, `--entwickler-k0`	Entwicklerschalter für Prüfungen, nicht für den Betrieb. **Netz-Automatik (G4-026):** Ohne Pult-Fenster (`--ohne-fenster`, `--entwickler-tasten`, `--entwickler-k0`) schaltet GRAFIK das Netz selbst: Netz-Ein am Anfang (ohne Hochlauf), Netz-Aus, sobald der Rechner 15 s lang keine Befehlsgrenze mehr durchlaufen hat (z. B. Eingabe-Halt vor einem leeren Leser). Ein teilgelesenes Band bleibt im Leserordner. Rückgabecode dann 0 (Strg+C: 8); kein eigener Schalter.

Herkunft: `--TON`, `--TASTATUR`, `--DEBUGLOG`, `--AUFZEICHNUNG` und `--WIEDERGABE` wurden im Zuge der Entwicklung vom Projektinhaber beauftragt (Auskunft 19.10.2026); Dok. 11 S5-05 führt sie nicht auf. Einzelheiten stehen bei §6 (`--TON`) und im Prüfverfahren §5 (Klickspur).

Wirkungslos, ohne Fehler: `--k0`, `--sw-ausgabe`, `--NO-TERMOUT`, `--zwischenablage`, `--SW-TERMINAL`, `--out`, `--knotentest`, `--lochbandleser1/2`, `--lochbandstanzer1`, `--pruefe-stanzer1`, `--pruefe-sw-ausgabe` und das positionale Argument `tests`

(Entscheidung 11 und 3 zu `--MODUS`; Review F-13, entschieden 16.10.2026). `--version` und `-h`/`--help` liefern in jedem Modus die Ausgabe des Interpreters (S5-07, Vorschlag).

Rückkehrcodes (`c8205_grafik.py`, `RC_OK, RC_ABBRUCH, RC_START = 0, 8, 16`): **0** normal; **8** Abbruch mit Strg+C oder durch Schließen eines Fensters; **16** Start nicht möglich (fehlendes Tkinter oder Anzeige, C-Modul nicht ladbar, ungültiger Schalterwert wie `--BASP` oder `--TASTATUR`, beide Leser auf demselben Ordner, Fehler in der Klickspur). Ein unbekannter `--MODUS` gibt **2** (siehe §1).

3 Bedienpult-Fenster

Inhalt: Layout, Lampen, Farben, Netzablauf, Lampentempo, Betriebsmodus, Bedienablauf, Zuordnung T/SW/SP/BS/LS, E unter M

Quellen (alt): Dok. 01 §1 bis §6, 08

Das Bedienpult-Fenster (Tkinter) gibt es nur im Modus GRAFIK. Es bildet das Foto des Bedienpults ab (Lampen, Tasten, Schlüsselschalter), zeigt den Zustand des Rechners an und nimmt die Bedienung entgegen. Die Regeln der Tastenbedienung stehen in §4, die Sperren und Verriegelungen in `C8205-Interpreter-Bedienpult_mit_Schaltern.md`; GRAFIK führt dafür nichts Eigenes ein: Sind die Tasten bedienbar, reagieren sie wie im TEST beschrieben (Review F-14, 16.10.2026). Alte Kennungen aus Dokument 01 und 08 stehen in Klammern.

3.1 Fenster, Layout, Bildschirm

- **Layout.** Als Vorlage dient das Foto des Bedienpults (Abb. 2) mit Lampenfeld, Tastatur (Abb. 3 und 4), Kanaltasten und Selektoren. Das Fenster ist **fest 1148 × 860 Pixel** groß und wird nicht skaliert (Entscheidungen 13.10. und 16.10.2026; Review F-05). Das Foto ist ein einziges Hintergrundbild und wird nie neu gezeichnet; jede Lampe und jede Taste ist ein eigenes Bildobjekt, dessen Grafik nur bei Zustandsänderung getauscht wird (Dok. 01 §6.2; am laufenden Fenster nicht bestätigt).
- **Teilgrafiken** liefert der Projektinhaber: Tasten (je `-dunkel` und `-hell`), große Lampe (`dunkel`, `hell`, `hell-grün`, `hell-rot`, `dunkel-rot`, 26 × 26 px), kleine Lampe (`Diode-dunkel`, `-hell`, 14 × 14 px), Schlüsselschalter (`schlüssel-ein`, `-aus`, 40 × 41 px). Neun Tasten (0, 2, 4, 6, 8, −, „,“, D, ZR) hat der Entwickler nach Vorbild ergänzt (Näherung); damit hat jede aktive Taste ein Grafikpaar. Lage und Zuordnung stehen im Manifest in `pult/` (Maßstab 860 px).
- **Bedeutung von „hell“** (bestätigt 16.10.2026, Entwurf 81): bei Tasten **gedrückt** (hell beschriftet, helle Umrandung, abgesenkt), bei Lampen **leuchtet**.
- **Entfallen** sind die vier unbeschrifteten unteren Selektortasten unter SP, BS, LS und SW; sie sind nur Teil des Fotos. Das Lösen eines Selektors geschieht durch erneuten Klick. Aktiv sind 38 Tasten.
- **Bildschirm.** Zielbildschirm 1920 × 1080; zu sehen sind drei getrennte Fenster: das Terminal, daneben das Pult, daneben darunter das SW-Fenster (§5). Pult und SW-Fenster dürfen sich überlappen. Auf kleineren Bildschirmen wird das Pult trotzdem in fester Größe geöffnet (Standardannahme, F-05). Die Fensterlage wird gemerkt (FL-001, `grafik_fensterlage.py`); Einzelheiten beim Füllen von §10 zu prüfen.
- **Schließen.** X am Pult ist wie Abbruch mit Strg+C; zweimal Strg+C beendet sofort (§4, §1).

3.2 Lampen

Die Konvention des Handbuchs gilt: Die Lampe heißt wie das Flag („Lampe = Anzeige“). **Rot und grün gibt es nur für die Lampen B und N**, alle anderen Lampen kennen hell und dunkel.

Lampe	Quelle	Anmerkung
E, A, T, SW, K1, K2, K3, FE, BS, US, H	`m.fl[...]` gleichen Namens	direkt
FÜ	`m.fl['FUE']`	anderer Schlüsselname
R	nicht Flag H	Lampe R leuchtet, wenn der Schalter R gedrückt ist (H und R sind gegeneinander verriegelt; Lampe R = nicht Lampe H), **unabhängig von Haltgründen** (Review F-09, entschieden 16.10.2026, berichtigt in Entwurf 33; ein Haltgrund US, BS, FÜ, FE, E, A ändert daran nichts; es gibt kein eigenes Flag R)
B	Betriebsanzeige	siehe 3.5; rot nur im Hochlauf

Lampe	Quelle	Anmerkung
N	Netz liegt an	leuchtet **immer grün**, solange das Programm am Terminal gestartet und das Pult sichtbar ist: vor dem Einschalten, im Hochlauf, im Betrieb und beim Ausschalten bis zum Schließen (Entwurf 80)
AR (12 kleine Lampen)	`m.adr`; bei `E+` ohne M, solange E gehalten ist: `m.BZ`	**Adreßteil des Befehlsregisters** (Lampenreihe „AR“), nicht das Register `m.AR` (Review F-10, 16.10.2026); Triaden mit den Gewichten 4, 2, 1. **Ausnahme (OP-10, entschieden 21.10.2026):** Nach `E+` ohne M zeigt die Reihe bis zum Loslassen von E den **Befehlszähler BZ** (→ §3.8); die Entscheidung trifft `Rechner.ar_lampen()` in `c8205_grafik.py`, das Pult liest sie dort
OR (9 kleine Lampen)	`m.OR`, Bit 8 … 0	Triaden B, A, Z
V1, V2	`m.fl['V1']`, `m.fl['V2']`	

Insgesamt 15 große und 23 kleine Lampen (12 AR, 9 OR, V1, V2). Die Lampen T und SW zeigen die Flags T und SW (Kanal K0, Teilflags). Bei Halt zeigen FE (Fehler Eingabe), FÜ (Überlauf), BS (bedingter Stopp) und US (unbedingter Stopp) den jeweiligen Haltzustand; H und R werden von der Stellung H–R gesteuert (Nr. 21 und 22 des Bedienablaufs, 3.7).

Hinweis: Die Datenstruktur-Doku sagt, im Zustand „Rechnen“ dürfe von den Lampen H, R, US, BS, FÜ, FE, E und A nur R leuchten. Das gilt nur, solange kein Haltgrund besteht; maßgeblich ist die Entscheidung F-09 (Register 16.10.2026).

3.3 Tasten, Schalter und Selektoren

38 aktive Tasten: Kanal K0 bis K3; Selektoren SP, BS, LS und **Selektor SW** (Manifest-Name `SW_umschalt`, rechts außen, nach dem Foto „SW“); Eingriff H, E, M, T, R, G, Ü, SW (Tastatur) und ST; Eingabe B, +, 1, 3, 5, 7, 9, „.“, „:“, D, −, 0, 2, 4, 6, 8, „,“, ZR, IS, IB, WM. D ist die Sondertaste für den Programmbaustein-Ansprung des BIS.

Wippe T-SW (Review F-06, entschieden 16.10.2026, Entwurf 50 und 71). „T-SW“ ist eine mechanisch verbundene Kombination zweier Tasten, T und SW der Tastatur, von denen immer genau eine gedrückt ist (wie H und R). Es gibt keine dritte Bedienstelle. Der **Selektor SW** rechts außen ist davon unabhängig. Die Tastatur hat keine weitere Taste SW. Die Lage von T und SW im Foto ist „geschätzt, Treffer unsicher“ (Manifest).

Element	Wirkung im Interpreter	Rastverhalten	bei R gesperrt
Taste T	Flag T := 1, Flag SW := 0 (Token `SW=0`)	rastet ein, löst SW	**ja** (Entwurf 71)
Taste SW	Flag SW := 1, Flag T := 0 (Token `SW=1`)	rastet ein, löst T	**ja** (Entwurf 71)
SP	Flag SP := 1, solange eingerastet	einrastend	nein
BS	nur Schalterstellung; Flag BS wird erst 1, wenn ein „Bedingter Stopp“ bei eingerastetem BS wirksam wird	einrastend	nein
LS	Stanzermotor läuft ständig; ohne Wirkung auf die Stanzerdatei, nur Ton (3.7)	einrastend	nein
Selektor SW	unterdrückt neue SW-Ausgabe (3.7)	einrastend, je für sich	nein

Die Taste G stellt den **Kanal der eingerasteten Kanaltaste** (K0 bis K3) ein, nicht immer K0 (Projektinhaber 27.10.2026, G4-019; vorher setzte G immer K0 an): ist K1, K2 oder K3 eingerastet, gilt dieser Kanal (Flags K1 bis K3 entsprechend, T und SW := 0); ist K0 eingerastet, was auch der Startzustand ist, gilt K0 und das Flag wird abgeleitet: Schalter SW gedrückt → SW := 1, sonst T := 1. Ein im Programm ausgeführter Anwahlbefehl ändert die Rastung der Kanaltaste nicht (OP-35a); nach G gilt wieder der Kanal der Taste. G bei gehaltenem E ändert den Kanal nicht (Nr. 6 unten). Die Stellung T/SW wirkt auf die **Eingabe**: Stellung SW heißt Eingabe nur im SW-Fenster, Stellung T nur über Klicks im Pult; beides gilt nur bei angewähltem K0 und aufgestelltem Eingabe-Halt (Vorgabe 6 zu `--MODUS`). Sie hat keinen Einfluss auf die Schreibwerk-Ausgabe über K2.

Startzustand der Tasten (entschieden 16.10.2026, Review F-14): alle Schalter und Tasten ungedrückt, **außer H und T-SW in Stellung T**; die Taste K0 ist eingerastet und Kanal K0 angewählt (Entwurf 96; ersetzt Entwurf 76). Lampe R ist dunkel (R = nicht H), andere Lampen als N und B sind dunkel, solange der Rechner aus ist.

Bahnsperrenschalter. Der rückseitige Bahnsperrenschalter (6 Schalter) gehört zur Trommel (`bahnsperre_setzen`, §7) und wird im Pult **nicht angezeigt** („gar nicht anzeigen“, 16.10.2026, Entwurf 86, O-9; keine Zusatzleiste). Die Stellungen kommen über `--BASP` und die Datei neben der Trommel.

3.4 Netzschalter und Netzablauf

Der Netzschalter ist der **Schlüsselschalter unter der Lampe N** (Grafiken `schlüssel-ein` grünlich, `schlüssel-aus` rötlich; „der Netzschalter ist rot“ heißt `schlüssel-aus`). Dok. 01 §3.4 nannte dies „nach meiner Lesart“ mit der Bitte um Bestätigung; **der Projektinhaber hat es am 19.10.2026 bestätigt** („ja“ auf den Vermerk in Checkpoint G6-006). Bestätigt („Ja, so ist es genau“, 16.10.2026) ist, dass „der Netzschalter ist rot“ die Grafik `schlüssel-aus` meint; der Bedienablauf (Review F-14) setzt den Schlüsselschalter als Netzschalter voraus.

Schritt	Verhalten
Start	Im Terminal wird der Interpreter im Modus GRAFIK gestartet; Pult- und SW-Fenster werden aufgeblendet. Rechner **aus**: Schlüsselschalter `aus`, N grün, B dunkel, Tasten wie 3.3; **nur der Netzschalter nimmt Klicks an** (Entscheidung 16.10.2026; ersetzt „gestartetes Skript = eingeschalteter Rechner“)
Einschalten (Klick)	Netzschalter sofort grün (`schlüssel-ein`), **B rot**, Ton `fade_in` läuft in voller Länge (**11,3 s**, die Länge der Aufnahme; die frühere Schätzung „5 bis 6 s“ ist überholt). Während des Hochlaufs nehmen die Tasten keine Klicks an
Hochlauf zu Ende	am Ende von `fade_in` wird **B grün**; erst jetzt gilt der Rechner als eingeschaltet und läuft der Interpreter. Die Tasten sind im Rahmen ihrer natürlichen Abhängigkeiten bedienbar. Das Grundgeräusch (Schleife) beginnt (§6)
Klick im Hochlauf	Netzschalter wieder rot, `fade_in` **bricht ab**, `fade_out` läuft (es läuft immer nur einer von beiden), **B dunkel**; nach Ende von `fade_out` endet das Programm. Seltener Fall (F-02 Nr. 2 und 4, ersetzt die Annahme „wirkungslos“)
Ausschalten bei laufendem Rechner (Klick)	`schlüssel-aus`, **B sofort dunkel** (nicht rot; Entwurf 45), N bleibt grün, `fade_out` läuft (rund 2 s); danach endet das Skript **komplett**, Pult- und SW-Fenster werden abgeräumt, Rückkehrcode 0

Technik: Das Fenster meldet `NETZ_AUS` über einen Rückkanal; der Interpreter beendet sich kontrolliert an einer Knotengrenze (Dateien schließen, Trommel ausschalten, Stanzer und SW-Datei sauber abschließen). B ist nur beim Einschalten rot. Mit `--STUMM` entfällt die Wartezeit des Hochlaufs; B wird nach dem Klick sofort grün (§2).

3.5 Lampentempo und ruhiges Bild

- **Tempo der Befehle** (Review F-07, entschieden und präzisiert 16.10.2026, Entwurf 82): Die Befehle laufen **ungebremst**; verlangsamt werden sie nur dadurch, dass auf Trommel, Stanzer, Leser und SW gewartet werden muss. Keine weitere künstliche Verzögerung, kein Takt-Schalter. Die Zahl „rund 1900 Einzelbefehle/s“ bei zeitoptimaler Lage ist keine Vorgabe, nur die Größenordnung.
- **Anzeigetakt:** Das Fenster fragt alle **40 ms** ab (höchstens 25 Bilder pro Sekunde); kommen mehrere Zustände an, wird nur der letzte gezeichnet. Bei jedem Halt (auch nach einem E-Schritt) wird der Stand **sofort** gemeldet. Der Takt betrifft nur die Anzeige, nicht die Befehle.
- **Ruhiges Bild** (Entwurf 84, Durchgang O-5; Wortlaut des Projektinhabers: „Lass die Lampen so leuchten, dass das Bild ruhig ist.“): Die Lampen springen im Dauerlauf nicht mit jeder Momentaufnahme. Das Wie ist dem Entwickler überlassen. **Bestätigt 21.10.2026 (OP-37):** Mittelung statt Momentaufnahme: Die Lampenzustände werden zeitgesteuert abgetastet, das Pult beurteilt jede Lampe über das Fenster der letzten 40 ms; sie leuchtet ab 60 % Anteil „an“, ist dunkel bis 40 %, dazwischen bleibt der bisherige Zustand (Hysterese). Es bleiben zwei Grafiken (hell, dunkel); ein abgestuftes Nachleuchten entfällt. Schwellen und Fenster sind Konstanten und werden bei der Handabnahme am laufenden Fenster geprüft (Prüfverfahren §7).

3.6 Bedienablauf: vom Einschalten bis zum Start eines Programms

Der Ablauf wurde mit dem Projektinhaber am 16.10.2026 Schritt für Schritt festgelegt (Review F-14) und dient als Abnahmeszenario (Prüfverfahren §7). Nr. 1 bis 6: siehe 3.4. Weiter:

Nr.	Schritt	Festlegung
7	Voraussetzung	Programmstart erst, wenn das **Minimalprogramm (MP)** geladen ist
8	MP laden: Tasten	H ist gedrückt; dann **G, M halten (M+), E, E, M lösen (M−), K1, R**; das **KEP** wird angesprungen. Mit Leser 2 statt Leser 1: **K2** statt K1
9	MP laden: Lochstreifen	In Leser 1 (K1), alternativ Leser 2 (K2), liegt als Datei im Leserordner ein **binär kodierter Lochstreifen mit dem BIS-Minimalprogramm**. Das KEP liest ihn und speichert das MP ab Adresse 0040; am Lochstreifenende bleibt es stehen
10	Sprung in Zelle 0000	beim Abspeichern trägt das KEP in Zelle 0000 einen Sprung zum MP ein
11	MP starten	**H, G, R**: G stellt BR auf „0000 000“, R führt ihn aus, der Ablauf springt ins MP
12	Eingabe eines Befehls mit Kennzeichen	Zuerst können **Leitadressen** gesetzt werden; es sind die verschiedenen Möglichkeiten, eine Leitadresse einzugeben: **20 aaaa** = Relativadresse, **21 aaaa** = 1. Leitadresse, … **29 aaaa** = 9. Leitadresse
13	Anwenderprogramm laden *(Bedeutung von `01.` gegen das BIS-Dokument nicht geprüft, außerhalb des Umfangs, OP-33)*	Leserkanal wählen (K1 oder K2), dann `01.` eingeben (Kennzeichen $K_1 = 0$, $K_2 = 1$: fortlaufende Eingabe von Objektkodebefehlen mit Addition der 1. Leitadresse). Die Adressen des Objektkodes werden entsprechend dem Kennzeichen LEITADRESSE (00 bis 09) mit der zuvor eingegebenen Adresse verschoben
14	Programmstart	`10 aaaa` verlässt das MP; `aaaa` ist die **Startadresse im Speicher**. *Beispiel 1:* Programm relativ bei 0000, `21 1200` gegeben → Start mit `10 1200`. *Beispiel 2:* relativ bei 0200, `21 1200` → Start mit `10 1400` (oktal)

Das KEP gelangt **nicht** über Pult oder GRAFIK auf Spur 0; es muss anderweitig dorthin (z. B. mit einem TEST oder von Hand in die Trommeldatei; Pflege durch den Projektinhaber, F-03). Der Objektkode-Lochstreifen kommt wie das MP als Datei in den Leserordner (Erzeugung: Assembler und `c8205_lochstreifen_umwandlung.py`). Die BIS-Einzelheiten (Kennzeichen, Leitadressen, Befehl `01.`) legt dieses Dokument nicht neu fest; sie stehen in der BIS-Dokumentation (in Dok. 01 nicht geprüft). In den Nr. 14 bis 25 sind einige Standardannahmen nicht ausdrücklich bestätigt (siehe §10).

3.7 Bedienung während des Programmlaufs und Ende

Das Pult ist in Teilen **interruptfähig**, also jederzeit bedienbar, auch während die Befehlsverarbeitung läuft (Dialog F-14, 16.10.2026, Entwurf 48):

Nr.	Element	Festlegung
15	Taste **H**	jederzeit; unterbricht die Befehlsverarbeitung; danach ist das Pult wieder normal bedienbar. Der Interpreter prüft an der Befehlsgrenze (Knoten 2); dass das auch in GRAFIK so ist, ist bestätigt (21.10.2026, OP-13)
16	**K0 bis K3**	jederzeit
17	**Selektor SW**	jederzeit; unterdrückt die SW-Ausgabe, der Rechner macht weiter (unten)
18	**SP, BS, LS**	jederzeit
19	**Netzschalter**	jederzeit (3.4)
20	**Wippe T-SW**	**nicht** im Lauf R: Rollensperre, nur bei H bedienbar (Entwurf 71)
21	Flags **FE, FÜ, BS, US**	die Lampen zeigen den Haltzustand
22	Stellung **H–R**	steuert die Lampen H und R

Selektor SW (Entwurf 49 bis 52, bestätigt). Eingerastet bleibt der bestehende Text im SW-Fenster sichtbar, neuer Text wird nicht ausgegeben (kein Text, kein Klang, nichts wird nachgeholt; **die Schreibwerkzeit läuft still weiter** (G4-004), der Rest eines laufenden Textes geht also nur so weit verloren, wie das Schreibwerk in der Zeit gebraucht hätte, und die Ausgabe beginnt beim Lösen wieder); der Rechner arbeitet die Ausgabebefehle weiterhin korrekt ab (Register, Flags, Ablauf wie ohne Selektor). **Keine Ausgabe heißt kein Warten:** die Schreibwerkzeit (10 Zeichen/s) entfällt, es gibt weder Tastengeräusch noch Tastenverzögerung. Wird der Selektor gelöst, geben Ausgabebefehle ab diesem Moment wieder aus, mit Geräusch und Verzögerung; unterdrückter Text wird nicht

nachgeholt. Der Selektor berührt weder Flags noch die Lampe SW und wirkt **nur auf das Schreibwerk** (Leser, Stanzer und Rechnergeräusch bleiben unberührt). Die Dateiausgabe `--sw-ausgabe` und die Terminalausgabe gibt es in GRAFIK nicht (Standardannahme).

Stanzer im Lauf (Entwurf 52 und 53). Der Selektor SW gilt für den Stanzer nicht; der Stanzer hat im Lauf keine eigene Bedienung. Der Schalter LS wirkt nur auf den **Ton**: Ist LS gesetzt, wird das Stanzergeräusch immer gespielt, sonst endet es **5 Sekunden** nach dem letzten Zeichen und beginnt mit dem nächsten wieder. Der Ton hat keine Wirkung auf die Stanzerdatei. Die 5 s gelten nur für das Geräusch (3 s sind schlecht hörbar); der Motor schaltet laut Projektdokumentation weiterhin nach 3 s ab. Diese Aufteilung ist eine Festlegung des Bearbeiters auf Delegation („mach wie du denkst“), nicht der Wortlaut des Projektinhabers (§6).

Ende des Programmlaufs (Entwurf 54).

Nr.	Ende	Festlegung
23	Programm hält selbst	Stopp-Befehl (Lampe US), bedingter Stopp bei eingerastetem BS (Lampe BS) oder Überlauf (Lampe FÜ): das Pult zeigt nur die Lampe und wartet auf ST, Ü, M, E wie im TEST. Nichts Zusätzliches: keine Meldung im Terminal, kein Ton
24	Eingriff am Pult	z. B. H (Nr. 15)
25	Netzschalter	Ablauf 3.4, Rückkehrcode 0

3.8 E und M+ E im Halt

Festlegung des Projektinhabers (18.10.2026): E und M+ E wirken **so, wie es im Interpreter programmiert ist** (TEST: `c8205_interp.py` 0.03-015; GRAFIK: `c8205_grafik.py` G2-012, Pult G2-010). Die erste Fassung vom 17.10.2026 („jedes E erhöht BZ um 1 …“) ist gestrichen. Es gilt nur im Halt; bei Stellung R sind E und M gesperrt.

1. **E ohne M: Einzelschritt, der Befehl im Befehlsregister wird ausgeführt** (Sprünge und Trommelzugriffe eingeschlossen). **Niederdrücken** (`E+`) gibt Phase IV und I frei, der Automat hält vor Phase II (Knoten 8). **Loslassen** (`E-`) durchläuft Phase II und III und hält vor Knoten 19. `E` ist der kurze Anschlag. BZ ändert sich durch die Befehlsentnahme (+1) oder einen Sprung, nicht pauschal um 1. Steht der Automat nach einem E unter M schon vor Phase II, gibt es beim Niederdrücken nichts freizugeben. **Anzeige (OP-10, entschieden 21.10.2026):** `E+` **ohne M zeigt, solange E gehalten ist, in den Lampen der Reihe AR den BZ** (Beispiel: Befehl `0100 211` aus Zelle 0000, nach `E+` zeigt die Reihe 0101, gemessen); das Register BR (`m.adr`, HW-Check-Zeile BR) bleibt dabei unverändert und zeigt den alten Befehl. Nach dem Loslassen zeigt die Reihe wieder den Adreßteil des neuen BR.
2. **M+ E (M gedrückt oder gehalten, dann E): Phase II entfällt.** Schon das Niederdrücken erhöht **BZ um 1 und setzt AR := BZ**. Der AC enthält im Adressfeld BZ+1 (übrige Stellen 0); die Lampenreihe AR (`m.adr`) zeigt das erhöhte AR (seit 0.03-015 / G2-011), auch solange E gehalten ist (OP-10, entschieden 21.10.2026: E+ mit gehaltenem M erhöht AR und zeigt es an; war schon so programmiert). Das Loslassen von E unter M ist ohne Wirkung. Nach G gilt das erste E unter M als Sonderfall: BZ := 1, AR := 1.
3. BZ und AR zählen in 12 Bit, nach 7777 (oktal) folgt 0000.
4. Prüfbar im HW-Check-Fenster (§8): Nach n Klicks auf E unter M+ sind BZ und AR um n gestiegen. Der Test `KepUeberPult` endet mit BZ=2404, AC=22060635124 (Pult-Tests 65 von 65 bestanden, Stand G2-012); Bedienablauf 3.6 Nr. 8 gilt unverändert.
5. TEST und GRAFIK verhalten sich gleich (Gleichwertigkeit 191 von 191, Stand G2-011); nur TEST kennt allein den ganzen Anschlag `E`. Die Klick-Zuordnung (`E+`, `E-`) steht in §4.3.
6. **G bei gehaltenem E (OP-11, entschieden 21.10.2026; Handbuch 4.1 Nr. 2 ist allein maßgeblich): G löscht nur den Überlauf.** Wirkung: Flag FÜ (`FUE`) := 0; BR, AR, BZ, AC, der angewählte Kanal und alle übrigen Flags bleiben (kein Generallöschen, kein US := 1, keine neue Tastenfolge); E bleibt gehalten. Das Loslassen von E verläuft danach wie üblich (Phase II und III). Ohne gehaltenes E gilt das volle Generallöschen unverändert (Handbuch Taste G). *Wann „FÜ gesetzt und E gehalten“ entsteht (untersucht 21.10.2026, gemessen):* `E+` hält an, wenn FÜ gesetzt ist und der Befehl im BR kein Überlaufsprung ist (Überlauftest vor Phase IV, Knoten 28, 44, 55, 65, oder vor Phase I, Knoten 80; Halt an Knoten 30, 46, 58, 66, 81, Meldung „Halt wegen Ueberlauf (FUE bereits 1)“). E ist dann noch gehalten, der Zustand entsteht also am Pult. Beispiel: `0000: 3000 211`, `0001: 3001 210`, `0002: 3002 211` mit 3000 = 37777777777 und 3001 = 1; nach `H G E E E` ist FÜ = 1, das nächste `E+` hält an Knoten 30. *Fortsetzung (bestätigt 21.10.2026, OP-41):* Löscht G in diesem Halt den Überlauf, wird der Halt aufgehoben; das Loslassen von E setzt Phase IV und I beim Prüfknoten fort (jetzt ohne Überlauf, der Befehl im BR wird ausgeführt) und durchläuft danach Phase II und III. Ohne G bleibt der Halt auch beim Loslassen (wie

bisher). Ein anderer Halt (US, BS, FE) bleibt bei G mit gehaltenem E bestehen, nur FÜ wird gelöscht. TEST ist nicht betroffen (kennt kein gehaltenes E).

7. Prüfbar: `testscripte/grafik_pult_tests.py`, Klasse `EGetrennt` (G bei gehaltenem E: sechs Tests; Lampenreihe: drei Tests); Handliste H-8 (Prüfverfahren §7.1).
8. **Abbruch eines E-Schritts im Eingabe-Halt (OP-20, behoben 21.10.2026):** Bricht H (oder ein Kanalwechsel) einen E-Schritt ab, der im Eingabe-Halt wartet, gilt die Taste E danach als losgelassen und K0 wartet nicht mehr; E, G und R wirken danach wie sonst. **Fortsetzung mit R (G4-016, 26.10.2026):** Ein durch H im Eingabe-Halt des R-Laufs abgebrochener Eingabebefehl wird mit R nur fortgesetzt, solange kein neuer BR aufgestellt wurde. Haben danach E-Schritte (oder M, ST, Ü) einen BR aufgestellt, beginnt R bei diesem BR; früher setzte R im alten Eingabeknoten fort und wartete erneut auf K0, die Befehle liefen nicht los. Prüfbar: Test `test_r_nach_h_und_e_beginnt_beim_aufgestellten_br`. Prüfbar: Klassen `K0AbbruchBeimWarten` und `EImEingabeHaltAbgebrochen` in `testscripte/grafik_pult_tests.py`; Einzelheiten in §10.5.

3.9 Aufbau der Pult-Software und Threads

Das Fenster läuft im Hauptthread, der Rechner in einem eigenen Thread. **Alle Pult-Tasten werden nur vorgemerkt und im Rechner-Thread ausgeführt**: Der Einzelschritt E kann im Eingabe-Halt auf ein K0-Zeichen warten; im Fenster-Thread blockierte er das ganze Fenster (Fehler des Projektinhabers, Strg+C ohne Wirkung). Strg+C setzt nur ein Flag, das Fenster beendet im Takt; **zweimal Strg+C beendet sofort**; eine Ausnahme im Takt oder im Rechner-Thread beendet weder Takt noch Thread.

Datei	Aufgabe
`grafik_pult.py`	Pult-Steuerung ohne Tkinter: Netzablauf, Tastenregeln, Lampen, Zeichenliste, Weitergabe an den Rechner
`grafik_pult_fenster.py`	Tk-Schicht: Canvas, Maus, Computertastatur, Schließen; Takt 40 ms
`grafik_ton.py`, `ton/`	Töne des Rechners (§6)
`grafik_hwcheck.py`	Hardware-Check-Fenster (§8)
`pult/`	Manifest, Foto, Teilgrafiken
`c8205_grafik.py`	Start öffnet das Pult-Fenster; Entwicklerschalter `--ohne-fenster`, `--netz-schnell`, `--pult-protokoll` (§2)

Die Zählung der GRAFIK-Module (G1 bis G5) steht in `C8205-Versionierung.md`; die Aufteilung in §11.

Nicht in §3 übernommen (noch zu verarbeiten, siehe Checkpoint G6-006): Dok. 01 §4 (Befunde), §7 (offene Entscheidungen), §8 (Erkenntnisse und Risiken), §9 (Plattformen) und §10 (Nächste Schritte); Dok. 08 §4 (offene Einschränkungen der Abnahme Stufe 2). Die Zeichenabbildung des SW-Fensters aus Dok. 01 §1.7 kommt nach §5.

4 Bedienung

Inhalt: Mausregeln, Computertastatur, Fokus, Touchscreen, Sperren (M-15 bis M-35)

Quellen (alt): Dok. 06

Das Bedienpult wird mit der Maus bedient; die Computertastatur wirkt nur auf die Eingabetasten und auf die Umschalttaste. Die Nummern in Klammern sind die alten Kennungen aus Dokument 06 (M-15 bis M-35). Welche Taste wann gesperrt ist, steht in `C8205-Interpreter-Bedienpult_mit_Schaltern.md` (Rollensperre, Entriegelung, Verriegelung H gegen R, Verriegelung von ST) und gilt für Maus und Tastatur gleich (M-17, keine eigene Regel).

4.1 Auslösung und Tastenarten

Jede Taste löst **beim Drücken der Maustaste** aus, nicht erst beim Loslassen. Jede Taste hat ihren eigenen Zustand und ihren eigenen Timer; ein zweiter Klick innerhalb der 0,25 s gilt als neuer Tastendruck und startet die 0,25 s neu (M-15). Die Zeit 0,25 s (statt früher 0,4 s) gilt seit dem 17.10.2026 und ist einstellbar.

Art	Tasten	Verhalten
Klick-Taste	B, +, 1, 3, 5, 7, 9, ·, :, IS, IB, G, Ü, D, −, 0, 2, 4, 6, 8, „,“, ZR, WM, ST	Klick: 0,25 s die gedrückte Grafik, dann wieder normal; die Maus muss nicht gehalten werden
Rastende Paare	H und R, T und SW	je Paar ist immer genau eine Taste eingerastet (dauerhaft hell); einfacher Klick auf die andere rastet sie ein und löst die erste; Klick auf die eingerastete ändert nichts; kein Aufleuchten, kein Umschalt (M-19, M-20)
Halte-Tasten	E, M	kurzer Klick: 0,25 s hell; Maustaste gehalten: gedrückt, solange gehalten; Umschalt+Klick ersetzt das Halten (4.3)
Kanalwahl und Selektoren	K0 bis K3; SP, BS, LS, Selektor SW	K0 bis K3 rasten ein, immer genau eine (wechselseitig, auch bei R); Start: K0 eingerastet. SP, BS, LS und SW rasten je für sich ein, erneuter Klick löst; kein Aufleuchten (M-34)

T-SW ist **ein** Schalter mit den Stellungen T und SW (Beschriftung „SW“) und ist von der Rollensperre betroffen: bei Stellung R gesperrt, nur bei H bedienbar, wie E, M, G und Ü (M-18, bestätigt 16.10.2026). Wechselt der Kanal per Befehl im Programm, folgen die Lampen K1 bis K3 bzw. T und SW, die Rastung der Taste nicht (M-34 Nr. 1; bestätigt 21.10.2026, OP-35 (a)). Wortlaut des Projektinhabers: Die Kanaltasten wirken nur im Moment ihres Drückens (beim Start automatisch durch die Voreinstellung); sie bleiben in der eingerasteten Stellung, unabhängig davon, welchen Kanal das Programm anwählt; das Drücken einer anderen Kanaltaste wird dagegen wieder zu einer sofortigen Wirkung.

4.2 Sperren bei Stellung R

Gesperrt sind bei R: E, M, G, Ü und der Schalter T-SW. Die acht Tasten K0 bis K3, SP, BS, LS und SW sind bei R **nicht** gesperrt (M-34 Nr. 3). Die 21 Eingabetasten sind bei R nicht mechanisch gesperrt, sondern **nur elektronisch verriegelt** (bestätigt 17.10.2026): Sie lassen sich klicken, wenn der Rechner auf eine K0-Eingabe wartet (Eingabe-Halt, Kanal K0, Stellung T), sonst nicht.

Ein Klick auf eine gesperrte Taste bewirkt **nichts**: kein Aufleuchten (auch nicht 0,25 s), kein Ton, keine Meldung, nichts wird an den Interpreter weitergegeben, kein Flag ändert sich (M-25).

4.3 E und M: Halten, Umschalt+Klick, E+ und E-

- **Gehaltene Maustaste (M-16, bestätigt).** Die Taste gilt als losgelassen, sobald die Maustaste losgelassen wird, egal wo der Zeiger steht; wandert der Zeiger bei gehaltener Maustaste weg, bleibt die Taste gedrückt; wurde kürzer als 0,25 s gehalten, bleibt sie trotzdem die vollen 0,25 s hell. *Befund 21.10.2026 (OP-16, Messung an* `PultSteuerung`*):* Bei **E** gilt das so. Bei **M** leuchtete die Taste nach dem Loslassen noch 0,25 s weiter (Code: `loslassen` rief für M `_blinken` auf). **Behoben 21.10.2026 (OP-16, Entscheidung des Projektinhabers: Weiterleuchten entfernen,** `grafik_pult.py` **G2-012):** M gilt wie E sofort als losgelassen; nur der 0,25-s-Timer vom Drücken läuft zu Ende. Test `test_m16_m_leuchtet_nach_dem_loslassen_nicht_weiter`; Gegenprobe (Aufruf wieder eingesetzt) macht ihn rot.
- **Umschalt+Klick (M-21, bestätigt).** Umschalt+Klick auf E oder M hält die Taste dauerhaft gedrückt und hell, wie eine eingerastete Taste; ein weiterer Umschalt+Klick lässt sie los; ein normaler Klick auf eine so gehaltene Taste ändert nichts. E und M können gleichzeitig gehalten werden (z. B. `M+ E … E M-`). Die Rollensperre gilt auch für Umschalt+Klick. *Auf der Computertastatur entspricht dem Umschalt+Klick* `Umschalt+E` *bzw.* `Umschalt+M`*, nicht das Liegenlassen des Fingers auf der Taste (4.5).*
- **R bei gehaltenem E oder M (berichtigt 16.10.2026).** Ein Klick auf R bleibt **wirkungslos** (wie 4.2), solange E oder M gehalten ist, weil R mechanisch die Rollensperre auslöst und das nur bei nicht gedrückten gesperrten Tasten geht. Ein Klick auf R innerhalb der 0,25 s nach einem kurzen Klick auf E oder M **wirkt sofort** (bestätigt 21.10.2026, OP-35 (b): „ja, wirkt sofort“); die 0,25 s sind nur die Darstellung eines kurzen Tastendrucks (Entscheidung des Entwicklers, vom Projektinhaber überlassen).
- **Andere Tasten (M-26).** Die Umschalttaste hat bei allen anderen Tasten keine Wirkung; Umschalt+Klick verhält sich dort wie ein einfacher Klick.
- **E+ und E- (M-35).** Niederdrücken und Loslassen von E sind zwei Ereignisse: Maustaste drücken und Umschalt-Klick (Einrasten) senden `E+` (Phase IV und I, Halt vor Phase II); Maustaste loslassen und Umschalt-Klick (Lösen) senden `E-` (Phase II und III, Halt vor Knoten 19). Ein kurzer Klick ist `E+` beim Drücken und `E-` beim Loslassen; die Taste zeigt dabei mindestens 0,25 s die gedrückte Grafik. M verhält sich entsprechend mit `M+` und

`M-`. Unter gehaltenem M erhöht `E+` BZ und AR um 1, `E-` ist ohne Wirkung (§3, E unter M). Der TEST-Interpreter kennt nur den ganzen Anschlag `E`.

4.4 Darstellung, Touchscreen, Fokus

- **Überfahren (M-24).** Beim Überfahren ändert sich nichts: keine Hervorhebung, auch gesperrte Tasten sehen unverändert aus, der Mauszeiger bleibt der normale Zeiger, kein Hinweistext. „Hell“ heißt damit eindeutig „gedrückt oder eingerastet“.
- **Touchscreen (M-22).** Nicht unterstützt; keine Touch-Gesten (langes Drücken, Doppeltipp).
- **Fensterfokus verloren (M-27, bestätigt).** Eine gerade mit der Maus gehaltene Taste (E oder M) wird sofort losgelassen (bei E sendet das `E-`); eingerastete Tasten (H oder R, T oder SW sowie mit Umschalt+Klick gehaltene E und M) bleiben unverändert; laufende 0,25-s-Timer laufen zu Ende.
- **Fokus des Pult-Fensters (M-30).** Das Pult erhält den Fokus nach dem Programmstart und im laufenden Programm, wenn keine Eingabe im SW-Fenster anliegt; liegt eine an, hat das SW-Fenster Vorrang. Der Fokus bleibt beim Pult. Wird er händisch woandershin gesetzt, ist am Pult keine Eingabe möglich (weder Maus noch Computertastatur). Erst ein Mausklick auf das Pult bringt ihn zurück; dieser Klick dient **nur dem Fokus** und löst auch dann keine Taste aus, wenn er auf einer Taste liegt (bestätigt). Für die Tastenbedienung ist ein weiterer Klick nötig (M-32 verweist darauf).
- **Angefangene Eingabe im SW-Fenster (M-33).** Entfällt: Eine Eingabe im SW-Fenster ist immer nur ein Zeichen und damit abgeschlossen.

4.5 Computertastatur

Die ursprüngliche Regel „nur Maus“ (M-23) ist für die Eingabetasten **aufgehoben**. Die Eingabetasten `B + 1 3 5 7 9 · :` und `D − 0 2 4 6 8 , ZR` sind auch über die gleichnamigen Tasten der Computertastatur bedienbar; dazu seit OP-47 (21.10.2026) die Buchstabentasten H, R, E, G, M, T und S (siehe unten); alle übrigen Pult-Tasten nur mit der Maus. Die Umschalttaste der Computertastatur wird für E und M gebraucht (4.3).

Zuordnung (M-28, bestätigt). Ziffern 0 bis 9 und die Zeichen +, −, „,“ und „:“ entsprechen den gleichen Zeichen der Computertastatur (der Doppelpunkt braucht auf deutscher Tastatur Umschalt, das ist für die Pult-Taste unschädlich, M-26); „·“ entspricht dem Punkt „.“; B und D entsprechen b und d unabhängig von Groß- und Kleinschreibung; ZR entspricht der Leertaste; die Ziffern des Ziffernblocks zählen wie die der Hauptreihe. Die Grafik zeigt „ZR“ (Zwischenraum), nicht „ZP“, und „IS, IB“, nicht „JS, JE“; die Dokumente sind entsprechend berichtigt.

Verhalten (M-28). Ein Tastendruck wirkt wie ein Mausklick auf die Pult-Taste: Auslösung beim Drücken, Taste 0,25 s hell. Gehaltene Computertasten wiederholen nicht; erst loslassen und erneut drücken löst wieder aus. *Umgesetzt 21.10.2026 (OP-16, Entscheidung des Projektinhabers „ja“,* `grafik_pult_fenster.py` *G4-009):* `PultFenster` merkt gehaltene Computertasten (Schlüssel: `keysym`, sonst Zeichen) und ignoriert jedes weitere Drücken derselben Taste, bis sie losgelassen ist. X11 meldet die Wiederholung als Paar „Loslassen, Drücken“ kurz hintereinander; ein Drücken weniger als 0,03 s nach dem Loslassen gilt deshalb noch als Wiederholung. Beim Fokusverlust werden alle gemerkten Tasten freigegeben (Loslass-Ereignisse gehen dort verloren). Die Sperre gilt im Pult-Fenster; Tasten, die das Pult an das SW-Fenster weitergibt, laufen davor und sind davon nicht betroffen. Tests: fünf neue in `Op16Maus` (`test_m28_*`). **Nicht geprüft:** das Verhalten am echten Tk-Fenster (Wiederholrate des Betriebssystems, Schwelle 0,03 s). Die Sperren gelten gleich (bei gesperrten Tasten passiert nichts, 4.2). Die Taste leuchtet auch, wenn die Maus woanders steht; mehrere Computertasten gleichzeitig sind möglich, jede mit eigenem Timer.

Wann sie wirkt (M-29, M-31). Nur wenn das Pult-Fenster den Fokus hat **und** der Rechnerzustand die Benutzung erlaubt (z. B. Eingabetasten nur im Eingabe-Halt). Sobald das Pult den Fokus hat, wirkt die Computertastatur sofort; ein weiterer Klick ist dafür nicht nötig. Lässt der Zustand Eingaben nicht zu, passiert nichts (wie M-25). Die Buchstabentasten (OP-47) folgen denselben Regeln. Die Belegung beim Abspielen einer Klickspur wählt `--TASTATUR` (§2).

Buchstabentasten (OP-47, Entscheidung des Projektinhabers 21.10.2026). Zusätzlich zu den Eingabetasten lassen sich bedienen: `H` (Taste H der Wippe H-R), `R` (Taste R), `E`, `G`, `M`, `T` (Taste T der Wippe T-SW), `S` (Taste SW der Wippe T-SW) sowie `Umschalt+E` und `Umschalt+M` (wie Umschalt+Klick: halten, beim zweiten Mal lösen). Groß- und Kleinschreibung gelten gleich; mit der Feststelltaste (CapsLock) ist E ein gewöhnliches E, nicht Umschalt+E. Tasten mit gehaltener Strg- oder Alt-Taste werden nicht ausgewertet.

Wichtig: „E gedrückt halten“ heißt `Umschalt+E` (ebenso M: `Umschalt+M`), nicht „den Finger auf `e` lassen“ (Klarstellung des Projektinhabers, 21.10.2026). Das dauerhafte Halten von E oder M, wie es das Handbuch für E+ und M+ verlangt (4.3), geschieht mit Umschalt+Taste: das erste Mal halten (Taste bleibt hell, Halter gesetzt, die Computertaste kann losgelassen werden), das zweite Mal lösen (`E-` bzw. `M-`). Ein einfacher Druck auf `e` oder `m` ist

dagegen ein **Anschlag wie ein einfacher Mausklick**: Drücken gibt `E+`, Loslassen der Computertaste gibt sofort `E-`; wer den Finger auf der Taste lässt, hält E nur so lange, wie die Taste unten ist, und das ist **kein** Halten im Sinne der Halter (die Sperren, die ein gehaltenes E voraussetzen, z. B. R wirkungslos, gelten dann nur während des Drückens). Zur Gegenüberstellung: Maus: Klick = Anschlag, Umschalt+Klick = Halten; Tastatur: `e` = Anschlag, `Umschalt+e` = Halten.

Die Tasten wirken **nur, wenn die Pult-Taste klickbar wäre**: Die Tastendrücke laufen durch dieselbe Stelle wie der Mausklick (`PultSteuerung.klick`), also gelten alle Sperren unverändert (Netz im Betrieb, Rollensperre bei Stellung R für E, M, G, Ü, T und SW, R wirkungslos bei gehaltenem E oder M, 4.2). **E und M** folgen dem Maus-Ablauf: Drücken der Computertaste = `E+` bzw. `M+`, Loslassen = `E-` bzw. `M-`; die Taste leuchtet mindestens 0,25 s, nach dem Loslassen sofort dunkel (M-16). Bei `Umschalt+E` und `Umschalt+M` ist das Loslassen ohne Wirkung. Die Sperre gehaltener Computertasten (M-28) gilt auch hier, und zwar ohne Unterschied der Schreibweise (wird die Umschalttaste vor E losgelassen, gilt die Taste dennoch als frei). **Aufzeichnung (`--DEBUGLOG`, `--AUFZEICHNUNG`):** Ein Tastendruck wird als Klick auf die jeweilige Pult-Taste festgehalten (wie ein Mausklick, mit Umschalt-Angabe und Wirkung); beim Abspielen wirkt er daher ohne Besonderheit. Das Log-Thema `TASTE` zeigt zusätzlich das Zeichen und `-> wirkt=….`

Vorrang bei angewähltem Kanal SW (OP-48, Rückmeldung des Projektinhabers 21.10.2026: nach `S` wirkte `R` nicht, es klang wie eine Eingabe im SW, obwohl der Fokus beim Pult lag). Ist der Kanal K0/SW angewählt, nimmt das SW-Fenster sonst jede Taste, die am Pult-Fenster ankommt, als Schreibwerk-Eingabe (`taste_vom_pult`, G4-008; der Fokus liegt zwischen zwei Eingabeaufforderungen beim Pult). Seit G4-011 hat eine Pult-Buchstabentaste (H R E G M T S) am Pult-Fenster **Vorrang**, wenn sie in diesem Augenblick als Pult-Taste wirken würde (`PultSteuerung.taste_klickbar`: Betrieb, Wippen, Rollensperre, R bei gehaltenem E oder M, wie beim Mausklick) **und** das SW keine Eingabe erwartet (kein offener Eingabe-Halt 011, Puffer leer). In allen anderen Fällen bleibt die Taste SW-Eingabe: erwartet das SW gerade ein Zeichen, gehören auch `h r e g m t s` dazu; wirkt die Pult-Taste nicht (z. B. `T` bei gerastetem T, `R` bei Stellung R, `E`, `G`, `M`, `T`, `S` bei Stellung R), geht sie an das SW. Das Loslassen einer vom Pult behandelten Taste geht nicht an das SW; eine gehaltene Pult-Taste wiederholt nicht und gelangt auch nicht als Wiederholung in die SW-Eingabe. **Grenze (Nebenwirkung, gewollt hingenommen):** Zwischen zwei Eingabeaufforderungen ist `h` im Lauf (Stellung R) eine Pult-Taste und hält an, ein in dieser Lücke getipptes `h` ginge also nicht in den Text. Liegt der Fokus im SW-Fenster, sind Buchstaben dort immer Eingabe. Tests: neun in `Op47Buchstaben` (`test_op48_*`), Gegenprobe (Vorrang abgeschaltet) macht fünf rot.

Umgesetzt 21.10.2026 (`grafik_pult.py` G2-013: Tabelle `TASTATUR_PULT`, `tastatur(…, umschalt)`, `tastatur_los`; `grafik_pult_fenster.py` G4-010: Umschalt, Strg/Alt, Loslassen, Schreibweise der Haltesperre). **Belegt:** neue Klasse `Op47Buchstaben` in `testscripte/grafik_pult_tests.py` (17 Tests, 107 → 124 von 124); Gegenproben (leere Tabelle: 12 Tests rot; Umschalt unterdrückt: 5 rot); Sammellauf `grafik_alle_tests.sh --schnell` 18 von 18 Blöcken. Der alte Test `test_computertastatur` verlangte, dass `G` nicht wirkt; er prüft jetzt `q`. **Nicht geprüft:** Verhalten am echten Tk-Fenster (Tastenwiederholung, Feststelltaste, Strg/Alt unter Windows, deutsche und englische Belegung).

Prüfung (OP-16, 21.10.2026). Durch Tests belegt sind jetzt: Sperren und Rastungen (Klasse `Tasten`), E+ und E- (`Tasten`, `EGetrennt`), und in der neuen Klasse `Op16Maus` (`testscripte/grafik_pult_tests.py`, sieben Tests): der zweite Klick startet die 0,25 s neu (M-15), Loslassen zählt überall im Fenster (M-16), laufende Timer enden nach Fokusverlust (M-27), ein Klick nur für den Fokus löst keine Taste aus (M-30), R wirkt sofort nach einem kurzen Klick auf E, keine Bindung für Überfahren und Touch (M-24, M-22), je Computertaste ein eigener Timer (M-28). Gegenproben: sieben Eingriffe in `grafik_pult.py` und `grafik_pult_fenster.py` an einer Kopie machen jeweils genau den passenden Test rot. **Nicht geprüft:** das Verhalten am echten Tk-Fenster (Zeiger, Fokuswechsel, Wiederholung gehaltener Tasten).

Hinweis zur Herkunft: Dokument 06 wurde am 18.10.2026 auf den Stand von Entwurf 96 und 99 gebracht, ohne dass die Fassung von Entwurf 96 vorlag; Tastenzeit 0,25 s, einrastende Kanaltasten und die Verriegelung der Eingabetasten bei R sind nach dem Register und Dokument 01 §1.7 nachgezogen. Maßgeblich bleibt das Register (Anhang A).

5 SW-Fenster

Inhalt: K0-Eingabe, Zeichenpuffer, Anfangslage, Eingabe-Regeln (S3-xx)

Quellen (alt): Dok. 09 §3, §7a, §10; Dok. 01 §1.7 (Zeichenabbildung); Dok. 10 §12 bis §15 (Puffer, Rücktaste, Weiterleitung vom Pult)

Das SW-Fenster (Tkinter) ist das zweite Fenster des Modus GRAFIK, im selben Prozess wie das Pult, und öffnet nach ihm. Es zeigt die Schreibwerk-Ausgabe des Rechners **und jedes Zeichen, das der Rechner als Eingabe angenommen hat**, nimmt die Tastatureingabe am SW an und führt die Schreibposition. Im Modus TEST gibt es

kein SW-Fenster; dort liefert die K0-Datei die Zeichen. Der Ton der SW-Ausgabe und der angezeigten Eingabe steht in §6. Alte Kennungen (S3-xx, U-xx, G3-/G4-xx) stehen in Klammern; die Zuordnung zu den Abschnitten folgt am Ende von §5.

Begriffe. *Eingabe-Halt:* Der Rechner wartet im Befehl `011` auf ein Zeichen; die Warteschlange K0 meldet dann „erwartet SW“. *Token:* was der Rechner über K0 bekommt, ein Zeichen oder `Um. KB` / `Um. GB`. *Lage des SW:* KB oder GB, so wie das SW selbst sie führt (nie die Lage des Rechners); nach dem Einschalten unbestimmt. *Schreibposition:* Es gibt nur eine; sie wird entweder durch eine Eingabe oder durch eine Ausgabe mit einem Zeichen belegt und rückt dann weiter. Ein „Echo“ im Sinne eines Rückschreibens durch das Programm gibt es beim C 8205 nicht (Festlegung 18.10.2026); das Fenster zeigt jedes angenommene Zeichen selbst an.

5.1 Fenster und Anzeige

- **Größe und Darstellung** (S3-02): 130 Spalten × 36 Zeilen, Schrift mit fester Breite, kein Zeilenumbruch. Der Text ist markierbar; Markieren und Strg+C legen ihn in die Zwischenablage (S3-03).
- **Schreibposition** (S3-05): 0 bis 130; Tabulatorstopp alle 12 Zeichen. Die Position zählt die schon beschriebenen Plätze (0 = vor dem ersten Platz). Hinter Position 130 gibt es keine Bewegung: Jedes weitere Zeichen ersetzt das Zeichen auf dem 130. Platz (Überdruck), kein Zeilenumbruch. Ein Tabulator ab Position 120 geht auf 130, ab 130 bleibt er stehen.
- **Zeilenschaltung und Wagenrücklauf** (S3-06): WRZL beginnt eine neue Zeile. `WR` setzt die Position auf 0, ohne Anzeige und ohne Zeilenvorschub; folgende Zeichen überdrucken. TAB füllt nur fehlende Spalten. *(WRZL: Entscheidung; WR und TAB: bestätigt 21.10.2026, OP-14. Der **Klang** von WR ist entschieden, 21.10.2026: §6.2, OP-42.)*
- **Nicht darstellbare Zeichen** (S3-07): ZV, BM, Irrung Zeichen, Irrung Satz, Transport, ≈, ≋ und △ werden nur ausgeführt, ohne Anzeige und ohne Bewegung der Schreibposition (Entscheidung 18.10.2026; „keine Bewegung“ bestätigt 21.10.2026, OP-14). *Befund 21.10.2026 (OP-14):* Die Rechnerausgabe kann davon nur Irrung Satz, ≈, ≋ und △ liefern (zusammen mit den Umschaltzeichen und `<WR>`); ZV, BM, Irrung Zeichen und Transport stehen nicht in der Ausgabetabelle des Schreibwerks (`SW_ZEILE`, Kodetabellen 1.4: „das Schreibwerk gibt weder ZV noch BM aus“) und kommen dort nie an. Die Auslegung „keine Bewegung“ betrifft in der Praxis also nur die vier erreichbaren Zeichen. Belegt durch `test_op14_benannte_zeichen_die_das_schreibwerk_wirklich_ausgibt`.
- **Ausgabe des Rechners:** Jedes Zeichen, das ein Ausgabebefehl sendet, wird angezeigt. Ausgabe und angenommene Eingabe laufen durch **eine** Warteschlange in die Anzeige; die Reihenfolge beider Quellen bleibt erhalten (S3-04, U-1).
- **Schließen** (S3-16): X am SW-Fenster wirkt wie Strg+C (Rückgabecode 8); Netz-Aus beendet mit 0.
- **Lage auf dem Bildschirm:** Das Fenster öffnet an der zuletzt gemerkten Stelle (FL-001, `grafik_fensterlage.py`, 19.10.2026); ohne gemerkte Stelle rechts unten wie das frühere Fenster „SW Ausgabe“ (S3-15, bestätigt 21.10.2026, OP-14). Einzelheiten zum Merken stehen in §3.1 und werden beim Füllen von §10 geprüft.

5.2 Wann eine Eingabe angenommen wird

Eine Taste im SW-Fenster wirkt nur, wenn **K0 angewählt, der Selektor SW am Pult gedrückt** (Stellung SW statt T) **ist** (S3-08). Dazu braucht das Programm nur `011`; ein `030` ist für die Anzeige nicht nötig (Wortlaut des Projektinhabers 18.10.2026: „Wenn im SW ein Zeichen eingegeben wird, DANN MUSS ES DARGESTELLT WERDEN, nur mit dem 011-Befehl“).

Zustand	Wirkung der Taste
K0 und SW angewählt, Rechner bei `011`	Das Zeichen geht an den Rechner und wird sofort angezeigt.
K0 und SW angewählt, Rechner noch nicht (wieder) bei `011`	Das Zeichen wartet im Puffer (§5.5) und wird erst angezeigt, wenn der Rechner es geholt hat.
K0 und SW angewählt, Rechner im Halt (Schalter `H`, Flag H) und kein `011` aktiv (G4-023)	**Nur der Klang:** das Zeichen wird weder vorgemerkt noch angezeigt, auch nicht nach `R`. Im Halt arbeitet der Rechner keinen Eingabebefehl ab. Ist im Halt ein `011` aktiv (Einzelschritt `E` wartet auf das Zeichen), gilt die Zeile „Rechner bei `011`“.
Kanal K1 bis K3 angewählt, Schalter SW gelöst (Stellung T)	**Gesperrt:** keine Eingabe, keine Anzeige, kein Ton, keine Meldung, nichts vorgemerkt.
Zeichen ohne Gegenstück im R300-Code	Still verworfen (§5.3), ohne Wirkung auf die Lage.

Es gibt keine Regel „Programmende“ (S3-18); maßgeblich ist allein, ob K0/SW angewählt ist und ein Eingabebefehl die Zeichen abholt. Ein angezeigtes Zeichen, das den Rechner nicht erreicht, kann nicht vorkommen: Anzeige und Eingabe gehören zusammen (Antwort 18.10.2026 07:44). Weil gesperrte Tasten still bleiben, sieht ein Anwender, dessen Selektor SW nicht gerastet ist (stattdessen T), keine Reaktion; das ist kein Fehler (Debug-Log-Befund 19.10.2026, Dok. 10 §15). Ein Hinweis erscheint nicht (bestätigt 19.10.2026, §10.5).

5.3 Zeichenabbildung

Ein- und ausgegeben werden können nur Zeichen des R300-Codes (`C8205-Interpreter-Kodetabellen.md`, Tabelle 1; Entscheidung 16.10.2026, F-16).

1. **Abbildung** (S3-09): Ein getipptes Zeichen wird über seinen ASCII-Wert in den Spalten „Ascii KB“ und „Ascii GB“ der Tabelle 1 gefunden. Angenommen werden genau 85 Zeichen: `a–z`, `A–Z`, `0–9`, `% : + / , . - ; ( ) ? ! < > " ' [ ] ~ # * =` und der Gravis. Nicht angenommen werden Umlaute, ß, §, €, ° ´ ² ³ µ, die AltGr-Zeichen `@ { } \ |` und alles ohne Gegenstück.
2. **Sondertasten** (S3-10): Eingabetaste → WRZL, Leertaste → ZR, Tab → Tabulator. Strg-/Alt-Kombinationen, Entf, Pfeile, Funktionstasten, Esc, Pos1 und Umschalt allein ohne Wirkung. Ausnahme Rücktaste: siehe §5.6.
3. **Nicht eingebbar** über die Computertastatur sind die Zeichen ohne Tastenentsprechung (ZV, BM, Irrung Zeichen, Irrung Satz, Transport, ≈, ≋, △).
4. **Nicht angenommenes Zeichen:** stiller Verwurf wie bei einer gesperrten Taste: keine Anzeige, kein Ton, keine Meldung, nichts an den Rechner; Lage und Puffer bleiben unverändert.
5. **Gravis:** Er kommt am echten Fenster als `grave` an und wird geliefert (Handlauf F-56). Mit der Wiedergabe des Tests (`event_generate`) ist er nicht erzeugbar; die Auswertung führt das als bekannte Eigenschaft (`[BEKANNT]`), nicht als Befund (→ Prüfverfahren §7).

5.4 Lage KB/GB und Umschaltzeichen

- **Die Umschalttasten der Computertastatur** (Umschalt, Feststelltaste, AltGr) werden nicht gesondert ausgewertet. Sie bestimmen nur, welches Zeichen getippt wird (`A` statt `a`); das Fenster erkennt am Zeichen, welche Lage es braucht. Das C-8205-Zeichen `Um. KB` oder `Um. GB` tippt der Benutzer nie selbst; **das Fenster erzeugt es** (S3-11; bestätigt 18.10.2026, Checkpoint 45).
- **Die Lage des Rechners spielt für die Eingabe am SW keine Rolle** (Festlegung des Projektinhabers 18.10.2026). Das Fenster führt nur die Lage des SW selbst.
- **Nach dem Einschalten (jedem Programmstart) steht das SW immer auf KB** (Festlegung des Projektinhabers 29.10.2026, G4-010; ersetzt „unbestimmt“, S3-12 und Entscheidung 18.10.2026). Grund: Host-Tastaturen haben keine Taste für KB, nur Umschalten nach GB; ein unbestimmter Vorzustand ließ Ziffern und Sonderzeichen scheinbar verschwinden. Das Umschaltzeichen geht nur noch bei Abweichung von der Lage des SW zum Rechner: `a` oder `1` als erstes Zeichen gehen allein, `A` oder `!` als erstes Zeichen als `Um. GB`, Zeichen; das SW steht danach auf GB. Der Rechner zählt Umschaltzeichen bei der Speicherung mit (z. B. KAT4EINGABE), die Anzeige zeigt sie nie.
- **Bei den folgenden Zeichen** entscheidet das vorhergehende: gleiche Lage → nur das Zeichen; andere Lage → Umschaltzeichen und Zeichen, der Rechner bekommt **zwei Eingaben** und muss dafür zwei `011` durchlaufen. Zeichen mit gleichem Code in beiden Lagen (ZR, TAB, WRZL u. a.) gehen allein und ändern die Lage nicht.
- **Abstand der zwei Eingaben** (S3-11, U-11, Z-2; Entscheidung 18.10.2026): kein fester Wert. Maßgebend ist das echte Drücken der Tasten. Ist eine Umschalttaste beteiligt (Umschalt, AltGr), ist der Abstand die Zeit zwischen deren Drücken und dem Drücken der Zeichentaste; sonst geht das Umschaltzeichen beim Drücken der Zeichentaste und das Zeichen beim Loslassen (Abstand = Haltezeit). Das Fenster fügt keine Zeit hinzu. Die „etwa 0,1 s“ vom 18.10.2026 waren ein Beispiel, keine Vorgabe. Zeichen „dazwischen“ gibt es nicht; kommt die nächste Taste, ist eine ausstehende zweite Eingabe frei, nichts wird verworfen (S3-14).
- `Um. KB` und `Um. GB` werden nie angezeigt und bleiben stumm.
- **Eingabeprogramme:** Jedes Programm bekommt als erstes Eingabezeichen ein Umschaltzeichen. Die älteren Testprogramme `ziffern_1`, `ziffern_2` und `sw_zwei_ziffern` kennen keines und bleiben unverändert; ihre Tests setzen die Lage des SW vorab (U-12, entschieden 18.10.2026: „alte Tests, mach wie du denkst“). `r300_eingabe` kennt die Umschaltzeichen.

5.5 Puffer (seit 19.10.2026)

Auf Wunsch des Projektinhabers (19.10.2026) gibt es einen kleinen Puffer (G4-007; Anlass: viele Zeichen gingen verloren, Großbuchstaben „immer“, weil Umschaltzeichen und Zeichen fast gleichzeitig kommen). Er **ersetzt** die Fassung vom 18.10.2026 („keine wartenden Zeichen“, S3-13, S3-14, S3-18) und die Vormerkung der Fassung vom 16.10.2026.

Punkt	Regel
Wann gepuffert	K0 mit SW ist angewählt, aber kein `011` aktiv oder der aktive hat sein Zeichen schon.
Größe	Höchstens 8 Eingaben (`PUFFER_MAX`); ein Umschaltzeichen zählt mit, ein Zeichen in der anderen Lage braucht 2 und kommt als Paar ganz oder gar nicht hinein. Ist der Puffer voll, geht die Taste verloren.
Weitergabe	Der Takt des Fensters (10 ms) gibt je aktivem `011` **eine** Eingabe weiter, in der Reihenfolge des Tippens; Paare frühestens nach dem echten Tastenabstand (§5.4).
Anzeige und Ton	**Anzeige:** erst wenn der Rechner die Eingabe geholt hat, nicht beim Tippen; sie zeigt nur, was den Rechner erreicht hat (S3-04). **Klang (OP-09, entschieden 21.10.2026, G4-008/G4-010):** beim Tastendruck, sobald das Zeichen im Puffer ist, auch wenn der Rechner es noch nicht geholt hat; je Zeichen genau ein Klang (bei der späteren Annahme kein zweiter). Ein Umschaltzeichen klingt nie; bei einem Paar (Umschaltzeichen, Zeichen) klingt das Zeichen beim Drücken, nicht erst beim Loslassen. Eine Taste, die verloren geht (Puffer voll), ist gesperrt oder nicht darstellbar ist, klingt nicht. Die Rücktaste nimmt ein vorgemerktes Zeichen zurück; sein Klang ist dann schon gespielt.
Kanalwechsel	Anwahl von K1 bis K3 oder Wegfall des Flags SW **verwirft** den Puffer; gezählt wird jeder echte Wechsel, auch ein kurzes K2 und zurück. **Ausnahme:** die zweite Hälfte eines Paares, dessen Umschaltzeichen schon beim Rechner ist, bleibt erhalten, damit das Umschaltzeichen nicht allein steht (z. B. Programme, die nach dem Umschaltzeichen eine Meldung über K2 ausgeben). Ohne K0/SW ist die Taste weiter gesperrt und nichts wird gepuffert.
Halt (G4-023)	Im Halt (Flag H) ohne aktiven `011` kommt **kein** neues Zeichen in den Puffer (nur der Klang), und ein bereits gepuffertes Zeichen **verfällt** (wie beim Kanalwechsel; die zweite Hälfte eines Paares mit schon gesendetem Umschaltzeichen bleibt). Nach `R` erscheint also nichts von dem, was im Halt getippt wurde. Fehlerbericht des Projektinhabers 30.10.2026: im Halt getippte Zeichen erschienen nach `R` alle im SW-Fenster.
Verfall	Kein zeitlicher Verfall im Lauf; Zeichen bleiben im Puffer, bis der Rechner wieder bei `011` ist oder der Kanal wechselt.
Rücktaste	Nimmt zuerst das zuletzt gepufferte Zeichen zurück (samt seinem Umschaltzeichen, wenn es noch im Puffer steht), siehe §5.6.

Stand der Entscheidung. Gewünscht hat der Projektinhaber: einen kleinen Puffer, der bei Kanalwechsel verworfen wird. Größe 8, die Ausnahme für Paare und das Fehlen eines zeitlichen Verfalls sind Annahmen des Entwicklers, vom Projektinhaber am 19.10.2026 bestätigt (§10.5). Auf die Frage nach einer Taste „dazwischen“ antwortete der Projektinhaber am 19.10.2026: „weiß ich nicht, die Eingabe muss funktionieren!“ Daraufhin steht `SwEingabe.EIN_TASTENDRUCK = False` (G4-009): Es gilt dieser Puffer. Die Alternative `True` (G4-008: der Puffer hält höchstens einen Tastendruck, ein Zeichen oder Umschalter und Zeichen) bleibt wählbar, verliert aber bei schnellem Tippen systembedingt Zeichen. Eine dritte, nicht gebaute Möglichkeit wäre: ein Platz Vormerkung, die Taste danach nicht verwerfen, sondern erst bei Abholung annehmen. Ob 8 reicht, soll der nächste Lauf mit `--DEBUGLOG` zeigen (Zeilen `SW Taste … voll`).

Wegfall der Vormerkung am 18.10.2026 und ihr Ersatz. Bis 18.10.2026 wartete ein Zeichen nach dem Umschaltzeichen als „Vormerkung“ bis zum nächsten Eingabe-Halt und verfiel mit der Eingabebedingung oder dem Programmende (O-11). Das entfiel am 18.10.2026 (Antworten 07:44 und 07:56: Sperre statt Warten; `verwerfen()`, B-6 und die Regel „Programmende“ sind entfernt). Der Puffer vom 19.10.2026 ist keine Rückkehr zu dieser Vormerkung: Er verfällt nicht mit dem Programmende, sondern nur mit dem Kanalwechsel, und die Anzeige folgt erst der Abholung.

5.6 Rücktaste (seit 19.10.2026)

Die Rücktaste hat **Löschwirkung** (G4-006; Antwort des Projektinhabers 19.10.2026: Löschwirkung ja, Anzeige eines Namens nein; der frühere Stand „ohne Wirkung“ war ein Lesefehler des Entwicklers):

- Gibt es ein gepuffertes Zeichen (§5.5), nimmt sie **zuerst dieses** zurück (das zuletzt gepufferte, samt Umschaltzeichen); es hat den Rechner nie erreicht und war nie angezeigt.
- Sonst löscht sie das zuletzt angezeigte Zeichen links der Schreibposition, auch ein Zeichen der Rechnerausgabe, und die Position geht um eins zurück. Am Zeilenanfang ohne Wirkung, keine Zeilenverschmelzung.

- Keine Anzeige eines Namens (kein `<Rücktaste>`), **nichts an den Rechner** (der R300-Code hat kein Rücktasten-Zeichen; „Irrung Zeichen“ ist ein Lochstreifen-Zeichen und nicht eingebbar), kein Ton.
- Sie wirkt unabhängig davon, ob ein `011` aktiv ist. Strg+Rücktaste und Alt+Rücktaste ohne Wirkung.

Die fünf Annahmen (Löschen links der Schreibposition auch bei Rechnerausgabe, am Zeilenanfang ohne Wirkung, nichts an den Rechner, kein Ton, wirkt ohne aktiven `011`*) hat der Projektinhaber am 21.10.2026 bestätigt (OP-31, „alles JA“; §10.5). Belegt durch die Klasse* `Ruecktaste` *und zwei Tests des Fensterblocks in* `grafik_sw_tests.py`*; „kein Ton“ hatte bis dahin keinen Test, jetzt* `test_op31_kein_klang_bei_ruecktaste`*. Nicht geprüft: das Verhalten am echten Fenster.*

5.7 Fokus, Titel und Weiterleitung vom Pult

- **Fokus** (S3-15; Entscheidung 16.10.2026, F-16 Teilantwort): Bei einem Eingabe-Halt (und bei gepufferten Zeichen) fordert das SW-Fenster den Fokus an; der Titel nennt dann „Eingabe erwartet“. Sonst hat das Pult den Fokus. Wird der Fokus vom Betriebssystem verweigert, erzwingt das Fenster ihn nicht; ein Mausklick in das Fenster holt ihn (der Klick dient nur dem Fokus). Nach der Eingabe geht der Fokus zum Pult zurück.
- **Weiterleitung vom Pult** (G4-008, 19.10.2026): Nach jedem Zeichen ging der Fokus zurück zum Pult und wurde erst bei der nächsten Aufforderung (rund 0,3 s später) wieder angefordert; Tasten in dieser Lücke erreichten das Pult und gingen verloren (Debug-Log: `TASTE Computertastatur … wirkt=False`). Seitdem reicht das Pult Tasten der Computertastatur an das SW-Fenster weiter, solange K0/SW angewählt ist (Umschalttasten immer, damit der echte Tastenabstand stimmt); sie laufen durch dieselbe Eingabelogik. Ohne K0/SW wertet das Pult die Taste selbst aus (Pult-Tastatur, Stellung T). Ob das die verschluckten Zeichen beseitigt, ist am echten Fenster noch nicht bestätigt (→ §10).
- **Entprellung:** Das SW-Fenster gibt den Fokus erst nach **0,4 s ohne Eingabeaufforderung** an das Pult zurück (`_fokus`, `_inaktiv_seit`; Dok. 10 §14, Schwelle im Code gegengelesen). Liegt der Abstand der Eingabebefehle um diese Schwelle (gemessen 350 bis 450 ms), bleibt der Fokus mal im SW-Fenster und wechselt mal zum Pult; Tasten gehen dabei seit G4-008 nicht mehr verloren (Weiterleitung). Die Schwelle ist nicht geändert.
- **Warnung:** Bei geöffnetem HW-Check-Fenster (§8) kann das SW-Fenster den Fokus nicht halten; das hängt vom Fenstermanager ab und ist als bekannte Eigenschaft akzeptiert (B-5, A-4).

5.8 Debug-Log und Spur

- **Debug-Log** (`--DEBUGLOG`, S3-17): je Taste eine Zeile `SW Taste char=… keysym=… -> geliefert | folgt | vorgemerkt | voll | blockiert | angehalten | verworfen (Puffer: n)`; bei Kanalwechsel `Puffer -> verworfen`; ferner die Annahme beim Rechner („Rechner uebernimmt Token“, `grafik_k0.py`), die Anzeige (`SWAUS`) und je Eingabeklang eine Zeile mit Schreibwerk und Mischer. Die Klickspur kennt die Ereignisart `tipp`. Keine Wirkung auf das Rechnerverhalten.
- **Spur:** Das Programm `r300_eingabe` schreibt je empfangenem Token dessen Ausgabewort ab Zelle 3000 in den Speicher; damit ist prüfbar, was der Rechner wirklich bekam. Die Auswertung `r300_eingabe_log_auswerten.py` vergleicht Anzeige, angenommene Zeichen und Tokens (Verfahren: Prüfverfahren §6).

5.9 Überholte Fassungen (zum Lesen der Altdokumente)

Altdokument sagt	Gilt seit	Ersatz
Kein Echo, Tastatureingaben nirgends zu sehen (Dok. 00, 01 §1.6, 03 §5, Entwurf 88)	18.10.2026	Jedes angenommene Zeichen wird angezeigt (S3-04 neu, U-1 bis U-8)
Das erste Zeichen setzt die Lage, ohne dass ein Umschaltzeichen geht (Dok. 01 §1.7 Nr. 2, Dok. 09 vor Checkpoint 45)	18.10.2026 (Checkpoint 45)	Das erste Zeichen mit bestimmter Lage bekommt immer das Umschaltzeichen (S3-12)
Umschaltzeichen aus der Lage des Rechners	18.10.2026	Nur die Lage des SW zählt (S3-11)
Vormerkung bis zum nächsten Eingabe-Halt, Verfall mit Programmende (Dok. 01 §1.7 Nr. 2, 4, 5; O-11)	18.10.2026	Sperre statt Warten (S3-13, S3-18), seit 19.10.2026 der Puffer (§5.5)
Fester Abstand 0,1 s der zwei Eingaben (`ABSTAND_ZWEITE`)	18.10.2026 (Checkpoint 38/39)	Abstand der echten Tasten (U-11)
Rücktaste ohne Wirkung (S3-10, Entwurf vor Checkpoint 58)	19.10.2026	Löschwirkung (§5.6)
Taste im Puffer: höchstens ein Tastendruck (G4-008, `EIN_TASTENDRUCK = True`)	19.10.2026 (G4-009)	Puffer bis 8 Eingaben (`False`)

5.10 Kennungen, Stand der Festlegungen

Alte Kennung	Gegenstand	Abschnitt
S3-01, S3-02, S3-03	Fenster, Größe, Markieren und Kopieren	5.1
S3-04, S3-08	Anzeige jedes angenommenen Zeichens, Bedingungen	5.1, 5.2
S3-05, S3-06, S3-07	Schreibposition, Tabulator, WR/WRZL, nicht darstellbare Zeichen	5.1
S3-09, S3-10	Zeichenabbildung, Sondertasten	5.3
S3-11, S3-12, U-11, U-12, Z-1, Z-2	Lage, Umschaltzeichen, Abstand	5.4
S3-13, S3-14, S3-18, G4-007, G4-009	Sperre und Puffer	5.2, 5.5
G4-006	Rücktaste	5.6
S3-15, G4-008, FL-001, B-5	Fokus, Weiterleitung, Fensterlage	5.1, 5.7
S3-16	Schließen	5.1
S3-17, RE-001, RLA-001	Debug-Log, Spur, Auswertung	5.8
U-1 bis U-10, B-1 bis B-9	Umsetzung und Befunde der Stufe 3	5.9, Prüfverfahren §10

Stand. Entscheidungen des Projektinhabers: S3-01 bis S3-05, S3-08 (die Sperre), S3-09 bis S3-14, S3-16, S3-18, Z-1, Z-2, U-9 (Ton für die angezeigte Eingabe: ja, §6) und die Rücktaste. **Standardannahmen, am 21.10.2026 bestätigt (OP-14, §10.5):** S3-06 (WR und TAB), S3-07 (keine Bewegung bei nicht darstellbaren Zeichen), S3-15 (Lage rechts unten). **Annahmen des Entwicklers seit 19.10.2026, am 19.10.2026 bestätigt (OP-07, §10.5):** Puffergröße 8, Ausnahme für Paare, kein Verfall, `EIN_TASTENDRUCK = False`. Dieser Abschnitt ist aus den Dokumenten übernommen; nur die Konstanten (Breite 130, Tabulator 12, `PUFFER_MAX` 8, `EIN_TASTENDRUCK = False`, Takt 10 ms, Fenster 130 × 36, Titel „Eingabe erwartet“, Taste `BackSpace`) habe ich im Code gegengelesen, nicht das Verhalten. Die Puffer-Änderungen vom 19.10.2026 sind am echten Fenster nicht erprobt.

6 Töne

Inhalt: Schreibwerk, Lochstreifenleser, Rechner-Grundgeräusch, Netzablauf, Stanzer, Haltzeit, Mischer, Tastengeräusche des Pults (§6.12)

Quellen (alt): Dok. 03, 10 §4; `ton/SW_Ton_Beschreibung.md` bleibt als Teil B bis D

Töne gibt es nur im Modus GRAFIK. Im Modus TEST gibt es keine Töne und keine Hochlaufwartezeit. Dieser Abschnitt beschreibt den **geltenden Stand** der Töne (Stufe 4, am 19.10.2026 vom Projektinhaber abgenommen; Windows: Teilabnahme am 26.10.2026, §9.2). Er ersetzt Dok. 03 (Stand 16.10.2026, in großen Teilen überholt) als Beschreibung. Die Herkunft der Klangdateien, die Messwerte und der Testrahmen `ton/sw_ton.py` stehen weiter in `ton/SW_Ton_Beschreibung.md` (Teil B bis D, nachgeführt am 19.10.2026). Alte Kennungen (S4-xx, O4-xx, Nr. aus Dok. 03 §5, G4-xx) stehen in Klammern.

6.1 Aufbau: ein Tonstrom, ein Mischer

- **Tonstrom** (S4-01): In GRAFIK ohne `--STUMM` wird beim Start **ein Ausgabestrom geöffnet, der bis zum Ende dauernd läuft** (bei Ruhe Stille). Es wird kein Abspielprozess je Klang gestartet. *Dauergeräusche* (Rechner-Grundgeräusch, Leser-Dauergeräusch, Stanzer) laufen als Schleife und werden nur über den Pegel (0 oder voll) stumm oder hörbar. *Einzelklänge* (Zeichen, TAB, WRZL, Leser-Einzelschritt, Auslaufen) werden beim Ereignis in den laufenden Strom eingemischt; gleichzeitige Klänge überlappen und werden summiert (auf 16 Bit begrenzt).
- **Mischer** (S4-01, S4-02): das C-Modul `tonmisch` (`c-module/tonmisch.c`), als statische Bibliothek `libtonmisch.a` **fest in das Erweiterungsmodul** `c8205_trommel` **eingebunden** (eine Datei zusammen mit der Trommel, §7). Blöcke zu 10 ms (441 Werte, 44,1 kHz, mono, 16 Bit), **eigener C-Thread ohne GIL**, höchstens 2 Blöcke Vorlauf (20 ms), bis 128 gleichzeitige Stimmen. Der Python-Mischer `grafik_tonmisch.py` bleibt als Rückfall (ohne gebautes Erweiterungsmodul meldet `c8205_grafik` das beim Start) und als Bezug der Prüfungen.
- **Senke** (S4-02): Linux: ein Abspielprogramm mit Roh-Audio über eine Röhre, vom Modul gestartet; der Mischer nimmt in dieser Reihenfolge das erste vorhandene von `paplay`, `pw-cat`, `aplay`, `ffplay` (`SPIELER_ROH` in `grafik_tonmisch.py`). Windows: `winmm` (`waveOut…`) mit vier Puffern zu 10 ms; seit 26.10.2026 mit MinGW-w64 übersetzt und im GRAFIK-Interpreter eingeschaltet (Grundgeräusch, Lochbandleser und Stanzer vom Projektinhaber gehört; Dauerbetrieb und Schreibwerkton nicht geprüft; §9.2, §6.8).

- **Tonkanäle** (S4-12): Rechner, Schreibwerk, Leser, Stanzer; je Kanal ein internes Tor. Die Tore dienen **Netz aus** und dem Auslaufen der Dauergeräusche. **Der Selektor SW schließt kein Tor** (§6.2). Es gibt kein neues Bedienelement.
- **Verzögerung** (S4-03): Ton und Zeichen im SW-Fenster erscheinen im Mittel gleichzeitig; der Ton setzt höchstens 50 ms nach dem Zeichen ein; das Zeichen wird nie wegen des Tons verzögert. In der ersten Hörprüfung war der Tastenklang „bemerkbar“ verzögert (geschätzt 0,1 s); daraufhin wurden Vorlauf (30 → 20 ms) und Puffer des Abspielprogramms (40 → 20 ms; `pw-cat` 100 → 20 ms) gesenkt (G4-004). Gemessen ist das am Gerät nicht; die zweite Hörprüfung war „ok“. Zu kleine Werte können knacken (`SPIELER_LATENZ_MS` in `grafik_tonmisch.py`, ohne Neubau änderbar).
- **Aufrufstellen** (S4-14): Der Interpreter für TEST bleibt unverändert; die Töne hängen im GRAFIK-Pfad an vorhandenen Stellen: Schreibwerk-Ausgabe (`SwSenke`), Leser (`GrafikLeser.bei_zeichen`), Stanzer (`GrafikStanzer.bei_zeichen`, Selektor LS über `PultSteuerung.bei_ls`), Netzablauf (`MischerRechnerTon`).

6.2 Schreibwerk

Gegenstand	Festlegung
Klangarten (S4-05; OP-42)	Zeichen, TAB, WRZL, WR. Zeichen: zufällig eine der 24 Dateien `zeichen_*.wav`, **nie zweimal hintereinander derselbe**; der Anschlag (etwa 60 ms) wird nicht abgeschnitten. WRZL: `wrzl.wav`, 360 ms. **WR** (Wagenrücklauf ohne Zeilenschaltung, kommt aus dem Rechner als `<WR>`): `wr.wav`, **120 ms**, nur die erste Hälfte des WRZL-Klangs vor dem markanten Ton (Entscheidung 21.10.2026, §6.10 Nr. 6). TAB: **nur ein Klang**, `tab_400.wav`, 400 ms, gleich für jeden Tabulatorweg (kurz und lang entfallen).
Zeichenklang (S4-05; O4-1, O4-8)	**100 ms.** Die 24 Dateien dauern 120 ms und bleiben unverändert; bei der Wiedergabe werden sie auf 100 ms gekürzt, die letzten 10 ms werden ausgeblendet (`ZEICHEN_AUSBLENDUNG_MS`).
Schreibwerkzeit (S4-17; O4-1)	Jedes **ausgegebene** Zeichen belegt das Schreibwerk **im Mittel 82,5 ms, je Zeichen zufällig 75 bis 90 ms** (gleichverteilt; seit 22.10.2026, vorher 70 bis 90 ms, weil `namen_raten` merklich zu schnell lief); **seit G4-003 (22.10.2026, Projektinhaber) wartet die Befehlsschleife nicht auf das Ende des Zeichens:** Der Ausgabebefehl stößt Klang und Zeichen an und kehrt sofort zurück. Erst der nächste Ausgabebefehl wartet, falls das Schreibwerk noch nicht frei ist, genau bis zum Ende der Schreibwerkzeit des vorigen Zeichens. Dieses Warten ist ein E/A-Halt (Haltzeit-Zeile 8.3). Die Schreibwerkzeit ist **nicht** die Tonlänge. TAB (400 ms), WRZL (360 ms) und WR (120 ms) bleiben Schreibwerkzeit (bei diesen drei ist sie gleich der Klanglänge). Das Zeichen erscheint im SW-Fenster zu Beginn der Schreibwerkzeit.
Überstand (O4-7)	Der Zeichenklang (100 ms) ist länger als die Schreibwerkzeit (75 bis 90 ms): Der Klang des folgenden Zeichens beginnt zur Schreibwerkzeit, der Rest des vorigen **überlappt** und klingt darunter zu Ende.
Rechter Anschlag (S4-06)	Ein Zeichen hinter Position 130 wird angeschlagen (Ton und Zeit wie jedes Zeichen) und im Fenster auf Position 130 überdruckt (§5.1).
Selektor SW (S4-07; O4-3, O4-10)	Bei eingerastetem Selektor SW (`SW_umschalt`) gibt es für die **Ausgabe des Rechners** keine Ausgabe, keine Schreibwerkzeit und keinen Klang; der Rechner arbeitet die Ausgabebefehle weiter ab; nach dem Lösen gibt es wieder Ausgabe mit Klang und Schreibwerkzeit; unterdrückter Text wird nicht nachgeholt. **Der Selektor SW wirkt nur auf die Ausgabe**; Leser, Stanzer und Rechnergeräusch und die angezeigte Eingabe bleiben unberührt. Nicht zu verwechseln mit der Wippe T-SW (Eingabequelle, §3.3, §5.2).
Angezeigte Eingabe (S4-08; U-9, O4-2, O4-5, O4-6)	Jedes im SW-Fenster angezeigte Eingabezeichen klingt **wie jede Ausgabe** auf das Schreibwerk (Klangwahl gemeinsam, ein Schreibwerk). Es hat **keine Schreibwerkzeit** und **keine Sperre**: der Klang läuft nur nebenher. **Umschaltzeichen haben keinen Ton**, nur die Zeichen. **Der Klang kommt beim Tastendruck** (OP-09, entschieden 21.10.2026), sobald das Zeichen im Puffer ist (§5.5), auch wenn der Rechner es noch nicht geholt hat; nur die **Anzeige** hängt an der Annahme durch den Rechner. Je Zeichen ein Klang, kein zweiter bei der Annahme. Bei einem Paar (Umschaltzeichen, Zeichen) klingt das Zeichen beim Drücken. Folgen Ein- und Ausgabe abwechselnd, laufen die Klänge ineinander. Die Eingabe klingt auch bei eingerastetem Selektor SW.
`--STUMM` (S4-13; O4-9)	Kein Ton, aber die Schreibwerkzeit gilt auch mit `--STUMM` (im Modus TEST nicht).

6.3 Lochstreifenleser

Der Leserton hängt an `GrafikLeser.bei_zeichen` und wird bei **jedem Zeichen** aufgerufen, das der Leser K1 oder K2 tatsächlich an den Rechner liefert (S4-09; genau ein Zeichen je Zugriff, vom Projektinhaber bestätigt 16.10.2026). Kein Aufruf bei Zeichen von Tastatur oder K0. Es gibt **einen** Leserton für K1 und K2 (O4-12).

- **Lesetempo 130 Zeichen/s** beim Leser 303 (Standard) und **750 Zeichen/s** beim Leser FS751 (`--LS-LESER-TYP`, G4-022; der Typ gilt für beide Leser; Ton je Typ: 303 `ls_einzel.wav`, `ls_dauer.wav`, `ls_ende.wav`, FS751 `ls_fs751_einzel.wav`, `ls_fs751_dauer.wav`, `ls_fs751_ende.wav`, G4-024; Ablauf Einzelton, Dauerton, Aus ist für beide Typen gleich), auch mit `--STUMM`, nicht im Modus TEST (O4-11); Rechenzeit wird angerechnet, die Tonlänge verzögert nichts.
- Das erste Zeichen nach mehr als 100 ms Pause ist ein **Einzelschritt** (`ls_einzel.wav`, etwa 150 ms). Folgt das nächste Zeichen höchstens 100 ms später, läuft das **Dauergeräusch** (Schleife `ls_dauer.wav`, der Einzelschritt wird abgebrochen). Bleiben die Zeichen länger als 100 ms aus, endet es mit dem **Auslaufen** (`ls_ende.wav`, 60 ms). Die Schwelle 100 ms ist eine Konstante, die nach der Hörprüfung nachzustellen ist (S4-16).
- Das Löschzeichen 11111111, das der Interpreter übergeht, zählt als gelieferte Zeichen: es klingt und kostet Lesezeit (O4-13).

6.4 Lochbandstanzer

Der Stanzerton hängt an `GrafikStanzer.bei_zeichen` (nach dem Schreiben des Zeichens) und wirkt **nicht** auf die Stanzerdatei. Der Ton hängt vom **Zustand**, nicht vom einzelnen Zeichen: ein Ton je Zeichen würde sich überlagern, weil die Einzeltöne viel länger als die Stanzzeit (20 ms) sind (Dok. 03 §5 Nr. 15).

- **Zustände:** *Aus* (kein Ton) – *Geräusch* (`ls_motor.wav`, Schleife) – *Dauerstanzen* (`ls_dauerstanzen.wav`, Schleife). Es gibt **einen** Geräuschzustand, nie zwei Geräusche gleichzeitig; einen Einzelschritt-Ton gibt es nicht (S4-10; Dok. 03 §5 Nr. 21).
- **Dauerstanzen** beginnt mit dem zweiten Zeichen, das weniger als 50 ms nach seinem Vorgänger ausgegeben wird, und läuft, solange das erhalten bleibt. 50 ms nach dem letzten Zeichen beendet ein Zeitgeber es (zurück zu *Geräusch*). **Kommt nach 5 s kein Zeichen, wird das Geräusch abgeschaltet**; das nächste Zeichen startet es wieder.
- **Selektor LS gedrückt:** Es läuft immer nur `ls_motor.wav`, dauernd und ohne Abschalten; `ls_dauerstanzen.wav` wird dann nie gespielt (Dok. 03 §5 Nr. 23). Das gilt auch ohne `--stanzer1` (O4-15). Beim Lösen von LS beginnt die 5-s-Frist neu (O4-16).
- **Stanztempo** (S4-18): etwa 50 Zeichen/s, ein Zeichen frühestens 20 ms nach dem vorigen (Rechenzeit angerechnet; wer langsamer liefert, wird nicht gebremst); auch mit `--STUMM`, nicht im Modus TEST. Es gilt für den Ausgabebefehl des Rechners, nicht für die Tonlänge. Bei schnellem Lauf entsteht damit Dauerstanzen (20 ms < 50 ms). Eine Streuung wie beim Schreibwerk ist nicht eingebaut.
- **3 s und 5 s:** Der **Motor** des Stanzers schaltet nach der Projektdokumentation weiter **nach 3 s** ohne Zeichen ab; die **5 s** gelten nur für das **Geräusch**, weil 3 s schlecht hörbar sind. Diese Aufteilung ist eine Festlegung des Entwicklers auf die Delegation des Projektinhabers (16.10.2026, „mach wie du denkst“), nicht sein Wortlaut.

6.5 Rechner: Grundgeräusch und Netzablauf

Der Ablauf von Netz-Ein bis Netz-Aus steht in §3.4. Die Töne dazu (S4-11; Dok. 03 §1 Nr. 4, 5; §5 Nr. 19, 20):

- **Einschalten:** `fade_in` in voller Länge (11,3 s, Länge der Aufnahme), danach B grün und **Grundgeräusch** als Schleife (Aufnahme 5,9 s, Pegel 0,68, seit 27.10.2026; vorher 0,8, Entscheidung 17.10.2026; kein Neustart wie in Stufe 2). Der Pegel am Ende von `fade_in` (etwa 0,073 RMS) liegt über dem am Schleifenanfang (etwa 0,037 seit der Absenkung auf 0,68; vorher 0,044); eine Überblendung ist nicht festgelegt und bleibt Gegenstand der Hörprüfung.
- **Ausschalten mit dem Klick:** `fade_out` (2 s) beendet `fade_in` und das Grundgeräusch und schließt die Tore von Schreibwerk, Leser und Stanzer (O4-17); danach Ende mit Code 0.
- **Strg+C, X am Pult oder SW-Fenster, Schließen** (Code 8): alle Töne hören sofort auf (Tore zu, Strom zu), **kein** `fade_out`.
- **Wiedereinschalten** heißt, das Skript neu zu starten; es beginnt wieder mit „Rechner aus“ und wartet auf den Klick auf den Netzschalter. Die Register sind danach weg, die Trommeldatei bleibt erhalten.
- **Haltzeit:** Die Haltzeit-Zeile des Hardware-Checks (HZ-003, nur reine Rechnerzeit) gehört zu §8, nicht zu den Tönen.

6.6 Lautstärke und Stummschaltung

- `--STUMM` (S4-13): keine Töne, **keine Hochlaufwartezeit** (B grün sofort nach dem Klick), keine Prüfung des Audiogeräts; Schreibwerkzeit, Lesetempo und Stanztempo bleiben.

- `--TON=pppp` (S4-19; Entscheidung 19.10.2026 nach der ersten Hörprüfung): vier Ziffern 0 bis 9 in der Reihenfolge **Pult, SW, LL, LS** (Pult = Rechnertöne und Tastengeräusche des Pults (§6.12); SW = Schreibwerk einschließlich der angezeigten Eingabe; LL = Lochstreifenleser; LS = Lochbandstanzer). Voreinstellung 5555; 0 = stumm. Faktor = Basis × Ziffer / 5. **Basis laut Code (`TON_BASIS` in `grafik_tonmisch.py`): Pult 1,0, SW 2,0, LL 1,3, LS 1,0.** SW 2,0, weil der Tastenanschlag doppelt so laut werden sollte (Hörprüfung 1); LL 1,3 seit G4-012 („Leser einen Tick lauter“, Hörprüfung 19.10.2026). Ungültige Angaben (nicht vier Ziffern) beenden den Start mit der argparse-Meldung (Code 2). Mit `--STUMM` wirkungslos; unter Windows wirkte der Wert Pult im `winsound`-Zweig nicht; mit dem Mischer (`winmm`) ist `--TON` nicht geprüft (§9.2.3). Die Zeichenklänge erreichen 0,4 des Vollausschlags; ab Faktor 2,5 (SW: Ziffer 6) übersteuert ein einzelner Klick.

6.7 Start und Fehlerfälle

Fehlt in GRAFIK ohne `--STUMM` ein Abspielprogramm oder ein nutzbares Audiogerät, bricht das Skript **vor dem Aufblenden der Fenster** mit der Meldung „Kein Abspielprogramm gefunden“ und **Rückkehrcode 16** ab (S4-04; Entscheidung 16.10.2026, bestätigt als O4-4 am 18.10.2026); es läuft **nicht** ohne Ton weiter. Dasselbe gilt, wenn der Strom **im Betrieb** ausfällt (Abspielprozess endet oder Schreiben schlägt fehl). Der Startfall ist mit leerem PATH und einem Ersatz für tkinter geprüft (G4-019); der Ausfall im Betrieb am echten Gerät ist nicht eigens geprüft.

6.8 Plattformen

Linux ist umgesetzt und vom Projektinhaber abgehört (Hörprüfung 2, 19.10.2026: ok). Windows: seit 26.10.2026 mit MinGW-w64 gebaut; der Mischer mit `winmm` läuft im GRAFIK-Interpreter (Grundgeräusch, Lochbandleser und Stanzer vom Projektinhaber gehört); offen sind Dauerbetrieb, Schreibwerkton und Lautstärken (B-2; → §9.2, §10, Prüfverfahren §9). Lässt sich `winmm` nicht öffnen, bleibt als Rückfall der Stand von Stufe 2: Rechnertöne über `winsound` (nur ein Klang gleichzeitig), Schreibwerkzeit und Lesetempo ja, sonst kein Ton, mit Hinweis und Grund beim Start (O4-18). Bekannte funktionierende Linux-Umgebung (Dok. 10 §10): Ubuntu 23.04, Python 3.11.4, gcc 12.3.0, `paplay`; das Erweiterungsmodul ist ein abi3-Modul (nach dem Absturz „deallocating None“ auf Python 3.11, behoben mit G4-005, benutzt es `Py_IncRef`/`Py_DecRef` statt der Makros). Ein Neubau am eigenen Rechner ist der sichere Weg: `cd c-module/python && rm -rf build && python3 setup.py build_ext --inplace`.

6.9 Konstanten und ihr Stand

Alle Konstanten sind nach der Hörprüfung nachstellbar (S4-16); der Projektinhaber kann Laufzeiten und Schwellen akustisch nicht stoppen.

Konstante	Wert	Fundstelle im Code	Stand
Schreibwerkzeit	82,5 ms, 75 bis 90 ms (22.10.2026; vorher 80 ms, 70 bis 90 ms)	`grafik_schreibwerk.py`	E 18.10.2026
Zeichenklang, Ausblendung	100 ms, 10 ms	`grafik_schreibwerk.py`	E 18.10.2026
TAB, WRZL	400 ms, 360 ms	`grafik_schreibwerk.py`	E (Nr. 17); `tab_400.wav` ohne eigene Abnahme
WR (Klang und Schreibwerkzeit), Ausblendung der Datei	120 ms, 5 ms	`grafik_schreibwerk.py` (`WR_MS`), `wr.wav` aus `ton/erzeuge_wr_klang.py`	E 21.10.2026 (OP-42); Schnitt nicht gehört, Hörprüfung offen (H-4)
Schwelle Leser, Auslaufen	100 ms, 60 ms	`grafik_geraeteton.py`	E; nachzustellen
Lesetempo	130 Zeichen/s (Typ 303), 750 Zeichen/s (Typ FS751)	`grafik_geraeteton.py` (`LESETEMPO_303`, `LESETEMPO_FS751`)	303: E (O4-11); FS751: Auftrag des Projektinhabers 30.10.2026
Dauerstanzen, Ende, Abschalten	50 ms, 50 ms, 5 s	`grafik_geraeteton.py`	E 16.10.2026 (Nr. 21)
Stanztempo	50 Zeichen/s	`grafik_geraeteton.py`	E 18.10.2026 („ungefähr so“)
Grundgeräusch-Pegel	0,68 (vorher 0,8)	`grafik_rechnerton.py` (`GRUND_PEGEL`, G4-005)	0,8: E 17.10.2026; 0,68: 27.10.2026, Auftrag des Projektinhabers nach der Hörprüfung („Töne im Verhältnis zum Grundgeräusch zu leise“, 15 % leiser); fade_in und fade_out unverändert; Hörprüfung offen

Konstante	Wert	Fundstelle im Code	Stand
Vorlauf, Puffer der Senke	2 Blöcke (20 ms), 20 ms	`grafik_tonmisch.py`	G4-004, am Gerät nicht gemessen
Pegel der Pult-Tastenklänge	1,76 (vorher 1,6; mal Gruppenfaktor Pult aus `--TON`)	`grafik_pultton.py` (`KLICK_PEGEL`, G4-002)	1,6: V 27.10.2026; 1,76: 27.10.2026, Auftrag des Projektinhabers nach der Hörprüfung („10 % lauter“); Hörprüfung offen

6.10 Befunde und Hinweise zu den Quellen

1. **Pegel LL:** Dok. 10 S4-19 nennt die Basis 1,0 für Pult, LL und LS; der Code hat für LL seit G4-012 1,3. Eine Beschreibung dieser Änderung liegt in den Dokumenten nicht vor; hier steht der Code-Wert, vom Projektinhaber am 19.10.2026 bestätigt (§10.5).
2. **Dok. 03** beschreibt in Kopf, §2, §3 und im Absatz „Stand 16.10.2026“ noch „nichts eingebaut“, „120 ms ±10 %“ und „Einsatz nicht festgelegt“; maßgeblich sind hier §6 und Dok. 10. Dok. 03 ist nur im Kopf und mit einer Berichtigung zu O4-1 bis O4-6 vermerkt (Dok. 10 B-1, B-8). `tab_kurz.wav` und `tab_lang.wav` liegen als Quelle, werden aber nicht mehr gespielt.
3. **Eingabeklang beim ersten Zeichen:** Der Projektinhaber berichtete (19.10.2026), dass für das erste Zeichen im Fenster kein Klick kam; eine Ursache wurde nicht gefunden (Dok. 10 §11, „Drei Beobachtungen“). Das Debug-Log schreibt dazu je Eingabeklang eine Zeile `SW Eingabeklang …`. **Untersucht 21.10.2026 (ohne Fenster, ohne Ton):** (1) Der Klangweg ist in Ordnung: `Schreibwerk.eingabe()` legt beim ersten Zeichen eine Stimme in den Mischer (Python- und C-Mischer gemessen: eine Stimme, Spitzenwert 26212 im ersten Block). Der Aufruf `_echo_eingabe` steht hinter `SwEingabe._geliefert()`, also erst **nach** der Annahme durch den Rechner. (2) **Einzige Besonderheit des ersten Zeichens:** Nach dem Einschalten ist die Lage des SW unbestimmt (`lage = None`), das erste Zeichen mit bestimmter Lage geht daher als **Paar** (`Um. KB` oder `Um. GB`, dann das Zeichen; Regel 18.10.2026, G3-008). Das Umschaltzeichen klingt und erscheint nie (O4-6); das Zeichen selbst wird ohne gehaltene Umschalttaste (Fall b, U-11) erst **beim Loslassen** der Taste frei und klingt erst dann, und nur wenn der Rechner bis dahin wieder bei einem `011` ist; fragt er nicht noch einmal, bleibt es im Puffer (kein Klang, keine Anzeige). Beim **Drücken** der ersten Taste ist es deshalb still. Das passt zur Beobachtung, ist aber **nicht als Ursache belegt**: ob der Klick am Fenster gefehlt hat oder nur beim Drücken ausblieb, ist nicht bekannt. Belegt durch zwei neue Tests (`test_op09_*` in `grafik_sw_tests.py`, 83 von 83 grün; Gegenprobe: mit sofort freigegebenem zweiten Zeichen werden drei Tests rot). Verhalten **nicht geändert** (Regel G3-008 und U-11 sind entschieden). (3) **Offen, nur am Fenster prüfbar:** Mit `--DEBUGLOG` zeigen die Zeilen die Ursache: `SW Taste … -> folgt`, dann `zweite Eingabe beim Loslassen -> geliefert`, dann `SW Eingabeklang …`. Fehlt die Zeile `Eingabeklang`, liegt es am Paar (Puffer oder fehlender zweiter `011`); steht sie da und es klingt trotzdem nichts, liegt es am Tonstrom (Abspielprogramm). **Lauf am Fenster 21.10.2026 (**`op09_eingabeklang.sh`**, Log des Projektinhabers):** Das erste Zeichen `3` kam bei 25,384 s mit „folgt“ (Puffer 1); die Zeile `SW Eingabeklang '3'` (Schreibwerk und Mischer vorhanden) folgt erst bei 27,172 s, also **1,8 s später**; ein zweites Zeichen `4` (Lage nun bekannt) wurde bei 30,577 s sofort geliefert und klang ohne Verzug. Eine Zeile „zweite Eingabe beim Loslassen“ fehlt. **Folgerung:** Das Zeichen geht nicht verloren, der Klang wird angefordert, Schreibwerk und Mischer sind vorhanden; der Klick des ersten Zeichens kommt verspätet, weil es als Paar zuerst das Umschaltzeichen braucht und erst frei wird, wenn Taste losgelassen und ein `011` aktiv ist. **Zweiter Lauf (gleiche Anordnung, noch mit der alten** `grafik_sw_fenster.py`**, die neuen Log-Felder fehlen):** Taste `2` bei 23,137 s, Eingabeklang bei 24,868 s (1,73 s); `3` sofort. Der Verzug ist also **wiederholbar** und fast gleich lang (1,8 s und 1,73 s), kein Zufall einer langen Haltezeit. **Offen bleibt, was die Zeit ausmachte** (lange Tastenhaltezeit oder ein Rechner, der nicht bei einem `011` war): das alte Log zeigte weder den Zeitpunkt des Loslassens ohne Lieferung noch den Zustand `011`. Dafür schreibt `grafik_sw_fenster.py` (G4-009) jetzt je Taste `011 aktiv`, beim Loslassen ohne Lieferung `Taste los … nichts geliefert (Puffer, 011 aktiv)` und je Eingabeklang `Stimmen im Mischer danach: n`. Kein Verhalten geändert. **Ursache gefunden 21.10.2026 (Probe** `testscripte/grafik_op09_rechnerzeit_probe.py`**, Rechner mit Schreibwerkzeit, Test** `sw_zwei_ziffern`**):** Der Rechner braucht nach dem Umschaltzeichen `Um. KB` **1,70 s** bis zum nächsten `011`, weil das Testprogramm das Zeichen nicht als Ziffer annimmt und die Aufforderung `Erste Ziffer 0-3:` mit Zeilenschaltung **noch einmal ausgibt** (17 Zeichen zu 80 ms und WRZL 360 ms, rund 1,7 s Schreibwerkzeit). Das erste Zeichen wartet im Puffer und wird erst dann angenommen, angezeigt und gespielt; die Zeit (1,73 bis 1,8 s in drei Läufen des Projektinhabers) stimmt damit überein. Der Klang ist also **nicht ausgefallen, sondern kam mit der Annahme** (Regel bis G4-009: Anzeige und Klang erst bei Annahme durch den Rechner, G4-007, U-9; seit G4-010 klingt das Zeichen beim Tastendruck, siehe unten). Ohne das Umschaltzeichen (zweites Zeichen) gibt es keinen Verzug. **Entschieden 21.10.2026 (Projektinhaber): Variante (b), Klang beim Tastendruck.**

Wortlaut: „Bei einer Eingabe ist das so. Dass das Zeichen vom SW in den Eingabepuffer kommt, kann die Eingabe noch nicht angenommen sein, aber der Tastendruck ‚klingt' schon." Umgesetzt in `grafik_sw.py` (G4-008): neuer Aufruf `klick` an `SwEingabe`; `tippen()` ruft ihn auf, sobald das Zeichen (bei einem Paar: das zweite Token) in den Puffer kommt, vor der Weitergabe an den Rechner. `echo` bleibt die Anzeige bei der Annahme (`_geliefert`). Im Fenster (`grafik_sw_fenster.py`, G4-010): `_klick_eingabe` spielt den Klang (Schreibwerk, ohne Schreibwerkzeit, ohne Sperre, auch bei Selektor SW), `_echo_eingabe` zeigt nur noch an. Folge: Das erste Zeichen nach dem Einschalten klingt beim Drücken, die Anzeige kommt nach dem Loslassen und der Annahme (am Testprogramm `sw_zwei_ziffern` rund 1,7 s später). Keine Taste, die der Puffer nicht aufnimmt (voll, gesperrt, nicht darstellbar), klingt. Tests: sechs neue in `grafik_sw_tests.py` (`test_op09_klick_*`, `test_op09_direkte_lieferung_*`, `test_op09_erstes_zeichen_klingt_beim_druecken_*`, `test_op09_kein_klang_*`, `test_op09_ruecktaste_*`, `test_op09_ohne_klick_*`), zwei im Fenster-Block von `grafik_stufe4_tests.py` (`test_eingabe_klick_klingt_auch_bei_selektor_sw` ersetzt `test_eingabe_echo_klingt_auch_bei_selektor_sw`, neu `test_eingabe_echo_zeigt_nur_an`); Gegenproben: ohne den Aufruf in `tippen()` werden fünf, mit dem Klang erst in `_geliefert` vier der sechs neuen Tests rot. **Nicht geprüft:** das Hören am Fenster (H-7, Prüfverfahren §7); `--DEBUGLOG` zeigt `SW Eingabeklang …` jetzt sofort nach `SW Taste … -> folgt` bzw. `vorgemerkt`, nicht mehr erst bei der Annahme. **Klarstellung des Projektinhabers (21.10.2026):** Seine Bemerkung, bei einem nicht angenommenen Zeichen sei „der Puffer voll, eine weitere Taste geht nicht", beschreibt das **echte C 8205**: Das SW hat dort einen elektronischen Ein-Zeichen-Puffer. Sie hat **nichts mit dem Software-Puffer** des SW-Fensters zu tun (§5.5, bis 8 Eingaben, OP-07 bestätigt); dieser bleibt unverändert, `EIN_TASTENDRUCK` bleibt `False`. Kein offener Punkt.

4. **Nicht an echten Geräten gemessen:** Verzögerung (50 ms), Schreibwerkzeit im Betrieb, Stanztempo. Windows nur teilweise geprüft (§9.2).
5. **Abgleich** `ton/SW_Ton_Beschreibung.md` **und Handbücher (OP-30 (b), OP-21; 21.10.2026):** Verglichen wurde mit §6, mit den Konstanten im Code (`grafik_schreibwerk.py`, `grafik_geraeteton.py`, `grafik_rechnerton.py`, `grafik_tonmisch.py`), mit Dauer und Format der WAV-Dateien und mit einem Lauf von `ton/sw_ton.py` (Beispiele §11.1 und §11.2 der Beschreibung). **Grenze:** Dok. 03 liegt nicht im Archiv; Nr. 11 bis 24 konnten nicht Satz für Satz verglichen werden, nur gegen das, was an seine Stelle trat. *Übereinstimmend:* alle Konstanten aus Teil B §7.3 (80 ms ±10 ms, 100 ms, 10 ms, 400 ms, 360 ms, 130 Zeichen/s, 100 ms), die Dauern der WAV-Dateien (Format überall 44,1 kHz, mono, 16 Bit), die Stanzerwerte D-6 und D-7 (50 ms, 5 s), das Beispiel §11.2 (Zeiten 0,000 bis 2,257 s reproduziert). *Berichtigt:* Teil D der `SW_Ton_Beschreibung.md` (Aufnahmen „nicht eingebaut", MP3, D-1, D-10 bis D-12 „offen") trug noch den Stand 15.10.2026; sie hat jetzt einen Nachführungsvermerk (0.01-004). In §6.1 fehlte `ffplay` in der Liste der Abspielprogramme (steht im Code und in `README.md`), in §6.5 war die Länge des Grundgeräuschs 6,0 s statt 5,9 s (Datei 5,94 s). *Neu gefunden:* OP-42 (Klang für `WR`; am 21.10.2026 entschieden und umgesetzt, Nr. 6). *Handbücher:* siehe OP-21 in §10.5.
6. **Klang für** `WR` **(OP-42, entschieden 21.10.2026):** Vorgabe des Projektinhabers: „für WR ohne Zeilenschaltung nur die erste Hälfte des WRZL-Tons nehmen, vor dem markanten Ton, und diesen neuen Ton 120 ms spielen." Umgesetzt als eigene Klangart `wr` in `grafik_schreibwerk.py` (G4-002): `_art('<WR>') = wr`, Klang `wr.wav`, Schreibwerkzeit 120 ms (`WR_MS`); WR gilt bei `--STUMM` wie jedes Zeichen als Schreibwerkzeit ohne Klang, und als angezeigte Eingabe klingt es ohne Schreibwerkzeit (§6.2). `wr.wav` entsteht mit `ton/erzeuge_wr_klang.py` aus `wrzl.wav`: die ersten 120 ms (5292 Samples), die letzten 5 ms linear ausgeblendet; 44,1 kHz, 16 Bit, mono wie die übrigen Dateien. **Auslegung, nicht gehört:** Als „markanter Ton" gilt der laute Schlag bei etwa 126 ms (Spitze 12941, der größte Einzelwert der Datei); davor liegt der Schnitt, und die 120 ms passen dazu. Ein zweiter lauter Schlag liegt bei etwa 224 ms (Effektivwert am höchsten); wäre er gemeint, läge der Schnitt später. Wo der Schnitt liegen soll, entscheidet das Ohr: `python3 ton/erzeuge_wr_klang.py --ende MS` erzeugt die Datei neu. Im Testrahmen `ton/sw_ton.py` gilt nur der Name `<WR>` als WR (eigener Klang); `\r`, ASCII 13 und `<WRZL>` bleiben WRZL, weil dort „nach Vorgabe CR = WRZL" gilt (`SW_Ton_Beschreibung.md` §14.1). Tests: fünf neue in `grafik_stufe4_tests.py` (`test_wr_*`) und die Prüfung der Datei; Gegenproben (WR als Zeichen, 360 ms statt 120 ms) schlagen an.

6.11 Kennungen

Alte Kennung	Gegenstand	Abschnitt
S4-01, S4-02, S4-03, S4-12, S4-14, G4-001 bis G4-005	Mischer, Senke, Tore, Verzögerung, Aufrufstellen	6.1
S4-05 bis S4-08, S4-17, O4-1 bis O4-10, U-9, OP-42	Schreibwerk, Selektor SW, angezeigte Eingabe, Klang von WR	6.2, 6.10 Nr. 6
S4-09, O4-11 bis O4-13	Lochstreifenleser	6.3
S4-10, S4-18, O4-14 bis O4-16, Dok. 03 §5 Nr. 15, 21, 23	Lochbandstanzer	6.4
S4-11, O4-17, Dok. 03 §1 Nr. 4, 5; §5 Nr. 19, 20	Rechner, Netzablauf	6.5
S4-13, S4-19, G4-012	`--STUMM`, `--TON`	6.6
S4-04, O4-4	Abbruch mit Code 16	6.7
S4-15, S4-16, O4-18, B-2	Plattformen, Hörprüfung, Konstanten	6.8, 6.9
G4-002 (`grafik_pultton.py`), G2-015	Tastengeräusche des Pults	6.12

Stand der Prüfung. Aus den Dokumenten übernommen; im Code gegengelesen sind nur die Konstanten der Tabelle in §6.9 und `TON_BASIS`. Logikprüfungen laufen ohne Audiogerät (`grafik_stufe4_tests`, 90 Prüfungen, am 21.10.2026 gezählt; vorher 80, die Angabe 69 war veraltet; gegen den Python-Mischer und gegen das C-Modul; Prüfverfahren §3). Hörprüfung 1 und 2 und der Lauf mit Tk-Fenster sind vom Projektinhaber am 19.10.2026 als „ok“ gemeldet, ohne Einzelbefunde.

6.12 Tastengeräusche des Pults

Das Pult macht Tastengeräusche (Entscheidung des Projektinhabers 27.10.2026). Sie gehören zur Gruppe **Pult** von `--TON` (§6.6) und gelten wie alle Töne nur in GRAFIK ohne `--STUMM`; unter Windows ohne `winmm` (Rückfall `winsound`) gibt es sie nicht.

- **Drei Klänge für die drei mechanischen Tastentypen** (Dateien in `ton/pult_toene/`, 44,1 kHz, 16 Bit, mono, Spitze 0,35; vom Projektinhaber am 27.10.2026 als gut bestätigt, Hörprüfung nach dem Einbau offen):

Tastentyp	Tasten	Klang	Datei
Typ 3 Impulstaste (Taster)	G, Ü, ST, die 21 Eingabetasten, E und M (beim Drücken, auch bei Umschalt+Klick)	`impuls`	`impuls_druck.wav` (60 ms, aus der Aufnahme des Projektinhabers geschnitten)
Typ 2 Ein-Tasten-Einrastschalter	SP, BS, LS, Selektor SW, Kanaltasten K0 bis K3	`einrast`	`einrast_aus.wav` (95 ms, synthetisch)
Typ 1 Radiotasten	H–R und T–SW	`radio`	`radio_wechsel.wav` (140 ms, synthetisch; ein Klang für beide Richtungen)

- **Nur wirksame Klicks klingen** (M-25: „kein Ton“): gesperrte Tasten (Rollensperre bei R, R bei gehaltenem E oder M, Eingabetaste ohne Eingabe-Halt), ein Klick auf eine schon eingerastete Taste der Wippen und jeder Klick außerhalb des Betriebs (nur der Netzschalter wirkt) bleiben stumm. Der Netzschalter hat keinen Tastenklang (er hat `fade_in` und `fade_out`).
- **Kein Ton beim Loslassen** (Entscheidung des Projektinhabers 27.10.2026): weder beim Loslassen einer Impulstaste noch bei E− und M− (Loslassen der Maus). Umschalt+Klick auf E oder M klingt beim Setzen und beim Lösen des Halters je einmal, weil beides ein Tastendruck ist.
- **Typ 2 klingt beim Einrasten und beim Lösen gleich** (`einrast_aus.wav`), weil nur dieser Klang bestätigt wurde; `einrast_ein.wav` (Erzeugung mit `erzeuge_pult_toene.py`) ist nicht eingebunden. K0 bis K3 klingen wie Typ 2 (Korrektur des Projektinhabers 27.10.2026); die wechselseitige Auslösung ist Logik, nicht Klang.
- **Tastatur und Maus** lösen denselben Klang aus, weil alles durch `PultSteuerung.klick` läuft. Die Zeichen im SW-Fenster haben ihren eigenen Klang (§6.2) und kommen nicht dazu.
- **Pegel gegen das Grundgeräusch:** Die Klicks werden mit `KLICK_PEGEL` 1,76 mal dem Gruppenfaktor Pult gespielt (Spitze 0,62; mit der Spitze des Grundgeräusches von 0,22 unter 1,0; bis 27.10.2026 1,6, auf Wunsch des Projektinhabers nach der Hörprüfung 10 % lauter). Gemessen mit `ton/pult_toene/pruefe_hoerbarkeit.py` (lautester 10-ms-Abschnitt des Klicks gegen die 10-ms-Fenster des Grundgeräusches mit Pegel 0,68, Band 2,5 bis 5 kHz): +13 bis +18 dB gegen das lauteste Fenster, +23 bis +27 dB gegen den Median (mit Klickpegel 1,6 und Grundgeräusch 0,8: +11 bis +16 und +20 bis +25 dB). Das Grundgeräusch hat seine Energie fast ganz

unter 500 Hz, die Klicks bei 2,5 bis 5 kHz. Der Faktor liegt im Code und nicht in den Dateien, damit `--TON` Grundgeräusch und Klicks gemeinsam regelt und das Verhältnis bei jeder Lautstärke bleibt. **Nur gemessen, nicht gehört.**

- **Impulstasten lauter (G4-003, 27.10.2026):** Der Klang `impuls` (G, Ü, ST, E, M, Eingabetasten) wird zusätzlich mit `IMPULS_FAKTOR` 1,4 (+2,9 dB) gespielt, weil der nur etwa 9 ms lange Tick in der Hörprobe zu leise war (Energie danach etwa wie `einrast`). Spitze 0,35 · 1,76 · 1,4 = 0,86; Stellschraube in `grafik_pultton.py`.
- **Umsetzung:** `grafik_pultton.py` (G4-002, Klasse `PultTastenTon`, Gruppe Pult = Tonkanal `rechner`); `PultSteuerung` (G2-015) ruft `taste_ton(art)` in `_klick` an den wirksamen Stellen auf; `c8205_grafik.py` (G4-018) lädt die Klänge und verbindet sie. Ein Fehler im Ton verhindert die Taste nicht (Eintrag im Debuglog). Prüfungen: `grafik_pult_tests.py` (Klasse `PultTastenTon`, 10) und `grafik_stufe4_tests.py` (Klasse `PultTastenTonPruefung`, 5; Dateien, Pegel, Mischung mit dem Grundgeräusch ohne Übersteuern).

7 Echte Trommel

Inhalt: C-Module, Regel „Schlitz ist Schlitz“, Takt, Plattformschicht, Anbindung, Zeitmodelle und Zähler der Trommelstatistik, Taktprotokoll (§7.11)

Quellen (alt): Dok. 02; Dok. 07 (Stufe 1); `C8205-Interpreter-Magnettrommel.md` verweist hierher

Im Modus GRAFIK läuft die Magnettrommel „echt“: Die Zugriffe des Knotengraphen gehen über C-Module, die als Hintergrund-Thread die drehende Trommel nachbilden und Lese- und Schreibaufträge nur bedienen, wenn die angeforderte Zelle gerade unter dem Kopf liegt. Im Modus TEST bleibt es beim rechnerischen Verfahren (alle Zeiten 0) ohne C-Modul; die Klasse `Trommel` des TEST-Interpreters bleibt unverändert (Entscheidung 15.10.2026). Den Aufbau der Trommel (128 Bahnen mit je 32 Sektoren, 4096 Worte zu 33 Bit, Verschränkung der Sektoren) beschreibt `C8205-Interpreter-Magnettrommel.md`; hier steht, wie GRAFIK sie nachbildet. Alte Kennungen (Dok. 02 Nr., O-7 bis O-9, G1-xx) stehen in Klammern.

7.1 Bausteine

Baustein	Aufgabe
`c-module/trommel.c/.h`	Hintergrund-Thread dreht die Trommel (Wortzeit **104 µs**, Reihenfolge `PERM` mit 32 Sektoren, `trommel_sektor` = untere 5 Bit der Adresse, `trommel_bahn` = obere 7 Bit); Auftragskanal `trommel_lesen_warten` / `trommel_schreiben_warten`
`c-module/bahnsperre.c/.h`	Hauptschalter und fünf Zusatzschalter; die Bahn ist gesperrt, wenn `bahn & ~zusatz == 0`; Bahnen ab 32 sind nie sperrbar. Speichert und lädt die Stellung in einer Datei neben der Trommel (§7.5)
`c-module/unterbrechung.c/.h`	Strg+C setzt `abbruch_angefordert`
`c-module/plattform.h`	Plattformschicht (§7.4)
`c-module/python/`	Python-Erweiterungsmodul `c8205_trommel` (statisch gelinkt, eine Datei `.so`/`.pyd`, Limited API ab CPython 3.8; enthält auch den Mischer `tonmisch`, §6.1) und Lader `trommel_lader.py`
`grafik_trommel.py`	Trommel-Klasse für GRAFIK über das C-Modul (`einschalten`, `--BASP`, Schalterdatei)

Eine **statische Bibliothek** kann zur Laufzeit nicht „geladen“ werden; nicht ladbar kann nur ein Python-Modul oder eine dynamische Bibliothek sein. Um die Absicht des Projektinhabers (eine Datei, bei Fehlen Fehlermeldung und Ende) zu erfüllen, gibt es `libtrommel.a` (`make`) und das **fest eingebundene Erweiterungsmodul** `c8205_trommel`; die dynamische Variante (`libtrommel.so`/`trommel.dll` über `ctypes`) bleibt gebaut, ist aber nicht der vorgesehene Weg. Das C-Modul wird **vorübersetzt mitgeliefert** (`-static` gelinkt; Entscheidung 16.10.2026), ein eigener Bau ist möglich (`cd c-module/python && python3 setup.py build_ext --inplace`; nötig sind ein C-Compiler, die Python-Entwicklungsdateien und `setuptools`).

7.2 Regeln der Nachbildung

- **Aktuelle Zelle** (Dok. 02 Nr. 2): Das Modul führt die aktuelle Sektornummer, die alle 104 µs wechselt. Ein Auftrag wird **nur ausgeführt, wenn der Sektor seiner Adresse gerade der aktuelle ist**. Die Wahl der Bahn kostet keine Zeit. `trommel_aktueller_sektor()` liefert den Sektor exakt aus der Uhr; die Variable `trommel_aktuelle_sektornummer` bleibt als reine Beobachtungsvariable lockfrei (formal ein Datenwettlauf; bestätigt 16.10.2026, O-7 Nr. 2).
- **„Schlitz ist Schlitz“** (Nr. 4): Ist eine Sektornummer aktuell, dann für die **ganzen 104 µs**. Trifft eine Anforderung in dieser Zeitspanne ein, wird sie bedient, auch wenn der Sektor schon einen Teil seiner Zeit läuft.

Das ist eine bewusst gewählte Gewohnheit (Ungenauigkeit) des Simulators. Folge: Trifft der Auftrag im Schlitz des richtigen Sektors ein, ist die **Wartezeit 0**; sonst wartet er auf den nächsten Tick, in dem sein Sektor aktuell ist, **1 bis 31 Ticks**. Zugriffe auf dieselbe Zelle innerhalb eines Schlitzes (Lesen und Zurückschreiben) kosten keine zusätzliche Zeit. (Das Makro `TROMMEL_ZUGRIFF_AUCH_IM_SCHLITZ=0` würde erst den nächsten Tickbeginn zulassen, 1 bis 32 Ticks; diese Variante ist nicht mehr getestet.)

- **Wartezeit:** `wartezeit` ist die Zahl der Ticks (Wortzeiten) vom Eintreffen bis zur Ausführung, 0 bis 31. Es ist immer nur ein Aufrufer gleichzeitig im Kanal. Die gemessenen Wartezeiten bestätigen unabhängig die Sektorreihenfolge: logisch nächste Adresse (Sektor +1) 5 Ticks, physisch benachbart (+15 oktal) 1 Tick.
- **Gesperrte Schreibzugriffe** verbrauchen ihre Wartezeit (wie nach `bahnsperre.h`; bestätigt 16.10.2026, O-7 Nr. 1).
- **Werte** werden auf 33 Bit begrenzt; eine Adresse über 07777 liefert `TROMMEL_FEHLER`.
- **Abbruch:** `abbruch_angefordert` (Strg+C) oder `trommel_warten_abbrechen()` beendet das Warten mit `TROMMEL_ABGEBROCHEN`; der Aufrufer prüft alle 50 ms. Das Flag wird schon beim Eintreffen jeder Anforderung geprüft und dabei verbraucht.
- **Startlage:** nach dem Einschalten **beliebig (zufällig)**, wie am echten Gerät. In GRAFIK gibt es **keinen Schalter** für eine feste Startlage (O-7 Nr. 3, 16.10.2026); `trommel_einschalten_start(pfad, index)` mit fester Startlage bleibt für Testrahmen.
- **Rückgabecodes** des Kanals: `TROMMEL_OK`, `TROMMEL_SCHREIBEN_GESPERRT`, `TROMMEL_ABGEBROCHEN`, `TROMMEL_FEHLER`.

7.3 Takt

Der Takt kommt aus der Monotonic-Uhr (Linux `clock_gettime`, Windows `QueryPerformanceCounter`) mit absoluten Zeitpunkten; die aktuelle Zelle ist eine Funktion der Uhr. Ein verspätet aufwachender Hintergrund-Thread **holt übersprungene Ticks der Reihe nach nach**, sodass kein Sektor ausgelassen wird. Unter Linux ist der Timer-Slack auf 1 ns gesetzt (der Standard von 50 µs wäre fast ein halber Tick). Gemessen unter Linux (ein Kern): mittlere Aufwachverspätung 25 µs, größte 1,9 ms (19 Ticks nachgeholt); die Tickzahl stimmt trotzdem mit der Uhr überein.

7.4 Plattformschicht

Aufgabe	Linux	Windows
Thread, Mutex, Bedingungsvariable	`pthread`	`CreateThread`, `CRITICAL_SECTION`, `CONDITION_VARIABLE`
Uhr	`clock_gettime(CLOCK_MONOTONIC)`	`QueryPerformanceCounter`
Schlafen bis zum nächsten Tick	`clock_nanosleep` (absolut), kein Spin	siehe unten
Strg+C	`sigaction(SIGINT)`	`signal(SIGINT)`

Kein aktives Warten (entschieden 16.10.2026, O-7 Nr. 4: „alle 20 µs prüfen reicht“; zuerst als 20 ms gelesen, gemeint sind Mikrosekunden): Der Hintergrund-Thread schläft immer; solange ein Auftrag ansteht, prüft er die Uhr in Abständen von höchstens 20 µs. Ein Tickbeginn darf bis etwa 20 µs spät erkannt werden; die Tickzahl und die Wartezeit in Wortzeiten bleiben exakt. Unter Windows arbeitet die Systemuhr nur grob (`timeBeginPeriod(1)` ist gesetzt); dort kann die **Echtzeit-Dauer** eines Zugriffs länger werden, die Wartezeit in Wortzeiten nicht. **Stand Dok. 02 (16.10.2026): Der Windows-Zweig (Parameter `spinnen` in `plattform.h`, Schalter `req_wartet` in `trommel.c`) war noch nicht angepasst; ob das inzwischen geschehen ist, habe ich nicht geprüft.** Bauen unter Windows: MinGW-w64 (Entscheidung 16.10.2026, O-8; kein MSVC, `build_msvc.bat` liegt nur bei) mit den Skripten `bau_trommel_mingw.bat` und `bau_ztmodul_mingw.bat` (ohne setuptools, `-static-libgcc`, gegen `python3.lib`; §9.2). Windows-Zweig seit 26.10.2026 mit MinGW-w64 (64 Bit) gebaut; `test_c8205_trommel.py` besteht unter Windows mit Python 3.13 (23 Tests, 1 übersprungen, der SIGINT-Test). Messwerte zur Windows-Uhr liegen nicht vor (§9.2.3).

7.5 Anbindung an den GRAFIK-Interpreter

Seit Stufe 1 umgesetzt und abgenommen (18.10.2026; Dok. 07). Dok. 02 §6 führt die Anbindung noch als „nicht umgesetzt“; das ist überholt.

- **Eigener Zweig:** GRAFIK benutzt die Wartezeit der echten Trommel, nicht zusätzlich die rechnerische Schleife der Knoten 040/041 (der Knotengraph der Dokumentation bleibt gemeinsam). Alle Zugriffe laufen über `Machine.rd` und `Machine.wr` auf die Klasse in `grafik_trommel.py`; das Modul ist der **einzige** Zugriff auf die Datei (kein Zwischenspeicher `mem`; so verlangt es Dok. 02 §6, im Code nicht gegengelesen; gegengelesen sind nur

Wortzeit 104 µs, `PERM` mit 32 Sektoren, `.basp` in `bahnsperre.c` und der Ladeweg mit Code 16 in `grafik_trommel.py`). Besonders zu beachten ist die Befehlsentnahme nach X-Abbruch als einzige Stelle, die von „alle Zeiten 0“ abweicht (`C8205-Interpreter-Überblick_und_Gesamtaufbau.md`, Abschnitt 4.1).

- **Start:** `lade_trommelmodul()`, dann Prüfung der Trommeldatei, dann `einschalten`, **als Erstes**, vor dem ersten Trommelzugriff. Am Ende (`finally`) `ausschalten()`; Strg+C beendet auch das C-Modul. Ist das Modul nicht ladbar oder die Trommel nicht einschaltbar: Fehlermeldung mit Bauanleitung und **Rückkehrcode 16**, wie bei fehlerhafter Trommeldatei (bestätigt 15.10.2026).
- **Trommeldatei:** `Trommel/trommel.txt` (4096 Zeilen `aaaa: ttttttttttt` mit LF, 18 Byte je Zeile, 73728 Byte), gemeinsam für TEST und GRAFIK (Entscheidung 15.10.2026); `--trommel PFAD` gilt in beiden Modi. Das Modul prüft Größe sowie erste und letzte Zeile und weist andere Dateien (das alte 17-Byte-Format, CRLF) mit Meldung ab. Pfade mit Umlauten sind unterstützt (UTF-8, unter Windows `_wfopen`).
- **Bahnsperre und** `--BASP` (Entscheidungen 15. und 16.10.2026, Entwurf 86, 87): `--BASP=abcdef` (sechs Ziffern 0/1, von links: Hauptschalter 0001, dann 0040, 0100, 0200, 0400, 1000) wird beim Hochfahren der Trommel an das C-Modul übermittelt, vor dem ersten Zugriff. Das Pult-Fenster stellt die Schalter **nicht** dar (ausdrücklich entschieden, O-9). Das Modul hat dabei drei Aufgaben: (1) wird eine Stellung übermittelt, übernimmt es sie und speichert sie in der **Schalterdatei neben der Trommel** (`trommel.basp`: Pfad der Trommeldatei ohne Endung plus `.basp`; eine Zeile mit sechs Zeichen 0/1 und LF); (2) fehlt `--BASP`, entnimmt es die Stellung aus dieser Datei; (3) existiert sie noch nicht, legt es sie mit `000000` an. Eine vorhandene, aber formal fehlerhafte oder nicht lesbare Datei wird nicht überschrieben (Meldung, Code 16); geschrieben wird bei jeder übermittelten Stellung. Der Kommentar in `bahnsperre.h`, es werde nie eine Konfigurationsdatei geben, ist damit überholt.
- **Startbelegung** (entschieden 15. und 16.10.2026): Die Trommel ist „immer so, wie sie ist“. In GRAFIK gibt es keine Vorbelegung und keinen Eingriff des Interpreters in die Trommeldatei; was darin steht, ist die Startbelegung. Für einen Kaltstart muss die Datei ein lauffähiges Minimalprogramm enthalten (das KEP braucht man nur, um das MP erstmals zu laden). **Den Zustand der Datei pflegt der Projektinhaber selbst;** das Paket liefert weder eine Vorlage `MP.txt` noch ein Zurücksetz-Skript. Wege zum Zurücksetzen (nur TEST oder Hilfsskript): leer mit `testscripte/erzeuge_leere_trommel.py`; auf das Minimalprogramm durch Laden mit `Tests/pj2-bis_minimalprogramm.test` auf leerer Trommel. **Jeder TEST-Lauf verändert die gemeinsame Datei und damit die Startbelegung von GRAFIK;** es gibt keine Warnung bei nicht lauffähiger Trommel und keinen Schutz gegen gleichzeitigen Zugriff.
- **Betriebsart:** Im Modus GRAFIK die echte Trommel; im Modus TEST und beim Claude-Interpreter das rechnerische Verfahren. Die Gegenprobe (Claude-Interpreter) vergleicht Ergebnisse, keine Echtzeit: Registerinhalte und Ausgaben dürfen nicht von der Trommelwartezeit abhängen.

7.6 Folgen

- **Tempo:** Jeder Zugriff dauert 0 bis 3,2 ms (0 bis 31 Wortzeiten). Bei zufälligen Adressen liegt die mittlere Wartezeit bei etwa 16 Ticks (rund 1,6 ms), bei nach der Trommelstruktur gewählten Adressen bei 0 bis 5 Ticks. Programme mit vielen Zugriffen laufen damit zum ersten Mal in realistischer Zeit; die Laufzeit hängt von der Programmierung ab. Gemessen ist das nur an den Testfolgen des Moduls.
- **Nicht reproduzierbar:** Die Wartezeit hängt von der Echtzeit ab, zu der der Interpreter einen Auftrag stellt. Läuft der Interpreter in Python langsamer als eine Wortzeit zwischen zwei Aufträgen, verpasst er „optimale“ Folgezellen und wartet bis zum nächsten Auftreten des Sektors. **Zeiten und Zyklenzahlen schwanken von Lauf zu Lauf.** Zeiten sind deshalb von der Gleichwertigkeit TEST/GRAFIK ausgenommen (Register, 16.10.2026); Zeitvergleiche in GRAFIK werden nicht gefordert. Einen deterministischen virtuellen Takt, der nicht nach der Wanduhr wartet, gibt es als wählbares Zeitmodell: Schalter `--trommelzeit` mit den Werten `echt` und `schnell` (§7.8). Der Standard bleibt `alt`; nur dort schwanken die Zeiten wie beschrieben.
- **Anzeige:** Ob eine Anzeige die aktuelle Sektornummer zeigen soll, ist nicht festgelegt. Das Lampentempo des Pults ist in §3.5 geregelt.

7.7 Prüfstand

Linux (gcc, ein Kern): `make test` mit 8 Prüfgruppen bestanden; 11 Python-Tests des Erweiterungsmoduls bestanden (Lesen mit Wartezeit 0 bis 31, Schreiben mit Begrenzung auf 33 Bit, Bahnsperre, Abbruch, Datei im alten Format, Pfad mit Umlaut, Strg+C mit Rückkehrcode 8, Lader mit Code 16); AddressSanitizer, UndefinedBehaviorSanitizer und ThreadSanitizer ohne Meldung (ein früher Datenwettlauf bei `req_wartet` ist durch atomare Zugriffe behoben); die alten Testrahmen `testframe.c` und `testframe_trommel.c` kompilieren, wurden mit der neuen Fassung nicht ausgeführt. Die Zahlen stammen aus Dok. 02 §5 (Stand 16.10.2026) und sind in dieser Sitzung **nicht neu gelaufen**. Die Meldung des Projektinhabers vom 19.10.2026 (Gesamtlauf `grafik_alle_tests.sh` 18 von 18 Blöcken) schließt die Trommel ein (Prüfverfahren §8).

7.8 Zeitmodelle und Zähler der Trommelstatistik

Quellen: `c-module/trommel.h`, `trommel.c`, `grafik_trommel.py`, `c8205_grafik.py`. *Aussagen mit „gemessen“ stammen aus dem Probelauf* `testscripte/grafik_trommel_zaehler_probe.py` *(TZ-001; Linux, Python 3.12.3, ein Kern; arbeitet auf einer Kopie der Trommeldatei). Alles andere ist aus dem Code gelesen. Windows ist nicht geprüft.*

Zwei Uhren. In GRAFIK laufen zwei voneinander unabhängige Uhren. Die Zähler der Statistik gehören zu verschiedenen:

- **Wanduhr (C-Modul).** Der Hintergrund-Thread in `trommel.c` schaltet alle 104 µs (`TICK_NS`) einen Sektor weiter, gemessen an der Monotonic-Uhr (§7.3). Alle Zähler mit Herkunft „C“ in der Tabelle unten beschreiben diesen Thread.
- **Virtuelle Zeit** `vt` **(Python).** `GrafikTrommel` (`grafik_trommel.py`) führt in den Modellen `echt` und `schnell` einen eigenen Zähler in Wortzeiten. Er beginnt beim Einschalten mit der Sektorlage der Trommel (`vt = PERM.index(aktueller Sektor)`); im Betrieb ist `start_index = -1`, die Lage also beliebig wie am echten Gerät (§7.2). Jede Phase der Befehlsschleife schreibt `vt` um eine Wortzeit fort (`tick()`; der Sprung über Knoten 63 → 6 zählt eine zusätzliche, weil Phase I übersprungen wird), jeder Zugriff um die Wartezeit bis zu seinem Sektor (`_warte_ticks()`). Die Wartezeit hängt also nur vom Programm und von `vt` ab, nicht von der Wanduhr. `vt` steht nicht in `statistik()`.

Zeitmodelle (Schalter `--trommelzeit`, §2; Standard `alt`):

Modell	Wartezeit bestimmt	Zugriff auf die Trommel	Tempo
`alt`	C-Modul nach der Wanduhr	`lesen_warten` und `schreiben_warten`: blockiert, bis der Sektor unter dem Kopf ist	Hardwaretempo; die Wartezeit hängt vom Zeitpunkt des Auftrags ab (§7.6)
`echt`	Python aus `vt`	direkt (`lesen`, `schreiben`), ohne Auftragskanal des C-Moduls	`_pace()` bremst auf Hardwaretempo: das Modell läuft nie merklich schneller als eine Wortzeit je 104 µs (ein Vorsprung bis 2 ms wird nicht ausgebremst). Ist der Rechner mehr als 20 ms im Rückstand, wird nichts aufgeholt, der Bezug beginnt neu (z. B. nach einem Halt)
`schnell`	Python aus `vt`	wie `echt`	kein Bremsen; der Lauf ist so schnell wie der Rechner

In `echt` und `schnell` läuft dieselbe Rechnung; sie unterscheiden sich nur im Bremsen. Deshalb ist die Wartezeitsumme (`wartezeit_summe_ticks_aufrufer`) in beiden gleich.

Zähler. `GrafikTrommel.statistik()` liefert die sieben Zähler des C-Moduls (`trommel_statistik_t`) und drei, die Python selbst führt. `c8205_grafik.py` schreibt sie bei Läufen mit `--ohne-fenster` oder `--entwickler-tasten` am Ende als Zeile `Trommel: {…}`.

Zähler	Herkunft	Bedeutung
`ticks`	C	Wortzeiten seit dem Einschalten der Trommel (beim Programmstart, nicht erst mit Netz-Ein am Pult), **aus der Wanduhr**. `trommel_statistik_lesen()` rechnet den Wert beim Abruf als (jetzt − Startzeit) / 104 µs. Der im Thread mitgeführte Wert hinkt nach, wenn der Thread verdrängt wird
`aufwachen`	C	Zahl der Aufwachvorgänge des Threads. Nach jedem Schlaf bis zum nächsten Tickbeginn wird er um 1 erhöht, gleich wie viele Ticks der Thread danach bearbeitet

Zähler	Herkunft	Bedeutung
`nachgeholte_ticks`	C	Ticks, die der Thread bearbeitet hat, obwohl schon ein späterer Tick lief. Wacht er verspätet auf, schaltet er die übersprungenen Sektoren der Reihe nach nach (§7.3), damit kein Sektor ausgelassen wird; jeder Tick eines Durchgangs außer dem letzten zählt hier. Bleibt 0, wenn der Thread immer pünktlich ist
`verspaetung_summe_ns`	C	Summe der Aufwachverspätungen in ns (tatsächliche minus geplante Aufwachzeit). Pünktliche Aufwachvorgänge tragen nichts bei
`verspaetung_max_ns`	C	größte einzelne Verspätung
`zugriffe`	C	vom Auftragskanal ausgeführte Aufträge, einschließlich gesperrter Schreibzugriffe
`wartezeit_summe_ticks`	C	Summe der vom Kanal gemeldeten Wartezeiten aller Zugriffe in Wortzeiten, vom Eintreffen des Auftrags bis zu seiner Ausführung
`rd_wr_zugriffe`	Python	Aufrufe von `rd()` und `wr()`
`gesperrte_schreibzugriffe`	Python	davon Schreibzugriffe auf eine gesperrte Bahn (Bahnsperre, §7.5; sie verbrauchen ihre Wartezeit)
`wartezeit_summe_ticks_aufrufer`	Python	Summe der Wartezeiten, wie `rd()` und `wr()` sie erhalten: in `echt` und `schnell` von `_warte_ticks()` in virtueller Zeit, in `alt` vom C-Modul gemeldet

Welcher Zähler in welchem Modell.

- `alt`**:** Alle Zugriffe laufen über den Auftragskanal des C-Moduls. `zugriffe` ist gleich `rd_wr_zugriffe` und `wartezeit_summe_ticks` gleich `wartezeit_summe_ticks_aufrufer` (gemessen: 800 und 800, 12 414 und 12 414).
- `echt` **und** `schnell`**:** Python rechnet die Wartezeit selbst und liest und schreibt direkt; der Auftragskanal des C-Moduls bleibt unbenutzt. `zugriffe` und `wartezeit_summe_ticks` bleiben 0 (gemessen). Maßgeblich sind `rd_wr_zugriffe` und `wartezeit_summe_ticks_aufrufer`.
- **Die Thread-Zähler** (`ticks`, `aufwachen`, `nachgeholte_ticks`, `verspaetung_summe_ns`, `verspaetung_max_ns`) beschreiben in allen Modellen nur den Thread, der die Trommel dreht. In `echt` und `schnell` hängt die Zeit des Modells nicht von ihnen ab. `ticks` ist dort die Wanduhr seit dem Programmstart und taugt **nicht** zum Vergleich von Läufen: In `schnell` ist der Wert bei kurzen Läufen winzig, weil der ganze Lauf nur Millisekunden dauert. In `echt` ist er etwa so groß wie die Zeit des Modells, solange nichts anhält; Halt, Eingabe-Halt und Hochlauf vergehen auf der Wanduhr, während `vt` steht, und bei einem zu langsamen Rechner wird nichts aufgeholt (`_pace()`, Tabelle der Zeitmodelle). Zum Vergleich von Läufen taugt `wartezeit_summe_ticks_aufrufer`.

Lesehilfen.

- **Mittlere Verspätung:** `verspaetung_summe_ns / aufwachen`, ein grober Wert dafür, wie pünktlich der Thread aufwacht. Ein hohes `verspaetung_max_ns` oder viele `nachgeholte_ticks` heißen, dass der Host den Thread ausgebremst hat. Die Sektorfolge bleibt trotzdem vollständig; in `alt` werden die Wartezeiten dann ungenauer.
- **Mittlere Wartezeit je Zugriff:** `wartezeit_summe_ticks_aufrufer / rd_wr_zugriffe` (0 bis 31 Wortzeiten; bei zufälligen Adressen etwa 16, §7.6).
- **Kontrolle der Thread-Zähler (Linux):** Jeder Durchgang des Threads bearbeitet einen Tick und die nachgeholten dazu. `aufwachen + nachgeholte_ticks` ist daher der Tickstand des Threads, und `ticks` minus diese Summe sein Rückstand beim Abruf. Gemessen: `alt` 12 293 + 161 = 12 454 = `ticks`; `echt` 13 960 + 61 = 14 021 = `ticks`. Unter Windows kehrt der Thread bei Auftragseingang früher aus dem Schlaf zurück (`plattform.h`); dort zählt `aufwachen` mehr, und die Gleichung gilt nur als Ungleichung. Nicht geprüft.

Streuung der Wartezeitsumme. In `echt` und `schnell` hängt `wartezeit_summe_ticks_aufrufer` nur von der Startlage ab. `vt` beginnt mit der zufälligen Lage der Trommel, und davon hängt allein die Wartezeit des **ersten** Zugriffs ab (0 bis 31 Wortzeiten); danach liegt `vt` auf dem Sektor dieses Zugriffs, und der Rest ist bei gleichem Programm festgelegt. Gemessen mit allen 32 festen Startlagen (`start_index` 0 bis 31): Summen von 12 797 bis 12 828, eine Spanne von 31 Wortzeiten (eine Umdrehung), bei dieser Summe 0,24 %. Die Schranke gilt **absolut** (höchstens 31 Wortzeiten), nicht in Prozent; bei Programmen mit wenigen Zugriffen ist die relative Streuung groß. Auch bei fester Startlage können zwei Läufe um wenige Wortzeiten abweichen, weil `vt` erst nach dem Einschalten aus der laufenden Uhr gelesen wird (gemessen: 12 821 in `schnell`, 12 822 in `echt`, beide Startlage 5). In `alt` gibt es keine solche Schranke (§7.6).

Messbeispiel (gemessen). Probelauf mit 400 Durchgängen aus `tick(2)`, `rd`, `tick(1)`, `wr` (800 Zugriffe), Startlage 5, auf einer Kopie der Trommeldatei (Aufruf: `python3 testscripte/grafik_trommel_zaehler_probe.py [--startlagen]`):

Zähler	schnell	echt	alt
Dauer des Laufs (Wanduhr)	1,8 ms	1458 ms	1295 ms
ticks	19	14 021	12 454
aufwachen / nachgeholte_ticks	1 / 1	13 960 / 61	12 293 / 161
verspaetung_max_ns	135 674	908 886	3 412 877
zugriffe	0	0	800
wartezeit_summe_ticks	0	0	12 414
rd_wr_zugriffe	800	800	800
wartezeit_summe_ticks_aufrufer	12 821	12 822	12 414
vt am Ende (nicht in statistik())	14 027	14 027	–

Lesart: In `echt` ist `ticks` (14 021) fast `vt` (14 027), denn 14 027 Wortzeiten sind 1458,8 ms, die der Lauf auf der Wanduhr auch gedauert hat; die 6 Wortzeiten Unterschied sind die Startlage. In `schnell` hatte der Thread auf dem einen Kern in den 1,8 ms nur einen Durchgang geschafft (Thread-Stand 1 + 1 = 2), während die Uhr schon bei 19 stand: der Fall „Thread hinkt nach“. Die Spalte `alt` ist mit den anderen nicht vergleichbar, weil im Probelauf die Phasen (`tick()`) dort nicht vergehen. Die Zahlen der Wanduhr schwanken von Lauf zu Lauf (§7.6); die der virtuellen Zeit nur um die Startlage.

7.9 Kennungen

Alte Kennung	Gegenstand	Abschnitt
Dok. 02 Nr. 1, 2, 5, 6, 8, 9, 10	Vorgaben: echte Trommel, Thread, Modi, Strg+C, Bibliothek	7.1, 7.2, 7.5
Dok. 02 Nr. 4, O-7 Nr. 1 bis 3	„Schlitz ist Schlitz“, gesperrte Zugriffe, Beobachtungsvariable, Startlage	7.2
O-7 Nr. 4, O-8	kein aktives Warten, MinGW-w64	7.4
O-9, Entwurf 86 und 87	`--BASP`, Schalterdatei `.basp`	7.5
G1-xx (Stufe 1), Dok. 07	`grafik_trommel.py`, Treiber	7.1, 7.5
`c8205_grafik.py` G4-009 (22.10.2026)	`--trommelzeit`, virtuelle Zeit, Zähler der Trommelstatistik	7.8
`c8205_grafik.py` G4-025, `grafik_takt.py` TP-001, `grafik_trommel.py` G5-002 (08.11.2026)	`--taktprotokoll`, `--takt-max`, Taktmitschrift	7.11
`testscripte/grafik_takt_schleife.py` TS-002, `grafik_takt_schleife_tests.py` TY-002 (08.11.2026)	Auswertung des Taktprotokolls: Schleife um einen Befehl, Takte (`--takte`)	7.11

7.10 Zusatztrommeln des experimentellen Prototyps C8205Z (`--C8205Z`)

Mit `./c8205_grafik.sh --C8205Z[=VERZ]` (auch zusammen mit `--HWCHECK`) hat GRAFIK vier Zusatztrommeln ZT1 bis ZT4 zu je 4096 Worten. Es handelt sich um einen **experimentellen Prototyp**; Zeitmodell und Modulspezifikation: `C8205Z/zt/Zusatztrommel_Zeitmodell.md`. Ohne den Schalter ist alles unverändert (`601`/`661` sind Leer-Operationen mit Löschen).

- **Befehle:** `5k00 030` (k = 1 bis 4; 5100 = ZT0 bis 5400 = ZT3) wählt die Zusatztrommel k; `zzz0 bbbb 601` überträgt die Haupttrommelspur bbbb auf die Spur zzz0 der gewählten ZT, `zzz0 bbbb 661` die Spur zzz0 der gewählten ZT auf die Haupttrommelspur bbbb. Jeder Befehl überträgt eine ganze Spur (32 Worte). Ohne Anwahl: Halt mit Flag A (601) bzw. E (661) wie in TEST.
- **Anbindung:** `grafik-interpreter-module/grafik_zt.py` (Klasse `GrafikZT`, G5-001) hängt das C-Modul `C8205Z/zt_modul` über das Erweiterungsmodul `c8205_ztmodul` an die Haupttrommel (Direktzugriffe ohne Koinzidenz). Der Python-Interpreter im Modus TEST rechnet dagegen mit Zeit 0 (`c8205_zt.py`).
- **Zeit:** Bei `--trommelzeit alt` wartet das C-Modul in Echtzeit (ein Befehl dauert 81 bis 86 ms). Bei `echt` und `schnell` rechnet das Modul nur; die Dauer wird auf die virtuelle Zeit der Haupttrommel aufgeschlagen, bei `echt`

abgeschlafen, bei `schnell` nicht. Ein Abbruch (X im Fenster, Netz-Aus, Strg+C) beendet auch einen laufenden Befehl.

- **Bauen:** `cd C8205Z/zt_modul/python && python3 setup.py build_ext --inplace` (Windows: `bau_ztmodul_mingw.bat` im selben Ordner; am 26.10.2026 gebaut und importiert, kein Funktionstest, §9.2.3). Fehlt das Modul, endet der Start mit einer Fehlermeldung und der Bauanleitung.
- **Prüfung:** `bash C8205Z/Tests_Z/zt_grafik_tests.sh` vergleicht TEST und GRAFIK (ohne Fenster) in allen drei Trommelzeiten Byte für Byte. Das Pult-Fenster mit `--C8205Z` ist nur gestartet, nicht abgenommen.
- **Anzeige:** Das Pult zeigt die Zusatztrommeln nicht. Der HW-Check (§8.7) zeigt sie nur mit `--C8205Z`: je ZT die zuletzt bearbeitete Adresse, hinter der angewählten ein kleiner grüner Punkt, und ein Inhaltsfeld für die angewählte ZT.

7.11 Taktprotokoll (`--taktprotokoll`)

Stand 08.11.2026 (G4-025, `grafik_takt.py` *TP-001,* `grafik_trommel.py` *G5-002). Auftrag des Inhabers: die Takte einer Programmschleife (z. B. der Eingabeschleife) ermitteln; die Messung muss abschaltbar sein. Geprüft mit* `testscripte/grafik_takt_tests.py` *(16 Prüfungen).*

Zweck. Die virtuelle Zeit `vt` (§7.8) hängt nur vom Programm und der Startlage ab. Mit `--taktprotokoll DATEI` schreibt GRAFIK jede Fortschreibung von `vt` als Zeile in eine Textdatei. Es wird **alles** mitgeschrieben, die Uhr läuft für alle Befehle; welche Schleife, welches Fenster und welche Summe gebildet werden, entscheidet erst ein Auswertungsskript. So bleibt jede Wartezeit richtig, auch die vor dem ersten Befehl der Schleife (sie hängt von der Trommellage und damit vom Befehl davor ab).

Aus und an. Ohne den Schalter wird das Modul nicht geladen; die Trommel prüft je `tick()` und je Zugriff nur `takt is None`. Verhalten, Ausgaben und Zähler sind dann unverändert. Mit dem Schalter ändert sich an `vt`, an der Wartezeitsumme und an den Registern nichts (geprüft: Lauf mit und ohne Mitschrift ist gleich). Gilt nur mit `--trommelzeit echt` oder `schnell`; mit `alt` meldet GRAFIK einen Fehler (Rückkehrcode 16), weil es dort keine virtuelle Zeit gibt. Der Knotenablauf ist nicht geändert: `Rechner.takt_einhaengen` umhüllt `exec_node` nur an der Instanz; deshalb ist in `c8205_interp.py` (TEST, alle Zeiten 0) nichts nachzuziehen.

Aufruf (Beispiel).

```
python3 c8205_grafik.py --ohne-fenster --STUMM --trommelzeit schnell --trommel EIGENE_TROMMEL.txt \
        --entwickler-tasten "H G R" --taktprotokoll takt.tsv
```

Am Ende erscheint zusätzlich die Zeile `Taktprotokoll: DATEI (N Zeilen, vt am Ende …, Zugriffe, Wartezeit).`

Format. UTF-8, LF; zwei Kopfzeilen mit `#`, dann je Ereignis eine Zeile, Spalten durch Tabulator getrennt:

Spalte	Inhalt
`lfd`	laufende Nummer ab 1
`vt`	virtuelle Zeit in Wortzeiten **vor** dem Ereignis
`art`	`T` Phase beginnt (eine Wortzeit; Knoten 63: die zusätzliche Wortzeit des Sprungs, Phase I wird übersprungen), `L` Lesezugriff, `S` Schreibzugriff, `Z` Lücke (`vt` wurde außerhalb von `tick()` und Zugriff fortgeschrieben, z. B. Zeitaufschlag der Zusatztrommeln, §7.10)
`knoten`, `phase`	Knoten des Graphen und zuletzt begonnene Phase (I bis IV)
`BZ`, `BR`, `OR`, `AC`	Befehlszähler, Adressteil des Befehlsregisters, Operationscode und Akkumulator im Augenblick des Ereignisses (oktal; die Register ändern sich während des Befehls, §7.5 und Knotenbeschreibung)
`n`	bei `T` und `Z`: Wortzeiten; bei `L` und `S`: Wartezeit bis der Sektor unter dem Kopf liegt (0 bis 31)
`adresse`	bei `L` und `S`: Trommeladresse (oktal); sonst `-`

Die Zeit nach einem Ereignis ist `vt + n`; die nächste Zeile beginnt dort (Stetigkeit, geprüft). Der Befehlsbeginn ist die Zeile `T` mit Knoten 2 (Phase I); der Sprung über Knoten 63 hat keine Phase I. Am Ende steht `# Ende: …`, bei Überschreiten von `--takt-max` stattdessen `# ABGEBROCHEN …` (die Uhr läuft weiter, nur die Mitschrift endet).

Grenzen (nicht erledigt).

- Eingabe-Halt und Halt stehen nicht im Protokoll: `vt` steht dort still (§7.8). Das gelesene Zeichen wird nicht eigens geschrieben; es steht, wenn der Befehl es in den Akkumulator übernommen hat, in der Spalte `AC` der folgenden Zeilen.

- Die Auswahl einer Schleife ist nicht Teil von GRAFIK; sie macht das Auswertungsskript `testscripte/grafik_takt_schleife.py` (Teil 1, seit 08.11.2026, unten). Die Takte je Zyklus bestimmt `--takte` (unten); noch offen: Summe je Zeichen (z. B. Punkt, zehn Ziffern) und Eintrittsadresse.
- Wie jede Zeit in GRAFIK hängt die erste Wartezeit von der Startlage ab (§7.8, Streuung bis 31 Wortzeiten). Die Takte von Befehl zu Befehl nicht, weil jeder Befehlsabruf auf dem Sektor seines Befehls beginnt.
- Windows ist nicht geprüft (reines Python, keine neue Plattformabhängigkeit).

Auswertung: Schleife um einen Befehl und ihre Takte (`testscripte/grafik_takt_schleife.py` **TS-002, Test** `grafik_takt_schleife_tests.py` **TY-002, 36 Prüfungen; Block des Sammellaufs).** Das Skript liest ein Taktprotokoll, wählt die Befehle, die mit einem durch seine Adresse angegebenen Befehl (Anker) eine Schleife bilden (Teil 1), und bestimmt mit `--takte` die Takte (Teil 2).

- **Befehlsadresse.** Jeder Befehl hat genau eine Zeile `L` mit Knoten 9 (Phase II, Befehl holen); ihre Spalte `adresse` ist die **Adresse des Befehls**. `BZ` taugt dafür nicht: nach einem Sprung (Knoten 63) steht dort die Rückkehradresse. OR und BR (Operationscode, Operandadresse) stehen in der ersten Zeile `T` mit Knoten 15 (Phase III) danach. Die Zeile des Befehls selbst hat in `adresse` nur `-`; das gilt auch für `011` (Eingabe): seine Adresse steht in der Zeile `L`/Knoten 9 davor, und sein OR wird während des Befehls zu `211`, der Operandzugriff folgt danach mit anderer Adresse. In einem Protokoll sind Befehle und Phase-III-Zeilen gleich viele (geprüft am Lauf der Eingabeschleife: 41 844 Befehle).
- **Zyklus und Variante.** Ausführungen des Ankers A in Protokollreihenfolge: e1, e2, … Ein *Zyklus* ist die Folge aller Befehle von e(i) einschließlich bis e(i+1) ausschließlich. Die Schleife ist die Menge der Befehlsadressen aller Zyklen. Gleiche Adressfolgen bilden eine *Variante* (z. B. Ziffer oder Zeilenende; Zahl der Durchläufe einer inneren Schleife). Unterprogramme und innere Schleifen zwischen zwei Ausführungen von A gehören zum Zyklus. Befehle vor der ersten und nach der letzten Ausführung von A bilden keinen Zyklus und werden nur gezählt.
- **Takte (Teil 2,** `--takte`**; Festlegung des Inhabers 08.11.2026).** Ein Befehl umfasst die Zeit vom Beginn seiner Phase III bis zum Beginn der Phase III des Folgebefehls, also die Wartezeit auf den Folgebefehl eingeschlossen. Die Takte eines Zyklus sind `vt_III` der nächsten Ausführung des Ankers minus `vt_III` dieser Ausführung; je Befehl `vt_III` des Folgebefehls minus eigenes `vt_III`. Ausgegeben werden je Variante Zyklen, kleinste, größte und mittlere Takte (mit ms bei 104 µs je Takt; „schwankt“, wenn kleinste und größte verschieden sind), dann über alle Zyklen Summe, Mittel je Zyklus und je Befehl. Die Wartezeit auf das Zeichen im Eingabe-Halt ist nicht enthalten (`vt` steht dort still). Die TSV-Datei hat je Befehl `vt_holen`, `wz_holen`, `vt_III` und `takte`. **Messwert (Lauf der Eingabeschleife,** `--trommelzeit schnell`**, FS751, Anker 0064):** Ziffernvariante V1 (11 Befehle, 0064 bis 0076) in allen 2391 Zyklen genau **352 Takte** (36,608 ms), wie in Lieferung 001; die Varianten über 0333 (Zeichenende, 41 bis 91 Befehle) 1216 bis 2816 Takte; Mittel über alle 2703 Zyklen 489,649 Takte. Die Varianten mit demselben Weg schwanken um bis zu 64 Takte (V2 1408 bis 1472), vermutlich wegen wechselnder Operandadressen (OR/BR wechselt); nicht untersucht.
- **Aufruf (aus dem Projektstamm).**

```
python3 testscripte/grafik_takt_schleife.py LOG --liste              # alle Befehlsadressen mit OR, BR, Häufigkeit
    (Anker finden)
python3 testscripte/grafik_takt_schleife.py LOG --befehl 0064        # Schleife um den Befehl auf Adresse 0064
    (oktal, mehrfach möglich)
python3 testscripte/grafik_takt_schleife.py LOG --op 011             # Anker = alle Adressen, an denen ein Befehl
    mit OR 011 steht
python3 testscripte/grafik_takt_schleife.py LOG --befehl 0064 --takte   # dazu die Takte je Variante und über alle
    Zyklen
```

Weitere Schalter: `--voll` (Befehle je Variante einzeln), `--varianten N` (Standard 8), `--tsv DATEI` (je Zyklus und Befehl eine Zeile: `anker zyklus variante pos adresse OR BR vt_holen wz_holen vt_III takte`, `takte` immer gefüllt). Ausgabe: Ausführungen, Zyklen, Varianten (Adressfolge in Bereichen, `xK` = derselbe Bereich K-mal hintereinander), Vereinigungsmenge mit Zahl der Zyklen und Ausführungen je Adresse, Kern (Adressen in jedem Zyklus). Rückkehrcode 0 in Ordnung, 2 Aufruf oder Datei falsch, 4 Anker nicht gefunden oder weniger als zwei Ausführungen.

- **Grenzen.** Ein Anker, der in einem langen Zyklus nur einmal je äußerem Durchlauf läuft (z. B. der Befehl 0051 im Lauf der Eingabeschleife, Zyklen bis 1398 Befehle), liefert den ganzen äußeren Durchlauf einschließlich der inneren Schleifen; die innere Schleife wählt man mit ihrem eigenen Anker (0064). Wechselt OR oder BR eines Befehls (Befehl zur Laufzeit gebaut), zeigt die Tabelle den häufigsten Wert und den Vermerk „wechselt“.

8 Hardware-Check

Inhalt: `--HWCHECK`, Fenster, Anzeige

Quellen (alt): Dok. 01 §1.9; `HWCHECK_HW-003.md`; Dok. 10 §13 bis §15 (Haltzeit)

Mit dem Schalter `--HWCHECK` (nur Modus GRAFIK, nur mit dem Pult-Fenster; Entscheidung 17.10.2026, Entwurf 97, erweitert Entwurf 98, Gruppierung geändert Entwurf 99) blendet das Programm **ein weiteres Fenster** auf, mit dem sich die Wirkung der Pult-Tasten auf die Hardware-Elemente genau ansehen lässt. Das Fenster ändert nichts am Rechner. Mit `--ohne-fenster`, `--entwickler-tasten` und `--entwickler-k0` ist `--HWCHECK` wirkungslos (Hinweis im Terminal). Alte Kennungen (HW-xxx, HZ-xxx, Entwurf-Nummern) stehen in Klammern.

8.1 Fenster

- **Größe und Lage:** nur so groß, wie die Anzeige braucht (keine Skalierung), **verschiebbar**; erscheint beim Start neben dem Pult-Fenster.
- **Dauer:** Das Fenster bleibt aufgeblendet, solange der GRAFIK-Interpreter gestartet ist. Es lässt sich **nicht schließen** (X am HW-Check-Fenster ohne Wirkung) und endet mit dem Pult-Fenster (Netz aus, X am Pult, Strg+C). „Aufgeblendet“ heißt nicht „immer im Vordergrund“.
- **Aktualisierung:** im 40-ms-Takt des Pult-Fensters (§3.5). **Keine Rückwirkung** auf den Rechner: Der Speicherinhalt wird ohne Koinzidenz direkt gelesen, ohne Wartezeit und ohne Änderung der Trommelstatistik (§7.8).
- **Vor dem Einschalten** (Rechner aus) zeigt das Fenster die Registerwerte: BZ, BR, `Spur Sektor` und AC stehen auf 0. Die Zeile `<<AR>>` zeigt dabei den **Inhalt der Zelle 0000 aus der Trommel**, nicht Null: Die Trommel wird beim Programmstart angelegt und eingeschaltet, nicht erst mit dem Netzschalter am Pult (gemessen 21.10.2026 am GRAFIK-Kern mit Trommelkopie: `<<AR>>  AR=0000` → `0000 2000 000`; Platzhalter `---- ---- ---` erscheint nur, wenn die Trommel geschlossen ist). Haltzeit: `----.- ms`.
- **Nicht Teil der Anforderung:** Schreiben in Register oder Speicher, Haltepunkte, weitere Elemente (Flags, V1/V2); sie lassen sich später ergänzen. Die umlaufende Trommellage (der Sektor unter dem Kopf, wechselt alle 104 µs) wäre im 40-ms-Takt nicht lesbar und ist nicht Teil der Anzeige; sie ließe sich bei Bedarf als eigene Zeile ergänzen.

8.2 Anzeige: sechs Zeilen untereinander

Alles oktal, außer der letzten Zeile (`grafik_hwcheck.py`, HW-004).

Zeile	Inhalt
`BZ`	Befehlszähler, 4 Stellen
`BR`	**AR-Teil und OR** des Befehlsregisters, 4 und 3 Stellen, wie die Lampenreihen AR und OR am Pult (im Programm `m.adr`, `m.OR`); die Zeile zeigt immer das Register, auch wenn die AR-Lampen bei `E+` ohne M den BZ zeigen (OP-10, §3.8)
`Spur Sektor`	Spurnummer (3 Stellen, 000 bis 177) und Sektornummer (2 Stellen, 00 bis 37) der **Adresse im Speicheradressregister AR** (`m.AR`): Spur = AR ÷ 32, Sektor = AR mod 32 (logische Sektornummer wie `sektor()` im Interpreter); Beispiel `054 13`
`AC`	Akkumulator, 11 Stellen in Vierergruppen
`<<AR>>`	**Speicherinhalt** an der Adresse im Register AR, 11 Stellen in Vierergruppen; der Wert von AR steht zur Kontrolle im Namen (`<<AR>>  AR=0001`)
`Haltzeit`	Millisekunden, dezimal (§8.3)

Vierergruppen werden von links gebildet: 11 Stellen = 4 + 4 + 3, z. B. `2206 0635 124`, Null: `0000 0000 000` (geändert in Entwurf 99 nach dem Test des Projektinhabers vom 17.10.2026; die Entwürfe 97 und 98 bildeten sie von rechts, 3 + 4 + 4). Der AR-Teil von BR ist der **Adressteil des Befehlsregisters** (die Pult-Lampenreihe AR); das AR in `<<AR>>` und in `Spur Sektor` ist dagegen das **Speicheradressregister**. Das sind zwei verschiedene Register.

8.3 Haltzeit (seit 19.10.2026)

Die sechste Zeile zeigt die **reine Rechnerzeit** in Millisekunden (eine Nachkommastelle) vom Start (Taste R wirkt) bzw. von der Fortsetzung nach einem Halt bis zum nächsten Halt, dazu „von → bis“, z. B. `85.0 ms  E/A-Halt -> E/A-Halt` oder `R -> E/A-Halt`. Ohne Messung steht `----.- ms`.

- **Einzelschritt (E, HZ-004, 20.10.2026):** Die Zeile zeigt die Zeit eines Schrittes (ein Durchlauf der Befehlsschleife) von der Taste E bis zum Halt danach, z. B. `0.6 ms  E -> Halt`; endet der Schritt in einem Stopp oder Fehler, steht dort `E -> Stopp` bzw. `E -> Fehler`. Vorher blieb die Zeile im Einzelschritt bei `---.- ms`, weil nur R die Messung startete. Bei E+ und E- wird jeder Teilschritt einzeln gemessen, bei E (kurzer Anschlag) beide zusammen.
- **Als Halt gilt:** Stopp (US/BS, Halt wegen Überlauf), Fehler (FE, Geräte- oder Ablauffehler), E/A-Halt (der Rechner wartet an einem Eingabebefehl auf ein Zeichen oder eine Lochbanddatei). Die Taste H zählt nicht.
- **Die Wartezeit im Halt zählt nicht** (Entscheidung des Projektinhabers 19.10.2026, HZ-003; `NUR_LAUF = True` in `grafik_haltzeit.py`; `False` gibt die Zeit mit Wartezeit zurück). Die frühere Fassung zeigte bei Halt → Halt die Zeit mit Wartezeit (HZ-001) und kurz auch beide Zeiten (HZ-002); beides entfällt. Im Debug-Log steht je Halt die Marke `HALT` mit Zeit und, falls bekannt, Laufzeit.
- **Messung:** `grafik_haltzeit.py` (`m.haltuhr`); `c8205_grafik.HaltMaschine` (Unterklasse von `Machine`, `c8205_interp.py` bleibt unberührt) meldet jedes Setzen von `halted` von False auf True; K0-Warteschlange und Lochbandleser haben dafür die Haken `bei_halt` und `bei_weiter`.
- **Genauigkeit gegenüber dem Original:** Die Zeit ist die Wanduhrzeit des Modells. Die Trommel dreht in Echtzeit (104 µs je Wortzeit), die Befehlsschleife braucht davon nur einen Bruchteil; zeitoptimale Programme zeigen deshalb die Originalzeit. Das Modell bedient aber auch Zugriffe, die unter dem dokumentierten Mindestabstand liegen (Sprung unter 4 Wortzeiten, Operand eines Einzelbefehls nach 1 Wortzeit); dort ist die angezeigte Zeit um bis zu eine Umdrehung (3,2 ms) je Zugriff kürzer als am Original. Die Mindestabstände werden bewusst nicht erzwungen (entschieden 22.10.2026, Projektinhaber; `trommel.c` bleibt unverändert). Messung und Zahlen: `C8205-Interpreter-GRAFIK_Pruefverfahren.md` 7.6.
- **Intervall (HZ-005, 22.10.2026):** Hinter dem Wert steht ab dem zweiten Halt das Intervall der letzten 10 Halts (`INTERVALL_N` in `grafik_haltzeit.py`), z. B. `0.7 ms  E -> Halt  (0.5 bis 2.9 ms, letzte 10)`. Verglichen werden dieselben Werte, die die Zeile als Hauptwert zeigt (reine Rechnerzeit, `NUR_LAUF`). Frühere Halts fallen heraus; das Fenster wird dafür breiter.
- **Weitere Wartestellen im Dauerlauf (22.10.2026):** Das Warten auf K0 und auf den Leser meldet der Haltzeit einen E/A-Halt (`bei_halt`/`bei_weiter`). **Seit G4-003 gilt das auch für das Schreibwerk:** Der Ausgabebefehl kehrt sofort zurück (die Befehlsschleife rechnet während der Schreibwerkzeit weiter); nur wenn der nächste Ausgabebefehl kommt, bevor das Schreibwerk frei ist (Zeichen 75 bis 90 ms, TAB 400 ms, WRZL 360 ms, WR 120 ms), wartet er bis zu deren Ende, und dieses Warten ist ein E/A-Halt: die Haltzeit-Zeile zeigt dann nur die Rechnerzeit (mit `NUR_LAUF = True`). Jedes solche Warten erzeugt einen Halt-Eintrag in der Zeile. Taste H wirkt, wenn der Rechner auf das Schreibwerk wartet, erst nach dem Ende des Wartens (bis 0,4 s). Der **Stanzer** hängt je Zeichen eine Zeile an die Datei an (Öffnen, Schreiben, Schließen, ohne Pause); die Dauer ist die des Dateisystems, er wartet nicht. Die Lampenabtastung bleibt richtig (der Rechner steht beim Warten), die Trommelzeit wird nicht berührt. Geprüft: `testscripte/grafik_stufe4_tests.py` (Klasse `SchreibwerkAsynchron`); nicht am Pult-Fenster erprobt.
- Die Haltzeit dient auch zur Beurteilung der SW-Eingabe: Der Abstand der Eingabebefehle (350 bis 450 ms gemessen, Zeit mit Wartezeit) liegt um die Fokus-Schwelle von 0,4 s (§5.7).

8.4 Fokus

Bei geöffnetem HW-Check-Fenster kann das SW-Fenster den Fokus bei einem Eingabe-Halt nicht immer halten; der Tk-Test meldet eine Warnung. Das hängt vom Fenstermanager ab und ist als bekannte Eigenschaft akzeptiert (B-5, A-4; kein Programmfehler). Im echten Lauf holte sich das SW-Fenster den Fokus bei 97 von 97 Tipps; ein HW-Check-Fenster, das während eines Eingabe-Halts erst öffnet, kommt in der Praxis nicht vor.

8.5 Stand, Annahmen, Prüfung

- **Abnahme:** Das Aussehen beurteilt der Projektinhaber; bei der Abnahme der Stufe 3 (18.10.2026) wurde keine Beanstandung gemeldet. Der Tk-Test (alle Punkte OK) lief am echten Fenster.
- **Standardannahmen aus Dok. 01 §1.9 Nr. 5, bestätigt 21.10.2026 (OP-15, §10.5):** (b) die beiden Bedeutungen von AR (§8.2); (c) Anzeige auch vor dem Einschalten; (d) „aufgeblendet“ = nicht schließbar; (f) `Spur Sektor` bezieht sich auf das Speicheradressregister AR, nicht auf die umlaufende Trommellage (Entwurf 98, zur Bestätigung). Die Gruppierung von links (a) ist durch den Test des Projektinhabers vom 17.10.2026 festgelegt. **Gegengelesen 21.10.2026 (OP-15):** (b) `BR` liest `m.adr` und `m.OR`, `<<AR>>` und `Spur Sektor` lesen `m.AR` (`hwcheck_zeilen`); (c) siehe §8.1, mit der Genauigkeit zu `<<AR>>`; (d) `WM_DELETE_WINDOW` ist leer belegt, das Fenster ist ein `Toplevel` des Pults und endet mit ihm; (f) Spur = AR ÷ 32, Sektor = AR mod 32, Randwerte 000 00 und 177 37 im Test. Bestätigt am 21.10.2026 (§10.5).

- **Rückfall beim Lesen** (HW-003): Hat die Trommel-Klasse keine Methode `einblick()` (älterer Stand von `grafik_trommel.py`), liest der HW-Check direkt über das C-Modul (`trommel.T.lesen`, ohne Koinzidenz, ohne Statistik); bei einem Lesefehler bleibt der Platzhalter `---- ---- ---`, der Grund steht in `grafik_hwcheck.LETZTER_FEHLER`. Ein Platzhalter im Fenster weist also auf einen Lesefehler hin.
- **Prüfung:** Die Formatierung (`hwcheck_zeilen`, `gruppen4`) braucht kein Tkinter und ist in `testscripte/grafik_pult_tests.py` (Klasse `HwCheck`, darin `test_rueckfall_ohne_einblick` und seit 21.10.2026 `test_op15_anzeige_vor_dem_einschalten`) geprüft; die Haltzeit-Prüfungen (z. B. `test_haltzeit_nur_rechnerzeit`) stehen nach Dok. 10 ebenfalls dort bzw. im Gesamtlauf (die Fundstelle habe ich nicht einzeln geprüft). Am echten Fenster ist der Hardware-Check mit dem Tk-Test (A-4) und der Hörprüfung 2 gelaufen (`./c8205_grafik.sh --HWCHECK`). Im Code gegengelesen habe ich nur den Kopf von `grafik_hwcheck.py` (HW-004, sechs Zeilen, Gruppierung von links), `NUR_LAUF` in `grafik_haltzeit.py`, die Fokus-Schwelle von 0,4 s in `grafik_sw_fenster.py` und das Vorhandensein der Klasse `HwCheck`.
- **Hinweis zu Dok. 01 §1.9:** Der Text nennt noch fünf Zeilen und keine Haltzeit; maßgeblich ist diese Beschreibung (HW-004, HZ-003).

8.6 Kennungen

Alte Kennung	Gegenstand	Abschnitt
Entwurf 97, 98, 99; Dok. 01 §1.9	Fenster, Zeilen, Gruppierung	8.1, 8.2
HW-003	Lesen des Speicherinhalts (Rückfall), Gruppierung von links	8.2, 8.5
HW-004, HZ-001, HZ-002, HZ-003, HZ-004, HZ-005	Haltzeit	8.3
HW-005 (25.10.2026)	Zusatztrommeln im HW-Check (`--C8205Z`)	8.7
B-5, A-4 (Dok. 09)	Fokus bei geöffnetem Fenster	8.4

8.7 Zusatztrommeln im HW-Check (nur mit `--C8205Z`, experimenteller Prototyp C8205Z)

Mit `./c8205_grafik.sh --HWCHECK --C8205Z` hat das Fenster fünf weitere Zeilen unter den sechs der §8.2 (`grafik_hwcheck.py` HW-005; Entscheidung des Inhabers 25.10.2026):

Zeile	Wert
`ZT1` bis `ZT4`	zuletzt bearbeitete Adresse der Zusatztrommel als `Spur Sektor` (oktal, wie die Zeile Spur Sektor; `--- --` = noch nichts übertragen). Hinter der **angewählten** Zusatztrommel steht ein kleiner grüner Punkt
`<<ZTk>>  aaaa`	**ein** Inhaltsfeld für alle vier (es ist immer nur eine Zusatztrommel aktiv): das Wort (11 Stellen, 4 + 4 + 3) an der zuletzt bearbeiteten Adresse der angewählten ZT k; ohne Anwahl `<<ZT>>` und `---- ---- ---`

Die „laufende Adresse“ ist die Adresse des zuletzt übertragenen Wortes (`zt_letzte_adresse` im C-Modul); sie läuft während eines Befehls mit (in `--trommelzeit alt` und `echt` sichtbar, in `schnell` nur als Ergebnis, weil ein Befehl dort keine Zeit dauert). Es ist nicht die Lage der Zusatztrommel unter dem Kopf. Ohne den Schalter bleibt das Fenster bei sechs Zeilen. Die Anzeige liest nur und verändert nichts. Prüfung ohne Fenster mit Tk-Attrappe: `python3 C8205Z/Tests_Z/zt_hwcheck_test.py`; das echte Fenster ist nicht abgenommen.

9 Plattformen

Inhalt: Linux (getestet), Windows (Teilabnahme 26.10.2026), Abhängigkeiten (Tk, ALSA/winmm)

Quellen (alt): Dok. 00 §3.2; Dok. 01 §9; Dok. 02 §5.3; Dok. 10 §10; Dok. 11 S5-12

Zielsysteme sind **Linux und Windows**, nicht von bestimmten Versionen abhängig. Vollständig getestet ist nur Linux; unter Windows hat der Projektinhaber am 26.10.2026 gebaut und teilweise geprüft (§9.2). Die Abnahme der Stufen 4 und 5 am 19.10.2026 erfolgte **ohne Windows**; Windows ist eine eigene Abnahme (O5-3, A5-8; Prüfverfahren §9), die erst teilweise erfolgt ist.

9.1 Linux

Gegenstand	Stand
Voraussetzungen	Python 3 mit Tkinter (unter Linux oft ein eigenes Paket, z. B. `python3-tk`); für die Töne eines der Abspielprogramme `paplay`, `pw-cat`, `aplay` (der Mischer nimmt das erste vorhandene in dieser Reihenfolge); ohne Abspielprogramm bricht GRAFIK ohne `--STUMM` mit Code 16 ab (§6.7). C-Modul: vorübersetzt mitgeliefert (`c8205_trommel.abi3.so`, statisch gelinkt); ein eigener Bau braucht gcc, `python3-dev` und `setuptools` (§7.1)
Start	`./c8205_grafik.sh [Schalter]` (§2)
Getestet	Trommel und Mischer in C (Linux, gcc, ein Kern), Logiktests ohne Fenster, Tk-Test am echten Fenster, Hörprüfungen 1 und 2 und echter Lauf mit Tk-Fenster durch den Projektinhaber (19.10.2026); Gesamtlauf `grafik_alle_tests.sh` 18 von 18 Blöcken auf seinem Rechner
Bekannte Umgebung	`hp7`, Ubuntu 23.04, Linux 6.2.0-27, Python 3.11.4, gcc 12.3.0, `paplay` (pacat 16.1), `pw-cat` (libpipewire 0.3.65), `aplay` 1.2.8
Python-Version	Das Erweiterungsmodul ist ein abi3-Modul für CPython ab 3.8. Bis Checkpoint 56 wurde es mit 3.12 gebaut und stürzte auf 3.11 ab (`deallocating None`); behoben in G4-005 (§6.8). Die Entwicklungsumgebung hat Python 3.12; der Absturz ließ sich dort nicht nachstellen
Bekannte Eigenschaften	Fokus bei geöffnetem HW-Check-Fenster hängt vom Fenstermanager ab (§8.4); der Strg+C-Test des C-Moduls schlägt fehl, wenn `grafik_alle_tests.sh` als Hintergrundauftrag (`&`) startet, weil die Shell dann SIGINT ignoriert; im Vordergrund ist er grün

9.2 Windows (Teilabnahme 26.10.2026)

Quelle der Windows-Angaben: Meldungen des Projektinhabers vom 26.10.2026 (Bau und Test auf seinem Rechner). Windows-Bau und Windows-Test laufen beim Inhaber, nie in der Cloud-Sitzung; die Angaben sind hier nicht nachgeprüft.

- **Stand:** Der Windows-Zweig der Trommel, der Mischer `tonmisch` mit `winmm` und die Zusatztrommel sind seit 26.10.2026 mit MinGW-w64 gebaut (Ergebnis, Bau, Fehlerbilder, Dateiablage und offene Punkte: unten und §9.2.1 bis §9.2.3). Für `c8205_interpreter.bat` liegt keine Meldung vor. Das Pult-Fenster (Tkinter) ist plattformneutral geschrieben; ob es unter Windows vollständig bedienbar ist, wurde nicht gemeldet und nicht geprüft.
- **Compiler:** MinGW-w64 (entschieden 16.10.2026, O-8; der Projektinhaber hat nur diesen installiert), kein MSVC. Bau ohne setuptools mit `bau_trommel_mingw.bat` (Ordner `c-module\python`) und `bau_ztmodul_mingw.bat` (Ordner `C8205Z\zt_modul\python`). Belegt ist nur der Bau mit diesen Skripten; der frühere Weg über `mingw32-make` und `python setup.py build_ext --inplace --compiler=mingw32` ist unter Windows nicht erprobt. Die Skripte benutzen nur `-static-libgcc`; ob mehr nötig ist, hängt von der DLL-Prüfung ab (§9.2.3). Das Modul benutzt die Limited API (`python3.lib`); ob dieselbe `.pyd` unter Python 3.8 bis 3.12 lädt, ist nicht geprüft (§9.2.3).
- **Auslieferung:** vorübersetzt, eine Datei je System (Entscheidung 16.10.2026). Die Windows-Dateien `c8205_trommel.pyd` und `c8205_ztmodul.pyd` liegen neben den Linux-Dateien (`.abi3.so`); Windows lädt nur `.pyd`, die `.abi3.so` wird dort ignoriert, beide dürfen nebeneinander liegen. Ob die Windows-Dateien als geprüft gelten, hängt von der DLL-Prüfung und den anderen Python-Versionen ab (§9.2.3).
- **Töne unter Windows:** `c8205_grafik.py` prüft beim Start mit `grafik_tonmisch_c.winmm_verfuegbar()`, ob das C-Modul mit `ton_*`-Methoden lädt und `winmm` sich öffnen lässt. Dann läuft derselbe Aufbau wie unter Linux (Pult-, Schreibwerk-, Leser- und Stanzerton über den C-Mischer, Senke `CWinmmSenke`, `neuer_mischer_winmm()`), mit der Startzeile „Hinweis: Toene unter Windows ueber den Mischer (winmm).“. Sonst gilt als Rückfall der `winsound`-Zweig von Stufe 2 (Rechnertöne, ein Klang zur Zeit, Schreibwerkzeit und Lesetempo, kein Leser- und Stanzerton) mit Hinweiszeile und einer Zeile „Grund: …“ (O4-18). Mit `--STUMM` wird `winmm` nicht geprüft. Fällt `winmm` im Betrieb aus, gilt wie unter Linux Code 16. `--TON` wirkt im `winsound`-Zweig für Pult nicht (§6.6); mit dem Mischer ist es nicht geprüft.
- **Trommel:** Die Systemuhr arbeitet unter Windows nur grob; die Wartezeit in Wortzeiten bleibt exakt, die Echtzeit-Dauer eines Zugriffs kann länger werden (§7.4). Die Trommeldatei muss LF-Zeilenenden haben (Werkzeuge im Textmodus würden CRLF schreiben; das Modul weist solche Dateien ab). Strg+C-Test des Erweiterungsmoduls wird unter Windows übersprungen.
- **Gemeinsam relevant:** Zeilenenden, Namen der Bibliotheken (`libtrommel.so`, `trommel.dll`), Pfadtrenner (Pfade mit Umlauten sind über `_wfopen` vorgesehen), Strg+C-Behandlung.
- **Abnahme:** Windows wird gegen das Prüfverfahren abgenommen (O5-3, O6-5; Prüfverfahren §9 mit B-2 und A5-8). Teilabnahme am 26.10.2026, Rest offen (§9.2.3, §10.1).

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Ergebnis (26.10.2026, Rechner des Projektinhabers)

Baustein	Bau	Laden	Test
`c8205_trommel.pyd` (Trommel, Bahnsperre, Unterbrechung, Tonmischer)	gebaut mit MinGW-w64	`import c8205_trommel` ok	`test_c8205_trommel.py`: 23 Tests, OK, 1 übersprungen
`c8205_ztmodul.pyd` (Zusatztrommeln, C8205Z)	gebaut mit MinGW-w64	`import c8205_ztmodul` ok	kein Python-Test vorhanden
GRAFIK-Interpreter (`c8205_grafik.bat`, Start aus `neu\`)	–	startet; Startzeile „Hinweis: Toene unter Windows ueber den Mischer (winmm).“	Töne vom Inhaber **gehört**: Grundgeräusch, Lochbandleser (LL) und Stanzer (LS) über den C-Mischer mit `winmm`

Umgebung: Python 3.13, 64 Bit (`...\Python313`), `gcc` aus MSYS2 `mingw64` (`gcc -dumpmachine` meldet `x86_64-w64-mingw32`; die gcc-Version wurde nicht festgehalten).

Der übersprungene Test ist der SIGINT-Test (`test_c8205_trommel.py`, Zeile 81); er ist unter Windows absichtlich ausgenommen. Die Meldung `trommel_einschalten: Dateigroesse 69632, erwartet 73728 …` mitten im Lauf gehört zum Test, der eine Datei im alten 17-Byte-Format (4096 × 17 = 69632) anlegt und deren Abweisung prüft (`TrommelFehler`, Zeilen 70 bis 73). Sie erscheint auch unter Linux und ist kein Fehler.

Ton unter Windows (Beobachtung des Inhabers, 26.10.2026): Beim Start von `c8205_grafik.bat` erscheint die Zeile „Hinweis: Toene unter Windows noch nicht ueber den Mischer (winmm fehlt): nur Rechnertoene (winsound) und Schreibwerkzeit.“ Die Töne waren hörbar. Sie kommen aber aus dem Stufe-2-Zweig (`os.name == 'nt'` in `c8205_grafik.py`, Klasse `RechnerTon` in `grafik_ton.py`, `winsound`, ein Klang zur Zeit). Der C-Mischer `tonmisch` mit `winmm` wurde damit **nicht benutzt**, weil dieser Zweig ihn umging; Leser- und Stanzerton waren dort aus. Das Hören belegt also `winsound`, nicht den Mischer.

Änderung danach (26.10.2026): `c8205_grafik.py` prüft unter Windows zuerst mit `grafik_tonmisch_c.winmm_verfuegbar()`, ob das C-Modul mit `ton_*`-Methoden lädt und `winmm` sich öffnen lässt. Dann läuft derselbe Aufbau wie unter Linux (Pult-, Schreibwerk-, Leser- und Stanzerton über den C-Mischer; Senke `CWinmmSenke`, `neuer_mischer_winmm()`), und die Hinweiszeile entfällt. Sonst gilt der alte `winsound`-Zweig mit der Hinweiszeile. Mit `--STUMM` wird `winmm` nicht geprüft. Fällt `winmm` im Betrieb aus, gilt wie unter Linux Code 16.
Ergebnis: Der Inhaber hat danach Grundgeräusch, Leser- und Stanzertöne gehört (Mischer mit `winmm` aktiv, Startzeile wie oben). Schreibwerkton, Lücken und Lautstärken (`--TON`) wurden nicht gemeldet und gelten als nicht geprüft.

Linux: Beide Module bauen mit der geänderten `plattform.h` unverändert; `test_c8205_trommel.py` läuft unter Linux mit 23 Tests ohne Auslassung und ohne Fehler (Python 3.12 in der Cloud-Sitzung, Build mit `gcc`, nicht mit `setup.py`). Die Änderungen stehen nur im `_WIN32`-Zweig und in `.bat`-Dateien.

Bauen unter Windows

Voraussetzungen: ein **64-Bit**-MinGW-w64 (MSYS2 `mingw64` oder gleichwertig) und ein 64-Bit-Python mit der Bibliothek `libs\python3.lib` (Standardinstallation von python.org). Ohne Setuptools:

```
set PATH=C:\Users\<name>\AppData\Local\Programs\Python\Python313;C:\msys64\mingw64\bin;%PATH%
gcc -dumpmachine                                  (muss x86_64-w64-mingw32 ausgeben)
cd <projekt>\c-module\python
bau_trommel_mingw.bat                             (ergibt c8205_trommel.pyd)
cd <projekt>\C8205Z\zt_modul\python
bau_ztmodul_mingw.bat                             (ergibt c8205_ztmodul.pyd)
python -c "import c8205_trommel; print('ok')"
python test_c8205_trommel.py
```

Beide Skripte rufen `gcc` mit `-shared -O2 -std=gnu11 -DPy_LIMITED_API=0x03080000 -D_WIN32_WINNT=0x0601 -DWINVER=0x0601 -static-libgcc`, binden gegen `python3.lib` und `winmm` und lesen den Python-Ordner über `sysconfig` und `sys.base_prefix`. Das `set PATH` gilt nur für das offene Terminalfenster.

9.2.1 Fehlerbilder und Abhilfe

Meldung	Ursache	Abhilfe
`unknown type name 'CONDITION_VARIABLE'`, danach implizite Deklarationen von `InitializeConditionVariable`, `WakeAllConditionVariable`, `SleepConditionVariableCS`, `YieldProcessor`	MinGW setzt sonst eine Windows-Zielversion unter Vista (`_WIN32_WINNT` 0x0502) und `windows.h` blendet diese Deklarationen aus	`-D_WIN32_WINNT=0x0601 -DWINVER=0x0601` im gcc-Aufruf der `.bat` (maßgeblich, weil `Python.h`, `bahnsperre.c` und `unterbrechung.c` `windows.h` selbst einbinden); zusätzlich setzt `plattform.h` die Werte vor `#include <windows.h>` und definiert `YieldProcessor` als `pause`, falls der Header es nicht liefert
`Recognised but unhandled machine type (0x8664) in Import Library Format archive`	Der gefundene `gcc` ist der alte 32-Bit-MinGW von mingw.org (`c:/mingw`, Pfad `mingw32/6.3.0`); `python3.lib` ist 64 Bit	64-Bit-MinGW-w64 vor den alten in `PATH` setzen; `gcc -dumpmachine` prüfen
`FEHLER: ...\libs\python3.lib fehlt` mit einem MSYS2-Pfad	Steht der MSYS2-Ordner vor Python im `PATH`, antwortet das Python von MSYS2 auf die Abfrage der `.bat`; es hat kein `libs\python3.lib`	Den Python-Ordner **vor** den MSYS2-Ordner setzen
`fatal error: ../tonmisch.h: No such file or directory`	`tonmisch.h` fehlt in `c-module`, oder `c8205_ton_methoden.h` liegt in `c-module` statt in `python` (dann zeigt `../tonmisch.h` eine Ebene zu hoch)	Ablage nach §9.2.2 herstellen
`fatal error: c8205_ton_methoden.h: No such file or directory`	Die Datei fehlt im Ordner `python`	Dorthin kopieren

9.2.2 Dateiablage

```
c-module\
    plattform.h            nur hier (nicht in python\)
    tonmisch.h             hier; c8205_ton_methoden.h bindet "../tonmisch.h" ein
    trommel.h, bahnsperre.h, unterbrechung.h, c8205_api.h, *.c
c-module\python\
    bau_trommel_mingw.bat
    c8205_ton_methoden.h   nur hier (nicht in c-module\)
    c8205_trommel_modul.c, setup.py, trommel_lader.py, test_c8205_trommel.py
C8205Z\zt_modul\python\
    bau_ztmodul_mingw.bat  liest plattform.h und c8205_api.h über ..\..\..\c-module
```

Dateinamen: Windows lädt nur `.pyd`. Die `.abi3.so` im selben Ordner ist der Linux-Build und wird unter Windows ignoriert; beide dürfen nebeneinander liegen.

9.2.3 Offen (nicht geprüft)

- **Weitere DLLs:** Der Importtest lief in demselben Terminal, in dem der MSYS2-Ordner im `PATH` stand. Fehlende Laufzeit-DLLs aus `C:\msys64\mingw64\bin` wären dadurch nicht aufgefallen. Prüfen: `objdump -p c8205_trommel.pyd | findstr "DLL Name"` und Importtest in einem frischen Fenster ohne MSYS2 im `PATH`.
- **Python-Versionen:** Gebaut und getestet nur mit Python 3.13. Das Modul benutzt die Limited API (`Py_LIMITED_API=0x03080000`) und `python3.lib`; ob dieselbe `.pyd` unter 3.8 bis 3.12 lädt, ist nicht geprüft. Der Inhaber nutzt sonst Python 3.11.
- **Tonmischer unter Windows:** `tonmisch.c` mit `winmm` läuft (gehört: Grundgeräusch, LL, LS). **Nicht geprüft:** Schreibwerkton und Tastenklänge über den Mischer, Lücken und Aussetzer bei längerem Betrieb (Puffer 4 × 10 ms), Lautstärke und `--TON`, Verhalten ohne Audiogerät (Probe `winmm_pruefen()` meldet dann den Grund), Netz aus mit `fade_out`. Unter Linux belegt: die Änderungen stehen nur im Windows-Zweig; `grafik_stufe4_tests.py` besteht mit beiden Mischern (103 Tests), und der Sammellauf `grafik_alle_tests.sh --schnell` ist so wie vorher (siehe nächster Punkt). Ob `test_c8205_trommel.py` den Mischer aufruft, wurde nicht geprüft.
- **Vorhandener Fehlschlag unter Linux (nicht von dieser Änderung):** `testscripte/grafik_sw_ende_zu_ende.py` (Block „Stufe 3 Ende-zu-Ende“) meldet 2 Fehlschläge und 1 Fehler, mit und ohne die Änderung gleich (15 Tests). Ursache nicht untersucht.

- **Zeitverhalten:** Der Test hat 0,43 s gedauert und bestanden; Messwerte der Windows-Uhr (Aufwachverspätung, Zähler `aufwachen`, `nachgeholte_ticks`, §7.8) liegen nicht vor.
- **Zusatztrommel:** nur gebaut und importiert, kein Funktionstest. In `C8205Z/zt_modul` liegt als Test nur `testframe_zt.c` (C-Testrahmen).
- **GRAFIK-Interpreter und Pult unter Windows:** Der Start über `c8205_grafik.bat` läuft bis zur Tonhinweiszeile und die Töne waren hörbar (`winsound`). Ob das Pult-Fenster (Tk) vollständig bedienbar ist, wurde nicht gemeldet und nicht geprüft.
- `-static`**:** Die Skripte benutzen nur `-static-libgcc`; ob mehr nötig ist, hängt vom Ergebnis der DLL-Prüfung ab.

9.3 macOS

Nicht Gegenstand der Anforderungen; im Code ist nur das Merken der Fensterlage (FL-001) für macOS vorgesehen (`~/Library/Application Support/C8205/`). Getestet ist nichts.

10 Offene Punkte

Dieser Abschnitt führt alles, was bei Abschluss der Stufen 1 bis 5 nicht entschieden, nicht bestätigt oder nicht geprüft war. **Entschiedenes steht nicht hier**, sondern in Anhang A (Register) und in den Abschnitten 1 bis 9. Die Punkte sind nach Art gruppiert; Kennungen in Klammern verweisen auf die alte Fundstelle (Anhang B). Wo ich einen Stand nicht gegengeprüft habe, steht es dabei.

10.0 Gestrichene Punkte (22.10.2026)

Der Projektinhaber hat am 22.10.2026 festgelegt, dass folgende Punkte aus den Arbeitsständen (Checkpoints) **nicht weiterverfolgt** werden: Hören der Leserton-Frist (`LS_AUS_S`), Lesedauer 101 s von `namen_raten.kep`, Umbau der Lampenabtastung, Wahl eines kritischsten Befehlsmixes, Messung mit Bedienung und Rechenzeit-Reserve der Knoten, Governor-Test und Erweiterung des Messskripts. Die Ergebnisse stehen im Prüfverfahren §7.6 bis §7.8. Windows bleibt eine eigene spätere Abnahme (10.1).

10.1 Windows (eigene Abnahme, Teilabnahme 26.10.2026)

Entschieden 19.10.2026 (O5-3): Stufe 5 wurde ohne Windows abgenommen; Windows ist eine eigene Abnahme, sobald ein Tester da ist (Dok. 00 §4: der Tester „kommt noch“, baut und testet selbst; Compiler MinGW-w64). Teilabnahme am 26.10.2026: Bau von `c8205_trommel.pyd` und `c8205_ztmodul.pyd` mit MinGW-w64, Laden, Test der Haupttrommel bestanden (§9.2). GRAFIK startet unter Windows; der Mischer mit `winmm` ist eingeschaltet, Grundgeräusch, Lochbandleser und Stanzer hat der Inhaber gehört. Offen: Schreibwerkton und Dauerbetrieb über den Mischer, Pult-Fenster, Zeitmessung, andere Python-Versionen, Zusatztrommel-Funktionstest. Die Abnahmekriterien stehen im Prüfverfahren §9.

Nr.	Punkt	Stand	Alte Fundstelle
OP-01	**Bau** `tonmisch` **mit MinGW-w64** und Mischer mit `winmm`	erledigt für Bau und Betrieb am 26.10.2026: übersetzt, gelinkt und im GRAFIK-Interpreter unter Windows eingeschaltet (`winmm_pruefen()`, `neuer_mischer_winmm()`; Rückfall `winsound`); Grundgeräusch, Lochbandleser- und Stanzerton über `winmm` vom Inhaber gehört. Offen: Dauerbetrieb, Schreibwerkton	B-2 (Dok. 10), S4-02, S4-15
OP-02	**Startskripte** `c8205_grafik.bat` und `c8205_interpreter.bat`	`c8205_grafik.bat` am 26.10.2026 unter Windows gestartet (Start aus `neu\`); für `c8205_interpreter.bat` keine Meldung, nicht geprüft	S5-12, A5-8 (Dok. 11)
OP-03	**Ton unter Windows**	am 26.10.2026 unter Windows gehört: erst Rechnertöne über `winsound` (Zweig ohne Mischer), danach über den Mischer mit `winmm` zusätzlich Lochbandleser- und Stanzertöne; die Startzeile nennt den Weg. Rückfall auf `winsound` mit Hinweis und Grund, wenn `winmm` nicht öffnet (O4-18, „wenn du meinst. ja“)	O4-18

Nr.	Punkt	Stand	Alte Fundstelle
OP-04	**Trommel, Plattformschicht Windows** (`plattform.h`, Win32): am 26.10.2026 mit MinGW-w64 übersetzt; `test_c8205_trommel.py` besteht unter Windows (Python 3.13, 23 Tests, 1 übersprungen). `plattform.h` setzt dafür `_WIN32_WINNT` und `WINVER` auf 0x0601 und definiert `YieldProcessor` (§9.2.1). **Offen:** ob das aktive Warten entfernt ist (Dok. 02 §4.4: „noch nicht angepasst“, Stand 16.10.2026; **nicht geprüft**, → §7.4); `-static` (die Skripte benutzen `-static-libgcc`); Python-Versionen 3.8 bis 3.12 (Limited API); Echtzeit-Prüfungen und Messwerte der Windows-Uhr; der Strg+C-Test wird unter Windows übersprungen	teilweise erledigt, Rest offen	Dok. 02 §4.4, §5.3
OP-05	**Tk-Verhalten unter Windows und WSL**, Bildschirmgrößen, Zwischenablage, X/Strg+C über die Pipe-Meldung (plattformneutral gewählt, Dok. 04 Nr. 15): nie geprüft	offen	Dok. 01 §8 Nr. 13, Dok. 04 Nr. 15
OP-26	**Kein Sammellauf unter Windows:** `grafik_alle_tests.sh` ist ein Bash-Skript, eine `.bat`-Entsprechung gibt es nur für `r300_eingabe_*`; ebenso weicht `build_msvc.bat` (MSVC) von S4-02 (MinGW-w64 für `tonmisch`) ab. Welcher Weg unter Windows gilt, ist offen (Prüfverfahren §9, W-2 und W-5). Aufgenommen am 19.10.2026 (Projektinhaber: „mir egal“, daher vom Entwickler eingetragen)	offen	Prüfverfahren §9, §10

10.2 Zu entscheiden oder zu bestätigen

Diese Punkte sind im Verhalten umgesetzt oder im Code festgelegt, aber vom Projektinhaber nicht ausdrücklich entschieden. OP-06, OP-07, OP-08 und OP-12 wurden am 19.10.2026 bestätigt, OP-42 am 21.10.2026 entschieden; sie stehen in §10.5.

Nr.	Punkt	Stand	Alte Fundstelle

10.3 Standardannahmen ohne Einzelbestätigung

Bei der Abnahme der Stufen nicht beanstandet, aber nie einzeln bestätigt. Sie gelten als Verhalten, bis jemand etwas anderes sagt.

Nr.	Annahme	Alte Fundstelle
OP-16	**Mausbedienung:** Dok. 06 ist nur so weit umgesetzt, wie Dok. 01 §1.7 es wiederholt. **Gegengelesen 21.10.2026:** Dok. 06 liegt im Archiv nicht vor; §4 führt M-15 bis M-35 vollständig (Zählung G6-014). Verglichen wurde §4 mit `grafik_pult.py` (G2-011), `grafik_pult_fenster.py` (G4-008) und den Tests. Die Regeln M-15 bis M-27, M-29 bis M-31 und M-34, M-35 stehen im Code; sieben davon, die kein Test festhielt, sind jetzt durch `Op16Maus` belegt (99 von 99 in `grafik_pult_tests.py`, sieben Gegenproben schlagen an, Kopie mit angeglichenen Umlautnamen). Das Aussehen am echten Fenster ist damit nicht geprüft. **Zwei Abweichungen vom beschriebenen Verhalten:** (1) **M-28:** „Gehaltene Computertasten wiederholen nicht“ war im Pult-Fenster nicht umgesetzt; **erledigt 21.10.2026** (Entscheidung „ja“, `grafik_pult_fenster.py` G4-009, fünf neue Tests, 104 von 104 in `grafik_pult_tests.py`, Gegenprobe macht zwei Tests rot; Einzelheiten §4.5). (2) **M-16:** M leuchtete nach dem Loslassen noch 0,25 s weiter, E nicht; **erledigt 21.10.2026** (Entscheidung „das Weiterleuchten entfernen, also der Beschreibung folgen“, `grafik_pult.py` G2-012, neuer Test, 105 von 105; §4.3). **Beide Abweichungen sind damit beseitigt; offen bleibt nur das Aussehen und Verhalten am echten Tk-Fenster.** Außerdem: der Test `test_klicktaste_04s` heißt noch nach der alten Zeit 0,4 s, prüft aber 0,25 s (nur der Name). Einmalig trat in einem Lauf eine Meldung über eine fehlende WAV-Datei in den Tonprüfungen auf, das Ergebnis blieb OK; in acht weiteren Läufen nicht wiederholt	Dok. 08 §4

10.4 Bekannte Einschränkungen und Befunde

Nr.	Punkt	Stand	Alte Fundstelle
OP-22	**Pult-Grafiken:** (a) richtige Grafik „BS gedrückt“ fehlt, die gelieferte zeigt ein „E“, im Fenster steht eine synthetische Fassung; (b) fünf Foto-Abgleiche unsicher (Ü, G, H, T, WM: Schätzwert Mittelpunkt + 13 px); (c) Positionen der Schlüsselschalter-Grafiken widersprechen sich um rund 20 px, beide liegen am selben Mittelpunkt; (d) gedrückte Tasten haben an manchen Stellen eine schwache Doppelkante. Blockieren nichts; Abgleich am laufenden Fenster	Hinweis	Dok. 01 §8 Nr. 2, 3, 5, 11, 12
OP-24	**Überholte Aussagen** in Dok. 01 §1.7 und §1.9, Dok. 02 §6 (§5.9, §7.5, §8.5) und Dok. 03 (§5 Nr. 9 und die Absätze „Stand 16.10.2026“ nennen „nicht eingebaut“; `tab_kurz.wav` und `tab_lang.wav` werden nicht mehr gespielt, Dok. 10 §6.3 B-1, B-2); sie fallen mit Stufe 6 weg	erledigt mit Stufe 6	Dok. 10 §6.3; ferner Dok. 00 §3 (Zeilen „ohne Echo“, 15.10.2026; seit 18.10.2026 überholt, Dok. 09 S3-04), Stufe-3-Checkpoint U-8
OP-25	**R300-Eingabe, Klickspur und Sollfolge:** `Tests/r300_eingabe_klickspur.txt` und `Tests/r300_eingabe_soll.txt` liegen nicht im Archiv (am 19.10.2026 gelöscht, weil die Regressionsschleife sie als Tests zählte). Der echte Lauf und die Auswertung starten erst, wenn der Erzeuger sie neu anlegt. **Entschieden 19.10.2026 (Projektinhaber: „lassen“): so lassen; bei Bedarf erzeugen und danach wieder löschen.** Die vier Testskripte bleiben unverändert	entschieden, kein Eingriff	Prüfverfahren §6.5, §10 B-4
OP-27	**Ausfall des Tonstroms im Betrieb am echten Gerät** (Rückkehrcode 16 mitten im Lauf, S4-04, Dok. 10 B-5): nur der Start ohne Abspielprogramm ist an einem echten Start geprüft; der Ausfall im Betrieb wurde in der Abnahme nicht eigens gemeldet	offen	Dok. 10 B-5
OP-28	**Zeiten am Gerät nicht gemessen:** Verzögerung des Tons höchstens 50 ms, Schreibwerkzeit 75 bis 90 ms, Lesetempo 130 Zeichen/s (Typ 303; Typ FS751: 750 Zeichen/s), Stanztempo etwa 50 Zeichen/s. Belegt ist nur die Hörprüfung 2 („ok“, 19.10.2026). Ebenso nicht am Gerät geprüft: die Aufrufe von `pw-cat` und `aplay` (am Rechner des Projektinhabers lief ein Abspielprogramm; welches, ist nicht festgehalten)	Hinweis	Dok. 10 §3a (Nicht geprüft), S4-03, S4-18, §6.9
OP-29	**Bedienablauf am Pult nie durchgespielt:** Dok. 01 §1.8 Nr. 8 bis 14 (Minimalprogramm laden, Leitadressen, `01.`, `10 aaaa`) wurde in Stufe 2 nur am KEP-Beispiel geprüft; ein Beleg für den ganzen Ablauf liegt mir nicht vor. Außerdem offen aus `AENDERUNG_E_unter_M` (0.03-015): der Hinweis „H statt R“ zu Handbuch 4.4 Nr. 2	offen	CP Stufe 2, AENDERUNG 0.03-015

Nr.	Punkt	Stand	Alte Fundstelle
OP-32	**Fensterlage (FL-001) und Fokus-Entprellung am echten Fenster:** Dok. 05 F-65 (b) bat um eine Prüfung unter X11 (Wandern der Position) und mit zwei Bildschirmen; eine Rückmeldung ist nicht belegt (**offen, nur am echten Fenster prüfbar**). **Entprellung des Fokus, Testlücke geschlossen 21.10.2026:** `grafik_sw_tests.py` hat drei neue Tests (`test_fokus_entprellung_bleibt_unter_0_4_s_im_sw_fenster`, `test_fokus_geht_nach_0_4_s_ohne_aufforderung_zum_pult`, `test_fokus_neue_aufforderung_setzt_die_uhr_zurueck`; Uhr und Aufforderung gesteuert, Ersatzmodul statt Tk); 81 von 81 grün. Gegenprobe: mit der Schwelle 0,2 s oder 0,8 s im Code werden zwei der drei Tests rot. Das echte Verhalten am Fenster (Abstände um 0,4 s, §5.7) bleibt davon unberührt und ist nicht geprüft	Hinweis, Rest offen	Dok. 05 F-43, F-65 (b)
OP-33	**Bedienablauf §3 Nr. 12 bis 14 gegen das BIS-Dokument:** Dok. 01 §1.8 stellte fest, dass die BIS-Dokumentation dem Paket nicht beilag und die Einzelheiten nicht geprüft sind. Sie liegt im Projekt (`Bausteininterpretiersystem BIS/markdown-dateien/C8205-Interpreter-BIS.md`) und wurde bei G6-014 **stichprobenartig** gelesen. Bestätigt: `2` K_2 `aaaa` mit $K_2 = 0$ Relativadresse, 1 Hauptleitadresse, 2 bis 9 die 2. bis 9. Leitadresse (BIS-2.2, Kennzeichentabelle); `1 0 aaaa …` ist ein Sofortbefehl ohne Adressenänderung (BIS-2.2.11: `10 3004 000.`). **Abweichungen, nicht geklärt:** (a) das BIS-Dokument schreibt die Eingaben mit Schlusspunkt und beim Sofortbefehl mit Befehlsteil (`1` K_2 $a_1a_2a_3a_4$ `BAZ .`); §3 Nr. 12 bis 14 und Dok. 01 zeigen die Kurzform `20 aaaa`, `10 aaaa` ohne Punkt; (b) Dok. 01 nennt `01.` einen „Befehl zur fortlaufenden Eingabe von Objektkodebefehlen“; nach BIS-2.2.3 ist `0` K_2 `…` ein **einzelnes Maschinenwort** (mit Addition der Leitadresse nach K_2), und die Programmstück-Eingabe des MP wird dort mit `025 eingabeprogst` aufgerufen. Ob `01.` am Pult das Einlesen des Bandes auslöst, habe ich nicht aus dem BIS-Dokument herleiten können. **Entschieden 19.10.2026 (Projektinhaber): Die Prüfung am Pult ist nicht möglich, weil dafür ein vollständiges BIS-Minimalprogramm nötig ist; das ist nicht Gegenstand dieser Anforderung.** §3 Nr. 12 bis 14 bleibt wie in Dok. 01 festgelegt (Kurzform), ungeprüft; die Abweichungen (a) und (b) bleiben als Hinweis stehen und werden bei der BIS-Abnahme geprüft	Hinweis, außerhalb des Umfangs	Dok. 01 §1.8 Nr. 12 bis 14 und „Lesart und Grenzen“ (b)

10.5 Als erledigt belegt

Punkte, die in den Anforderungsdokumenten offen standen und durch spätere Abnahmen erledigt sind. Sie bleiben hier mit Beleg, damit sie nicht als Verlust erscheinen.

Alter Punkt	Erledigt durch	Beleg
Töne nicht hörbar geprüft (Stufe 2); Mischung mit späteren Tönen	Mischer `tonmisch` (Stufe 4); Hörprüfung 2 unter Linux am 19.10.2026: ok	Dok. 10 B-6
Grundgeräusch: kurze Lücke beim Schleifenneustart	Grundgeräusch ist im Mischer eine Schleife ohne Neustart (G4-002)	Dok. 10
Tk-Fenster lief nur beim Projektinhaber (Linux)	Tk-Lauf vom Projektinhaber als „läuft“ gemeldet (B-3 bis B-5), Stufe 4 abgenommen	Dok. 10, G5-007
Vormerkung im SW-Fenster, Stau (F-35, F-36, S3-13, S3-14)	ersetzt durch den Puffer (19.10.2026, §5.5); Entscheidung 18.10.2026 07:44: keine wartenden Zeichen, Sperre bei K0 weg	Dok. 09 §6, Dok. 10 §15
Freigabe zum Neuprogrammieren (Dok. 04)	Stufen 1 bis 5 umgesetzt und abgenommen, Stufe 5 am 19.10.2026 ohne Windows	Dok. 11
Hörprüfung und Nachstellen der Konstanten (Dok. 03 §5 Nr. 22)	Hörprüfung 2 unter Linux: ok (B-6); Konstantenstand in §6.9	Dok. 10
OP-06 Leserpegel LL 1,3 (G4-012)	**Bestätigt 19.10.2026** („ja das ist alles ok“, auf die Frage, ob 1,3 als gewollter Wert gilt); Dok. 10 S4-19 (1,0) ist überholt	Antwort 19.10.2026

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Alter Punkt	Erledigt durch	Beleg
OP-07 Puffer im SW-Fenster: 8 Eingaben, Ausnahme für Umschaltzeichen und Zeichen, kein Verfall	**Bestätigt 19.10.2026** (gleiche Antwort); am echten Fenster weiter nicht erprobt, `--DEBUGLOG` zeigt es an Zeilen `SW Taste … voll`	Antwort 19.10.2026
OP-08 Hinweis bei gesperrter Taste (Selektor SW nicht gerastet)	**Bestätigt 19.10.2026: bleibt still, kein Hinweis** (gleiche Antwort; gelesen als „so bleibt es“)	Antwort 19.10.2026
OP-12 C-Modul vorübersetzt mitliefern oder bauen	Antwort „ja das ist alles ok“ auf die Frage mit vier Punkten; eine Wahl zu diesem Punkt wurde nicht genannt. **Gelesen als: der jetzige Stand gilt** (vorübersetzte `c8205_trommel.abi3.so` liegt bei, `c-module/python/setup.py` baut sie; unter Windows baut `bau_trommel_mingw.bat` die `c8205_trommel.pyd`, die seit 26.10.2026 auch vorübersetzt beiliegt). Bei Widerspruch bitte sagen	Antwort 19.10.2026
Testhandbuch nennt 201 Tests und die beiden r300-Dateien	`r300_eingabe_klickspur.txt` und `r300_eingabe_soll.txt` gelöscht (199 Tests); PDFs über `make_all_pdf.sh` neu erzeugt und bestätigt	Dok. 11, Dok. 04
Gravis ungeprüft (Dok. 09 §5b/§5d, Vorschlag „aus S3-09 streichen“)	Handlauf F-56: am echten Fenster kommt `grave` an und wird geliefert; in der Wiedergabe nicht erzeugbar, daher `[BEKANNT]`. Keine Streichung	Dok. 09 §5d (`gravis.log`), Prüfverfahren §10 B-5
`ziffern_2` bei Um. KB (Dok. 10 §16)	Kein Fehler des Interpreters; die Prüfung `grafik_sw_ende_zu_ende.py` hatte eine falsche Erwartung und wurde berichtigt (15 von 15). `ziffern_*` kennen kein Umschaltzeichen (U-12) und bleiben unverändert	Dok. 10 §16
Vormerkung verfällt nicht bei Programmende (Dok. 09 B-6, `verwerfen()` nie aufgerufen)	Die Vormerkung entfiel am 18.10.2026; seit 19.10.2026 gilt der Puffer, es gibt keine Regel „Programmende“ (§5.5)	Dok. 09 §6, §11
`KepUeberPult.test_kep` rot im Gesamtlauf (Dok. 09 §11 Nr. 6, Dok. 10 §12)	Gesamtlauf am 19.10.2026: 18 von 18 Blöcken bestanden (im Vordergrund; als Hintergrundauftrag scheitert der Strg+C-Test, siehe §8)	Dok. 11 A5-1
# im Modus TEST nicht eingebbar (Dok. 09 B-4)	Eigenschaft der K0-Datei (# beginnt dort einen Kommentar), keine Änderung; in GRAFIK wird # getippt	Dok. 09 §6
Flag BS wird beim Einrasten gesetzt, anders als im Dokument (CP Stufe 2, „Offen im TEST-Modus“)	Interpreter 0.03-014: `BS+` setzt nur die Schalterstellung `bs_schalter`, das Flag BS erst ein wirksamer bedingter Stopp (§3, Tabelle der Schalter)	`c8205_interp.py` Kopf, CP Stufe 2
Handbuch nennt 0.03-016 nicht (A-6; AENDERUNG 0.03-016 „Frage 3 offen“)	`Python_Interpreter.md`, `Bedienpult_mit_Schaltern.md` und beide README nennen 0.03-016; PDFs neu erzeugt (A5-7)	Dok. 11 A5-7
„ohne Echo“ in den Dokumenten berichtigen (U-8)	`Python_Interpreter.md` trägt den Hinweis (Echo nur im Modus TEST, in GRAFIK zeigt das SW-Fenster jedes angenommene Zeichen); Rest in Dok. 00 fällt mit Stufe 6 weg (OP-24)	Dok. 09 §10
E unter M: Niederdrücken und Loslassen von E trennen (AENDERUNG 0.03-015 „Noch offen“)	umgesetzt als `E+` und `E-` (M-35, §4)	Dok. 08, Beschreibung §4
HW-Check zeigt `---- ---- ---` (HW-003), `grafik_trommel.py` G2-007 gegen G2-010	HW-Check liest über das C-Modul, wenn `einblick()` fehlt; HW-Check in Stufe 2 abgenommen	HWCHECK_HW-003

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Alter Punkt	Erledigt durch	Beleg
Auswertung der R300-Eingabe an einem neuen Log nicht erprobt; altes Log passt nicht zur neuen Sollfolge (93 statt 92 Token)	Läufe F-55 bis F-58 mit neuer Sollfolge, Auswertung gelaufen	Dok. 09 §5b bis §5e
Dok. 05 Kurzliste F-01 bis F-19 und Durchgang O-1 bis O-13	im Durchgang 16.10.2026 beantwortet (Entwurf 24 bis 93); die Antworten stehen in Anhang A und in den Abschnitten 1 bis 9	Dok. 05 §0, §8
F-35 bis F-53 (Vormerkung, Stau, Testlücken F-40, F-43, F-44, Handliste F-46, Wiedergabe F-37)	Vormerkung und Stau entfielen am 18.10.2026 (Sperre, später Puffer); Wiedergabe und Tk-Test berichtigt (SWTK-003); Handliste neu gefasst; Tests für den Puffer vorhanden (`test_puffer_*`)	Dok. 05 §9, Dok. 09 §5d, §7
F-63 (Lautstärke `--TON`, Verzögerung 20 ms, Hörprüfung von Leser, Stanzer, TAB, WRZL, Eingabe)	Hörprüfung 2 unter Linux: ok (19.10.2026); `--TON` am 19.10.2026 entschieden (S4-19)	Dok. 10 §16
F-39 Fokus bei geöffnetem HW-Check-Fenster („zu bestätigen“)	als bekannte Eigenschaft akzeptiert (B-5, A-4, §8.4)	Dok. 09 §6
F-02 Standardannahmen Nr. 1 bis 9 (Netzablauf)	ersetzt oder bestätigt durch den Dialog zu F-14 (Entwurf 42, 43, 54) und die Entscheidungen 16.10.2026; Hochlauf und Klick im Hochlauf stehen in §3	Dok. 05 F-02
Dok. 02 §8 Nr. 1 bis 10 (gesperrte Schreibzugriffe, Beobachtungsvariable, Startlage, C-Modul mitliefern, Code 16, Startbelegung)	am 14. bis 16.10.2026 entschieden oder bestätigt („nicht widersprochen“); Nr. 6 (aktives Warten, Compiler, Tester) in OP-01 bis OP-04	Dok. 02 §8, Dok. 05 §8 (O-7, O-8)
Dok. 03 §5 Nr. 12 bis 24 (Tonstrom, Ausfall, C-Modul, Stanzer, Hörprüfung)	Nr. 15 und Nr. 21 (Stanzer: 40/60 ms) sind durch 50 ms und 5 s (S4-10) ersetzt; Nr. 22 und Nr. 24 bestätigt („die anderen Punkte sind ok“, 16.10.2026); Hörprüfung 2 ok	Dok. 03 §5 Nr. 24, Dok. 10 S4-10, B-6
Trommel-Testergebnisse (Dok. 02 §5), Lücke aus G6-009	übernommen in Prüfverfahren §7.4 (Sollwerte, nicht neu gemessen)	Prüfverfahren §7.4
Dok. 07 §5: Beispiellauf `kep_eingabe.txt` (BZ=2404, AC=22060635124) „nicht gegen einen Erwartungswert verglichen“	Der Pult-Test `KepUeberPult` endet mit denselben Werten (BZ=2404, AC=22060635124), Pult-Tests 65 von 65	Dok. 01 §1.10 Nr. 6
Dok. 07 §6: Freigabe für Stufe 2 steht aus; Dokumentation für GRAFIK nicht angepasst	Stufen 1 bis 3 am 18.10.2026 abgenommen (Dok. 00 §6, Entwurf 119, 120); Anpassung der Dokumentation ist Stufe 6 selbst	Dok. 00 §6
Dok. 01 §1.8 Nr. 17: `--sw-ausgabe` und Terminalausgabe in GRAFIK (Standardannahme)	Schalterliste F-13 am 16.10.2026 entschieden: `--sw-ausgabe` wirkungslos ohne Fehler (§2)	Dok. 05 F-13
Dok. 01 §1.9 Nr. 5 (a): Vierergruppen von links	Test des Projektinhabers 17.10.2026, Entwurf 99	Dok. 01 §1.9
Dok. 06 M-15 bis M-35	alle 21 Nummern kommen in §4 vor (Zählung G6-014); „Noch offen“ in Dok. 06 verweist nur auf Dok. 05 §8, das in dieser Triage gelesen ist	§4
Dok. 00 §6, Vermerke „Offen:“ der Entwürfe 35 bis 140	alle durch spätere Zeilen erledigt (F-12, F-14, F-15, F-16, F-17, F-18, Stufe 3 bis 5 abgenommen) oder oben als OP-34, OP-35 übernommen; PDFs und Testhandbuch: Dok. 11 A5-7	Dok. 00 §6

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Alter Punkt	Erledigt durch	Beleg
OP-23 Assemblerdokumentation sagt, die Oberfläche benutze `node.js`	**Entschieden 19.10.2026 (Projektinhaber): „aus der .odt ableiten“.** `C8205_Assembler_Dokumentation.md` neu abgeleitet (Version 1.00-008): `node.js`-Satz entfernt, `-WARN=SYS` und `spa` (neuer Abschnitt 3.3.10) übernommen, Abschnitte 3.3.11 und 3.3.12 umnummeriert, Verweise auf das Pragma auf 3.3.12 berichtigt (die .odt verweist selbst noch auf 3.3.11). **Rest:** das PDF `C8205-Assemblerdokumentation_A4.pdf` nennt `node.js` noch (nach Entscheidung 16.10.2026 unverändert); die .odt hat die falschen Verweise 3.3.11. **Rest gestrichen 21.10.2026 (Projektinhaber: Schritt „node.js im Assembler-PDF (OP-23)“ entfernt).** Gelesen als: Das PDF bleibt, wie es ist (Entscheidung 16.10.2026), und der Rest wird nicht mehr als offen geführt; die .odt wurde nicht angefasst. Bei Widerspruch bitte sagen	Dok. 01 §8 Nr. 1, Dok. 00 §6 (Entwurf 90 bis 92)
`AENDERUNG_E_unter_M_…0_03-015.md` und `AENDERUNG_K0_…0_03-016.md`: Inhalt nur dort; „Frage 3“ (Handbuch nennt 0.03-016 nicht)	„Noch offen: E+/E- trennen“ ist umgesetzt (M-35), „H statt R“ steht in OP-29, die drei trommelabhängigen Protokolle in OP-30	–
Dok. 01 §2 (Artefakte) und §3 (Element-Zuordnung): „Bitte bestätigen“ zum Netzschalter, Tabelle in §3.5 „abgeleitet, nicht bestätigt“	Netzschalter = Schlüsselschalter bestätigt (Entwurf 81); T-SW, SP, BS, LS, Selektor SW entschieden (Entwurf 50, 71, 96); `pult-fenster/` aus §2 gibt es als Ordner nicht mehr (Stufe 2 neu programmiert), §2 ist Verlauf	Dok. 00 §6 (Entwurf 50, 71, 81), §4
Dok. 00 §6, Entwurf 34 bis 114: F-11 (a) bis (f), F-12, F-13, O-1 bis O-13	alle beantwortet; F-11 (c), (d) ohne Anforderung erledigt (Entwurf 36, 37), (e), (f) „JA“ (Entwurf 38); Rest siehe OP-34, OP-37	Dok. 00 §6
U-3, U-4, U-6 (Stufe 3): `r300_eingabe` ohne Echo, Auswertung und Handliste an die neue Eingabe-Anzeige anpassen	umgesetzt (Checkpoint 40, 41); echte Läufe F-55 und F-58 ohne Befund, Tk-Test SWTK-004: 49 OK, 1 WARNUNG (Fokus, B-5), 0 FEHLER	Dok. 09 §5b bis §5e, §10
OP-10 E+ und BR/BZ (Anzeige nach `E+`)	**Entschieden 21.10.2026 (Projektinhaber): Es muss unterschieden werden.** `E+` **ohne M+ zeigt im gehaltenen Zustand den BZ in den Lampen der Reihe AR;** `E+` **mit gehaltenem M+ erhöht AR und zeigt es an.** Der zweite Fall war programmiert und ist unverändert. Der erste ist neu: `Rechner.ar_lampen()` (`c8205_grafik.py`, G4-005) liefert BZ bei `E+` ohne M (E gehalten, Automat vor Phase II), sonst `m.adr`; `grafik_pult.py` (G2-011) liest es für `lampen()` und die Lampenabtastung. Gemessen: Befehl `0100 211` aus Zelle 0000, nach `E+` zeigt die Reihe 0101 (BR-Register bleibt 0100 211). Tests: `test_lampenreihe_waehrend_E_gehalten`, `test_lampenreihe_E_unter_M_zeigt_erhoehtes_AR`, `test_lampenreihe_ohne_E_zeigt_BR_Adressteil`; Gegenprobe schlägt an. Der HW-Check (Zeile BR) zeigt das Register und bleibt unverändert	Antwort 21.10.2026, `testscripte/grafik_pult_tests.py`

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Alter Punkt	Erledigt durch	Beleg
OP-11 G bei gehaltenem E	**Entschieden 21.10.2026 (Projektinhaber): Nach dem Handbuchtext zu 4.1 Nr. 2 löscht G bei gehaltenem E nur den Überlauf; das ist allein richtig.** Umgesetzt in `c8205_grafik.py` (G4-005, Zweig `G` in `_taste`, `_e_taste`, `run_e_nieder(ab=…)`): FÜ := 0, sonst keine Wirkung; ein Halt wegen Überlauf von `E+` wird aufgehoben, das Loslassen setzt fort (Auslegung, OP-41). Ersetzt den bisherigen Test `test_G_bei_gehaltenem_E`; sechs Tests in `EGetrennt`, drei Gegenproben schlagen an. Der Befund vom 21.10.2026 (Abweichung bestätigt, Zustand „FÜ und E gehalten“ entsteht am Pult) steht in §3.8 Nr. 6	Antwort 21.10.2026, `testscripte/grafik_pult_tests.py`
OP-41 Fortsetzung nach G bei gehaltenem E im Halt wegen Überlauf	**Bestätigt 21.10.2026 (Projektinhaber):** Das Loslassen von E setzt Phase IV und I beim Prüfknoten fort (der Befehl im BR wird ausgeführt) und durchläuft dann Phase II und III; ohne G bleibt der Halt. Umsetzung wie in §3.8 Nr. 6, Test `test_G_bei_gehaltenem_E_im_Halt_wegen_Ueberlauf`	Antwort 21.10.2026, `testscripte/grafik_pult_tests.py`
OP-38 Versionskennung der GRAFIK-Module uneinheitlich	**Erledigt 21.10.2026:** Maßgeblich ist die Konstante `VERSION_*` im Code (Entscheidung des Projektinhabers); neun Kopfzeilen `Version:` angeglichen (u. a. `c8205_grafik.py` G2-004 → G4-004); `grafik_k0.py` und `grafik_trommel.py` tragen neu G5-001 (Entscheidung des Projektinhabers). Stand je Modul: Kopfzeile des Moduls	`C8205-Versionierung.md`, Abschnitt 2
OP-18 `sw_eingabe_leser_stanzer.test` liefert im Gesamtlauf den Rückkehrcode 8	**Ursache gefunden und Test entfernt, 21.10.2026 (Entscheidung des Projektinhabers):** Der Test war eine unfertige Ableitung aus `sw_eingabe_lage`. In der Quelle stand `beginn: 000 anf` und `anf: 030 04600` (ANWAHL Leser 1) vor `equ 02000`; Zelle 0000 sprang dadurch nach Zelle 0001, danach lief der Rechner über die leere Zelle 0002 zurück nach 0000 in eine Endlosschleife, und der Lauf endete mit „Knotenlimit erreicht“ (Rückkehrcode 8). Der Programmteil `anfang` bei 02000 wurde nie erreicht. Der Test hatte weder `--leser1` noch eine Prüfung und deckte dasselbe ab wie `k1_leser_stanzer` (dort steht `ANWAHL LSL1` richtig am Anfang von `anfang`). Entfernt wurden die Testdatei, die Assemblerdateien, das Protokoll, die Zeile in der Testübersicht und die Nennung im Testhandbuch-Skript; die Zahl der Tests in `Tests/` sinkt um 1 (das Testhandbuch zählt sie beim Erzeugen selbst). Ein neuer Gesamtlauf aller Tests ist nach der Entfernung nicht erfolgt; das Testhandbuch-PDF und die Testübersicht-PDF sind noch neu zu erzeugen (`make_all_pdf.sh`).	`C8205-Interpreter-GRAFIK_Pruefverfahren.md` (0.95-002), `CSV/test_tabelle.md` (0.95-002)

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Alter Punkt	Erledigt durch	Beleg
OP-34 Leserdateien: Sortierung bei gleicher Änderungszeit, Namensgleichheit im `LOCHSTREIFENKORB`	**Entschieden 21.10.2026 (Projektinhaber: „OP34 da mache A“): Variante (a), so lassen.** Sortiert wird nach Änderungszeit, bei Gleichstand nach Dateiname (`grafik_geraete.py`: `(st_mtime_ns, name)`; Test `test_gleiche_zeit_dann_dateiname`). Liegt im Korb schon eine Datei gleichen Namens, wird sie **ohne Meldung überschrieben**, der alte Inhalt geht verloren (unter Linux gemessen, jetzt durch `test_gleicher_name_im_korb_wird_ueberschrieben` festgehalten; 19 von 19 grün). Gelesen als Antwort auf den ganzen Punkt, also auch auf die Sortierung bei Gleichstand; bei Widerspruch bitte sagen. Unter Windows nicht geprüft (§10.1)	Antwort 21.10.2026, `testscripte/grafik_geraete_tests.py`
OP-37 Lampenmittelung („ruhiges Bild“): Fenster 40 ms, 60 % / 40 %, Hysterese	**Bestätigt 21.10.2026 (Projektinhaber: „OP-37 JA“).** Werte im Code (`grafik_pult.py`: 0,04 s, 0,60, 0,40) stimmen mit §3.5 überein; sechs Tests (`grafik_pult_tests.py`, Klasse `Beruhigung`) belegen Konstanten, Schwellen, Zwischenbereich und Fenster; 75 von 75 grün, fünf Gegenproben schlagen an. Die Handprüfung am laufenden Fenster (H-12, Prüfverfahren §7.5) ist beschrieben, aber nicht durchgeführt; die Konstanten bleiben dort einstellbar	Antwort 21.10.2026, `testscripte/grafik_pult_tests.py`, Prüfverfahren §7.5
OP-36 `CHECKPOINT_Stufe1.md` fehlt im Archiv (Dok. 07 §4: „Annahmen zu Leserdatei und LOCHSTREIFENKORB“)	**Als nicht beschaffbar geschlossen, 21.10.2026:** Die Datei liegt im Projekt nirgends vor und fällt mit dem Löschen der Checkpoint-Dateien (S6-11) ohnehin weg. Ihr Inhalt ist anderweitig belegt: die Regeln zu Leserdatei und `LOCHSTREIFENKORB` stehen im Entscheidungsregister (15.10.2026, 17.10.2026), die Sortierung und die Namensgleichheit sind mit OP-34 entschieden, das Verhalten steht in 19 Tests (`grafik_geraete_tests.py`). Nicht gegen die Datei geprüft; bei Widerspruch bitte sagen	Suche im Archiv 21.10.2026, Register, `grafik_geraete_tests.py`
OP-30 (b) `ton/SW_Ton_Beschreibung.md` nicht mit Dok. 03 Nr. 11 bis 24 abgeglichen	**Abgeglichen 21.10.2026, mit Einschränkung:** Dok. 03 liegt nicht im Archiv, verglichen wurde gegen §6, den Code, die WAV-Dateien und einen Lauf von `sw_ton.py` (Ergebnis in §6.10 Nr. 5). Teile A bis C stimmen mit dem Code überein; Teil D trug den Stand 15.10.2026 und hat einen Nachführungsvermerk, ferner Zusätze in §17, §19, §21.1, §21.2 (D-1, D-10 bis D-12) und im Änderungsverzeichnis. Rest von OP-30: (a) und (c) bleiben offen. Bei Widerspruch bitte sagen	`ton/SW_Ton_Beschreibung.md` (0.01-004), `grafik_*.py`, `ton/sw_ton.py`

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Alter Punkt	Erledigt durch	Beleg
OP-21 Handbücher auf Töne durchgesehen (S6-05)	**Durchgesehen 21.10.2026, keine Änderung nötig.** `erzeuge_bedienungsanleitung_pdf.py` (2763 Zeilen): Suche nach Ton, Töne, Klang, Geräusch, STUMM, `--TON`, hörbar, Lautstärke ohne Treffer; die Anleitung beschreibt den Modus TEST (ohne Töne), die Zeile `--MODUS` nennt GRAFIK als Pult- und SW-Fenster, das ist nicht falsch; `--STUMM` und `--TON` stehen dort nicht, sondern in §2. Ebenso ohne Treffer: `Bedienpult_mit_Schaltern`, `Magnettrommel`, `Durchführung_eines_Tests` und die übrigen PDF-Skripte. Mit Treffer und gelesen: `README.md` und `README.en.md` (stimmen mit §6: 80/100/400/360 ms, Stanzer, Code 16, `--TON`, Abspielprogramme), `Projektstruktur` (stimmt, nennt auch die ungenutzten Kopien in `toene_rechner_stanzer/`), `Lochstreifenleser` A7 (stimmt mit §6.3), `Python_Interpreter`, `Überblick`, `Projektüberblick`, `Einstieg` (nennen Töne nur als Merkmal von GRAFIK). **Grenze:** gelesen wurden die Treffer der Suche und die Optionstabelle der Anleitung, nicht jedes Handbuch von vorn bis hinten	Suche im Archiv 21.10.2026
OP-42 Klang für `WR` (Wagenrücklauf ohne Zeilenschaltung)	**Entschieden 21.10.2026 (Projektinhaber):** „für WR ohne Zeilenschaltung nur die erste Hälfte des WRZL-Tons nehmen, vor dem markanten Ton, und diesen neuen Ton 120 ms spielen.“ Umgesetzt: eigene Klangart `wr` (`grafik_schreibwerk.py` G4-002), Klang `ton/sw_toene/wr.wav` (die ersten 120 ms von `wrzl.wav`, 5 ms Ausblendung, erzeugt mit `ton/erzeuge_wr_klang.py`), Schreibwerkzeit 120 ms; Testrahmen `ton/sw_ton.py` kennt `<WR>` als vierte Klangart. Der Schnitt vor dem Schlag bei etwa 126 ms ist eine Auslegung und nicht gehört (§6.10 Nr. 6). Fünf neue Tests, `grafik_stufe4_tests.py` 85 von 85 (Python- und C-Mischer), zwei Gegenproben schlagen an	Antwort 21.10.2026, `testscripte/grafik_stufe4_tests.py`
OP-09 Eingabeklang beim ersten Zeichen	**Entschieden und umgesetzt 21.10.2026 (Projektinhaber), Variante (b): Klang beim Tastendruck.** Ursache der Verspätung war die erneute Aufforderung des Testprogramms nach dem Umschaltzeichen (1,70 s Schreibwerkzeit); der Klang kam bisher erst bei der Annahme. Jetzt klingt das Zeichen, sobald es im Puffer ist, die Anzeige kommt weiter bei der Annahme (§5.5, §6.2, §6.10 Nr. 3). `grafik_sw.py` G4-008, `grafik_sw_fenster.py` G4-010; sechs neue Tests in `grafik_sw_tests.py` (89 von 89), zwei im Fenster-Block von `grafik_stufe4_tests.py` (90 von 90 an einer Kopie mit angeglichenen Umlautnamen); Gegenproben schlagen an. **Nicht geprüft:** das Hören am Fenster (Prüfverfahren H-7); der Ein-Zeichen-Puffer, den der Projektinhaber nannte, gehört zum echten Gerät und ändert den Software-Puffer nicht (§6.10 Nr. 3)	Antwort 21.10.2026, `testscripte/grafik_sw_tests.py`

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Alter Punkt	Erledigt durch	Beleg
OP-14 Stufe 3, Standardannahmen: S3-06 (WR, TAB), S3-07 (keine Bewegung bei nicht darstellbaren Zeichen), S3-15 (Lage rechts unten)	**Bestätigt 21.10.2026 (Projektinhaber: „OP-14 ja“).** WR setzt die Schreibposition auf 0 (ohne Anzeige, ohne Zeilenvorschub, folgende Zeichen überdrucken); TAB füllt nur fehlende Spalten bis zum nächsten Stopp (alle 12, rechts bei 130); nicht darstellbare Zeichen werden ausgeführt, ohne Anzeige und ohne Bewegung; ohne gemerkte Stelle öffnet das Fenster rechts unten. Im Code und in Tests so festgehalten (90 von 90 in `grafik_sw_tests.py`). Befund: die Rechnerausgabe liefert von den nicht darstellbaren Zeichen nur Irrung Satz, ≈, ≋ und △ (§5.1). Die Bezeichnung ZV/ZL in den Kodetabellen ist mit OP-45 angeglichen (kein Einfluss auf das SW)	Antwort 21.10.2026, `testscripte/grafik_sw_tests.py`
OP-13 Stufe 1, Standardannahmen (1) bis (3)	**Bestätigt 21.10.2026 (Projektinhaber: „OP-13 alle ja“).** (1) Nach einem Halt bleibt der Rechner stehen; H und R heben das auf. (2) H im Lauf wirkt an der Befehlsgrenze (Knoten 2), im Eingabe-Halt sofort; R setzt danach den Befehl fort. (3) Kanal- und Schalterwechsel im Lauf wirken an der Befehlsgrenze bzw. im Eingabe-Halt (Wiederaufsetzen des Eingabebefehls). Belegt durch die Messungen der Probe `testscripte/grafik_op13_probe.py` (GRAFIK-Kern mit Rechner-Thread, leere Trommelkopie, Linux; **die Probe liegt im Archiv** `projekt_C8205.zip` **nicht vor**, obwohl `Projektstruktur` sie nennt; hier sind die festgehaltenen Messwerte wiedergegeben, nicht neu gemessen): Halt bleibt 0,6 s unverändert, H und R heben ihn auf, G allein nicht; H im Lauf wirkt nach 2 ms, im Eingabe-Halt sofort; K1 im Lauf wirkt nach 2 bis 4 ms, SW=1 im Eingabe-Halt führt den Befehl neu aus. **Nicht belegt (unverändert):** ob das dem Handbuch entspricht; Wartezeiten der echten Trommel bei längeren Befehlen; Kanalwechsel im Lauf zwischen zwei Eingabebefehlen; das Verhalten an der echten Anlage. Der Kopfkommentar von `c8205_grafik.py` vermerkt die drei Annahmen jetzt als bestätigt (G4-007, nur Kommentar, kein Verhalten geändert)	Antwort 21.10.2026, `testscripte/grafik_op13_probe.py`
OP-15 HW-Check, Standardannahmen (b), (c), (d), (f)	**Bestätigt 21.10.2026 (Projektinhaber: „OP-15 alle Ja“).** (b) Die Zeile `BR` zeigt AR-Teil und OR des Befehlsregisters (`m.adr`, `m.OR`), `<<AR>>` und `Spur Sektor` beziehen sich auf das Speicheradressregister `m.AR`; (c) die Anzeige steht auch vor dem Einschalten: BZ, BR, `Spur Sektor` und AC auf 0, `<<AR>>` zeigt den Inhalt der Zelle 0000 aus der Trommel (Trommel läuft ab Programmstart); (d) das Fenster bleibt aufgeblendet und lässt sich nicht schließen; (f) Spur = AR ÷ 32, Sektor = AR mod 32, nicht die umlaufende Trommellage. Code und Tests so festgehalten (`grafik_pult_tests.py`, Klasse `HwCheck`, 92 von 92; neuer Test `test_op15_anzeige_vor_dem_einschalten`). **Nicht geprüft:** Aussehen und Verhalten am echten Fenster in dieser Sitzung (§8.5)	Antwort 21.10.2026, `testscripte/grafik_pult_tests.py`

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Alter Punkt	Erledigt durch	Beleg
OP-31 Rücktaste, Annahmen (1) bis (5)	**Bestätigt 21.10.2026 (Projektinhaber: „OP-31 alles JA“).** (1) Sie löscht das Zeichen links der Schreibposition, auch ein Zeichen der Rechnerausgabe; (2) am Zeilenanfang ohne Wirkung; (3) der Rechner erfährt nichts; (4) kein Ton; (5) sie wirkt auch ohne aktiven `011`. Code und Tests so festgehalten (`grafik_sw_tests.py`, Klasse `Ruecktaste`, 91 von 91; neuer Test `test_op31_kein_klang_bei_ruecktaste`, Gegenprobe mit Klang in der Rücktaste schlägt an). Kein Programmcode geändert. **Nicht geprüft:** Verhalten am echten Fenster	Antwort 21.10.2026, `testscripte/grafik_sw_tests.py`
OP-35 (a) Kanal per Befehl, Rastung der Kanaltaste	**Bestätigt 21.10.2026 (Projektinhaber: „OP-35 nein, das ist so richtig“, mit Erläuterung, Wortlaut in §4.1).** Wählt das Programm einen Kanal an, bleibt die Kanaltaste in ihrer Rastung; die Lampen K1 bis K3 bzw. T und SW folgen dem Programm; eine andere Kanaltaste wirkt beim Drücken sofort. **Gelesen als:** Die Antwort sprach von den Tasten; dass die Lampen dem Programm folgen, war Teil der Frage und blieb unwidersprochen (bei Widerspruch bitte sagen). Programmcode nicht geändert; neuer Test `test_op35a_kanal_per_befehl_aendert_die_rastung_nicht` (`grafik_pult_tests.py`, 106 von 106), Gegenprobe (Rastung folgt dem Programm) macht ihn und `test_kanaltasten_genau_eine_rastet` rot. **Nicht geprüft:** Verhalten am echten Fenster	Antwort 21.10.2026, `testscripte/grafik_pult_tests.py`
OP-35 (b) R innerhalb der 0,25 s nach einem Klick auf E oder M	**Bestätigt 21.10.2026 (Projektinhaber: „ja, wirkt sofort“).** Die 0,25 s sind nur die Darstellung eines kurzen Tastendrucks; R wirkt sofort. Bei gehaltenem E oder M (Umschalt+Klick) bleibt R wirkungslos (M-21). Programmcode nicht geändert; `test_r_wirkt_sofort_nach_kurzem_klick_auf_e` bestand, neu `test_op35b_r_wirkt_sofort_nach_kurzem_klick_auf_m` (`grafik_pult_tests.py`, 107 von 107). Gegenprobe (R nach kurzem Klick gesperrt) macht sechs Tests rot, darunter beide. **Nicht geprüft:** Verhalten am echten Fenster	Antwort 21.10.2026, `testscripte/grafik_pult_tests.py`
OP-17 Klicks im Dauerlauf R	**Bestätigt 21.10.2026 (Projektinhaber: „OP-17 ja“, gelesen als: am echten Fenster erprobt und in Ordnung).** Das Pult meldet jeden Klick sofort, der Interpreter fragt an jeder Befehlsgrenze ohne Warten nach; die Wirkung folgt spätestens nach 50 ms. Der frühere Stand „vorläufig entschieden, zu erproben“ (16.10.2026, Dok. 01 §7 Nr. 14) ist damit erledigt. **Einschränkung, nicht neu geprüft:** die 50 ms sind eine Entwurfsangabe, im Code steht keine Konstante dafür; sie gelten zwischen zwei Befehlen. Läuft gerade die Schreibwerkzeit eines Zeichens, wirkt H erst danach, höchstens nach 400 ms (TAB), siehe OP-39. Programmcode und Tests nicht geändert. Eine Messung durch den Entwickler gibt es dazu nicht	Antwort 21.10.2026, OP-39

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Alter Punkt	Erledigt durch	Beleg
OP-30 Abgleiche, die nie stattfanden	**Erledigt 21.10.2026.** (a) Die drei trommelabhängigen Protokolle (`acbz-003_AC_Ue_E_Transport`, `bsp_5_6_vordere_bits_als_adresse`, `pj2-bis_minimalprogramm`) liefern im TEST- und im GRAFIK-Lauf dasselbe, dreimal verglichen mit `testscripte/grafik_gleichwertigkeit.py` (Register und Flags, Trommelinhalt Byte für Byte, Schreibwerk-Ausgabe, Stanzerdatei, Prüfergebnis; die gemeinsame Trommeldatei blieb unverändert). **Grenze:** kein alter Interpreter vor 0.03-015 zum Vergleich, also Gleichwertigkeit der beiden Zweige, nicht der Änderung gegen den alten Stand; andere Trommelstände nicht probiert; Lauf an einer Kopie mit angeglichenen Umlautnamen. (b) `ton/SW_Ton_Beschreibung.md` abgeglichen (siehe OP-30 (b) unten und §6.10 Nr. 5). (c) `r300_eingabe_trommel_nach_lauf.txt` (Echtlauf F-54, 73 KB) als nicht beschaffbar geschlossen, Projektinhaber 21.10.2026; ein neuer Echtlauf ist bei Bedarf erzeugbar (OP-25)	Antwort 21.10.2026, `testscripte/grafik_gleichwertigkeit.py`
OP-43 Stand der Dateien im Archiv `projekt_C8205.zip`	**Erledigt 21.10.2026 (Entscheidung des Projektinhabers: „mach was du denkst“, Empfehlung (a)):** `c8205_grafik.py` (G4-005) liegt jetzt im Wurzelordner und `grafik_pult.py` (G2-011) in `grafik-interpreter-module/`; die losen Kopien in `testscripte/` sind entfernt. Die ältere Pult-Testdatei im Modulordner (70 Tests) ist entfernt; ihr Test `test_haltzeit_einzelschritt` (HZ-004) steht jetzt in `testscripte/grafik_pult_tests.py` (83 Tests, vorher 82). Prüfung an einer Kopie mit angeglichenen Umlaut-Dateinamen: `grafik_pult_tests.py` 83 von 83 grün; Sammellauf `grafik_alle_tests.sh --schnell` 16 von 16 Blöcken grün (vorher 15 von 16); die beiden Gleichwertigkeitsläufe wurden nicht gestartet. Hinweis: Im Archiv tragen Dateien mit Umlauten im Namen (z. B. in `markdown-dateien/`) Namen wie `#U00e4` statt `ä`; die Tests, die diese Dokumente lesen, laufen erst nach Angleichen der Namen	Antwort 21.10.2026, Versionierung 0.95-014

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Alter Punkt	Erledigt durch	Beleg
OP-40 Schreibwerkzeit und Laufzeit der GRAFIK-Tests	**Untersucht 21.10.2026 (Projektinhaber: „mach weiter“): Im Sammellauf wirkt die Schreibwerkzeit nicht.** Die Gleichwertigkeitsläufe und die Testblöcke benutzen den GRAFIK-Kern `Rechner` mit der `SwSenke` ohne Schreibwerk (`schreibwerk=None`, `c8205_grafik.py`); das Schreibwerk mit Schreibwerkzeit (S4-17, gilt auch bei `--STUMM`) wird nur in `main()` des Treibers angeschlossen. Messung mit dem neuen Skript `testscripte/grafik_sw_zeit_messung.py` (Mittelwerte 80 ms je Zeichen, TAB 400 ms, WRZL 360 ms, WR 120 ms): von den 199 Tests in `Tests/` schreiben 33 auf das Schreibwerk, zusammen 1233 Zeichen. Liefen alle über den echten Treiber, kämen **rund 110 s Schreibwerkzeit** hinzu; die GRAFIK-Läufe der Gleichwertigkeit dauern ohne sie rund 91 s (Summe der Kindprozesse, Messung dieses Tages). Die längsten Tests sind `st-004_STARTE_1000_druck_interrupt`, `st-005_STARTE_1000_druck_mit_druckroutine` und `namen_raten` mit je rund 11 s, `ziffern_1` mit rund 8 s. Folge: Die Gleichwertigkeitsläufe bleiben ohne Schreibwerk (sonst etwa doppelte Laufzeit, ohne dass die Gleichheit der Ergebnisse besser belegt würde; die Zeiten sind ausgenommen, §1). Nur echte Treiberläufe wie `r300_eingabe_grafik.sh` warten die Schreibwerkzeit ab. **Nicht untersucht:** gemessen wurde die Rechnung aus der Ausgabe, nicht ein echter Lauf mit Schreibwerk; die Zufallsstreuung (70 bis 90 ms) ist im Mittel angesetzt	Messung 21.10.2026 (`/tmp`-Bericht, Zahlen hier festgehalten), `testscripte/grafik_sw_zeit_messung.py` **Nachtrag 22.10.2026 (G4-003):** Das Schreibwerk läuft jetzt asynchron zur Befehlsschleife; die berechnete Schreibwerkzeit ist deshalb nur noch eine obere Grenze der zusätzlichen Laufzeit (sie überlappt mit der Rechenzeit). Die Messung selbst wurde nicht wiederholt.
OP-39 Rechner-Thread in der Schreibwerkzeit	**Erprobt 21.10.2026 (Projektinhaber: „mach weiter“): H und Abbruch werden durch die Schreibwerkzeit nicht blockiert.** (1) Ein Ausgabebefehl schreibt genau ein Zeichen oder einen `<Name>` (`sw_schreibe(m, sw_zeichen(m, z))` in `c8205_interp.py`), die Schreibwerkzeit je Befehl beträgt also höchstens 400 ms (TAB); die frühere Angabe „bis 0,4 s“ stimmt. (2) Die Pult-Taste H wird im Thread-Modus nur vorgemerkt (`pult_taste` → `_ereignisse`, `_h_angefordert`) und kehrt sofort zurück; sie wirkt wie bisher an der nächsten Befehlsgrenze (OP-13 (2)), im ungünstigsten Fall also nach der laufenden Schreibwerkzeit von höchstens 400 ms. (3) Der Abbruch (`abbrechen()`, X, Strg+C) wartet nicht auf den Rechner-Thread: dieser ist ein Daemon-Thread und wird nirgends mit `join` abgewartet. Neu in `grafik_stufe4_tests.py` (Klasse `SchreibwerkzeitUndH`, vier Tests, Echtzeit einmal 0,4 s): `pult_taste('H')` und `abbrechen()` kehren während einer 400-ms-Schreibwerkzeit in unter 50 ms zurück; Gegenprobe (Sperre in der Schreibwerkzeit gehalten) gibt 350 ms und lässt den Test rot werden. Prüfungen 89 von 89, Python- und C-Mischer. **Nicht geprüft:** am echten Fenster mit Hörprobe; ob H nach höchstens 400 ms im Dauerlauf gewünscht ist, ist keine offene Frage (Standardannahme OP-13 (2))	Messung und Tests 21.10.2026, `testscripte/grafik_stufe4_tests.py`

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Alter Punkt	Erledigt durch	Beleg
OP-20 E-Schritt im Eingabe-Halt, abgebrochen durch H oder Kanalwechsel	**Behoben 21.10.2026 (Projektinhaber: „ok, mach weiter“, gelesen als Variante (a), meine Empfehlung).** Befund vor der Korrektur: Die Taste E blieb nach dem Abbruch tot (`e_gedrueckt` wurde nicht zurückgesetzt, `HUnterbrechung` verlässt `run_e_nieder` über `_e_taste`), und `K0Warteschlange.naechstes` ließ `_wartet_auf` stehen, sodass `erwartet('T')` wahr blieb und ein Zeichen angenommen wurde, das den Rechner nicht erreichte (auch bei H im R-Lauf). Umsetzung: (1) `c8205_grafik.py` (G4-006, `leerlauf`): bei `HUnterbrechung` und `Wiederaufsetzen` einer E-Taste (`E`, `E+`, `E-`) werden `e_gedrueckt` und `e_nieder_unter_m` zurückgesetzt, E gilt nach dem Abbruch als losgelassen; (2) `grafik_k0.py` (G5-002, `naechstes`): wird das Warten durch eine Ausnahme verlassen (H, Kanalwechsel, Abbruch), wird `_wartet_auf` gelöscht. Wirkung (Probe `testscripte/grafik_e_im_eingabe_halt_probe.py`, KEP-Programm, siebter E-Schritt): nach H ist `e_gedrueckt` falsch und K0 wartet nicht mehr; ein erneutes E führt den Befehl wieder in den Eingabe-Halt, ein Zeichen wird übernommen (AC := 4000…, BZ 0032); G und dann E wirkt wie nach einem Generallöschen; R setzt den Befehl fort und nimmt das Zeichen (AC := …1); H im R-Lauf lässt K0 nicht wartend. Tests: `testscripte/grafik_pult_tests.py`, neue Klassen `K0AbbruchBeimWarten` (drei Tests) und `EImEingabeHaltAbgebrochen` (fünf Tests); 91 von 91 grün (vorher 83). Gegenproben: mit dem alten Kern allein werden zwei, mit dem alten `grafik_k0.py` allein sechs der acht neuen Tests rot. Sammellauf `grafik_alle_tests.sh --schnell`: 16 von 16 Blöcken grün; die beiden Gleichwertigkeitsläufe wurden nicht gestartet (kein Code des TEST-Zweigs berührt). **Nicht geprüft:** der Abbruch durch Kanalwechsel (`Wiederaufsetzen`) nimmt im Code denselben Weg wie H (gleiche Zeilen in `leerlauf`), ist aber nicht eigens nachgestellt; das Verhalten an der echten Anlage ist nicht belegt; der halbe Befehl bleibt nach dem Abbruch liegen, ein erneutes E führt ihn wieder in den Eingabe-Halt (gemessen); ob das dem Handbuch entspricht, ist nicht geprüft	Antwort 21.10.2026, `testscripte/grafik_pult_tests.py`

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Alter Punkt	Erledigt durch	Beleg
OP-19 Trommeldatei wird von Testläufen verändert	**Umgesetzt 21.10.2026 (Projektinhaber: „mache b)“, Variante (b), Wächter im Sammellauf).** Neu: `testscripte/grafik_trommel_waechter.py` (`merken` und `pruefen`; SHA-256 von `Trommel/trommel.txt` und `Trommel/trommel.basp`; eine fehlende Datei zählt als eigener Zustand; nur der Inhalt zählt, nicht die Änderungszeit; Rückgabewert 0 unverändert, 1 verändert, 2 Stand oder Aufruf unlesbar; die Datei wird nicht zurückgesetzt). `grafik_alle_tests.sh` merkt den Stand vor dem ersten Block und vergleicht ihn im letzten Block (Block 20; Block 19 ist der Selbsttest `testscripte/grafik_trommel_waechter_tests.py` mit elf Prüfungen und Gegenproben in einem Temp-Ordner); beides läuft auch bei `--schnell` (jetzt 18 Blöcke, voll 20). Lässt sich der Stand vor dem Lauf nicht merken, zählt das als Fehler. Belegt: Sammellauf `--schnell` 18 von 18, Prüfsummen von `trommel.txt` und `trommel.basp` danach gleich; **Gegenprobe am Skript selbst** (Kopie des Skripts mit einem Block, der an `trommel.txt` anhängt): Wächter meldet FEHLER, Rückgabewert 1; ohne diesen Block 0. Der volle Lauf mit den Gleichwertigkeitsläufen wurde nicht gestartet. Das Verhalten der Läufe außerhalb des Sammellaufs ist unverändert: Läufe im Modus TEST ohne `--trommel` verändern `Trommel/trommel.txt` (gewollt, §1), für reproduzierbare Läufe vorher sichern oder `--trommel EIGENE_KOPIE` (Prüfverfahren B-2). Nicht abgedeckt: GRAFIK-Läufe mit Fenster, Windows (der Wächter ist Python und läuft dort voraussichtlich, der Sammellauf selbst ist ein Bash-Skript, OP-26)	Antwort 21.10.2026, `testscripte/grafik_trommel_waechter*.py`, Prüfverfahren §3.2

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Alter Punkt	Erledigt durch	Beleg
OP-44 PDF-Erzeugung der `GRAFIK_Beschreibung` bricht mit `LayoutError` ab (Lauf von `make_all_pdf.sh`, 21.10.2026, Tabelle auf Seite 43: „Flowable … Table 3 rows x 3 cols … too large on page 43“)	**Behoben 21.10.2026 (Projektinhaber: „los gehts“, erster Schritt der Abarbeitung).** Ursache: `table_block` in `md2pdf.py` bindet die **letzten beiden Tabellenzeilen** mit `NOSPLIT` aneinander (Regel „letzte Zeile nie allein auf der Folgeseite“, PDF.md 3a.6). Die letzten beiden Zeilen von §10.5 sind OP-20 und OP-19; beide sind inzwischen so lang, dass sie samt wiederholter Kopfzeile 903 pt brauchen, die Satzfläche hat 736 pt. Reportlab konnte den Block nicht trennen und brach ab. Der Fehler trat erst auf, nachdem OP-19 und OP-20 am 21.10.2026 ihre langen Belegtexte bekamen. **Änderung:** in `table_block` gilt `NOSPLIT` nur noch, wenn Kopfzeile und die beiden letzten Zeilen zusammen höchstens 80 % der Satzflächenhöhe (`H - 2*M - HEAD - FOOT`) brauchen; sonst entfällt die Regel für diese eine Tabelle, alle anderen Tabellen setzen sich unverändert. Geändert in `PDF/pdf-layout-scripte/md2pdf.py` und in den eingebetteten Kopien in `erzeuge_grafik_beschreibung_pdf.py` und `erzeuge_grafik_pruefverfahren_pdf.py`. **Wirkung:** OP-20 steht jetzt allein auf Seite 44, OP-19 auf Seite 45 unter wiederholter Kopfzeile. **Belegt:** Lauf von `erzeuge_grafik_beschreibung_pdf.py` an der Quelle 0.95-042 ohne Fehler (68 Seiten; mit dieser Fassung 0.95-043 69 Seiten), ebenso `erzeuge_grafik_pruefverfahren_pdf.py` (10 Abschnitte); Seite 44 angesehen. **Nicht geprüft:** der Gesamtlauf `make_all_pdf.sh` (die übrigen Skripte brauchen Werkzeuge, die hier fehlen, z. B. Graphviz); die **vierzehn anderen** `erzeuge_*`-Skripte und `build_testhandbuch.py` tragen denselben Tabellencode weiter in der alten Fassung, sie liefen im Lauf vom 21.10.2026 fehlerfrei und sind unverändert geblieben. Wird dort einmal eine Tabelle mit sehr langen Endzeilen gebraucht, ist dieselbe Änderung nachzuziehen	Lauf `make_all_pdf.sh` 21.10.2026, `PDF/pdf-layout-scripte/md2pdf.py`

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Alter Punkt	Erledigt durch	Beleg
OP-45 Bezeichnung ZV/ZL in den Kodetabellen	**Angeglichen 21.10.2026 (Projektinhaber: „ja, mache deinen Vorschlag“).** Befund: Dasselbe Zeichen (Zeilenvorschub ohne Wagenrücklauf, Eingabeadresse 72, ohne Sprung-Umstellung) heißt in Tabelle 1 „ZV“ (Zeile 52, vom Projektinhaber am 11.10.2026 ergänzt) und in Tabelle 2 „ZL“ (Zeile 59, aus der Vorlage unverändert übernommen; „ZL“ = Zeilenschaltung, wie in „WRZL“). Die Lesehilfe zu Tabelle 2 sagte „ZL = ZV“, Tabelle 1 kannte „ZL“ nicht. **Änderung (nur Text, in `C8205-Interpreter-Kodetabellen.md`, 0.95-001 → 0.95-002):** (1) die Abkürzungszeile unter Tabelle 1 nennt ZL als Namen der Vorlage; (2) der Absatz „ZV und BM“ in 1.4 erklärt beide Namen und sagt, welcher wo gilt (ZV in Tabelle 1 und im Programm, ZL nur in Tabelle 2); (3) die Lesehilfe zu Tabelle 2 verweist auf Zeile 52 und 1.4. **Keine Tabellenzelle geändert** und keine der beiden Bezeichnungen umbenannt: Tabelle 2 soll nach ihrem eigenen Hinweis der Vorlage folgen, und der Interpreter (`c8205_interp.py`) und die Testdateien benutzen nur „ZV“; Code und Tests lesen die Namen nicht aus dem Dokument (Suche im Archiv). **Belegt:** `erzeuge_kodetabellen_pdf.py` läuft fehlerfrei (10 Seiten), die drei Stellen stehen im PDF. **Nicht geprüft:** ob du lieber einen der beiden Namen überall vereinheitlichen willst; das wäre eine Ein-Wort-Änderung in einer Tabelle, sagst du es, mache ich es	Suche im Archiv 21.10.2026, `C8205-Interpreter-Kodetabellen.md`
OP-46 Code im Archiv hinter der Dokumentation (G2-012, G4-009 fehlten)	**Behoben 21.10.2026 (Projektinhaber: „los gehts“, Schritt 1).** Befund: `grafik_pult_fenster.py` trug G4-008 und `grafik_pult.py` G2-011; die in OP-16 als erledigt geführten Änderungen M-28 (Sperre gehaltener Computertasten) und M-16 (M leuchtet nach dem Loslassen nicht weiter) fehlten im Code des Archivs, vier Tests der Datei `grafik_pult_tests.py` schlugen an. Nachgeholt: `grafik_pult.py` G2-012, `grafik_pult_fenster.py` G4-009 (neu nach §4.5 und den Tests geschrieben, das Original lag nicht vor). **Belegt:** 107 von 107 grün (Kopie mit angeglichenen Umlautnamen). **Nicht geprüft:** Verhalten am echten Tk-Fenster	Archivprüfung 21.10.2026, `testscripte/grafik_pult_tests.py`, Versionierung 0.95-045

Alter Punkt	Erledigt durch	Beleg
OP-47 Pult-Tasten über die Buchstabentasten des Rechners (H, R, E, G, M, T, S, Umschalt+E, Umschalt+M)	**Umgesetzt 21.10.2026 (Projektinhaber: Anforderung „alternativ auch die Buchstabentasten“, Sperren sind zu beachten).** Beschreibung in §4.5. `grafik_pult.py` G2-013, `grafik_pult_fenster.py` G4-010, Tests `Op47Buchstaben` (17), 124 von 124 grün, Sammellauf 18 von 18. Vorrang der SW-Eingabe bei angewähltem Kanal SW bleibt (OP-48). **Nicht geprüft:** Verhalten am echten Fenster	Anforderung 21.10.2026, `testscripte/grafik_pult_tests.py`, Versionierung 0.95-046
OP-48 Buchstabentasten bei angewähltem Kanal SW	**Behoben 21.10.2026 (Rückmeldung des Projektinhabers: „G und S (SW) funktionieren, R funktioniert nicht, da klickt es sogar wie Eingabe in SW, obwohl der Fokus noch beim Pult liegt“).** Ursache: `taste_vom_pult` (G4-008) nahm bei Kanal SW jede Taste am Pult als SW-Eingabe, auch `R`, `H` und `T`. Gelesen als: Variante (b) mit Schutz für die SW-Eingabe (Beschreibung §4.5): Pult-Taste hat Vorrang, wenn sie klickbar wäre und das SW keine Eingabe erwartet. `grafik_pult.py` G2-014 (`taste_klickbar`), `grafik_pult_fenster.py` G4-011 (`_pult_vorrang`). 133 von 133 grün, Sammellauf 18 von 18. **Nicht geprüft:** Verhalten am echten Fenster. Bei Widerspruch (z. B. Vorrang auch bei erwarteter Eingabe) bitte sagen	Rückmeldung 21.10.2026, `testscripte/grafik_pult_tests.py`

10.6 Arbeitsvermerk Stufe 6 (entfällt nach Abschluss)

Ausnahme für S6-12 (entschieden 19.10.2026): `testscripte/r300_eingabe_log_auswerten.py` behält die Verweise „Dok. 09 5b“ (Kommentar und Berichtstext). Ein Berichtstext ändert die Ausgabe des Skripts; Stufe 6 ändert keinen Code. Die Suche unter A6-3 nach „Dok. 09“ meldet dort zwei Treffer, die gewollt sind. Bedeutung: Dok. 09 §5b = Prüfverfahren §6 (R300-Eingabe).

Die Triage nach S6-04 ist mit G6-014 **durchgesehen**. **Vollständig gelesen** sind Dok. 05, 06, 07 §4 bis §6, 09, 10, 11, Dok. 01 §1.8 bis §1.10, Dok. 02 §5 und §8, Dok. 03 §5, `CHECKPOINT_Stufe2.md`, `CHECKPOINT_Stufe3.md`, `CHECKPOINT_G5-001.md`, `AENDERUNG_*` und `HWCHECK_HW-003.md`; ferner durchgesehen Dok. 00 §4, Dok. 01 §4 bis §10, Dok. 04 §2 und §5, Dok. 08 §4 und die Checkpoints G5-002 bis G5-010. **Daraus kamen OP-25 bis OP-36 und die Erledigungen in §10.5.**

Grenzen (ehrlich): (1) **Dok. 00 §6** (alle Entwürfe 1 bis 140) und **Dok. 01 §2, §3** sind seit G6-015 vollständig gelesen; daraus kamen OP-23 (erweitert), OP-34 (erweitert) und OP-37. (2) **Dok. 06**: dass alle M-Nummern in §4 vorkommen, ist **gezählt**, nicht Satz für Satz verglichen; die Satzebene gehört zu A6-1 (Stichprobe des Projektinhabers). (3) Das BIS-Dokument wurde nur an den Stellen gelesen, die OP-33 nennt; die Prüfung von `01.` und `10 aaaa` am Pult ist entschieden außerhalb des Umfangs (OP-33). (4) Dok. 01 §4 bis §10 wurden schon früher durchgesehen (nicht in G6-015 wiederholt). (5) Versionierung (S6-03) bei G6-017 geprüft: der Interpreter hat die Builds 0.03-013 bis 0.03-017 lückenlos; für die `.md`-Dateien ist „je Änderungsschritt +1“ nicht prüfbar (keine Historie im Archiv); `projekt_C8205_1.zip` ist nicht untersucht. **„Keine weiteren offenen Punkte“ gilt für die gelesenen Quellen.** Aus Sicht der Triage ist das Löschen (S6-11) freigegeben, **erst nach** Versionierung (S6-03), Verweisen (S6-07) und Sicherung (S6-11).

10.7 Kennungen

Alte Kennung	Gegenstand	Abschnitt
B-2 (Dok. 10), S4-15, S5-12, A5-8, O4-18	Windows	10.1
G4-007, G4-009, G4-012, O4-17	Puffer, Leserpegel	10.5
F-15 (a), Dok. 01 §7 Nr. 14	Klicks im Dauerlauf	10.5 (OP-17, bestätigt)
S3-06, S3-07, S3-15	Standardannahmen Stufe 3	10.5 (OP-14, bestätigt)
Stufe 1, Annahmen (1) bis (3); Dok. 08 §4	Halt, H im Lauf, Kanal- und Schalterwechsel	10.5 (OP-13, bestätigt)
G5-009, Dok. 11 „Beim Abnehmen festgestellt“	Testbefunde	10.4, 10.5 (OP-18, erledigt)
Dok. 10 B-5, S4-03, S4-18	Tonstrom im Betrieb, Zeiten am Gerät	10.4 (OP-27, OP-28)
Prüfverfahren §9 (W-2, W-5)	Windows: Bauweg, Sammellauf	10.1 (OP-26)
CP Stufe 2, AENDERUNG 0.03-015/-016, HWCHECK_HW-003	Bedienablauf, nie durchgeführte Abgleiche	10.4 (OP-29), 10.5 (OP-30, erledigt)
Dok. 02 §5.3, §8; Dok. 03 §5	Windows-Zusätze zu OP-04, Erledigungen	10.1 (OP-04), 10.5
Dok. 05 F-64, F-65, F-43	Rücktaste-Annahmen, Fensterlage, Fokus-Entprellung	10.5 (OP-31, bestätigt), 10.4 (OP-32)
Dok. 01 §1.8 Nr. 12 bis 14, BIS-Dokument	Schreibweise und Bedeutung von `20 aaaa`, `01.`, `10 aaaa`	10.4 (OP-33, außerhalb des Umfangs)
Dok. 00 §6 (Entwurf 35), Dok. 06 M-21, M-34	Leser-Gleichstand, Pult-Auslegungen	10.5 (OP-35 (a) und (b), OP-34, erledigt)
Dok. 07 §4	`CHECKPOINT_Stufe1.md` fehlt	10.5 (OP-36, geschlossen)
Dok. 00 §6 (Entwurf 84), Dok. 01 §6.2	Lampenmittelung	10.5 (OP-37, bestätigt)
Versionierung, Abschnitt 2	Versionskennung der GRAFIK-Module	10.5 (OP-38, erledigt)
CP Stufe 3 (Schreibwerkzeit), Dok. 10 §6.3	Rechner-Thread in der Schreibwerkzeit, Testlaufzeit	10.5 (OP-39, erprobt; OP-40, untersucht)
`SW_Ton_Beschreibung.md` §14.1, `grafik_schreibwerk.py`	Klang für WR	6.2, 6.10 Nr. 6, 10.5 (OP-42, entschieden)
Versionierung 0.95-010 und 0.95-014, `projekt_C8205.zip`	Stand der Dateien im Archiv	10.5 (OP-43, erledigt)
`make_all_pdf.sh` (Lauf 21.10.2026), `md2pdf.py`	`LayoutError` bei der Tabelle in §10.5	10.5 (OP-44, behoben)
`C8205-Interpreter-Kodetabellen.md` 1.4, 2.2	Bezeichnung ZV/ZL; Rest von OP-23 gestrichen	10.5 (OP-45, angeglichen; OP-23, Rest gestrichen)

11 Aufbau der GRAFIK-Module

Dieser Abschnitt erklärt kurz, **warum** es neben `c8205_interp.py` den Treiber `c8205_grafik.py`, den Ordner `grafik-interpreter-module/` und den Ordner `c-module/` gibt, und was in welcher Datei steht. Entscheidung O6-4: ein kurzer Abschnitt, kein Parallelkapitel. Die Dateiliste ist aus den Kopfkommentaren der Dateien gelesen; das Verhalten habe ich nicht geprüft.

11.1 Warum eine zweite Programmfamilie

`c8205_interp.py` ist ein **Stapelbetrieb**: erst die Tastenfolge einer Testdatei, dann die Knotenschleife bis zum Halt, dann die Ergebnisprüfung (Dok. 04 §2.2). GRAFIK ist **ereignisgesteuert**: Klicks am Pult, ein laufender Rechner, Halt heißt Warten. Das ist kein Austausch einzelner Klassen, sondern ein neuer Treiber. Damit TEST unverändert bleibt, liegt GRAFIK in neuen Dateien, die `c8205_interp.py` als Bibliothek nutzen (`load_graph`, Tabellen, `FUNCS`, `Machine`, `Leser`, `Stanzer`). Die Auswahl trifft allein `--MODUS` (§1, §2); der Interpreter ruft bei `GRAFIK` `c8205_grafik.hauptprogramm()` auf (`c8205_interp.py` 0.03-017).

11.2 Schichten

Schicht	Dateien	Aufgabe
Interpreter (beide Modi)	`c8205_interp.py`	Knotenautomat für TEST, Tabellen, `Machine`; Modus-Weiche `--MODUS`
Treiber GRAFIK	`c8205_grafik.py`	Rechner im Dauerbetrieb, Tastenlogik mit dauerhaftem Zustand, Start der Fenster und Töne, Abbruch
Logik ohne Tk	`grafik_pult.py`, `grafik_sw.py`, `grafik_k0.py`, `grafik_haltzeit.py`, `grafik_schreibwerk.py`, `grafik_geraeteton.py`, `grafik_tastatur.py`	testbar ohne Fenster und ohne Audiogerät
Tk-Schicht	`grafik_pult_fenster.py`, `grafik_sw_fenster.py`, `grafik_hwcheck.py`, `grafik_fensterlage.py`	Zeichnen, Maus, Computertastatur, Fenster merken; ein Tk-Prozess, zwei Fenster
Geräte	`grafik_geraete.py`, `grafik_trommel.py`	Leser und Stanzer als Ordner, Trommel über das C-Modul
Geräte	`grafik_zt.py`	Anbindung der Zusatztrommeln des experimentellen Prototyps C8205Z (`--C8205Z`, Klasse `GrafikZT`) über das Erweiterungsmodul `c8205_ztmodul` (§7.10)
Töne	`grafik_tonmisch.py` (Rückfall), `grafik_tonmisch_c.py`, `grafik_rechnerton.py`, `grafik_ton.py` (Stufe 2)	Mischer und Rechnertöne
Fehlersuche	`grafik_debuglog.py`	`--DEBUGLOG`
C-Module	`c-module/` (`trommel.c`, `bahnsperre.c`, `unterbrechung.c`, `tonmisch.c`, `plattform.h`), Erweiterungsmodul `c-module/python/c8205_trommel_modul.c`	echte Trommel mit Koinzidenz, Bahnsperren, Abbruch, Tonmischer; statisch in **ein** Erweiterungsmodul gebunden (abi3)

Die Trennung „Logik ohne Tk“ und „Tk-Schicht“ gilt, damit die Logik in der Entwicklungsumgebung ohne Anzeige geprüft werden kann (Dok. 09, Dok. 10; Prüfverfahren §2).

Modulübersicht. Alle 19 Dateien in `grafik-interpreter-module/`, nach Schichten geordnet, je Datei ein Satz. Die Sätze sind aus den Kopfkommentaren der Dateien gelesen; das Verhalten habe ich nicht geprüft. Der Treiber `c8205_grafik.py` liegt in der Wurzel und steht in der Schichtentabelle.

Schicht	Modul	Aufgabe in einem Satz
Kern und Geräte	`grafik_k0.py`	Eingabewarteschlange des Kanals K0, die der Rechner im Eingabe-Halt unbegrenzt abwartet und die vom Pult (Quelle T) und vom SW-Fenster (Quelle SW) gefüllt wird.
Kern und Geräte	`grafik_geraete.py`	Lochbandleser und Lochbandstanzer als Ordner (Leser nach Änderungszeit, aktives Warten auf eine Datei im Eingabe-Halt).
Kern und Geräte	`grafik_trommel.py`	Trommel-Klasse mit der Schnittstelle der Interpreter-Trommel, die über das C-Modul `c8205_trommel` auf die echte Trommel mit Koinzidenz und Bahnsperren zugreift.
Logik ohne Tk	`grafik_pult.py`	Pult-Steuerung ohne Tkinter: Netzablauf, Tastenregeln, Lampen und Weitergabe der Tasten an Rechner und K0-Warteschlange.
Logik ohne Tk	`grafik_sw.py`	Logik des SW-Fensters ohne Tkinter: Schreibposition und Tabulator der Ausgabe sowie Abbildung der getippten Zeichen auf K0-Zeichen samt Zeichenlage KB/GB.
Logik ohne Tk	`grafik_tastatur.py`	Tastaturbelegung des Rechners (`--TASTATUR=de|en`), die beim Abspielen einer Klickspur bestimmt, wie ein Zeichen getippt wird (AltGr).
Logik ohne Tk	`grafik_haltzeit.py`	Zeitmessung Start → Halt, Halt → Halt und Einzelschritt → Halt in Millisekunden für die Zeile „Haltzeit“ des HW-Check-Fensters.
Tk-Schicht	`grafik_pult_fenster.py`	Tk-Schicht des Pult-Fensters: Zeichnen, Maus, Computertastatur und Schließen (Logik in `grafik_pult.py`).
Tk-Schicht	`grafik_sw_fenster.py`	Tk-Schicht des SW-Fensters (130 × 36 Zeichen) als Toplevel des Pult-Fensters (Logik in `grafik_sw.py`).
Tk-Schicht	`grafik_hwcheck.py`	Zweites, kleines Hardware-Check-Fenster (`--HWCHECK`) mit BZ, BR, Spur/Sektor, AC, Speicherinhalt an AR und Haltzeit.

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Schicht	Modul	Aufgabe in einem Satz
Tk-Schicht	`grafik_fensterlage.py`	Merkt sich die letzte Position von Pult- und SW-Fenster in einer JSON-Datei im Benutzerordner.
Töne	`grafik_tonmisch_c.py`	Python-Hülle (`CMischer`) um den Tonmischer im C-Modul, mit Rückfall auf den Python-Mischer.
Töne	`grafik_tonmisch.py`	Python-Mischer (44,1 kHz, Blöcke zu 10 ms) als Rückfall, Prüfbezug und Quelle gemeinsamer Hilfen für beide Mischer.
Töne	`grafik_schreibwerk.py`	Schreibwerkzeit (82,5 ms ± 7,5 ms) und Klangwahl des Schreibwerks, einschließlich TAB, WRZL und WR.
Töne	`grafik_geraeteton.py`	Zustandsautomaten für die Töne von Lochstreifenleser und Lochbandstanzer (Einzelschritt, Dauergeräusch, Auslaufen).
Töne	`grafik_rechnerton.py`	Rechnertöne über den Mischer: Hochlauf, Grundgeräusch als Schleife und Auslauf beim Ausschalten.
Töne	`grafik_ton.py`	Frühere Rechnertöne (Stufe 2) mit je einem Abspielprozess je Klang; bleibt unter Windows als Rückfall, wenn `winmm` nicht verfügbar ist (§9.2).
Fehlersuche	`grafik_debuglog.py`	Zeitgestempeltes Fehlersuch-Protokoll (`--DEBUGLOG`) sowie Aufzeichnung und Wiedergabe der Pult-Bedienung (Klickspur).
Prüfung	`grafik_pult_tests.py`	Prüfungen der Pult-Steuerung ohne Tkinter und der Tk-Schicht mit Ersatzmodul; liegt hier zusätzlich zur Fassung in `testscripte/`, die davon abweicht.

11.3 Was man wissen muss

- **Kopie statt Herauslösen (Punkt 18):** `exec_node` und `run_e` sind in `c8205_grafik.py` **Kopien** der verschachtelten Funktionen aus `lauf()` in `c8205_interp.py`. Jede Änderung am Knotenablauf ist in beiden Fassungen nachzuziehen; die Gleichwertigkeit von TEST und GRAFIK (ohne Zeiten) prüft `testscripte/grafik_gleichwertigkeit.py` (Prüfverfahren §4). Entschieden war „erst kopieren, nach bestandenen Tests herauslösen“; nach dem Kopfkommentar von `c8205_grafik.py` stehen die Kopien noch.
- **Zwei Zählungen:** `c8205_interp.py` zählt `y.xx-zzz` (derzeit `0.95-…`), die GRAFIK-Module zählen je Datei eigene Kennungen (G1-, G2-, … G5-, FL-, HW-, HZ-, DL-). Maßgeblich ist die Konstante `VERSION_*` im Code (eine Kopfzeile `Version:` gibt es seit 10.11.2026 nicht mehr); der Stand je Modul steht in dieser Konstante.
- **Rechnerische Wartezeit bleibt:** Die Knoten 040/041 behalten ihre rechnerische Schleife; die echte Trommel liefert Daten und Echtzeit (Punkt 13, Dok. 04).
- **Modulpfad:** `c8205_grafik.py` setzt `grafik-interpreter-module/` selbst in den Suchpfad; Start über `c8205_grafik.sh/.bat` oder `python3 c8205_interp.py` (Standard `GRAFIK`).
- **TEST nicht umgebaut:** Nach Dok. 04 §6 blieb TEST bis Stufe 5 unberührt; `--MODUS` kam mit Stufe 5 (0.03-017). Ob die Änderungen 0.03-015 und 0.03-016 den TEST-Zweig berühren, habe ich nicht geprüft.

11.4 Kennungen

Alte Kennung	Gegenstand	Abschnitt
Dok. 04 §2 bis §4, O6-4	Modulbefunde, Aufteilung	11.1, 11.2
Punkte 13, 18 (Dok. 04 §5)	Wartezeit, Kopie von `exec_node`/`run_e`	11.3
G1-xxx bis G4-xxx, FL-, HW-, HZ-, DL-	Modulversionen	11.2, 11.3

Anhang A Entscheidungsregister

Die 114 Zeilen aus Dokument 00 §2, **unveränderte Nummern und Daten, unveränderter Wortlaut** (O6-2). Die Spalte „Dokument“ nennt die alten Dokumentnummern; Anhang B.1 ordnet sie den neuen Abschnitten zu.

<!-- REGISTER-ANFANG: 114 Datenzeilen, Kopf und Trenner zählen nicht mit -->

Datum	Entscheidung	Dokument
13.10.2026	Abb. 3 und 4 als interaktive Grafik in separatem Fenster; als Layout dient das **Foto (Abb. 2)**	01
13.10.2026	Fensterhöhe etwa **860 px**; Grafik vorher auf diese Höhe skalieren (1148 × 860)	01

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Datum	Entscheidung	Dokument
13.10.2026	Teilgrafiken je Bildelement (gedrückt/losgelassen, hell/dunkel) werden vom Projektinhaber geliefert	01
13.10.2026	Die vier **unbeschrifteten unteren Selektortasten** werden nie gedrückt: entfallen	01
13.10.2026	**Rot/grün nur für Lampen B und N**; gestartetes Skript = Rechner eingeschaltet = grün	01
13.10.2026	Klick auf Netzschalter: **B rot, 2 Sekunden Anzeige, dann beendet sich das Skript komplett** *(B-Farbe ersetzt am 16.10.2026: B wird dunkel, siehe Eintrag „Hochlauf, KEP, Abbruch im Hochlauf“)*	01
13.10.2026	Die **Trommel soll echter werden**: Zugriffe laufen über die hochgeladenen C-Module	02
13.10.2026	Das Modul merkt sich die **aktuelle Zelle** (wechselt alle **104 µs**) und führt Lese-/Schreibanforderungen nur aus, wenn diese Zelle gerade die aktuelle ist; deshalb **selbständiger Hintergrundprozess** (umgesetzt als Hintergrund-Thread im Python-Prozess; falls ein eigener Prozess gemeint war, weicht das ab, Rückfrage offen)	02
13.10.2026	**Dateiformat:** Das Modul wird auf das Format der Interpreter-Trommeldatei (18 Byte je Zeile) umgestellt	02
13.10.2026	**„Schlitz ist Schlitz“:** Ist eine Sektornummer aktuell, dann für die ganzen 104 µs (digital). Trifft in dieser Zeitspanne eine Anforderung ein, wird sie bedient. (Gewohnheit/Ungenauigkeit des Simulators, bewusst so festgelegt.)	02
13.10.2026	**Betriebsart:** Die echte Trommel läuft **immer**, außer beim **Claude-Interpreter**. **Eingeschränkt am 15.10.2026:** nur im Modus `--MODUS=GRAFIK`; im Modus `--MODUS=TEST` gibt es keine echte Trommel und kein C-Modul (siehe 3.6)	02, 00 (3.1, 3.6)
13.10.2026	**Zielsysteme: Linux und Windows**	00 (3.2), 02
13.10.2026	Die Frage nach dem **Lampentempo** (Echtzeitausgabe von AR) war zurückgestellt; **aufgehoben 16.10.2026** (Review F-07), Entscheidung siehe Zeile „Tempo in GRAFIK“ weiter unten	01 (6.2)
14.10.2026	**Strg+C:** Wird der Interpreter mit Strg+C unterbrochen, wird auch das C-Modul beendet (gilt nur im Modus GRAFIK; im Modus TEST gibt es kein C-Modul)	02
14.10.2026	**Bibliothek:** Eine statische Bibliothek ist denkbar; ist das C-Modul nicht ladbar, gibt es eine **Fehlermeldung und der Interpreter beendet sich** (kein Rückfall). Umsetzung: Python-Erweiterungsmodul mit fest eingebundenen C-Modulen (Dokument 02, Abschnitt 4.5). Gilt nur im Modus GRAFIK	02
13.10.2026	Die Dokumentation wird **aufgespalten** (Hauptdokument, Trommel, Bedienpult; weitere können folgen)	00
15.10.2026	**Der Interpreter läuft erst, wenn Lampe B auf grün schaltet.**	03, 01 (5)
15.10.2026	**Einschalten:** Klick auf den Netzschalter → B rot → `fade_in` läuft → B grün. (Weicht vom bisherigen Start mit B und N grün in Dokument 01 ab.)	03, 01 (5)
15.10.2026	**Betriebsmodi:** Der Python-Interpreter soll die zwei Modi `--MODUS=TEST` und `--MODUS=GRAFIK` haben	00 (3.6), 01 (1.6), 02, 03
15.10.2026	`--MODUS=TEST` ist der bisherige Stapelbetrieb **ohne** Pult-Fenster, **ohne** Töne (also auch ohne die Wartezeit des Hochlaufs), **ohne** die C-Module der Trommel und **ohne** das interaktive Eingabefenster für das SW	00 (3.6), 01, 02, 03
15.10.2026	**SW-Fenster:** Das bisherige Fenster „SW Ausgabe“ (`--SW-TERMINAL`) gibt es nur im Modus TEST. Im Modus GRAFIK erscheint ein dazu **analoges Fenster**, das nur SW-Eingaben und -Ausgaben darstellt und **immer auch die SW-Eingabe übernimmt**; es wird aufgeblendet, wenn der Interpreter im Modus GRAFIK am Terminal gestartet wird, und verschwindet, wenn das Programm endet	00 (3.6), 01 (1.6)
15.10.2026	**Vorgehen:** Nach einer Prüfung bleibt der bisherige Interpreter für TEST **unverändert**; für GRAFIK werden die betroffenen Module **neu programmiert**, danach wird nur noch umgeschaltet	00 (3.6), 02
15.10.2026	**SW-Fenster in GRAFIK, Aussehen:** genau wie das bisherige Fenster „SW Ausgabe“: 130 × 36 Zeichen, Markieren und Kopieren; **ohne Echo: Tastatureingaben sind nirgends zu sehen, auch nicht in Dunkelgelb** (Berichtigung 16.10.2026, Entwurf 88) **[überholt am 18.10.2026, Entwurf 104: das SW-Fenster zeigt jedes angenommene Zeichen selbst an, Dok. 09 S3-04 neu]**	00 (3.6), 01 (1.6)

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Datum	Entscheidung	Dokument
15.10.2026	**Eingaben in GRAFIK:** SW-Eingaben erfolgen im SW-Fenster, wenn der Rechner im Eingabe-Halt-Befehl steht, **Kanal K0 angewählt** und der Schalter **SW** gedrückt ist (Kanal von „Kanal 2“ auf K0 mit Schalter SW **berichtigt**, 3.6 Punkt 9). **Alle Eingaben des K0 (Pult-Tastatur) erfolgen mit Klicks im interaktiven Bedienpult.** Bei Eingabe-Halt unterscheidet T-SW also zwischen dem interaktiven Bedienpult (T) und dem interaktiven SW-Fenster (SW)	00 (3.6), 01 (1.6)
15.10.2026	**Kein Testdokument in GRAFIK:** Es gibt kein Test-Anforderungs-Dokument (Testdatei), daher keine Makros (z. B. `#STARTE`) und kein bisheriges Testprotokoll. Wortlaut des Projektinhabers: „bei TEST“; als GRAFIK gelesen (**bestätigt**, 3.6 Punkt 10)	00 (3.6)
15.10.2026	**X am SW-Fenster** (Modus GRAFIK) ist wie **Abbruch mit Strg+C**: der Interpreter und das C-Modul beenden sich (Entscheidung vom 14.10.2026 gilt)	00 (3.6), 01, 02
15.10.2026	**Lage in GRAFIK:** drei getrennte Fenster, nichts eingebettet: das Terminal, daneben das Bedienpult, daneben darunter das SW-Fenster	00 (3.6), 01 (1.6)
16.10.2026	**Bildschirm in GRAFIK (Review F-05):** Zielbildschirm **1920 × 1080**; Pult in **fester Größe 1148 × 860 px, nicht skaliert**; Pult- und SW-Fenster dürfen sich **überlappen, wenn der Platz fehlt**. Lage der Fenster sonst wie am 15.10.2026 (keine Pixelvorgabe)	00 (3.6), 01 (1.6, 8), 05
15.10.2026	`--SW-TERMINAL` **im Modus GRAFIK:** wird ohne Fehlermeldung ignoriert (der Schalter kann ohnehin nur beim Aufruf des Interpreters mitgegeben werden)	00 (3.6)
15.10.2026	**Geräte-Schalter in GRAFIK:** nur `--leser1`, `--leser2`, `--stanzer1`, je nur ein Pfad (Leser: Ordner mit nachrückenden Lochstreifendateien, gelesene wandern in `LOCHSTREIFENKORB`, siehe Register unten); `--k0`, `--sw-ausgabe`, `--NO-TERMOUT`, `--zwischenablage` wirkungslos ohne Fehler	00 (3.6)
15.10.2026	**Programmquelle in GRAFIK:** Objektprogramm-Lochstreifen über Leser 1/2 mit dem Minimalprogramm (Befehlseingabe zur laufenden Speicherung); danach Start durch einen Sofortbefehl am Bedienpult	00 (3.6), 01
15.10.2026	**Terminal in GRAFIK:** zeigt gegebenenfalls nur Fehlermeldungen	00 (3.6)
15.10.2026	**Standardmodus:** Ohne `--MODUS` gilt **GRAFIK**; bisherige Stapelaufrufe brauchen künftig `--MODUS=TEST`	00 (3.6)
15.10.2026	**Unbekannter Modus** (z. B. `--MODUS=XYZ`): Fehlermeldung, Rückkehrcode 16, kein Start	00 (3.6)
15.10.2026	**Fehlerfälle in GRAFIK:** Tkinter oder Anzeige fehlt → Fehlermeldung, Rückkehrcode 16, Beenden; C-Modul nicht ladbar → Rückkehrcode 16 (bestätigt)	00 (3.6), 01, 02
15.10.2026	**Einzelschalter in GRAFIK:** nur `--STUMM` (Töne aus, dann auch keine Hochlaufwartezeit); `--PULT-FENSTER` entfällt (ersetzt durch `--MODUS=GRAFIK`); `--BASP` wirkt in GRAFIK über das C-Modul	00 (3.6), 01, 02, 03
15.10.2026	**Claude-Interpreter:** arbeitet nach den Regeln des Modus TEST	00 (3.1, 3.6)
15.10.2026	**Prüfung der betroffenen Module:** durch Claude als nächster Schritt, nur lesend; Prüfbericht als Dokument 04; danach Freigabe durch den Projektinhaber	00 (3.6), 04
15.10.2026	**X** am Pult-Fenster wie am SW-Fenster: Abbruch wie Strg+C (so gelesen; ersetzt für GRAFIK die Annahme „X = Netz-Aus“ des Prototyps)	00 (3.6), 01 (7, Punkt 4)
15.10.2026	**Eigener Zweig für GRAFIK:** Für alle betroffenen Teile des Python-Programms wird im Modus GRAFIK ein eigener Zweig angefertigt, wenn eine unveränderte Nutzung nicht möglich ist; TEST bleibt unverändert. Daraus (so gelesen): Punkt 13 des Prüfberichts (rechnerische Wartezeit der Knoten 040/041 gegen die echte Trommel) und Punkt 18 (`exec_node`/`run_e` für GRAFIK kopieren) sind erledigt; der Knotengraph in der Doku bleibt gemeinsam	04 (5), 00 (3.6)
15.10.2026	**Trommel in GRAFIK „immer so wie sie ist“:** keine Vorbelegung und kein Eingriff des Interpreters in die Trommeldatei. **Die Trommeldatei wird unverändert übernommen; was darin steht, ist die Startbelegung (Präzisierung 15.10.2026, ersetzt den Nachsatz „das KEP muss schon in der Datei stehen“). Hinweis: Für einen Kaltstart muss die Datei ein lauffähiges Minimalprogramm enthalten; das KEP braucht man nach BIS-2.1 nur, um das MP erstmals zu laden** (Punkt 17 des Prüfberichts erledigt)	04 (5), 02
15.10.2026	**Gemeinsame Trommeldatei:** TEST und GRAFIK nutzen dieselbe `Trommel/trommel.txt` (kein eigener Dateiname für GRAFIK). Folge: Jeder TEST-Lauf schreibt in die Datei, die GRAFIK als Startbelegung übernimmt; ein Schutz gegen gleichzeitigen Zugriff von TEST und GRAFIK ist nicht vorgesehen	02 (8, Punkt 9), 00 (3.6)

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Datum	Entscheidung	Dokument
15.10.2026	**Standard von** `--BASP` **in GRAFIK:** Ohne Angabe gilt `000000` (wie im Interpreter), also ist Spur 0 ungeschützt; geschützt wird nur auf ausdrückliche Angabe von `--BASP`. **Ersetzt 16.10.2026 (Entwurf 86): Ohne Angabe gilt die Stellung aus der Schalterdatei neben der Trommel; fehlt sie, wird sie mit** `000000` **angelegt (siehe Zeile unten, O-9).**	02 (6), 00 (3.6)
15.10.2026	`--trommel PFAD` gilt für TEST und GRAFIK; ohne Angabe gilt die gemeinsame `Trommel/trommel.txt`	02, 00 (3.6)
15.10.2026	**Verhalten der Leser in GRAFIK (entschieden 15.10.2026):** Für Leser 1 und 2 wird je **ein Pfad** angegeben, der nach dieser Beschreibung ein Ordner ist, in dem Lochstreifendateien erscheinen (bestätigt). Liegt ein Eingabe-Halt-Befehl für den Kanal Leser 1 bzw. Leser 2 vor, wartet der Leser **mit leuchtender Lampe E**, bis im Pfad eine Lochstreifendatei vorhanden ist; er prüft das **aktiv**. Ist das Ende einer Datei erreicht und der Eingabe-Halt noch aktiv, wird eine eventuell vorhandene **zweite Datei** (usw.) genommen, andernfalls wird gewartet. Liest der Rechner nicht alle Zeichen einer Datei, bleibt der Leser **an dieser Stelle stehen**; ein späterer Eingabebefehl liest dort weiter. Streifenwechsel im Betrieb ist damit durch Ablegen weiterer Dateien im Pfad möglich. **Reihenfolge und Verbleib (entschieden 15.10.2026):** Mehrere Dateien werden in der **zeitlichen Reihenfolge ihres Zeitstempels** genommen. Gelesene Dateien bleiben **nicht** im Ordner liegen, sie werden in den Unterordner `LOCHSTREIFENKORB` verschoben, der bei Bedarf anzulegen ist (bestätigt 15.10.2026: ältester Zeitstempel zuerst, Änderungszeit, Verschieben sobald das Dateiende gelesen ist; eine nicht zu Ende gelesene Datei bleibt liegen)	00 (3.6, Punkt 11), 04 (2.4)
16.10.2026	**Leser in GRAFIK, Ergänzung (Review F-11 (e), (f), entschieden 16.10.2026):** Nach Netz-Aus bleibt eine teilgelesene Datei im Leserordner liegen und wird beim nächsten Start von vorn gelesen (entspricht dem Neustart des Rechners). Zeigen `--leser1` und `--leser2` auf denselben Ordner, ist das ein Startfehler (Konfigurationsfehler, Code 16).	00 (3.6, Punkt 11), 04 (2.4), 05 (F-11)
16.10.2026	**Stanzer in GRAFIK (Review F-12, entschieden 16.10.2026):** Der Stanzer legt seine Datei an, wenn er nach dem Start des Interpreters zum ersten Mal zu schreiben beginnt und im Stanzerordner noch keine Datei liegt; Standardname `stanzer.ls`. Gibt es die Datei bereits, wird sie **fortgeschrieben** (angehängt), nie überschrieben. Der Stanzerpfad `--stanzer1` ist danach ein Ordner (aus der Antwort abgeleitet, nicht gesondert bestätigt). Abweichung vom heutigen Interpreter, der die Datei `stanzer-ausgabe.txt` beim ersten Zeichen jedes Aufrufs überschreibt; in TEST bleibt es beim heutigen Verhalten (Standardannahme).	00 (3.6, Punkt 11), 04 (2.5), 05 (F-12)
16.10.2026	**Standardordner der Geräte in GRAFIK (Review F-12, entschieden 16.10.2026):** In GRAFIK sollen `--leser1`, `--leser2` und `--stanzer1` angegeben werden. Fehlt ein Schalter, gilt der Standardordner aus dem Ordner `virtuelle_Geraete/` (Projektstruktur, `C8205-Projektstruktur.md`): `Leser1/`, `Leser2/` bzw. `Stanzer1/`; die Lage dieser Ordner wie bei der Trommel (Vorgabepfad neben dem Skript, aus der Projektstruktur abgeleitet, nicht gesondert bestätigt). Damit gibt es in GRAFIK keinen Stanzbefehl ohne Stanzerordner; die Frage entfällt.	00 (3.6, Punkt 11), 04 (2.5), 05 (F-12)
16.10.2026	**Schalterliste in GRAFIK (Review F-13, entschieden 16.10.2026):** Für GRAFIK gelten genau die Schalter `--MODUS`, `--leser1`, `--leser2`, `--stanzer1`, `--trommel`, `--STUMM` und `--BASP`; dazu `--mdverz`, weil der Graph ohne das Verzeichnis der Knotendoku nicht geladen werden kann. Alle anderen Schalter des Interpreters (u. a. `--out`, `--knotentest`, `--pruefe-*`, `--version`, `--k0`, `--sw-ausgabe`, `--NO-TERMOUT`, `--zwischenablage`, `--SW-TERMINAL`) sind in GRAFIK wirkungslos und ohne Fehler; das positionale Argument `tests` entfällt. Die Aliasse `--lochbandleser1/2` und `--lochbandstanzer1` sind in der Vorschlagsliste nicht genannt und damit in GRAFIK wirkungslos (nicht gesondert besprochen). Der entfallene `--PULT-FENSTER`, die Groß-/Kleinschreibung des Modus-Werts und `--STUMM`/`--BASP` in TEST wurden nicht gesondert besprochen; es bleibt beim heutigen Verhalten.	00 (3.6, Punkt 11), 04, 05 (F-13)

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Datum	Entscheidung	Dokument
15.10.2026	**Rollensperre und Entriegelung** (Erklärung des Projektinhabers): Rollensperre bedeutet nur Sperren. Die Taste R löst sie aus, danach ist keine andere Taste der Gruppen mit Rollensperre bedienbar, nur H hebt sie auf; R verriegelt gleichzeitig die Eingabetasten. Nicht betroffen: K0 bis K3, ST, BS, SP, LS. Steht in `C8205-Interpreter-Bedienpult_mit_Schaltern.md` (0.03-007) und im PDF. **Entriegelung entschieden (Antwort B):** Die Eingabetasten werden durch einen aufgestellten Eingabe-Halt-Befehl entriegelt, auch bei R. **T-SW (entschieden 16.10.2026, Entwurf 49):** Der Schalter T-SW (Tasten T und SW, `SW_umschalt`) unterliegt **nicht** der Rollensperre; die Ableitung vom 16.10.2026 (Review F-06, „ja“) ist ersetzt. Nicht betroffen sind damit K0 bis K3, ST, BS, SP, LS, der Selektor SW und T-SW	01 (1.7, 3.5, Punkt 7)
16.10.2026	**Schalter T, SW,** `SW_umschalt` **(Review F-06):** Logisch ein Schalter T-SW mit den Stellungen T und SW; Bedienstellen im Fenster: Tasten T und SW (Tastatur, gegenseitig verriegelt) und `SW_umschalt` oben rechts (wechselt die Stellung). **Berichtigt in Entwurf 50:** T-SW ist eine Wippe aus nur den zwei Tasten T und SW; `SW_umschalt` ist der Selektor SW. Stellung SW → Flag SW := 1, T := 0; Stellung T → SW := 0, T := 1. Aus der Pult-Doku abgeleitet, nicht ausdrücklich bestätigt; Tabelle in Dokument 01, Abschnitt 3.5. **Entwurf 49:** keine Rollensperre für T-SW (entschieden); der **Selektor SW** rechts außen ist ein eigener Selektor (unterdrückt die SW-Ausgabe)	01 (3.5), 05
16.10.2026	**Tempo in GRAFIK (Review F-07, aus den Unterlagen abgeleitet, nicht ausdrücklich bestätigt):** Rückstellung aufgehoben. **Zielwert ist das Originaltempo** des C 8205 (Interpreter-Doku „Überblick und Gesamtaufbau“, Abschnitt 5): Einzelbefehl bei zeitoptimaler Lage 5 Wortzeiten (0,52 ms, rund 1900 Befehle/s), Organisationsbefehl 4 Wortzeiten (0,42 ms, rund 2400/s), Gruppen- und Wiederholungsbefehle bis rund 9000/s. Es wird **nicht künstlich gebremst** und es gibt keinen Takt-Schalter: Das Tempo ergibt sich aus der echten Trommel (Wartezeit 0 bis 31 Wortzeiten zu 104 µs, mittlere Zugriffszeit 1,67 ms) und dem Python-Tempo der Phasen ohne Trommelwartezeit. Die Geräte behalten ihr Originaltempo (Leser 130 Zeichen/s, Stanzer 50 Zeichen/s, Schreibwerk 10 Zeichen/s; Dokument 03; ob das auch bei `--STUMM` gilt, regelt F-17). **Lampen:** Im Dauerlauf R zeigt das Pult höchstens alle 40 ms den jeweils letzten Zustand (Weg 2 aus Dokument 01 Abschnitt 6.2); bei jedem Halt und nach jedem E-Schritt sofort den Stand. Kein abgestuftes Nachleuchten (Weg 3, keine dritte Lampengrafik); **ergänzt in Entwurf 84: ruhiges Lampenbild** (Zeile „Lampenbild ruhig“). **Abnahme (Standardannahme, nicht bestätigt):** Messung auf dem Zielrechner im Dauerlauf R mit einem Programm aus zeitoptimal gelegten Einzelbefehlen; erwartet werden mindestens 90 % von 1900 Befehlen/s. Braucht Python je Phase länger als eine Wortzeit, verlängern sich die Trommelwartezeiten (Dokument 02 §7); das ist hingenommen, es gibt dazu weder Eingriff noch Meldung.	01 (1.5, 6.2), 02 (7), 05
16.10.2026	**Rückkehrcodes und Beenden in GRAFIK (Review F-08, entschieden 16.10.2026 mit „ja“ zum Vorschlag): Netz-Aus** (Schlüsselschalter, 2 s Auslauf) → Code **0**. **X** am Pult- oder SW-Fenster und **Strg+C** → Code **8** (wie Strg+C in TEST). Ein **Fehler, bei dem der Rechner hält** (z. B. Stub, nicht nutzbares Gerät) und der danach per X beendet wird → **8**. **Start- und Konfigurationsfehler** (Tkinter oder Anzeige fehlt, C-Modul nicht ladbar, fehlerhafte Trommeldatei, unbekannter Modus, **kein Abspielprogramm oder Audiogerät ohne** `--STUMM`, Entwurf 55) bleiben **16**. Die Codes 0, 4 und 6 der Testauswertung (`Prüfungen:`) entfallen in GRAFIK, weil es dort keine Testdatei gibt. **Aufräumen** bei Netz-Aus, X und Strg+C: Stanzer-, SW- und Trommeldatei werden geschlossen, beide Fensterprozesse und das C-Modul beendet; Netz-Aus endet kontrolliert an einer Knotengrenze, X und Strg+C lösen `KeyboardInterrupt` sofort aus und räumen im `finally` auf (gleiches Aufräumen, anderer Zeitpunkt). Bei **hartem Ende** (kill, Absturz) beenden sich die Fensterprozesse selbst, sobald ihre Verbindung zum Interpreter wegfällt (Standardannahme zu diesem letzten Satz).	00 (3.6), 01 (7 Nr. 2, 8 Nr. 8), 04 (Punkt 14), 05

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Datum	Entscheidung	Dokument
16.10.2026	**Lampe R und „AR“-Lampen (Review F-09 und F-10, entschieden 16.10.2026): Lampe R leuchtet, wenn der Schalter R gedrückt ist** (Flag H = 0; H und R sind gegeneinander verriegelt, es ist immer genau eine der beiden Tasten gedrückt, Lampe R = nicht Lampe H). Ein Haltgrund (US, BS, FÜ, FE, E, A) ändert daran nichts. Es gibt kein eigenes Flag R, die Lampe wird aus `m.fl['H']` abgeleitet. Meine Fassung vom selben Tag („es gibt keine Lampe R“) beruhte auf einem Missverständnis und ist berichtigt; der Vorschlag „leuchtet nur ohne Haltgrund“ bleibt verworfen. **Die unteren Lampen „AR“ zeigen den Adreßteil des Befehlsregisters,** `m.adr` (Schreibweise im Text: „Adreßteil des BR (Lampenreihe ‚AR‘)“, nicht zu verwechseln mit dem Register `m.AR`). Hinweis: Die Datenstruktur-Doku (Abschnitt Anzeige des Arbeitszustands) sagt, im Zustand „Rechnen“ dürfe von den Lampen H, R, US, BS, FÜ, FE, E und A nur R leuchten; das entspricht der Regel nur, solange kein Haltgrund besteht. Maßgeblich ist die Entscheidung des Projektinhabers (Lampe R = Schalter R gedrückt)	01 (3.1, 3.2, 7 Nr. 5 und 6), 05
15.10.2026	`--BASP` **in GRAFIK** wird als Erstes an das Trommelmodul gemeldet (vor dem ersten Zugriff), damit das Modul bei allen Schreibzugriffen des Knotengraphen darauf zurückgreift. Format nach `Magnettrommel.md`: `--BASP=abcdef`, links nach rechts Hauptschalter 0001, 0040, 0100, 0200, 0400, 1000, 1 = EIN. (Im Chat als `--BSP=111000` geschrieben, als Tippfehler gelesen, **bestätigt 15.10.2026: Name** `--BASP`)	02 (6), 00 (3.6)
15.10.2026	**Bedienung des Pults per Klick:** Tasten wie 0, 1, + und B, ebenso E, M und G, zeigen nach dem Klick 0,4 s (einstellbar) die gedrückte Grafik und gehen dann in den Ausgangszustand zurück; Schalter (z. B. H, SW, K1) rasten ein und zeigen einen anderen Grafikzustand. Das **Tastenverhalten selbst** (Sperren, Verriegelungen) steht in `C8205-Interpreter-Bedienpult_mit_Schaltern.md` und wird hier **nicht doppelt beschrieben**	01 (1.7, 7)
15.10.2026	**Halter für M und E (E bestätigt 15.10.2026):** Umschalt-Klick auf M oder E hält die Taste (Grafik dauerhaft gedrückt, die Maus muss nicht gehalten werden); bei gehaltenem M ist E mehrfach klickbar (je 0,4 s), bei gehaltenem E ist G klickbar („E halten, G drücken“); ein weiterer Umschalt-Klick löst die Taste und die Grafik geht in die Ausgangslage zurück	01 (1.7)
15.10.2026	**Prüfbericht Punkt 14 und 19 (Antwort „ja“):** Fehler in GRAFIK (`Stub`, `LaufFehler`, `KonfigFehler`): Der Rechner hält, die Meldung erscheint im Terminal, beendet wird nur durch X, Strg+C oder Netzschalter (14). K0-Quelle: T nur vom Pult, SW nur vom SW-Fenster, die andere Quelle wird ignoriert (19)	04 (5)
15.10.2026	**Prüfbericht Punkt 15 und 16 (Ausführung dem Entwickler überlassen):** Vorhanden sind nur das Terminal, zwei Fenster und die C-Module der Trommel als selbständiger Hintergrundprozess. X löst den Abbruch durch eine Meldung vom Fenster an den Interpreter aus, der Interpreter führt den Strg+C-Abbruch selbst aus (plattformneutral); Pult und SW-Fenster laufen in einem gemeinsamen Tk-Prozess mit zwei Fenstern; das SW-Fenster erhält bei erwarteter Eingabe den Fokus	04 (5), 01 (1.7)
16.10.2026	**Startzustand in GRAFIK (Review F-01, Schritt 1):** Das Skript startet mit **ausgeschaltetem Rechner**: Schlüsselschalter „aus“, alle Lampen dunkel (auch B und N), das Pult wartet auf den Klick auf den Netzschalter. Danach Hochlauf (B rot → `fade_in` → B grün); der Interpreter läuft erst bei B grün. Ersetzt für GRAFIK die frühere Annahme „gestartetes Skript = eingeschalteter Rechner, B und N grün“ (Dokument 01, 1.3)	01 (1.3, 5, 7), 03 (3, 5), 05 (F-01)
16.10.2026	**Zustand des Pults vor dem Einschalten (Review F-14, Dialog, entschieden 16.10.2026):** Terminal: Interpreter gestartet; die Fenster Bedienpult und SW-EA werden aufgeblendet; im Pult sind alle Schalter ungedrückt außer H und T-SW (Stellung T); die Taste K0 ist nicht eingerastet, Kanal K0 ist elektronisch angewählt (berichtigt in Entwurf 76); Lampe N grün, Lampe B dunkel; Netzschalter rot (Schlüsselschalter „aus“); nur der Netzschalter nimmt Klicks an. **Ändert** den Eintrag vom 16.10.2026 zum Startzustand (dort „alle Lampen dunkel, auch B und N“) und die Standardannahme zur Startstellung (F-02 Nr. 8). Der Dialog zum Einschalten und Programmlauf wird fortgeschrieben	01 (1.3, 1.8, 3.5, 5), 05 (F-14)

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Datum	Entscheidung	Dokument
16.10.2026	**Einschalten und Laden des Minimalprogramms (Review F-14, Dialog, entschieden 16.10.2026):** Klick auf den Netzschalter: er wird sofort grün, B rot, `fade_in` läuft (Projektinhaber: ca. 5 bis 6 s); in dieser Zeit nehmen die Tasten keine Klicks an, nur der Netzschalter; ein erneuter Klick macht ihn wieder rot, `fade_out` läuft, danach wird das komplette Interpreterprogramm beendet und beide Fenster werden abgeräumt. Nach `fade_in` wird B grün; ab dann sind die Tasten bedienbar. Das MP wird geladen mit der Tastenfolge G, M halten, E, E, M lösen, K1 (Leser 2: K2), R (H ist schon gedrückt); das KEP wird angesprungen, liest den binär kodierten Lochstreifen mit dem BIS-Minimalprogramm und speichert ihn ab 0040; am Lochstreifenende bleibt es stehen. Der Programmstart ist erst danach möglich. **Ersetzt** F-02 Nr. 2 und Nr. 4. (Dauer und KEP geklärt, siehe nächste Zeile)	01 (1.3, 1.8, 5), 03 (5), 04 (2.7), 05 (F-14)
16.10.2026	**Hochlauf, KEP, Abbruch im Hochlauf (Review F-14, Dialog, entschieden 16.10.2026):** Hochlauf: `fade_in` läuft in voller Länge (11,3 s, die Länge der Aufnahme; „solange es läuft“), B wird **am Ende von** `fade_in` grün. Die frühere Angabe „ca. 5 bis 6 s“ war eine Schätzung. KEP: Das Pult-Fenster und der Interpreter in GRAFIK bringen das KEP nicht auf die Spur 0. Es muss **anderweitig** dorthin gelangen, z. B. mit einem TEST oder von Hand in die Trommeldatei (Pflege durch den Projektinhaber, F-03). Der Widerspruch zu Dokument 04 §2.7 ist damit aufgelöst. Klick im Hochlauf: `fade_in` **bricht ab**, `fade_out` läuft (es läuft immer nur einer von beiden), der Netzschalter wird rot, **B wird dunkel**; danach Ende des Programms. Die Tasten bleiben im Hochlauf gesperrt (bestätigt). **Farbe B beim Ausschalten bei laufendem Rechner (entschieden 16.10.2026, Entwurf 45):** B wird **gleich dunkel**, nicht rot; der Netzschalter wird rot, `fade_out` läuft, danach endet das Programm. Ersetzt den Eintrag vom 13.10.2026 („B rot, 2 Sekunden Anzeige“). Offen bleibt der Programmlauf nach dem Laden des MP *(am 16.10.2026 weitergeführt, nächster Eintrag)*	01 (1.3, 1.8, 5), 03 (5), 04 (2.7), 05 (F-14)
16.10.2026	**Programmlauf nach dem Laden des MP (Review F-14, Dialog, entschieden 16.10.2026, berichtigt in Entwurf 47):** Das KEP trägt beim Abspeichern des MP in die Zelle 0000 einen Sprung zum Minimalprogramm ein; **H, G, R** startet das MP (Start auf Zelle 0000). Das MP erreicht die Stelle „Eingabe eines Befehls mit Kennzeichen“; dort werden zuerst Leitadressen eingegeben: **20 aaaa** = Relativadresse, **21 aaaa** = 1. Leitadresse, **22 aaaa** = 2. Leitadresse usw. bis **29 aaaa** (die verschiedenen Eingabemöglichkeiten). Dann wird der Leserkanal **K1 oder K2** gewählt und `01`. eingegeben (Kennzeicheneingabe K_1K_2; Befehl zur fortlaufenden Eingabe von Objektkodebefehlen mit Addition der 1. Leitadresse); der Objektkode-Lochstreifen wird nach dem Kennzeichen LEITADRESSE (00 bis 09) und der eingegebenen Adresse **ab Speicherplatz aaaa** gespeichert. **Programmstart:** Eingabe `10 aaaa` im MP verlässt das MP, die Steuerung geht an das geladene Programm. `aaaa` in `10 aaaa` ist die **Startadresse im Speicher** (Entwurf 48): häufig gleich der (1.) Leitadresse; Beispiel: relativer Start 0200, `21 1200` gegeben → Start mit `10 1400`; Einzelheiten BIS: `C8205-Interpreter-BIS.md` (nicht im Paket)	01 (1.8 Nr. 10 bis 14), 05 (F-14)
16.10.2026	**Bedienung während des Programmlaufs (Review F-14, Dialog, entschieden 16.10.2026, Entwurf 48):** Das Pult ist **interruptfähig** bezüglich **H** (jederzeit Unterbrechung der Befehlsverarbeitung, danach wieder normal bedienbar), der **Kanalanwahltasten**, der **Taste SW rechts neben LS** (unterdrückt jederzeit die SW-Ausgabe, der Rechner läuft weiter), **SP, BS, LS**, des **Netzschalters** und – nur bei Stellung H, bei R gesperrt (Entwurf 71) – des **Umschalters T ↔ SW**. Das Pult reagiert auf die Flags **FE, FÜ, BS, US** (Lampen); die Stellung **H–R** steuert die Lampen H und R. **Widerspruch geklärt in Entwurf 49:** Die Taste SW rechts außen ist ein eigener Selektor; T-SW unterliegt nicht der Rollensperre *(in Entwurf 71 ersetzt: bei R gesperrt)*	01 (1.8 Nr. 15 bis 22, 3.5), 05 (F-14)
16.10.2026	**Selektor SW, T-SW und Tastenwirkung (Review F-14 und F-06, Dialog, entschieden 16.10.2026, Entwurf 49): SW rechts außen ist ein eigener Selektor** (neben SP, BS, LS). **T ↔ SW unterliegt nicht der Rollensperre** *(in Entwurf 71 ersetzt: bei R gesperrt)*. Sind die Tasten bedienbar, **reagieren sie wie im TEST beschrieben** (Pult-Doku, Durchführung eines Tests). Ersetzt die Ableitung zu F-06 (Entwurf 29, „Rollensperre ja“)	01 (1.8, 3.5, 7), 05 (F-06, F-14)
16.10.2026	**Wippe T-SW und Selektor SW (Review F-06 und F-14, Dialog, entschieden 16.10.2026, Entwurf 50): T-SW ist eine mechanisch verbundene Kombination zweier Tasten** (T und SW) **wie eine Wippe**, immer nur alternativ nach unten zu drücken; die Tastatur hat **keine weitere Taste SW**. Das Manifest-Element `SW_umschalt` **ist der Selektor SW** (rechts außen in der Reihe SP, BS, LS, SW, Abbildung 4 der Pult-Doku) und **rastet ein wie SP, BS und LS**. Ersetzt die Lesart „`SW_umschalt` = dritte Bedienstelle von T-SW“ (Entwurf 29). Abbildung im Paket: `pult-fenster/original-upload/abb4_anwahltasten_selektoren_sperrschalter.png`	01 (3.3, 3.5, 1.8), 05 (F-06, F-14)

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Datum	Entscheidung	Dokument
16.10.2026	**Wirkung des Selektors SW (Review F-14, Dialog, entschieden 16.10.2026, Entwurf 51; ergänzt in Entwurf 52):** Bei eingerastetem Selektor SW bleibt der **bestehende Text im Fenster „SW Ausgabe“ sichtbar**, **neuer Text wird nicht ausgegeben**, der Rechner **arbeitet die Ausgabebefehle weiterhin korrekt ab**, das Schreibwerk macht **keinen Ton**; **keine Ausgabe heißt kein Warten** (keine Schreibwerkzeit). Nach dem Lösen wird wieder ausgegeben (Tastengeräusch, Tastenverzögerung), **unterdrückter Text wird nicht nachgeholt**. Flags und Lampe SW bleiben unberührt; Leser, Stanzer und Rechnergeräusch sind nicht berührt	01 (1.8 Nr. 17, 3.5), 03 (1), 05 (F-14)
16.10.2026	**Stanzer im Lauf und Schalter LS (Review F-14, Dialog, entschieden 16.10.2026, Entwurf 52):** Der Selektor SW gilt für den Stanzer nicht; keine eigene Bedienung. Bei gesetztem LS ist das **Stanzergeräusch immer** zu senden, sonst **endet es 5 Sekunden nach dem letzten Zeichen** und beginnt erst mit dem nächsten Zeichen wieder; der Ton hat **keine Wirkung auf die Stanzerdatei**. Abweichung zur Projektdokumentation (3 Sekunden Abschaltverzögerung des Motors) **geklärt in Entwurf 53**: 5 s nur für das Geräusch, weil 3 s schlecht hörbar sind; Motor weiter 3 s; Projektdokumentation unverändert	01 (1.8, 3.5), 03 (1), 05 (F-14)
16.10.2026	**Ende des Programmlaufs (Review F-14, Dialog, entschieden 16.10.2026, Entwurf 54):** Das Ende kann **durch Halt des Programms** (US, BS, FÜ), **durch Eingriff am Pult** oder **durch den Netzschalter** erfolgen (alle drei möglich). Nach einem Halt zeigt das Pult **nur die Lampe** und wartet auf die Tasten **ST, Ü, M, E wie im TEST beschrieben**; **nichts Zusätzliches** (keine Terminalmeldung, kein Ton). **Damit ist der Beispielablauf F-14 abgeschlossen**	01 (1.8 Nr. 23 bis 25), 05 (F-14)
16.10.2026	**Kein Abspielprogramm oder Audiogerät (Review F-17, Dialog, entschieden 16.10.2026, Entwurf 55): Fehlt in GRAFIK (ohne `--STUMM`) ein Abspielprogramm oder ein nutzbares Audiogerät, bricht das Skript mit Fehlermeldung und Rückkehrcode 16 ab** (wie bei fehlendem Tkinter, fehlender Anzeige oder nicht ladbarem C-Modul), und läuft **nicht** ohne Ton weiter. Standardannahmen (nicht gefragt, nicht bestätigt): (1) Mit `--STUMM` wird kein Abspielprogramm gebraucht und nichts geprüft. (2) Die Prüfung erfolgt beim Start, zusammen mit den anderen Startfehlern, vor dem Aufblenden der Fenster. (3) Als „nicht nutzbar“ gilt: kein Abspielprogramm gefunden oder der Probeaufruf beim Start schlägt fehl; ein nicht erkennbares Audiogerät lässt sich nicht sicher feststellen.	03 (5 Nr. 11), 05 (F-17)
16.10.2026	**Dauernd laufender Tonstrom (Review F-17, Dialog, Vorschlag des Projektinhabers 16.10.2026, Entwurf 56):** Der Projektinhaber schlägt vor, **alle Töne ständig auszugeben** und nur zwischen **stumm und hörbar** umzuschalten, um die Anlaufverzögerung zu sparen. Daraus abgeleitet, nicht ausdrücklich bestätigt: (1) In GRAFIK ohne `--STUMM` wird beim Start **ein Ausgabestrom geöffnet, der bis zum Ende dauernd läuft** (bei Ruhe Stille); es wird **kein Abspielprozess je Klang** gestartet. Der Probeaufruf aus Nr. 11 ist das Öffnen dieses Stroms. (2) **Dauergeräusche** (Rechner-Grundgeräusch, Motor, Dauerstanzen) laufen als Schleife immer mit und werden nur über den Pegel (0 oder voll) stumm oder hörbar geschaltet; die Schleifenlage beim Einschalten ist beliebig. (3) **Einzelklänge** (Zeichen, TAB, WRZL, Leser) können nicht stumm mitlaufen, weil Beginn und Länge zum Ereignis gehören; sie werden **beim Ereignis in den laufenden Strom eingemischt**. Damit sind auch gleichzeitige Töne (Nr. 7) im Strom lösbar. (4) **Anforderung an die Verzögerung:** Ton und Zeichen im SW-Fenster erscheinen **im Mittel gleichzeitig**; der Ton setzt gegenüber dem Zeichen **höchstens 50 ms** später ein (Standardannahme; Zeichenabstand des Schreibwerks 100 ms). Das Zeichen wird nie wegen des Tons verzögert. Der Puffer des Stroms darf diese Grenze nicht überschreiten. (5) **Stummschaltung im Betrieb:** je Tonkanal (Rechner, Schreibwerk, Leser, Stanzer) ein **internes Tor**; es dient dem Selektor SW, Netz aus und dem Auslaufen der Dauergeräusche. **Kein neues Bedienelement** und keine neue Schalterstellung (Standardannahme; `--STUMM` bleibt Startschalter und öffnet keinen Strom). (6) Ausfall des Stroms im Betrieb und Umsetzung unter Windows: siehe Nr. 13 (Dok. 03 §5).	03 (5 Nr. 12), 05 (F-17)

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Datum	Entscheidung	Dokument
16.10.2026	**Ausfall des Tonstroms im Betrieb und Windows-Umsetzung (Review F-17, Dialog, 16.10.2026, Entwurf 57): (a) Fällt der Ausgabestrom in GRAFIK (ohne `--STUMM`) mitten im Lauf aus, bricht das Skript mit Fehlermeldung und Rückkehrcode 16 ab**, wie beim Start (Nr. 11); es läuft nicht ohne Ton weiter. Antwort „ja“ auf den Vorschlag. Standardannahmen (nicht gefragt, nicht bestätigt): Als Ausfall gilt, wenn der Abspielprozess endet oder das Schreiben in den Strom bzw. ein Aufruf der Windows-Tonschnittstelle fehlschlägt; der Abbruch läuft wie jeder andere Abbruch mit Code 16 (Trommeldatei und Protokoll wie dort). **(b) Windows (Vorschlag des Assistenten, noch nicht bestätigt):** `winsound` entfällt für GRAFIK. Der Strom wird mit der Windows-Schnittstelle `winmm` **(**`waveOut…`**) über** `ctypes` geöffnet; das ist Teil von Python, braucht keine zusätzliche Software und kennt gleichzeitig mehrere Puffer. Das Skript mischt die Klänge selbst zu Blöcken (44,1 kHz, mono, 16 bit) und reicht sie in einem Ring aus vier Puffern zu je 10 ms an die Schnittstelle (höchstens 40 ms Verzögerung, innerhalb der Grenze 50 ms aus Nr. 12). Unter Linux ist die Senke ein Abspielprogramm mit Roh-Audio über die Standardeingabe (`paplay`, `pw-cat` oder `aplay`), dasselbe Mischen. **(c) Vorbehalte:** Windows ist weder übersetzt noch getestet; ein Windows-Tester **kommt noch** (Antwort 16.10.2026, F-18). Das Mischen in Python würde den Interpreter belasten (GIL); deshalb ist das Mischen **gleich im C-Modul** vorgesehen (Nr. 14). Fehlt `winmm` oder schlägt das Öffnen fehl, gilt Nr. 11 (Abbruch mit Code 16).	03 (5 Nr. 13), 05 (F-17)
16.10.2026	**Mischer im C-Modul, statische Bibliotheken, Umwandlung MP3 → WAV** *(Review F-17 und F-18, Dialog, 16.10.2026, Entwurf 58)*: **(a) Windows-Tester:** Er „kommt noch“ (Antwort des Projektinhabers); bis dahin bleibt der Windows-Zweig ungetestet. **(b) Das Mischen kommt gleich in ein C-Modul** (Antwort „wenn möglich, gleich C-Modul vorsehen“), nicht erst nach einer Messung. Anforderungen: Das Modul `tonmisch` enthält Mischer (Dauerschleifen mit Pegel 0 oder voll, Einzelklänge, Ausgabe in Blöcken zu 10 ms), Ringpuffer und die **Senke** (Linux: Abspielprogramm mit Roh-Audio über eine Röhre, vom Modul gestartet; Windows: `winmm`/`waveOut…`). Es läuft in **einem eigenen C-Thread**, sodass weder die GIL noch das Python-Tempo den Ton stören. Die Schnittstelle zu Python bleibt schmal: Klangdaten übergeben, Ereignis auslösen, Pegel je Kanal setzen, Öffnen, Schließen, Ausfall melden (→ Nr. 13a). **(c) Statische Bibliotheken** (Wunsch „static libraries“): Das Modul wird wie die Trommel als **statische Bibliothek** gebaut (`libtonmisch.a`, unter MSVC `tonmisch.lib`) und **fest in das Python-Erweiterungsmodul eingebunden**, möglichst **zusammen mit der Trommel in einer einzigen Datei** (`.so` bzw. `.pyd`); es wird keine zusätzliche Bibliothek zur Laufzeit nachgeladen. Der Testrahmen unter Linux wird mit `-static` gebunden. Grenze (Standardannahme): Das Erweiterungsmodul selbst ist eine ladbare Datei; die Systembibliotheken (C-Laufzeit, unter Windows `winmm.dll`, die es nur als DLL gibt) bleiben dynamisch. **(d) Klangdaten:** Python liest die WAV-Dateien (Standardbibliothek `wave`) und übergibt die Rohdaten; das C-Modul kennt kein Dateiformat (Standardannahme, nicht gefragt). **(e) MP3 → WAV, einmalig umgewandelt** (Antwort „ja, einmal mundgerecht umwandeln“): Die fünf Aufnahmen liegen jetzt als WAV (44,1 kHz, mono, 16 bit; Stereo zu Mono gemittelt, unverändert im Pegel) in `ton/toene_rechner_stanzer/` vor: `rechner_grundgeraeusch.wav` (5,94 s), `netz_ein_fade_in.wav` (11,29 s), `netz_aus_fade_out.wav` (1,95 s), `ls_motor.wav` (4,96 s), `ls_dauerstanzen.wav` (18,06 s). Die MP3-Quellen liegen nicht mehr im Projekt (Ordner `ton/aufnahmen/` am 21.10.2026 entfernt); maßgeblich sind die WAV-Dateien. Zur Laufzeit wird nichts umgewandelt, ein MP3-Decoder ist nicht nötig. Die Längen weichen um bis zu 0,05 s von den MP3-Angaben ab (Decoderverzögerung); bei `fade_in` ist 11,3 s weiter ausreichend. Die WAV sind nur gemessen, **nicht angehört**.	03 (5 Nr. 14), 02 (4.5), 05 (F-17, F-18)

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Datum	Entscheidung	Dokument
16.10.2026	**Stanzertöne: Dauerstanzen, Einzelzeichen, Aufnahmen** *(Review F-17, Dialog, 16.10.2026, Entwurf 59; Entscheidung dem Assistenten überlassen, daher nicht ausdrücklich bestätigt)*: **Grundsatz (Hinweis des Projektinhabers):** Die Einzeltöne sind **viel länger als die Stanzzeit eines Zeichens** (20 ms bei 50 Zeichen/s) und passen nur näherungsweise. Der Stanzerton wird deshalb **nicht je Zeichen, sondern nach Zustand** gespielt; ein Ton je Zeichen würde sich überlagern. **(a) Zustände und Schwelle:** *Aus*: kein Ton. *Motor*: `ls_motor.wav` als Schleife (Einsatz und Ende nach Entwurf 52 und 53: bei LS immer, sonst bis 5 s nach dem letzten Zeichen). *Dauerstanzen*: `ls_dauerstanzen.wav` als Schleife, sobald **zwei aufeinanderfolgende Zeichen höchstens 40 ms Abstand** haben; Ende, wenn **60 ms** lang kein Zeichen kommt; dann zurück zu *Motor*. Die Zahlen 40 ms und 60 ms sind Standardannahmen (Stanzer 50 Zeichen/s = 20 ms Abstand; Schwelle = zwei Zeichenzeiten, Ende = drei) und mit der Hörprüfung zu überprüfen. **(b) Einzelzeichen:** Es gibt **keinen Einzelschritt-Ton** (D-8 entschieden: nicht gebraucht, es liegt auch keine Aufnahme vor); ein einzelnes oder langsam folgendes Zeichen hört man nur als Motor. **(c) Motor und Dauerstanzen:** Eine Messung der Aufnahmen (16.10.2026) ist nicht eindeutig: Das Motorgeräusch hat über 5 kHz fast nichts (81 dB gegen 146 dB bei Dauerstanzen) und unter 200 Hz mehr (135 dB gegen 124 dB); die Feinspektren unter 1 kHz stimmen mäßig überein (Korrelation 0,64). **Standardannahme (D-9):** `ls_dauerstanzen.wav` enthält den Motor schon; während *Dauerstanzen* wird der Motor-Pegel daher auf 0 gesetzt (Überblendung 50 ms), beim Zurückschalten umgekehrt. Falls die Hörprüfung (Nr. 8) das Gegenteil zeigt, laufen beide gleichzeitig; das ist nur eine Konstante im C-Modul. **(d) Umsetzung:** über die Tore des Mischers (Nr. 12 und 14), im C-Modul; die Schwellenzeiten liegen als Konstanten dort. **Berichtigt durch Nr. 21 (Entwurf 66): Schwelle, Ende, Motor und Dauerstanzen.**	03 (5 Nr. 15), 05 (F-17)
16.10.2026	**Echo über K0 mit Ton** *(Review F-17, Dialog, 16.10.2026, Entwurf 60)*: Das **Echo von Eingaben über K0** bekommt **denselben Ton** wie jede andere Ausgabe auf das Schreibwerk (Antwort „ja“ auf den Vorschlag). Begründung: Das Schreibwerk schreibt dort tatsächlich; die Gleichheit von TEST und GRAFIK (F-04) betrifft Ergebnisse, nicht den Ton. Es gelten Nr. 12 (Verzögerung), der Selektor SW und `--STUMM` unverändert. Standardannahme (nicht gefragt): Der Aufruf gehört an dieselbe Stelle wie bei der Ausgabe (`sw_schreibe`), sofern das Echo dort durchläuft; die Prüfung des Echo-Pfades im Interpreter steht beim Einbau noch aus. **Hinfällig seit 16.10.2026 (Entwurf 88): Das Echo gibt es in GRAFIK nicht** („Tastatureingaben sind nirgendwo zu sehen“); folglich auch kein Ton dafür.	03 (5 Nr. 16), 05 (F-17)
16.10.2026	**Tabulator, Schreibposition und rechter Anschlag** *(Review F-17, Dialog, 16.10.2026, Entwurf 61; Festlegung des Projektinhabers)*: **(a) Tabulatorstopp alle 12 Zeichen** (eingetragen, hardwareseitig). Gibt der Rechner ein TAB aus, geht die Schreibposition auf den **nächsten Stopp** (12, 24, 36 …). Beispiele des Projektinhabers: nach `<wrzl>Alle Vö` (7 Zeichen) macht TAB **5 Leerschritte**; `<Tab>12<Tab>` macht beim zweiten TAB **10 Leerschritte** (Position 14 → 24). **(b) Ton:** Es gibt **nur noch einen TAB-Ton**, gleich für jeden Tabulatorweg, **400 ms lang** (etwa drei normale Zeichen). Damit entfällt die Wahl kurz oder lang und der Aufrufer braucht keinen Tabulatorweg (Beschreibung §16 und D-Punkt zu TAB erledigt). Die Datei `ton/sw_toene/tab_400.wav` ist aus `tab_lang.wav` (325 ms) durch tonhöhentreue Dehnung auf 400,0 ms gemacht (letzte 10 ms ausgeblendet); nur gemessen, **nicht angehört**. `tab_kurz.wav` und `tab_lang.wav` bleiben als Quelle liegen, werden aber nicht mehr gespielt; der Parameter `tab_lang` von `sw_ton()` entfällt beim Einbau. **(c) Rechter Anschlag:** Wird **Position 130 überschritten, wird keine Bewegung der Schreibposition mehr ausgeführt**; das gilt **auch für normale Zeichen**. **(d) Annahmen des Assistenten,** vom Projektinhaber bestätigt am 16.10.2026 (Entwurf 62)**:** Position = Spalte 0 bis 130 seit Zeilenanfang; WRZL und WR setzen sie auf 0. Ein TAB, dessen Stopp hinter 130 läge (ab Position 120), bewegt die Position **bis 130** und nicht weiter. Das Zeichen hinter dem Anschlag wird **angeschlagen (Ton läuft, TAB 400 ms, Zeichen 120 ms)** und **im Fenster auf Position 130 überdruckt** (ersetzt dort das letzte Zeichen), es entsteht **kein Zeilenumbruch** mehr (bisher `wrap='char'` bei 130 Spalten). Die Schreibwerkzeit eines TAB ist **400 ms** (Wartezeit = Tonlänge, wie bei `sw_ton()` heute). **(e) Geltungsbereich (**bestätigt 16.10.2026, Entwurf 62: nur im SW-Fenster**):** Die Regel gilt für die **Darstellung im SW-Fenster in GRAFIK**. Die Schreibwerk-Datei (`m.sw_datei`) und die Terminalausgabe behalten das Zeichen unverändert (TAB bleibt ein TAB); die Vergleichsdateien in `Test-Ergebnisvergleiche/` (zwei enthalten TAB: `bsp_5_15_lochband_druck…`) und damit die Gleichheit von TEST und GRAFIK (F-04) bleiben unberührt. Das Fenster rechnet die Position selbst mit.	03 (5 Nr. 17), 01 (SW-Fenster), 05 (F-17)

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Datum	Entscheidung	Dokument
16.10.2026	**Aufrufstelle des Lesers** *(Review F-17, Dialog, 16.10.2026, Entwurf 63)*: `ls_ton()` wird **bei jedem Zeichen aufgerufen, das der Leser K1 oder K2 tatsächlich an den Rechner liefert** (Antwort „dein Vorschlag passt“). Der Projektinhaber bestätigt, dass der Interpreter dort **genau ein Zeichen je Zugriff** liefert; das erledigt die Prüfung, die beim Einbau offen war. Kein Aufruf bei Zeichen von Tastatur oder K0. Im Modus TEST kein Ton. Das Tempo bleibt bei **130 Zeichen/s**; Einzelschritt, Dauergeräusch und Auslaufen richten sich wie bisher nach dem Abstand der Zeichen (Schwelle 100 ms, noch im echten Lauf zu prüfen). Annahme des Assistenten (nicht gefragt): Im Tonstrom (Nr. 12 und 14) ist der Einzelschritt ein Einzelklang, das Dauergeräusch eine Schleife mit Tor und das Auslaufen ein Einzelklang (`ls_ende.wav`).	03 (5 Nr. 18), 05 (F-17)
16.10.2026	**Netz aus und** `fade_out` *(Review F-17 und F-02, Dialog, 16.10.2026, Entwurf 64)*: `fade_out` **(2 s) wird nur beim normalen Ausschalten mit dem Klick auf den Netzschalter gespielt** (Antwort „ja“ auf den Vorschlag); danach endet das Skript mit **Code 0**. Bei **Strg+C, X und Schließen des Fensters** (Code 8) hören **alle Töne sofort auf**: Tore zu, Tonstrom zu, **kein** `fade_out`. Beendet der Tonstrom dabei selbst den Prozess nicht, gilt Nr. 13 nicht als Ausfall (das Schließen ist gewollt). Damit ist D-5 erledigt.	03 (5 Nr. 19), 05 (F-02, F-17)
16.10.2026	**Wiedereinschalten** *(Review F-17 und F-02, Dialog, 16.10.2026, Entwurf 65)*: **„Wieder einschalten“ heißt, das Skript neu zu starten** (Antwort „ja“ auf den Vorschlag). Es beginnt wieder mit „Rechner aus“ und wartet auf den Klick auf den Netzschalter. Die Register sind danach weg, die Trommeldatei bleibt erhalten. Damit ist D-1 erledigt; im Skript gibt es kein Wiedereinschalten nach dem Ausschalten.	03 (5 Nr. 20), 05 (F-02, F-17)
16.10.2026	**Stanzerton, Berichtigung durch den Projektinhaber** *(Review F-17, Dialog, 16.10.2026, Entwurf 66; ersetzt Nr. 15 (a) und (c))*: **(a) Dauerstanzen beginnt mit dem zweiten Zeichen, das weniger als 50 ms nach seinem Vorgänger ausgegeben wird, und läuft, solange diese Bedingung erhalten bleibt** (ersetzt die Schwelle 40 ms). **(b) Ende von Dauerstanzen** (Annahme des Assistenten): Sobald 50 ms nach dem letzten Zeichen kein weiteres kommt, gilt die Bedingung nicht mehr; der Ton wird vom Zeitgeber beendet, nicht erst beim nächsten Zeichen. Er fällt dann auf das Motorgeräusch zurück (ersetzt die 60 ms). **(c) Abschalten:** Kommt **nach 5 Sekunden kein neues Zeichen**, wird der Ton abgeschaltet (der Projektinhaber schrieb „5?“; übernommen als 5 s, wie in Entwurf 52 und 53). **(d) Motor und Dauerstanzen sind das gleiche Geräusch:** Es gibt **einen** Stanzergeräusch-Zustand und nie zwei Geräusche gleichzeitig; während Dauerstanzen läuft, ist der Motor nicht zusätzlich zu hören (D-9 entschieden, die Messung der Aufnahmen ist damit gegenstandslos). **(e) LS gedrückt:** Der **Motor läuft dauernd**; das ist die **dominierende Bedingung**: Das Geräusch wird nie abgeschaltet, auch nicht nach 5 s. Auslegung des Assistenten (nicht gefragt): Auch bei gedrücktem LS wählt die Bedingung (a) weiter zwischen `ls_dauerstanzen.wav` und `ls_motor.wav`. Umsetzung unverändert über die Tore des Mischers (Nr. 12 und 14); 50 ms und 5 s sind Konstanten im C-Modul.	03 (5 Nr. 21), 05 (F-17)
16.10.2026	**Hörprüfung** *(Review F-17, Dialog, 16.10.2026, Entwurf 66)*: Vorschlag des Assistenten, vom Projektinhaber **nicht ausdrücklich bestätigt**, aber mit Korrekturen zu zwei Punkten der Liste beantwortet und deshalb so weitergeführt (bei Widerspruch ändern): Die Hörprüfung ist eine **Abnahme nach dem Einbau**, vom Projektinhaber am Rechner, unter Linux und später unter Windows; sie ist **kein Hindernis für die Freigabe zum Neuprogrammieren**. Danach werden als Konstanten angepasst: die 50 ms und 5 s des Stanzers, die 100 ms Schwelle des Lesers, der Klang von `tab_400.wav` und der Übergang von `fade_in` in die Grundgeräusch-Schleife. Nicht mehr Gegenstand der Hörprüfung: Motor und Dauerstanzen (Nr. 21 (d)).	03 (5 Nr. 22), 05 (F-17)
16.10.2026	**LS gedrückt und Abhören der Aufnahmen** *(Review F-17, Dialog, 16.10.2026, Entwurf 67; berichtigt die Auslegung in Nr. 21 (e))*: **Bei gedrücktem LS wird immer nur** `ls_motor.wav` **gespielt**; die 50-ms-Bedingung wählt dann nicht mehr zwischen den Aufnahmen, `ls_dauerstanzen.wav` wird bei gedrücktem LS nie gespielt. Der Projektinhaber hat **die Geräusche abgehört**; sie **scheinen in Ordnung zu sein**. Laufzeiten und Schwellen (50 ms, 5 s, 100 ms des Lesers) kann er **akustisch nicht stoppen**; sie bleiben Konstanten, die nach dem Einbau (Nr. 22) nachgestellt werden. Nicht ausdrücklich gesagt, welche Aufnahmen gemeint sind; `tab_400.wav` (neu in Entwurf 61) gilt daher weiter als nicht bestätigt.	03 (5 Nr. 23), 05 (F-17)

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Datum	Entscheidung	Dokument
16.10.2026	**Zielsysteme und Python-Version** *(Review F-18, Dialog, 16.10.2026, Entwurf 66)*: **macOS entfällt** (Antwort des Projektinhabers); Zielsysteme sind **Linux und Windows**; die macOS-Zweige im Code (`pbcopy`, `afplay`) werden im Neuprogrammieren nicht berücksichtigt. **Python-Mindestversion: 3.8 oder neuer** (passt zur Limited API des Erweiterungsmoduls). Die Tk-Version ist nicht festgelegt (Standardannahme: die mit Python gelieferte). **Auslieferung des C-Moduls: entschieden, siehe Eintrag Entwurf 67.**	05 (F-18)
16.10.2026	**Auslieferung des C-Moduls** *(Review F-18, Dialog, 16.10.2026, Entwurf 67; schließt Dokument 02 §8 Nr. 3)*: **Das Paket liefert die fertig übersetzten Dateien mit**; der Anwender braucht weder Compiler noch Python-Entwicklungsdateien (Antwort „so soll es sein“). **Die Dateien sind** `-static` **gelinkt** (Wunsch des Projektinhabers). Festlegungen des Assistenten dazu (nicht gefragt): (1) **Eine Datei je System** (Linux, Windows), die Trommel und Tonmischer (Dokument 03 §5 Nr. 14) enthält; Limited API, daher **unabhängig von der Python-Version ab 3.8**. (2) **Statisch** heißt: eigene Quellen, `libtrommel.a`, `libtonmisch.a`, unter Linux `libgcc`, unter Windows die C-Laufzeit (`/MT` bzw. MinGW `-static`) werden in die Datei gebunden. **Technische Grenze:** Unter Linux bleibt `glibc` dynamisch (ein Python-Erweiterungsmodul ist selbst eine ladbare Datei), unter Windows `winmm.dll`. Die Linux-Datei wird auf einem **älteren System** gebaut, damit sie auf neueren läuft. (3) **Architektur x86-64** (Standardannahme, nicht gefragt). (4) **Die Windows-Datei wird erst mitgeliefert, wenn der Windows-Tester sie geprüft hat** („kommt noch“); der Quelltext mit Makefile und `build_msvc.bat` bleibt im Paket zum Nachbauen, ist aber keine Voraussetzung. (5) Nicht ladbares Modul: weiter Fehlermeldung und **Code 16** ohne Rückfall (Entscheidung 14./15.10.2026).	02 (8 Nr. 3), 05 (F-18)
16.10.2026	**Hintergrund-Thread statt Hintergrundprozess** *(Review F-19, Dialog, 16.10.2026, Entwurf 68)*: Die Drehung der Trommel (und der Tonmischer, Dokument 03 §5 Nr. 14) laufen als **Hintergrund-Thread, ein C-Thread im Erweiterungsmodul, im selben Prozess** wie der Interpreter; **kein eigener Betriebssystem-Prozess** (Antwort „ja einverstanden“). Der Begriff „Hintergrundprozess“ wird in den Dokumenten 00 (Verzeichnis) und 02 durch „Hintergrund-Thread“ ersetzt; frühere Registerzeilen vom 13. und 15.10.2026 bleiben als Geschichte unverändert und gelten in diesem Sinn. Begründung: passt zu den statisch gelinkten Dateien und zu Strg+C; ein Prozess brächte nur Aufwand für den Datenaustausch.	02, 03 (5 Nr. 14), 05 (F-19)
16.10.2026	**Klick- und Tastenbedienung des Pults (Review F-15, Dialog, 16.10.2026, Entwurf 69)**: **Auslösung beim Drücken der Maustaste. Klick-Tasten** (B, +, 1, 3, 5, 7, 9, ·, :, IS, IB, G, Ü, D, −, 0, 2, 4, 6, 8, „,“, ZR, WM, ST): 0,4 s hell, dann normal. **Wippen H–R und T–SW:** einfacher Klick rastet die Taste ein und löst die andere, immer genau eine eingerastet, kein Aufleuchten, kein Umschalt. **E und M:** Klick 0,4 s, gehaltene Maustaste = gedrückt, **Umschalt+Klick = Halter** (zweiter Umschalt+Klick löst, E und M gleichzeitig möglich); Umschalt wirkt **nur** bei E und M. **Gesperrte Taste:** ein Klick bewirkt nichts (kein Aufleuchten, kein Ton, keine Weitergabe); beim Überfahren ändern sich Taste, Mauszeiger und Hinweistext nicht. **Touchscreen wird nicht unterstützt.** Fokusverlust: gehaltene Maustaste wird losgelassen, Rastungen bleiben, 0,4-s-Timer laufen aus. *Zweiter Klick innerhalb der 0,4 s zählt als neuer Tastendruck und startet die Zeit neu: Vorschlag des Assistenten, nicht ausdrücklich bestätigt.* Dokument 01 §1.7, Dokument 05 F-15.	
16.10.2026	**Computertastatur für die Eingabetasten (Review F-15, Dialog, 16.10.2026, Entwurf 69; ändert die Aufgabenteilung in Dokument 01 §1.6)**: Die Tasten `B + 1 3 5 7 9 · :` und `D − 0 2 4 6 8 , ZR` sind auch über die **gleichnamigen Tasten der Computertastatur** bedienbar, alle anderen Pult-Tasten nur mit der Maus. Zuordnung: Ziffern, + − , : wie auf der Computertastatur; „·“ = Punkt „.“; B und D = b und d (Groß/Klein gleich); ZR = Leertaste; Ziffernblock wie Hauptreihe. Wirkung wie ein Mausklick (Auslösung beim Drücken, 0,4 s hell), kein Wiederholen bei gehaltener Taste, dieselben Sperren, mehrere Tasten gleichzeitig mit je eigenem Timer. Die Computertastatur wirkt nur, wenn das **Bedienpult-Fenster den Fokus hat und der Rechnerzustand Eingaben zulässt**.	

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Datum	Entscheidung	Dokument
16.10.2026	**Fokus des Bedienpult-Fensters (Review F-16, Teilantwort, Dialog, 16.10.2026, Entwurf 69)**: Das Pult erhält den Fokus nach dem Programmstart und im laufenden Programm, wenn keine Eingaben im SW-Fenster anliegen (dort hat das SW-Fenster Vorrang); der Fokus verbleibt beim Pult. Wird er von Hand woandershin gesetzt, ist am Pult keine Eingabe möglich (weder Maus noch Computertastatur). Ein **Mausklick auf das Pult** bringt den Fokus zurück und **löst keine Taste aus**; für die Mausbedienung ist ein weiterer Klick nötig; die Computertastatur wirkt danach sofort, sofern der Rechnerzustand Eingaben zulässt. Eine Eingabe im SW-Fenster ist immer nur **ein Zeichen**, daher gibt es keine angefangene Eingabe beim Fokuswechsel.	
16.10.2026	**Rollensperre für T–SW (Review F-15 (b), Dialog, entschieden 16.10.2026, Entwurf 71):** Das Seitenabbild der Originaldokumentation „die Tasten E, M, G, Ü und der Schalter T-SW lassen sich nur bei Stellung H betätigen“ ist maßgebend; alles Abweichende wird angepasst. T-SW ist bei Stellung R gesperrt (Klick ohne Wirkung) und nur bei Stellung H bedienbar. **Ersetzt** den Eintrag vom Entwurf 49 („T ↔ SW unterliegt nicht der Rollensperre“) und die Aussage zur Interruptfähigkeit des Umschalters T ↔ SW (Entwurf 48)	01 (1.8 Nr. 20, 3.5, 7 Nr. 7 und 15), 05 (F-06, F-15), Pult-Doku 0.03-008
16.10.2026	**Zeichenabbildung im SW-Fenster (Review F-16, Dialog, entschieden 16.10.2026, Entwurf 72):** Ein- und ausgegeben werden können nur die Zeichen des R300-Codes; die Umschalttasten der Computertastatur gelten ganz normal (sie bestimmen das getippte Zeichen), die C-8205-Zeichen `Um. KB`/`Um. GB` werden selbständig im Zusammenhang mit dem Zeichen gehandhabt (Zeichenlage wird mitgeführt, das passende Umschaltzeichen geht dem Zeichen voraus, das Zeichen bleibt vorgemerkt und wird beim nächsten Eingabe-Halt geliefert, ohne erneutes Tippen). **[Überholt seit 18.10.2026 (Dok. 09 U-10, Checkpoint 42): Es gibt keine Vormerkung. Das Fenster sendet die zwei Eingaben im echten Tastenabstand; jede wird nur angenommen, wenn im Moment der Eingabe ein `011` aktiv ist, sonst ist sie blockiert (S3-11, S3-13, S3-18). Wortlaut bleibt zur Nachvollziehbarkeit stehen.]** Weitere Einzelheiten (Sondertasten, stiller Verwurf nicht angenommener Zeichen, Vormerkung, Fokus bei verweigerter Fokusübernahme) hat der Projektinhaber dem Entwickler überlassen, **Standardannahmen, zu erproben**	01 (1.7), 05 (F-16)
18.10.2026	**Zeichenlage bei der Eingabe am SW (Festlegung des Projektinhabers, GRAFIK G3-004):** Für die Eingabe am SW zählt nur die Zeichenlage des SW selbst (Vorzustand), nie die des Rechners; die Umschalttasten der Computertastatur werden nicht gesondert ausgewertet, das Fenster erkennt die Lage am Zeichen. Ein Umschaltzeichen geht nur bei Abweichung vom Vorzustand an den Rechner (dann zwei Eingabezeichen: Umschaltzeichen, Zeichen). Nach dem Einschalten ist der Vorzustand unbestimmt, das erste Zeichen setzt ihn. Ändert 01 (1.7 Nr. 2) und 05 (F-16); Test `testscripte/grafik_sw_pult_test.py` (Tasten des Bedienpults)	01 (1.7), 05 (F-16)
16.10.2026	**Review F-02 (Netzablauf, Schritt 2):** Der Projektinhaber bezeichnet die Fragen zu F-02 als bereits geklärt („das ist alles schon geklärt“); keine neue Entscheidung. Es gelten die eingetragenen Festlegungen (13.10., 15.10., 16.10.); wo die Dokumente nichts festlegen, gelten vorläufig die Standardannahmen in Dokument 05 (F-02), nicht gesondert bestätigt	05 (F-02), 01 (5, 7), 03 (5)
16.10.2026	**Startbelegung der Trommel in GRAFIK pflegen (Review F-03, Schritt 3):** Den Zustand der Trommeldatei vor einem GRAFIK-Lauf pflegt der **Projektinhaber selbst**; das Paket legt weder eine Vorlage `Trommelvorlagen/MP.txt` noch ein Zurücksetz-Skript an. Ein Schutz gegen Doppelstart bleibt nach der Entscheidung vom 15.10. („nicht vorgesehen“); eine Warnung bei nicht lauffähiger Trommel gibt es nicht (Standardannahme zu „Trommel so wie sie ist“, nicht gesondert gefragt)	02 (6, 8 Nr. 10), 05 (F-03)
16.10.2026	**Gleichwertigkeit TEST und GRAFIK (Review F-04, Schritt 4):** Bei gleichem Programm müssen TEST und GRAFIK **dieselben Ergebnisse** liefern (Registerinhalte, Ausgaben). Ausnahmen nur die Zeiten (echte Trommel in GRAFIK) und die Befehlsentnahme nach X-Abbruch (`Überblick_und_Gesamtaufbau.md`, 4.1); sonstige Abweichungen gelten als Fehler des GRAFIK-Zweigs. **Prüfweg nicht entschieden** (nur von Hand abnehmen oder Einspielmodus für Pult-Klicks): Der Projektinhaber antwortete „beide TEST und GRAFIK müssen die gleichen Ergebnisse liefern“ und wählte keinen der beiden Wege; vorläufig Handabnahme (Standardannahme)	00 (3.6), 04 (4), 05 (F-04)

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Datum	Entscheidung	Dokument
16.10.2026	**Lampenbild ruhig** *(Durchgang O-5, Dialog, 16.10.2026, Entwurf 84)*: Wortlaut des Projektinhabers: „Lass die Lampen so leuchten, dass das Bild ruhig ist.“ Im Dauerlauf R sollen die Lampen also **nicht mit jeder Momentaufnahme springen oder flackern**. Das ergänzt die Festlegung „höchstens alle 40 ms den letzten Zustand“ (Dokument 01 §6.2); der 40-ms-Takt bleibt, er betrifft nur die Anzeige. Bei Halt und nach E-Schritt gilt weiter sofort der genaue Stand. **Wie** das Bild beruhigt wird, ist dem Entwickler überlassen. **Standardannahme (nicht gefragt, nicht bestätigt, jederzeit zu ändern):** Mittelung statt Momentaufnahme; eine Lampe leuchtet, wenn sie im Fenster der letzten 40 ms überwiegend an war (Schwellen 60 % an, 40 % aus, dazwischen bleibt der bisherige Zustand); nur die beiden vorhandenen Grafiken (hell, dunkel), keine dritte Grafik; die Zwischenabtastung ist zeitgesteuert und bremst die Befehle nicht. Abnahme: Handabnahme am laufenden Fenster, Dauerlauf R, Bild ruhig.	01 (1.5, 6.2, 7 Nr. 8), 05
16.10.2026	**Trommel: vier Punkte aus Dokument 02 §8** *(Durchgang O-7, Dialog, 16.10.2026, Entwurf 84)*: **(1)** Gesperrte Schreibzugriffe verbrauchen ihre Wartezeit (Vorschlag, nicht widersprochen). **(2)** Die Beobachtungsvariable `trommel_aktuelle_sektornummer` bleibt lockfrei (Vorschlag, nicht widersprochen). **(3)** Es gibt **keinen Schalter für eine feste Startlage**; nach dem Einschalten liegt die Trommel beliebig, wie am echten Gerät (Vorschlag, nicht widersprochen); `trommel_einschalten_start` bleibt nur für Testrahmen. **(4) Kein aktives Warten** (Spin). Antwort des Projektinhabers: „kein aktives Warten, alle 20 µs prüfen reicht“ (zuerst als „20 ms“ gelesen; gemeint sind **Mikrosekunden**, berichtigt am 16.10.2026). Der Hintergrund-Thread schläft immer. Solange ein Auftrag ansteht, prüft er die Uhr in Abständen von **höchstens 20 µs**; ein Tickbeginn darf also bis etwa 20 µs spät erkannt werden (eine Wortzeit sind 104 µs). Die Tickzahl und damit die Wartezeit in Wortzeiten (0 bis 31) bleiben exakt, weil der Takt eine Funktion der Uhr ist und übersprungene Ticks nachgeholt werden (Dokument 02 §4.3). Ohne anstehenden Auftrag ändert sich nichts. **Folgen:** Linux: der Zweig spinnt nicht, `clock_nanosleep` bleibt; die in Dokument 02 §5.1 gemessene mittlere Aufwachverspätung von 25 µs liegt knapp über dem Wert und berührt die Tickzahl nicht. Windows: Das aktive Warten entfällt; ein Schlafen in 20-µs-Schritten ist dort nicht zu erwarten (Zeitgeberraster im Millisekundenbereich, der Code setzt `timeBeginPeriod(1)`), der Thread prüft so oft, wie das System es zulässt; die Echtzeit-Dauer eines Zugriffs kann dadurch länger werden, die Wartezeit in Wortzeiten nicht. **Ungetestet.** Der Code (`plattform.h`, Windows-Zweig, Schalter `req_wartet`) ist **noch nicht angepasst**.	02 (4.4, 5.3, 6, 8), 05
16.10.2026	**Windows: Compiler MinGW-w64** *(Durchgang O-8, Dialog, 16.10.2026, Entwurf 85; schließt Dokument 02 §8 Nr. 6, soweit es den Compiler betrifft)*: Vorschlag war Visual Studio (MSVC) für die ausgelieferte Windows-Datei; Antwort des Projektinhabers: „ja, aber ich habe nur MinGW-w64 installiert“. **Entschieden: Windows wird mit MinGW-w64 gebaut** (`mingw32-make`, Erweiterungsmodul mit MinGW-w64, ausgelieferte Datei `-static`). Das ist auch der Compiler, mit dem das Projekt den Assembler unter Windows baut (`projekt/Assembler/make_c82ass.bat`). `build_msvc.bat` bleibt unverändert im Paket, ist aber **nicht der vorgesehene Weg** und nicht Teil der Auslieferung. Die übrigen Punkte des Vorschlags sind mit „ja“ übernommen: **der Windows-Tester baut und testet selbst** und meldet das Ergebnis zurück (er „kommt noch“, Entwurf 58); hier gibt es keinen Windows-Compiler. **Folgen, aus den Bauskripten gelesen (nicht gebaut, nicht getestet):** (a) `setup.py` und das Makefile setzen **kein** `-static`, obwohl die Auslieferung statisch gelinkt sein soll (Entwurf 67); anzupassen. (b) Mit nur MinGW-w64 muss der Compiler beim Bau des Erweiterungsmoduls ausdrücklich gewählt werden (`python setup.py build_ext --inplace --compiler=mingw32`), weil setuptools unter Windows sonst MSVC sucht (nach meiner Kenntnis von setuptools, zu prüfen). (c) **Risiko für „eine Datei für alle Python ab 3.8“ (Entwurf 67):** Die Zeile `#pragma comment(lib, "python3.lib")` in `Python.h`, die bei Limited API gegen die versionsunabhängige Bibliothek bindet, gilt nur für MSVC; unter MinGW-w64 wird das Modul voraussichtlich gegen die versionsgebundene Bibliothek (`python3xx`) gelinkt, die Datei gälte dann nur für die Python-Version, mit der gebaut wurde. Vermutung, beim ersten Bau zu prüfen (Linkziel `python3.dll` anstreben). (d) Hier war kein MinGW-w64-Compiler vorhanden und ließ sich nicht beschaffen; der Windows-Zweig bleibt **ungetestet** (Dokument 02 §5.3).	02 (5.3, 6, 8, 9), 00 (3.2), 05

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Datum	Entscheidung	Dokument
16.10.2026	**Bahnsperrenschalter: nicht anzeigen, Schalterdatei** *(Durchgang O-9, Dialog, 16.10.2026, Entwurf 86; schließt Dokument 01 §7 Nr. 11)*: **(1)** Die rückseitigen Bahnsperrenschalter erscheinen **gar nicht** im Pult-Fenster, keine Zusatzleiste neben dem Foto. **(2)** Sie werden über `--BASP` des GRAFIK-Interpreters gesetzt und beim Hochfahren der Trommel an das C-Modul übermittelt. **(3)** Neue Aufgaben des C-Moduls: übermittelte Stellungen in einer **Datei neben der Trommel speichern**; werden keine übermittelt, die Stellung **aus dieser Datei entnehmen**; existiert die Datei noch nicht, mit `000000` **anlegen** und `000000` übernehmen. Ersetzt den Standard `000000` ohne Angabe (15.10.2026) und die Aussage „nie eine Konfigurationsdatei“ in `bahnsperre.h`. Dateiname, Format und Fehlerfall als Standardannahmen in Dokument 02 §6, **bestätigt 16.10.2026 (Entwurf 87, „ja, alles ok“)**	01 (3.4, 7), 02 (3, 6), 04, 05 (O-9)
16.10.2026	**Töne: Standardannahmen bestätigt, Echo entfällt** *(Durchgang O-10, Dialog, 16.10.2026, Entwurf 88)*: Die Annahmen zu Dokument 03 §5 Nr. 12 bis 14, 17, 21 und 22 sind bestätigt (Einzelheiten Dokument 03 §5 Nr. 24). **Das Echo der Eingaben gibt es in GRAFIK nicht** („Tastatureingaben sind nirgendwo zu sehen“): kein dunkelgelbes Echo im SW-Fenster, kein Ton dafür; Nr. 16 hinfällig. **[überholt am 18.10.2026, Entwurf 104: das SW-Fenster zeigt jedes angenommene Zeichen selbst an, Dok. 09 S3-04 neu]**	03 (5 Nr. 16, 24), 01 (1.6), 05 (O-10)
18.10.2026	**SW: Eingabe wird angezeigt, Sperre bei K0 weg** *(Dialog, 18.10.2026, Entwurf 105)*: „Wenn im SW ein Zeichen eingegeben wird, dann muss es dargestellt werden, nur mit dem 011-Befehl.“ („JA, NUR DAS IST RICHTIG.“) **Ersetzt** „ohne Echo“ (Entwurf 88). Ist der Kanal K0 da, wird eingegeben und angezeigt; ist er weg, ist die Taste blockiert (keine Eingabe, keine Anzeige); **es gibt keine wartenden Zeichen** und **kein Echo** beim C 8205; eine Schreibposition wird durch eine Eingabe oder eine Ausgabe belegt und rückt weiter; „Programmende“ als eigene Regel entfällt. **Zeichen in der anderen Lage (07:56):** Die Tastatur sendet zwei Signale (Umschalten, Zeichen, etwa 0,1 s Abstand); der Rechner bekommt zwei Eingaben und durchläuft zwei `011`; ist für die zweite kein `011` aktiv, ist sie blockiert (bei korrektem Eingabeprogramm kommt das nicht vor). Kein Warten, keine Vormerkung.	09 (S3-04, S3-13, S3-18, §7, §10), 05 (F-48 bis F-51)
16.10.2026	**Klick und SW-Eingabe: Standardannahmen bestätigt** *(Durchgang O-11, Dialog, 16.10.2026, Entwurf 89, „alles bestätigt“)*: F-15 Nr. 1 (zweiter Klick innerhalb 0,4 s = neuer Tastendruck, Timer neu) und F-16 Nr. 3 bis 6 (Sondertasten, stiller Verwurf nicht angenommener Zeichen, Vormerkung, Fokus mit Titel „Eingabe erwartet“)	05 (F-15, F-16, O-11), 01 (1.6, 1.7)
16.10.2026	**Pflege (Durchgang O-12, Dialog, 16.10.2026, Entwurf 90):** `node.js`-Satz in `projekt/Assembler/Dokumentation/C8205_Assembler_Dokumentation.md` **entfernen** (zuerst nur die `.md`, am 16.10.2026 auf Anweisung auch in der maßgeblichen `.odt`, Entwurf 91); **PDF werden nicht geändert**; keine Anforderungs-IDs („mach was du willst“: Register und Teildokumente bleiben); Überschrift und Punkt 3 in §3.6 bereinigt; Aufnahmelänge des Grundgeräuschs 6,0 s (Schleife, endlos)	00 (3.6, 7), 03, 05 (O-12)
16.10.2026	**Vorgehen beim Neuprogrammieren: fünf Stufen** *(Durchgang O-13, Dialog, 16.10.2026, Entwurf 93, „so wollen wir vorgehen“)*: Stufe 1 Kern und Trommel (ohne Fenster, ohne Ton), Stufe 2 Pult-Fenster, Stufe 3 SW-Fenster, Stufe 4 Töne, Stufe 5 Umschaltung `--MODUS` und Aufräumen; Paket und Abnahme nach jeder Stufe; bis Stufe 5 eigenes Startskript für GRAFIK, TEST unberührt. Freigabe für den Beginn von Stufe 1 noch nicht ausdrücklich erteilt *(Stand 18.10.2026, Entwurf 120: Stufen 1, 2 und 3 abgenommen)*	04 (6), 05 (O-13)
17.10.2026	`LOCHSTREIFENKORB` **in jedem Leser-Ordner** *(Dialog, 17.10.2026, Entwurf 94; zu Stufe 1)*: In **jedem Leser-Ordner** gibt es einen Unterordner `LOCHSTREIFENKORB`, **durchgehend in Großbuchstaben** (bestätigt schon 16.10.2026, Entwurf 35: gelesene Dateien wandern dorthin). **Leere Korb-Ordner werden nicht mit ins Paket gelegt**; der Leser legt sie bei Bedarf selbst an (so ist es im Programm). Eine gewöhnliche **Datei** namens `LOCHSTREIFENKORB` im Leserordner: **kann bleiben** wie bisher (wird beim Suchen wie ein Lochstreifen behandelt, beim Verschieben Fehlermeldung); keine Änderung	07 (3a)
17.10.2026	**Tastenzeit 0,25 s** *(Test des Projektinhabers, 17.10.2026, Entwurf 96; ändert 15./16.10.2026)*: Alle Tasten mit Aufleuchten (Klick-Tasten, E, M, G, Ü, ST, Eingabetasten) zeigen die gedrückte Grafik **0,25 s** statt 0,4 s; weiterhin einstellbar. Dokument 01 §1.7, Dokument 06 angepasst; die älteren Zeilen dieses Registers mit „0,4 s“ gelten dadurch nicht mehr	01 (1.7), 06

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Datum	Entscheidung	Dokument
17.10.2026	**Kanaltasten K0 bis K3 rasten ein, immer genau eine** *(Test des Projektinhabers, 17.10.2026, Entwurf 96; ersetzt Entwurf 76 und 77)*: Radio-Verhalten, auch bei Stellung R; Start: K0 eingerastet (vorher: keine Kanaltaste rastet). Dokument 01 (§1.3, §1.7, §1.8 Nr. 3), Dokument 06 angepasst	01 (1.3, 1.7, 1.8), 06
17.10.2026	**Töne des Rechners schon in Stufe 2** *(Test des Projektinhabers, 17.10.2026, Entwurf 96; zieht Teile von Stufe 4 vor)*: `fade_in` beim Einschalten (B rot), Grundgeräusch als Schleife ab B grün, `fade_out` beim Ausschalten, alles aus bei X und Strg+C; `--STUMM` ohne Ton. Schreibwerk-, Leser- und Stanzertöne bleiben Stufe 4. Nicht hörbar geprüft	03, 04 (6), 08
17.10.2026	**Eingabetasten bei Stellung R** *(bestätigt 17.10.2026, Entwurf 96)*: Die Eingabetasten sind nicht mechanisch gesperrt, sondern nur elektronisch verriegelt: Sie lassen sich klicken, **wenn der Rechner gerade auf eine K0-Eingabe wartet**, sonst nicht. Ändert die Zeile „Rollensperre und Entriegelung“ (15.10.2026, „R verriegelt gleichzeitig die Eingabetasten“) und Dokument 01 §3.5	01 (3.5)
17.10.2026	**Thread-Regel im Pult-Betrieb (Fehlerbehebung, Entwurf 96):** Pult-Tasten werden im Rechner-Thread ausgeführt, nie im Fenster-Thread (E im Eingabe-Halt blockierte das Fenster); Strg+C zweimal beendet sofort	08
17.10.2026	**Hardware-Check-Fenster** `--HWCHECK` *(Dialog, 17.10.2026, Entwurf 97)*: zweites, verschiebbares Fenster in Mindestgröße mit BZ, BR (AR-Teil und OR), AC und <<AR>> (Speicherinhalt an der Adresse in AR), alles oktal, AR/AC/<<AR>> in Vierergruppen; bleibt aufgeblendet, solange der Grafik-Interpreter läuft (nicht schließbar); ohne Rückwirkung auf den Rechner. Standardannahmen in Dokument 01 §1.9 Nr. 5, nicht bestätigt	01 (1.9), 08
17.10.2026	**HW-Check-Fenster um Spurnummer und Sektornummer erweitert** *(Dialog, 17.10.2026, Entwurf 98)*: neue Zeile „Spur Sektor“ zwischen BR und AC (Adresse im Speicheradressregister AR; Spur = AR ÷ 32, Sektor = AR mod 32, oktal 3 und 2 Stellen); jetzt fünf Zeilen. Auslegung Dokument 01 §1.9 Nr. 5 (f), nicht bestätigt	01 (1.9), 08
17.10.2026	**Wirkung von E und M+ E im Halt** *(Dialog, 17.10.2026, Entwurf 99; **berichtigt 18.10.2026**)*: Erste Fassung („jedes E erhöht BZ um 1, unabhängig vom Trommelinhalt“) **gestrichen**. Es gilt, **wie im Interpreter programmiert** (0.03-015, GRAFIK G2-012): E ohne M führt den Befehl aus (Niederdrücken `E+` = Phase IV und I, Loslassen `E-` = Phase II und III); unter M erhöht das Niederdrücken BZ und AR um 1 (AR := BZ), AC-Adressfeld = BZ+1, Lampenreihe AR zeigt AR; Loslassen unter M ohne Wirkung. Dokument 01 §1.10 neu gefasst	01 (1.10), 06 (M-35), 08
17.10.2026	**HW-Check-Fenster: Vierergruppen von links** *(Test des Projektinhabers, 17.10.2026, Entwurf 99; ersetzt „von rechts“ aus Entwurf 97)*: AC und <<AR>> (11 Stellen) werden als **4 + 4 + 3** gruppiert, z. B. `2206 0635 124`, Null als `0000 0000 000` (vorher `220 6063 5124`, `000 0000 0000`). Platzhalter bei nicht lesbarem Speicher bleibt `---- ---- ---`. Dokument 01 §1.9 Nr. 5 (a) berichtigt	01 (1.9), 08
20.10.2026	**Verzeichnisse der Geräte neu geordnet:** `virtuelle_Geraete/` heißt jetzt `Test-virtuelleGeraete/` und gehört nur dem TEST-Interpreter (`Leser1/`, `Leser1_02/`, `Leser2/`, `Stanzer1/`, `SW-Ausgabe/`, `T-SW-Eingabe/`). Neu ist ein leeres `virtuelle_Geraete/` mit `Leser1/`, `Leser2/`, `Stanzer1/` als Standardordner von GRAFIK. Die Trommel liegt in `Trommel/` (`trommel.txt`, `trommel.basp`, `trommel_r300_eingabe.txt`) und ist für beide Interpreter gemeinsam; die Einträge vom 15.10.2026 gelten mit dem neuen Pfad `Trommel/trommel.txt`	Projektstruktur
21.10.2026	**E+ ohne M zeigt BZ in der Lampenreihe AR; G bei gehaltenem E löscht nur den Überlauf** *(Projektinhaber, 21.10.2026, OP-10 und OP-11)*: (1) Es wird unterschieden: `E+` ohne M+ zeigt im gehaltenen Zustand den BZ in den AR-Lampen, `E+` mit gehaltenem M+ erhöht AR und zeigt es an. (2) Nach Handbuch 4.1 Nr. 2 löscht G bei gehaltenem E nur den Überlauf; das ist allein richtig. Umsetzung in §3.8 und §10.5 (OP-10, OP-11); Fortsetzung des Halts wegen Überlauf am 21.10.2026 bestätigt (OP-41)	Dialog 21.10.2026
21.10.2026	**Eingabeklang beim Tastendruck** *(Projektinhaber, 21.10.2026, OP-09, Variante b)*: Der Klang einer Eingabe kommt, sobald das Zeichen im Puffer ist, auch wenn der Rechner es noch nicht angenommen hat; die Anzeige kommt weiter erst bei der Annahme. Umsetzung in §5.5, §6.2, §6.10 Nr. 3 und §10.5 (OP-09). Klarstellung: Der Ein-Zeichen-Puffer, von dem der Projektinhaber sprach, ist der des echten Geräts; der Software-Puffer (8 Eingaben) bleibt	Dialog 21.10.2026

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Datum	Entscheidung	Dokument
27.10.2026	**Tempo der Knotenausführung (G4-017):** Der Knotengraph wird weiter wortlautgetreu aus der `.md` gelesen, aber beim Start einmal in eine Tabelle übersetzt (`Rechner._baue_plan`: Konnektor-Sprünge, Funktionen, Ausgänge); Normalknoten laufen ohne Neuauflösung. Knoten 2, Konnektor mit BR-Zeile, Endknoten, Knoten ohne Funktion und alle Knoten bei `--knotentest` laufen unverändert über `exec_node_lang`, der Ausgabe-Halt ohne Gerät über `_ausgabe_halt`. Verhalten, Zeitgeber (`_tick`) und Knotenzahl unverändert; gemessen rund 1,5-faches Tempo bei einer Rechenschleife (9,0 s → 5,9 s). Dasselbe in `lauf()` des TEST-Interpreters (0.95-012, dort 1,85-fach). Die Zeichenlage KB/GB des Schreibwerks bleibt in GRAFIK zufällig; nur der TEST-Interpreter setzt sie fest auf KB, und `grafik_gleichwertigkeit.py` setzt sie für den GRAFIK-Vergleichslauf ebenfalls auf KB.	
27.10.2026	**Tastengeräusche des Pults (G4-018, G2-015,** `grafik_pultton.py` **G4-002; Klickpegel 1,76; Grundgeräusch-Pegel 0,8 auf 0,68,** `grafik_rechnerton.py` **G4-005):** Drei Klänge für die drei Tastentypen (Impulstaste, Einrastschalter und K0 bis K3, Radiotasten), nur bei wirksamem Klick, **kein Ton beim Loslassen** (Projektinhaber 27.10.2026), Gruppe Pult von `--TON`, Pegel 1,6 gegen das Grundgeräusch; Einzelheiten §6.12. Hörprüfung offen.	
27.10.2026	**G stellt den Kanal der eingerasteten Kanaltaste ein (G4-019, Fehlerbehebung; Projektinhaber 27.10.2026):** Nach G galt immer K0, auch wenn K1, K2 oder K3 eingerastet war. Jetzt gilt der Kanal der Taste; nur wenn K0 eingerastet ist (Startzustand), gilt K0 mit T oder SW. Umsetzung `Rechner.kanaltaste` (c8205_grafik.py); Beschreibung in §3.3 bei „Die Taste G“. Prüfungen `grafik_kern_tests.py` (Klasse `GKanal`, 5).	
29.10.2026	**Vorzustand des SW nach dem Einschalten ist KB (Festlegung des Projektinhabers, G4-010):** ersetzt den unbestimmten Vorzustand vom 18.10.2026 (G3-008). Erstes Zeichen `a`, `1` ohne Umschaltzeichen; `A`, `!` mit `Um. GB`. Ändert §5.4; `grafik_sw.py` G4-010, `testscripte/grafik_sw_tests.py` (92 Tests)	§5.4
30.10.2026	**Zweiter Lochbandleser-Typ, Schalter** `--LS-LESER-TYP=303|FS751` **(G4-022,** `grafik_geraeteton.py` **G4-006,** `c8205_interp.py` **0.95-015; Auftrag des Projektinhabers):** 303 = Lochbandleser Modell 303 (elektrisch über Metallbürsten, 130 Zeichen/s, Standardleser, Standard des Schalters), FS751 = optoelektronisch, 750 Zeichen/s. Der Schalter gilt immer für beide Leser, im Modus TEST ist er erlaubt und ohne Wirkung, der Ton bleibt zunächst gleich. Der Auftrag schrieb den Typ einmal „FS751“, einmal „FS715“; angenommen werden beide Schreibweisen, die Doku nennt FS751 (Bestätigung des Namens steht aus). Nicht am Gerät mit Ton geprüft.	
07.11.2026	**Leserton für FS751 (G4-024,** `grafik_geraeteton.py` **G4-007; Auftrag des Projektinhabers, Aufnahme** `lochstreifenleser_optisch.wav`**, 5 s, Stereo, 44,1 kHz, hochgeladen):** Der Leser FS751 klingt anders als der 303. Aus der Aufnahme (gleichmäßiges Laufgeräusch mit etwa 0,3 s Anlauf und etwa 0,5 s Auslauf) wurden drei Mono-Klänge erzeugt (Mittel beider Kanäle): `ls_fs751_einzel.wav` (148 ms aus dem Anlauf, 5 ms ein- und ausgeblendet), `ls_fs751_dauer.wav` (3,55 s aus dem gleichmäßigen Teil 0,6 bis 4,2 s, Naht mit 50 ms Überblendung schleifenfähig) und `ls_fs751_ende.wav` (0,55 s, Auslauf der Aufnahme); alle drei Klänge auf 70 % abgesenkt (Lautstärke um 30 % verringert, Wunsch des Projektinhabers; Faktor in den Dateien, nicht im Pegel von `--TON`). Ablauf, Zeiten und Tempo unverändert (Einzelton, Dauerton, Aus nach `LS_AUS_S`); `--LS-LESER-TYP=303` spielt weiter `ls_einzel.wav`, `ls_dauer.wav`, `ls_ende.wav`. Die Aufteilung der Aufnahme in drei Klänge ist eine Annahme von Claude (Bestätigung steht aus). Nicht am Gerät mit Ton gehört.	
30.10.2026	**Tasten im Halt (G4-023, Fehlerbehebung; Projektinhaber 30.10.2026):** Mit `H` (Rechner im Halt) getippte Zeichen klangen zwar, wurden aber gepuffert und erschienen nach `R` alle im SW-Fenster. Jetzt: im Halt ohne aktiven `011` nur der Klang, nichts gepuffert, vorgepufferte Zeichen verfallen (§5.3, §5.5); mit aktivem `011` (Einzelschritt) wird angenommen. `grafik_sw.py` G4-011, `c8205_grafik.py` G4-023, Prüfungen `testscripte/grafik_sw_tests.py` (Klasse `HaltZustand`).	

<!-- REGISTER-ENDE -->

Anhang B Übersetzungstabelle alt → neu

B.1 Dokumente

Altdokument / Gegenstand	Ziel (Abschnitt)
00 Hauptdokument	§2 Register → Anhang A der GRAFIK_Beschreibung; §3.1 bis §3.5 → Beschreibung §9, Bestandsdokumente; §3.6 → Beschreibung §1; §4 offene Entscheidungen → Offene Punkte; §6 Änderungsverzeichnis → entfällt (Entwurfsverlauf, siehe S6-09)
01 Pult-Fenster	→ Beschreibung §3, §8, §9; Offenes → §10
02 Trommel	→ Beschreibung §7; Testergebnisse → Prüfverfahren §7
03 Ton	→ Beschreibung §6; Bezug zu `ton/SW_Ton_Beschreibung.md`
04 Prüfbericht Module	Befunde zur Modulaufteilung → Beschreibung („Aufbau der GRAFIK-Module“, Kurzfassung in §1 oder eigener Abschnitt, Entscheidung O6-4); Rest entfällt
05 Review (F-01 bis F-23 und folgende)	Entscheidungen stehen im Register; **offene** Befunde → Offene Punkte; Rest entfällt
06 Mausbedienung	→ Beschreibung §4 (M-15 bis M-35 mit Nummern)
07 bis 09 Stufen 1 bis 3	Anforderungen und Ergebnisse → Beschreibung; Abnahmekriterien und echte Läufe → Prüfverfahren §8; Handliste Dok. 09 §9 → Prüfverfahren §7
10 Stufe 4	Anforderungen S4-xx → Beschreibung §6; Abnahme, Hörprüfung, B-2 → Prüfverfahren §7, §8, §9
11 Stufe 5	S5-xx → Beschreibung §1, §2; A5-x → Prüfverfahren §8; A5-8 → §9
12 (dieses)	nach Abschluss von Stufe 6 entfällt es mit den übrigen (Ergebnis steht im Prüfverfahren)
`CHECKPOINT_Stufe2.md`, `CHECKPOINT_Stufe3.md`	**Triage** (S6-04): „NICHT geprüft“-Abschnitte und Warnungen prüfen; durch spätere Abnahme erledigte entfallen, offene und Warnhinweise → Offene Punkte bzw. Hinweise
`CHECKPOINT_G5-001` bis `G5-010`	Offene Punkte und „Nicht geprüft“ → Offene Punkte; sonst entfallen (Verlauf steht in Dokument 00)
`AENDERUNG_…0_03-015.md`, `…0_03-016.md`	Inhalt → `C8205-Versionierung.md` (S6-03); **erst danach löschen**
`HWCHECK_HW-003.md`	Ursachen und Behebung → Beschreibung §8 (kurz)
`ton/SW_Ton_Beschreibung.md`	**bleibt** (Teil B bis D); Teil A bekommt den Hinweis, dass die Anforderungen in die GRAFIK_Beschreibung §6 übernommen sind; Verweise auf Dok. 01 bis 03 umstellen
`Pult_Tastentest_GRAFIK.md`	wird Prüfverfahren §5; die Datei entfällt danach, die Verweise werden umgestellt

B.2 Kennungsfamilien

Gezählt per Textsuche in Dok. 00 bis 12, `CHECKPOINT_*`, `AENDERUNG_*`, `HWCHECK_HW-003.md`, `ton/SW_Ton_Beschreibung.md`, `Pult_Tastentest_GRAFIK.md`. „Erwähnungen“ sind Fundstellen, nicht verschiedene Nummern. Die Spalte „Zielabschnitt“ ist ein **Vorschlag** und wird beim Schreiben je Nummer bestätigt (Familien mit unsicherer Bedeutung sind so vermerkt). Dokument-/Abschnittsnummern der Ziele: Beschreibung §n, Prüfverfahren §n.

Familie	Bedeutung	Bereich	Erwähnungen	Fundstellen (Auswahl, Anzahl)	Zielabschnitt (Vorschlag)
F-	Review-Befunde (Dok. 05) und Entscheidungsdatei; **F-15 und F-16 doppelt belegt** (Review-Befunde und Entscheidungsdatei F-15 bis F-33), im Register unterscheidbar nur über Datum und Text	F-01 bis F-65	683	Dok. 00 (281), Dok. 05 (171), CP Stufe3 (85), Dok. 09 (60), Dok. 01 (27), Dok. 03 (24)	Entschiedenes: Register (Anhang A) und das jeweilige Sachthema in §1 bis §9; Offenes: §10. In B.1 nur als Bereich F-01 bis F-65 geführt, Feinzuordnung beim Schreiben
M-	Mausbedienung (Dok. 06, M-15 bis M-35 und weitere)	M-15 bis M-35	77	Dok. 06 (51), Dok. 00 (8), Dok. 12 (6), Dok. 01 (3), Dok. 05 (3), CP Stufe2 (2)	Beschreibung §4

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Familie	Bedeutung	Bereich	Erwähnungen	Fundstellen (Auswahl, Anzahl)	Zielabschnitt (Vorschlag)
S3-	Anforderungen Stufe 3 (Dok. 09: SW-Fenster, K0-Eingabe)	S3-01 bis S3-18	311	Dok. 09 (136), CP Stufe3 (81), Dok. 00 (51), Dok. 05 (18), Dok. 10 (13), Dok. 01 (10)	Beschreibung §5; Handlisten und Abnahme: Prüfverfahren §7 und §8
S4-	Anforderungen Stufe 4 (Dok. 10: Töne)	S4-01 bis S4-19	124	Dok. 10 (86), Dok. 00 (15), CP Stufe3 (15), Dok. 12 (3), SW_Ton (2), Dok. 05 (1)	Beschreibung §6; Abnahme und Hörprüfung: Prüfverfahren §7 bis §9
S5-	Anforderungen Stufe 5 (Dok. 11: Betriebsmodi, Start)	S5-01 bis S5-12	39	Dok. 11 (27), CP G5-004 (6), Dok. 00 (3), Dok. 12 (2), CP G5-010 (1)	Beschreibung §1 und §2
S6-	Anforderungen Stufe 6 (Dok. 12, dieses Vorhaben)	S6-01 bis S6-12	56	Dok. 12 (40), Dok. 00 (8), CP G6-001 (4), CP G6-002 (2), CP G6-003 (2)	entfällt mit Dok. 12; Ergebnis im Prüfverfahren §8
O4-	Offene Fragen Stufe 4 (Dok. 10)	O4-01 bis O4-18	168	Dok. 10 (67), Dok. 05 (40), CP Stufe3 (32), Dok. 00 (19), Dok. 03 (5), Dok. 04 (4)	entschiedene: Register und §6; offene: §10
O5-	Offene Fragen Stufe 5 (Dok. 11)	O5-01 bis O5-05	33	Dok. 11 (9), Dok. 00 (6), CP G5-005 (5), CP G5-004 (4), CP G5-006 (4), Dok. 10 (1)	entschiedene: Register und §1/§2; offene (O5-3 Windows): Prüfverfahren §9
O6-	Offene Fragen Stufe 6 (Dok. 12)	O6-01 bis O6-07	50	Dok. 12 (14), Dok. 00 (12), CP G6-002 (11), CP G6-001 (7), CP G6-003 (6)	entfallen mit Dok. 12; Entscheidungen sind in den Zieldokumenten umgesetzt
U-	in Dok. 09 und 10 (z. B. U-9, U-10, U-11 neben S3-14 und Z-1); genaue Bedeutung beim Schreiben je Fundstelle klären	U-01 bis U-12	243	CP Stufe3 (94), Dok. 09 (76), Dok. 00 (38), Dok. 05 (16), Dok. 10 (7), Dok. 03 (5)	Beschreibung §5 (Dok. 09) bzw. §6 (Dok. 10); Handprüfung: Prüfverfahren §7
B-	Befunde (Dok. 09, 10, 11, Checkpoints); **B-2 = Windows-Bau tonmisch**	B-01 bis B-09	174	Dok. 09 (52), Dok. 10 (33), CP Stufe3 (33), Dok. 00 (18), Dok. 05 (8), CP G5-007 (8)	Prüfverfahren §10 (Bekannte Befunde); B-2: Prüfverfahren §9
A-	Abnahmekriterien (A-1 bis A-8; Dok. 09)	A-01 bis A-08	79	Dok. 09 (35), CP Stufe3 (25), Dok. 00 (18), Dok. 05 (1)	Prüfverfahren §8
A5-	Abnahmekriterien Stufe 5 (Dok. 11); **A5-8 = Windows**	A5-01 bis A5-08	100	Dok. 11 (20), CP G5-010 (14), CP G5-009 (13), Dok. 00 (11), CP G5-008 (9), Dok. 12 (7)	Prüfverfahren §8; A5-8: §9
A6-	Abnahmekriterien Stufe 6 (Dok. 12)	A6-01 bis A6-08	27	Dok. 12 (23), Dok. 00 (2), CP G6-001 (2)	Dok. 12 entfällt; Ergebnis im Prüfverfahren §8
G1-	Änderungskennung (Verlauf Dok. 00 §6), zur Stufe 1; Bedeutung des Buchstaben-Zahlen-Schemas beim Schreiben bestätigen	G1-02 bis G1-02	2	Dok. 07 (1), CP Stufe2 (1)	Versionierung (Zählung der GRAFIK-Module, S6-03/S6-09)
G2-	Änderungskennung (Verlauf Dok. 00 §6), z. B. G2-003 `ton/`, G2-011/-012 E und M	G2-02 bis G2-12	38	CP Stufe2 (18), Dok. 01 (5), AENDERUNG 0.03-015 (4), Dok. 00 (2), Dok. 06 (2), Dok. 10 (2)	Versionierung; Offenes aus CHECKPOINT_Stufe2: §10 (Triage S6-04)
G3-	Änderungskennung (Verlauf Dok. 00 §6), z. B. G3-004 Zeichenlage SW	G3-01 bis G3-08	72	CP Stufe3 (38), Dok. 09 (18), Dok. 00 (9), AENDERUNG 0.03-016 (4), Dok. 05 (2), Dok. 10 (1)	Versionierung; Offenes aus CHECKPOINT_Stufe3: §10 (Triage S6-04)

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Familie	Bedeutung	Bereich	Erwähnungen	Fundstellen (Auswahl, Anzahl)	Zielabschnitt (Vorschlag)
G4-	Änderungskennung (Verlauf Dok. 00 §6), z. B. G4-005, G4-007	G4-01 bis G4-19	52	Dok. 10 (37), Dok. 09 (5), Dok. 00 (4), CP Stufe3 (2), Dok. 01 (1), Dok. 05 (1)	Versionierung; Offenes: §10
G5-	Änderungskennung Stufe 5 (nicht zu verwechseln mit den Dateien `CHECKPOINT_G5-001` bis `-010`)	G5-01 bis G5-10	59	Dok. 11 (17), Dok. 00 (8), Dok. 12 (7), Dok. 10 (5), Dok. 04 (3), CP G5-004 (3)	Versionierung; Offenes: §10 (Triage S6-04)
G6-	Stufe 6 (Dateien `CHECKPOINT_G6-001` bis `-004`; Dok. 12 nennt „Checkpoint G6-nnn“)	G6-01 bis G6-04	15	Dok. 12 (5), Dok. 00 (4), CP G6-003 (3), CP G6-002 (2), CP G6-001 (1)	entfallen nach Abschluss; Meilenstein in der Versionierung
SWTK-	Tk-Testläufe des SW-Fensters (Tk-Testskripte)	SWTK-01 bis SWTK-04	46	Dok. 09 (23), CP Stufe3 (15), Dok. 00 (4), Dok. 05 (4)	Prüfverfahren §3 (Testskripte) und §7
Z-	Rückfragen in Dok. 09 (Z-1, Z-2; Z-1 am 07:56 beantwortet, betrifft Vormerkung)	Z-01 bis Z-02	33	Dok. 09 (12), CP Stufe3 (12), Dok. 00 (7), Dok. 05 (2)	Beschreibung §5; Prüfverfahren §10
VM-	Änderungskennung zur Vormerkung (VM-001, Stufe 3)	VM-01 bis VM-03	45	CP Stufe3 (19), Dok. 09 (18), Dok. 05 (5), Dok. 00 (3)	Beschreibung §5
RLA-	Testlauf-Kennung (RLA-001/-002; RLA-002 weist den Gravis als `[BEKANNT]` aus)	RLA-01 bis RLA-02	12	Dok. 09 (6), CP Stufe3 (5), Dok. 00 (1)	Prüfverfahren §6 (R300-Eingabe)
DL-	Änderungskennung (DL-003)	DL-01 bis DL-03	8	CP Stufe3 (4), CP Stufe2 (2), Dok. 00 (1), Dok. 09 (1)	Beschreibung §2 (Schalter), Prüfverfahren §3
HZ-	Änderungskennung zur Haltzeit (HZ-001/-002, `grafik_haltzeit.py`)	HZ-01 bis HZ-04	5	Dok. 10 (5)	Beschreibung §8.3 (Haltzeit; Verweis in §6.5)
FL-	Änderungskennung zur Fensterlage (FL-001, `grafik_fensterlage.py`)	FL-01 bis FL-01	3	Dok. 00 (1), Dok. 05 (1), Dok. 10 (1)	Beschreibung §3
HW-	Änderungskennung zum Hardware-Check (HW-003)	HW-01 bis HW-04	20	Dok. 12 (5), Dok. 00 (3), AENDERUNG 0.03-015 (3), Dok. 08 (2), CP Stufe2 (2), CP Stufe3 (2)	Beschreibung §8
E-	E-1 bis E-4 (Dok. 05/09: E unter M) und E-001 (Änderung)	E-01 bis E-04	13	CP Stufe3 (8), Dok. 00 (2), Dok. 09 (2), Dok. 05 (1)	Beschreibung §3; Versionierung 0.03-015
D-	Entscheidungspunkte Ton (D-8, D-9, D-13; Dok. 03, SW_Ton)	D-01 bis D-13	65	Dok. 00 (20), SW_Ton (19), Dok. 03 (18), Dok. 05 (8)	Beschreibung §6 und `ton/SW_Ton_Beschreibung.md` (bleibt)
O-	O-1 bis O-12 der ersten Pakete (Dok. 00 bis 03, 05, 09)	O-01 bis O-13	79	Dok. 00 (35), Dok. 05 (18), Dok. 02 (13), Dok. 09 (5), Dok. 01 (4), CP Stufe3 (3)	entschiedene: Register; offene: §10
SW-	SW-Kennungen (SW_Ton)	SW-01 bis SW-07	14	SW_Ton (14)	`ton/SW_Ton_Beschreibung.md` (bleibt)
LS-	Lochstreifen/Leser-Töne (SW_Ton)	LS-01 bis LS-05	11	SW_Ton (11)	`ton/SW_Ton_Beschreibung.md` (bleibt)

Familie	Bedeutung	Bereich	Erwähnungen	Fundstellen (Auswahl, Anzahl)	Zielabschnitt (Vorschlag)
ALL-	Sammelkennung (SW_Ton)	ALL-01 bis ALL-03	6	SW_Ton (6)	`ton/SW_Ton_Beschreibung.md` (bleibt)
PTK-	Pult-Tastentest (nur in CHECKPOINT_Stufe3 erwähnt)	PTK-01 bis PTK-01	3	C8205-Interpreter-Pult_Ta (2), CP Stufe3 (1)	Prüfverfahren §5

Nicht aufgenommen (Treffer ohne Kennungscharakter, z. B. Normen, Zeichen, Teilenummern): BIS, C, RE, UTF.

B.3 Verweise im Code und in Dokumenten (später umzustellen)

Code (Kommentare und Docstrings; Zählung per Textsuche, die Zahlen weichen von Dok. 12 §2 ab, weil dort ein weiteres Muster mitgezählt wurde: hier 58 Treffer in 22 Dateien):

Datei	Treffer	Maßnahme
`c-module/python/c8205_ton_methoden.h`	1	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`c-module/tonmisch.c`	1	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`c-module/tonmisch.h`	1	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`c8205_grafik.py`	8	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`grafik-interpreter-module/grafik_geraete.py`	2	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`grafik-interpreter-module/grafik_geraeteton.py`	2	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`grafik-interpreter-module/grafik_k0.py`	2	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`grafik-interpreter-module/grafik_pult.py`	10	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`grafik-interpreter-module/grafik_pult_fenster.py`	2	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`grafik-interpreter-module/grafik_rechnerton.py`	1	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`grafik-interpreter-module/grafik_schreibwerk.py`	2	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`grafik-interpreter-module/grafik_sw.py`	7	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`grafik-interpreter-module/grafik_sw_fenster.py`	5	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`grafik-interpreter-module/grafik_ton.py`	1	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`grafik-interpreter-module/grafik_tonmisch.py`	2	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`grafik-interpreter-module/grafik_tonmisch_c.py`	1	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`grafik-interpreter-module/grafik_trommel.py`	2	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`testscripte/grafik_checkpoint5_tests.py`	1	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`testscripte/grafik_gleichwertigkeit.py`	1	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`testscripte/grafik_sw_pult_test.py`	2	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`testscripte/grafik_sw_vormerkung_s3_13.py`	2	auf neue Abschnitte umstellen (S6-07, nur Kommentare)
`testscripte/r300_eingabe_log_auswerten.py`	2	auf neue Abschnitte umstellen (S6-07, nur Kommentare)

C 8205 – GRAFIK-Interpreter: Beschreibung

Quelle: C8205-Interpreter-GRAFIK_Beschreibung.md · im Wortlaut der Quelle

Dokumente außerhalb der Anforderungspakete:

Datei	Treffer	Maßnahme
README.md	2	Verweise umstellen (S6-07/S6-08)
README.en.md	2	Verweise umstellen (S6-07/S6-08)
markdown-dateien/C8205-Interpreter-Python_Interpreter.md	1	Verweise umstellen (S6-07/S6-08)
markdown-dateien/C8205-Interpreter-Bedienpult_mit_Schaltern.md	1	Verweise umstellen (S6-07/S6-08)
markdown-dateien/C8205-Projektstruktur.md	5	Verweise umstellen (S6-07/S6-08)
markdown-dateien/C8205-Interpreter-Pult_Tastentest_GRAFIK.md	2	Verweise umstellen (S6-07/S6-08)
markdown-dateien/C8205-Interpreter-Lochstreifenleser.md	1	Verweise umstellen (S6-07/S6-08)
ton/SW_Ton_Beschreibung.md	15	Verweise umstellen (S6-07/S6-08)

Knotengrenze in GRAFIK (G4-014/G4-015, 26.10.2026)

In GRAFIK ist der Endlosschleifen-Schutz eine Rechenzeit: 1000 Knoten entsprechen 1 ms (Annahme von Claude, Auftrag des Projektinhabers), die Grenze je `lauf()` ist seit G4-015 18 000 000 ms (5 Stunden), das sind 18 000 000 000 Knoten; GRAFIK muss 5 Stunden am Stück laufen können (Auftrag des Projektinhabers) (`KNOTEN_JE_MS`, `MAX_ZEIT_MS` in `c8205_grafik.py`). Der Schalter `--max-knoten` gibt es nur im Modus TEST (Standard dort 2 000 000).